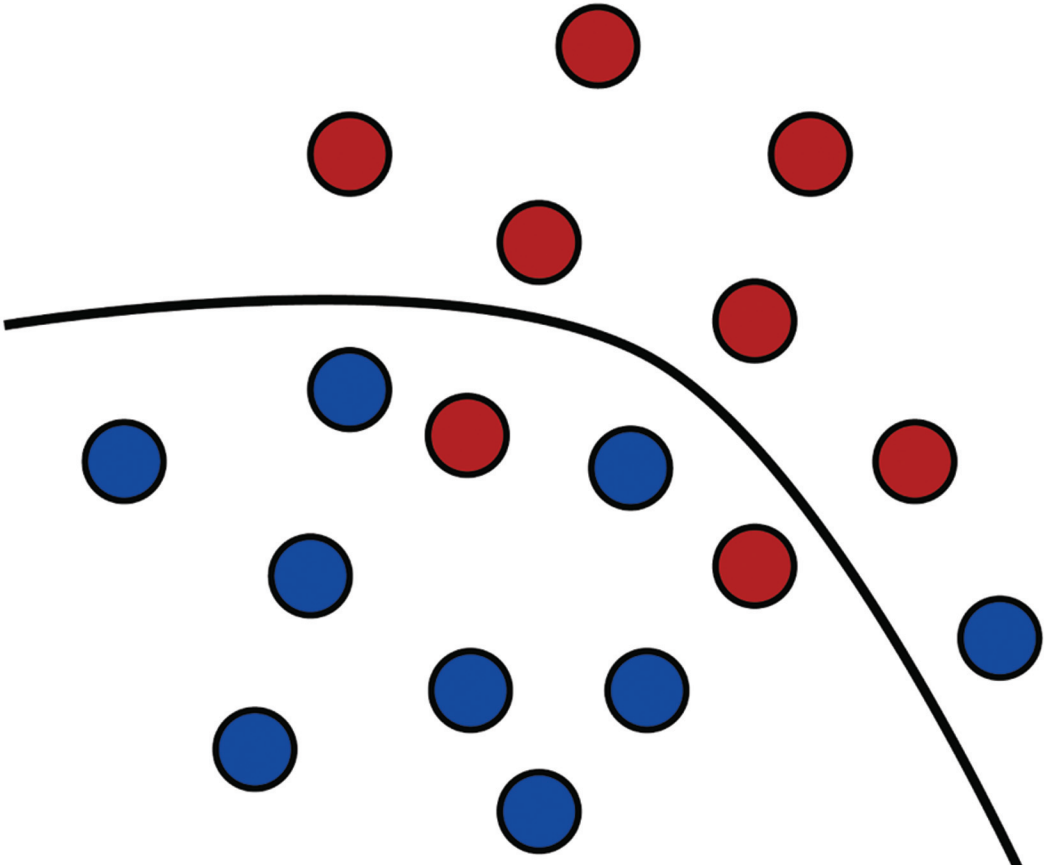
Foundations of Machine Learning



Mehryar Mohri,
Afshin Rostamizadeh,
and Ameet Talwalkar

Foundations of Machine Learning

Adaptive Computation and Machine Learning

Thomas Dietterich, Editor

Christopher Bishop, David Heckerman, Michael Jordan, and Michael Kearns, Associate Editors

A complete list of books published in The Adaptive Computations and Machine Learning series appears at the back of this book.

Foundations of Machine Learning

Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar

The MIT Press
Cambridge, Massachusetts
London, England

MIT Press books may be purchased at special quantity discounts for business or sales promotional use. For information, please email special_sales@mitpress.mit.edu or write to Special Sales Department, The MIT Press, 55 Hayward Street, Cambridge, MA 02142.

This book was set in LaTeX by the authors. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Mohri, Mehryar.
Foundations of machine learning / Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar.
p. cm. - (Adaptive computation and machine learning series)
Includes bibliographical references and index.
ISBN 978-0-262-01825-8 (hardcover : alk. paper) 1. Machine learning. 2. Computer algorithms. I. Rostamizadeh, Afshin. II. Talwalkar, Ameet. III. Title.
Q325.5.M64 2012
006.3'1-dc23

2012007249

10 9 8 7 6 5 4 3 2 1

Contents

Preface

This book is a general introduction to machine learning that can serve as a textbook for students and researchers in the field. It covers fundamental modern topics in machine learning while providing the theoretical basis and conceptual tools needed for the discussion and justification of algorithms. It also describes several key aspects of the application of these algorithms.

We have aimed to present the most novel theoretical tools and concepts while giving concise proofs, even for relatively advanced results. In general, whenever possible, we have chosen to favor succinctness. Nevertheless, we discuss some crucial complex topics arising in machine learning and highlight several open research questions. Certain topics often merged with others or treated with insufficient attention are discussed separately here and with more emphasis: for example, a different chapter is reserved for multi-class classification, ranking, and regression.

Although we cover a very wide variety of important topics in machine learning, we have chosen to omit a few important ones, including graphical models and neural networks, both for the sake of brevity and because of the current lack of solid theoretical guarantees for some methods.

The book is intended for students and researchers in machine learning, statistics and other related areas. It can be used as a textbook for both graduate and advanced undergraduate classes in machine learning or as a reference text for a research seminar. The first three chapters of the book lay the theoretical foundation for the subsequent material. Other chapters are mostly self-contained, with the exception of chapter 5 which introduces some concepts that are extensively used in later ones. Each chapter concludes with a series of exercises, with full solutions presented separately.

The reader is assumed to be familiar with basic concepts in linear algebra, probability, and analysis of algorithms. However, to further help him, we present in the appendix a concise linear algebra and a probability review, and a short introduction to convex optimization. We have also collected in the appendix a number of useful tools for concentration bounds used in this book.

To our knowledge, there is no single textbook covering all of the material presented here. The need for a unified presentation has been pointed out to us

every year by our machine learning students. There are several good books for various specialized areas, but these books do not include a discussion of other fundamental topics in a general manner. For example, books about kernel methods do not include a discussion of other fundamental topics such as boosting, ranking, reinforcement learning, learning automata or online learning. There also exist more general machine learning books, but the theoretical foundation of our book and our emphasis on proofs make our presentation quite distinct.

Most of the material presented here takes its origins in a machine learning graduate course (*Foundations of Machine Learning*) taught by the first author at the Courant Institute of Mathematical Sciences in New York University over the last seven years. This book has considerably benefited from the comments and suggestions from students in these classes, along with those of many friends, colleagues and researchers to whom we are deeply indebted.

We are particularly grateful to Corinna Cortes and Yishay Mansour who have both made a number of key suggestions for the design and organization of the material presented with detailed comments that we have fully taken into account and that have greatly improved the presentation. We are also grateful to Yishay Mansour for using a preliminary version of the book for teaching and for reporting his feedback to us.

We also thank for discussions, suggested improvement, and contributions of many kinds the following colleagues and friends from academic and corporate research laboratories: Cyril Allauzen, Stephen Boyd, Spencer Greenberg, Lisa Hellerstein, Sanjiv Kumar, Ryan McDonald, Andres Muñoz Medina, Tyler Neylon, Peter Norvig, Fernando Pereira, Maria Pershina, Ashish Rastogi, Michael Riley, Umar Syed, Csaba Szepesvári, Eugene Weinstein, and Jason Weston.

Finally, we thank the MIT Press publication team for their help and support in the development of this text.

1 Introduction

Machine learning can be broadly defined as computational methods using experience to improve performance or to make accurate predictions. Here, *experience* refers to the past information available to the learner, which typically takes the form of electronic data collected and made available for analysis. This data could be in the form of digitized human-labeled training sets, or other types of information obtained via interaction with the environment. In all cases, its quality and size are crucial to the success of the predictions made by the learner.

Machine learning consists of designing efficient and accurate prediction *algorithms*. As in other areas of computer science, some critical measures of the quality of these algorithms are their time and space complexity. But, in machine learning, we will need additionally a notion of *sample complexity* to evaluate the sample size required for the algorithm to learn a family of concepts. More generally, theoretical learning guarantees for an algorithm depend on the complexity of the concept classes considered and the size of the training sample.

Since the success of a learning algorithm depends on the data used, machine learning is inherently related to data analysis and statistics. More generally, learning techniques are data-driven methods combining fundamental concepts in computer science with ideas from statistics, probability and optimization.

1.1 Applications and problems

Learning algorithms have been successfully deployed in a variety of applications, including

- Text or document classification, e.g., spam detection;
- Natural language processing, e.g., morphological analysis, part-of-speech tagging, statistical parsing, named-entity recognition;
- Speech recognition, speech synthesis, speaker verification;
- Optical character recognition (OCR);
- Computational biology applications, e.g., protein function or structured predic-

tion;

- Computer vision tasks, e.g., image recognition, face detection;
- Fraud detection (credit card, telephone) and network intrusion;
- Games, e.g., chess, backgammon;
- Unassisted vehicle control (robots, navigation);
- Medical diagnosis;
- Recommendation systems, search engines, information extraction systems.

This list is by no means comprehensive, and learning algorithms are applied to new applications every day. Moreover, such applications correspond to a wide variety of learning problems. Some major classes of learning problems are:

- *Classification*: Assign a category to each item. For example, document classification may assign items with categories such as *politics, business, sports*, or *weather* while image classification may assign items with categories such as *landscape, portrait*, or *animal*. The number of categories in such tasks is often relatively small, but can be large in some difficult tasks and even unbounded as in OCR, text classification, or speech recognition.
- *Regression*: Predict a real value for each item. Examples of regression include prediction of stock values or variations of economic variables. In this problem, the penalty for an incorrect prediction depends on the magnitude of the difference between the true and predicted values, in contrast with the classification problem, where there is typically no notion of closeness between various categories.
- *Ranking*: Order items according to some criterion. Web search, e.g., returning web pages relevant to a search query, is the canonical ranking example. Many other similar ranking problems arise in the context of the design of information extraction or natural language processing systems.
- *Clustering*: Partition items into homogeneous regions. Clustering is often performed to analyze very large data sets. For example, in the context of social network analysis, clustering algorithms attempt to identify "communities" within large groups of people.
- *Dimensionality reduction* or *manifold learning*: Transform an initial representation of items into a lower-dimensional representation of these items while preserving some properties of the initial representation. A common example involves preprocessing digital images in computer vision tasks.

The main practical objectives of machine learning consist of generating accurate predictions for unseen items and of designing efficient and robust algorithms to produce these predictions, even for large-scale problems. To do so, a number of algorithmic and theoretical questions arise. Some fundamental questions include:

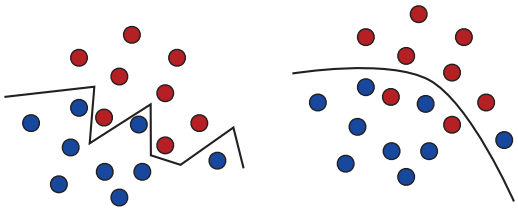


Figure 1.1 The zig-zag line on the left panel is consistent over the blue and red training sample, but it is a complex separation surface that is not likely to generalize well to unseen data. In contrast, the decision surface on the right panel is simpler and might generalize better in spite of its misclassification of a few points of the training sample.

Which concept families can actually be learned, and under what conditions? How well can these concepts be learned computationally?

1.2 Definitions and terminology

We will use the canonical problem of spam detection as a running example to illustrate some basic definitions and to describe the use and evaluation of machine learning algorithms in practice. Spam detection is the problem of learning to automatically classify email messages as either SPAM or non-SPAM.

- *Examples*: Items or instances of data used for learning or evaluation. In our spam problem, these examples correspond to the collection of email messages we will use for learning and testing.
- *Features*: The set of attributes, often represented as a vector, associated to an example. In the case of email messages, some relevant features may include the length of the message, the name of the sender, various characteristics of the header, the presence of certain keywords in the body of the message, and so on.
- *Labels*: Values or categories assigned to examples. In classification problems, examples are assigned specific categories, for instance, the SPAM and non-SPAM categories in our binary classification problem. In regression, items are assigned real-valued labels.
- *Training sample*: Examples used to train a learning algorithm. In our spam problem, the training sample consists of a set of email examples along with their associated labels. The training sample varies for different learning scenarios, as described in section 1.4.
- *Validation sample*: Examples used to tune the parameters of a learning algorithm

when working with labeled data. Learning algorithms typically have one or more free parameters, and the validation sample is used to select appropriate values for these model parameters.

- *Test sample*: Examples used to evaluate the performance of a learning algorithm. The test sample is separate from the training and validation data and is not made available in the learning stage. In the spam problem, the test sample consists of a collection of email examples for which the learning algorithm must predict labels based on features. These predictions are then compared with the labels of the test sample to measure the performance of the algorithm.

- *Loss function*: A function that measures the difference, or loss, between a predicted label and a true label. Denoting the set of all labels as $\mathcal{Y}$ and the set of possible predictions as $\mathcal{Y}'$, a loss function L is a mapping $L\colon \mathcal{Y} \times \mathcal{Y}' \to \mathbb{R}_+$. In most cases, $\mathcal{Y}' = \mathcal{Y}$ and the loss function is bounded, but these conditions do not always hold. Common examples of loss functions include the zero-one (or misclassification) loss defined over $\{-1, +1\} \times \{-1, +1\}$ by $L(y, y') = 1_{y' \neq y}$ and the squared loss defined over $I \times I$ by $L(y, y') = (y' - y)^2$, where $I \subseteq \mathbb{R}$ is typically a bounded interval.

- *Hypothesis set*: A set of functions mapping features (feature vectors) to the set of labels $\mathcal{Y}$. In our example, these may be a set of functions mapping email features to $\mathcal{Y} = \{\text{SPAM}, \text{non-SPAM}\}$. More generally, hypotheses may be functions mapping features to a different set $\mathcal{Y}'$. They could be linear functions mapping email feature vectors to real numbers interpreted as *scores* ($\mathcal{Y}' = \mathbb{R}$), with higher score values more indicative of SPAM than lower ones.

We now define the learning stages of our spam problem. We start with a given collection of labeled examples. We first randomly partition the data into a training sample, a validation sample, and a test sample. The size of each of these samples depends on a number of different considerations. For example, the amount of data reserved for validation depends on the number of free parameters of the algorithm. Also, when the labeled sample is relatively small, the amount of training data is often chosen to be larger than that of test data since the learning performance directly depends on the training sample.

Next, we associate relevant features to the examples. This is a critical step in the design of machine learning solutions. Useful features can effectively guide the learning algorithm, while poor or uninformative ones can be misleading. Although it is critical, to a large extent, the choice of the features is left to the user. This choice reflects the user's *prior knowledge* about the learning task which in practice can have a dramatic effect on the performance results.

Now, we use the features selected to train our learning algorithm by fixing different values of its free parameters. For each value of these parameters, the algorithm

selects a different hypothesis out of the hypothesis set. We choose among them the hypothesis resulting in the best performance on the validation sample. Finally, using that hypothesis, we predict the labels of the examples in the test sample. The performance of the algorithm is evaluated by using the loss function associated to the task, e.g., the zero-one loss in our spam detection task, to compare the predicted and true labels.

Thus, the performance of an algorithm is of course evaluated based on its test error and not its error on the training sample. A learning algorithm may be *consistent*, that is it may commit no error on the examples of the training data, and yet have a poor performance on the test data. This occurs for consistent learners defined by very complex decision surfaces, as illustrated in figure 1.1, which tend to memorize a relatively small training sample instead of seeking to generalize well. This highlights the key distinction between memorization and generalization, which is the fundamental property sought for an accurate learning algorithm. Theoretical guarantees for consistent learners will be discussed with great detail in chapter 2.

1.3 Cross-validation

In practice, the amount of labeled data available is often too small to set aside a validation sample since that would leave an insufficient amount of training data. Instead, a widely adopted method known as *n-fold cross-validation* is used to exploit the labeled data both for *model selection* (selection of the free parameters of the algorithm) and for training.

Let $\boldsymbol{\theta}$ denote the vector of free parameters of the algorithm. For a fixed value of $\boldsymbol{\theta}$, the method consists of first randomly partitioning a given sample S of m labeled examples into n subsamples, or folds. The ith fold is thus a labeled sample $((x_{i1}, y_{i1}), \ldots, (x_{im_i}, y_{im_i}))$ of size m_i. Then, for any $i \in [1, n]$, the learning algorithm is trained on all but the ith fold to generate a hypothesis h_i, and the performance of h_i is tested on the ith fold, as illustrated in figure 1.2a. The parameter value $\boldsymbol{\theta}$ is evaluated based on the average error of the hypotheses h_i, which is called the *cross-validation error*. This quantity is denoted by $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ and defined by

$$\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \underbrace{\frac{1}{m_i} \sum_{j=1}^{m_i} L(h_i(x_{ij}), y_{ij})}_{\text{error of } h_i \text{ on the } i\text{th fold}} .$$

The folds are generally chosen to have equal size, that is $m_i = m/n$ for all $i \in [1, n]$. How should n be chosen? The appropriate choice is subject to a trade-off and the topic of much learning theory research that we cannot address in this introductory

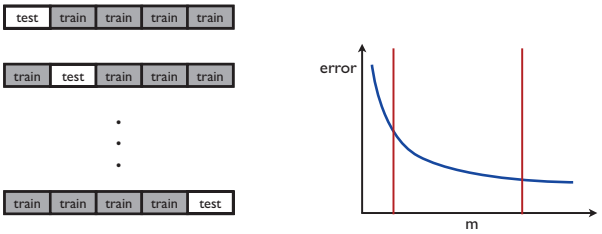


Figure 1.2 n-fold cross validation. (a) Illustration of the partitioning of the training data into 5 folds. (b) Typical plot of a classifier's prediction error as a function of the size of the training sample: the error decreases as a function of the number of training points.

chapter. For a large n, each training sample used in n-fold cross-validation has size $m - m/n = m(1-1/n)$ (illustrated by the right vertical red line in figure 1.2b), which is close to m, the size of the full sample, but the training samples are quite similar. Thus, the method tends to have a small bias but a large variance. In contrast, smaller values of n lead to more diverse training samples but their size (shown by the left vertical red line in figure 1.2b) is significantly less than m, thus the method tends to have a smaller variance but a larger bias.

In machine learning applications, n is typically chosen to be 5 or 10. n-fold cross validation is used as follows in model selection. The full labeled data is first split into a training and a test sample. The training sample of size m is then used to compute the n-fold cross-validation error $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ for a small number of possible values of $\boldsymbol{\theta}$. $\boldsymbol{\theta}$ is next set to the value $\boldsymbol{\theta}_0$ for which $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ is smallest and the algorithm is trained with the parameter setting $\boldsymbol{\theta}_0$ over the full training sample of size m. Its performance is evaluated on the test sample as already described in the previous section.

The special case of n-fold cross validation where $n = m$ is called *leave-one-out cross-validation*, since at each iteration exactly one instance is left out of the training sample. As shown in chapter 4, the average leave-one-out error is an approximately unbiased estimate of the average error of an algorithm and can be used to derive simple guarantees for some algorithms. In general, the leave-one-out error is very costly to compute, since it requires training n times on samples of size $m - 1$, but for some algorithms it admits a very efficient computation (see exercise 10.9).

In addition to model selection, n-fold cross validation is also commonly used for performance evaluation. In that case, for a fixed parameter setting $\boldsymbol{\theta}$, the full labeled sample is divided into n random folds with no distinction between training and test samples. The performance reported is the n-fold cross-validation on the full sample as well as the standard deviation of the errors measured on each fold.

1.4 Learning scenarios

We next briefly describe common machine learning scenarios. These scenarios differ in the types of training data available to the learner, the order and method by which training data is received and the test data used to evaluate the learning algorithm.

- *Supervised learning*: The learner receives a set of labeled examples as training data and makes predictions for all unseen points. This is the most common scenario associated with classification, regression, and ranking problems. The spam detection problem discussed in the previous section is an instance of supervised learning.
- *Unsupervised learning*: The learner exclusively receives unlabeled training data, and makes predictions for all unseen points. Since in general no labeled example is available in that setting, it can be difficult to quantitatively evaluate the performance of a learner. Clustering and dimensionality reduction are example of unsupervised learning problems.
- *Semi-supervised learning*: The learner receives a training sample consisting of both labeled and unlabeled data, and makes predictions for all unseen points. Semi-supervised learning is common in settings where unlabeled data is easily accessible but labels are expensive to obtain. Various types of problems arising in applications, including classification, regression, or ranking tasks, can be framed as instances of semi-supervised learning. The hope is that the distribution of unlabeled data accessible to the learner can help him achieve a better performance than in the supervised setting. The analysis of the conditions under which this can indeed be realized is the topic of much modern theoretical and applied machine learning research.
- *Transductive inference*: As in the semi-supervised scenario, the learner receives a labeled training sample along with a set of unlabeled test points. However, the objective of transductive inference is to predict labels only for these particular test points. Transductive inference appears to be an easier task and matches the scenario encountered in a variety of modern applications. However, as in the semi-supervised setting, the assumptions under which a better performance can be achieved in this setting are research questions that have not been fully resolved.
- *On-line learning*: In contrast with the previous scenarios, the online scenario involves multiple rounds and training and testing phases are intermixed. At each round, the learner receives an unlabeled training point, makes a prediction, receives the true label, and incurs a loss. The objective in the on-line setting is to minimize the cumulative loss over all rounds. Unlike the previous settings just discussed, no distributional assumption is made in on-line learning. In fact, instances and their labels may be chosen adversarially within this scenario.

- *Reinforcement learning*: The training and testing phases are also intermixed in reinforcement learning. To collect information, the learner actively interacts with the environment and in some cases affects the environment, and receives an immediate reward for each action. The object of the learner is to maximize his reward over a course of actions and iterations with the environment. However, no long-term reward feedback is provided by the environment, and the learner is faced with the *exploration versus exploitation* dilemma, since he must choose between exploring unknown actions to gain more information versus exploiting the information already collected.
- *Active learning*: The learner adaptively or interactively collects training examples, typically by querying an oracle to request labels for new points. The goal in active learning is to achieve a performance comparable to the standard supervised learning scenario, but with fewer labeled examples. Active learning is often used in applications where labels are expensive to obtain, for example computational biology applications.

In practice, many other intermediate and somewhat more complex learning scenarios may be encountered.

1.5 Outline

This book presents several fundamental and mathematically well-studied algorithms. It discusses in depth their theoretical foundations as well as their practical applications. The topics covered include:

- Probably approximately correct (PAC) learning framework; learning guarantees for finite hypothesis sets;
- Learning guarantees for infinite hypothesis sets, Rademacher complexity, VC-dimension;
- Support vector machines (SVMs), margin theory;
- Kernel methods, positive definite symmetric kernels, representer theorem, rational kernels;
- Boosting, analysis of empirical error, generalization error, margin bounds;
- Online learning, mistake bounds, the weighted majority algorithm, the exponential weighted average algorithm, the Perceptron and Winnow algorithms;
- Multi-class classification, multi-class SVMs, multi-class boosting, one-versus-all, one-versus-one, error-correction methods;
- Ranking, ranking with SVMs, RankBoost, bipartite ranking, preference-based

ranking;

- Regression, linear regression, kernel ridge regression, support vector regression, Lasso;
- Stability-based analysis, applications to classification and regression;
- Dimensionality reduction, principal component analysis (PCA), kernel PCA, Johnson-Lindenstrauss lemma;
- Learning automata and languages;
- Reinforcement learning, Markov decision processes, planning and learning problems.

The analyses in this book are self-contained, with relevant mathematical concepts related to linear algebra, convex optimization, probability and statistics included in the appendix.

2 The PAC Learning Framework

Several fundamental questions arise when designing and analyzing algorithms that learn from examples: What can be learned efficiently? What is inherently hard to learn? How many examples are needed to learn successfully? Is there a general model of learning? In this chapter, we begin to formalize and address these questions by introducing the *Probably Approximately Correct* (PAC) learning framework. The PAC framework helps define the class of learnable concepts in terms of the number of sample points needed to achieve an approximate solution, *sample complexity*, and the time and space complexity of the learning algorithm, which depends on the cost of the computational representation of the concepts.

We first describe the PAC framework and illustrate it, then present some general learning guarantees within this framework when the hypothesis set used is finite, both for the *consistent* case where the hypothesis set used contains the concept to learn and for the opposite *inconsistent* case.

2.1 The PAC learning model

We first introduce several definitions and the notation needed to present the PAC model, which will also be used throughout much of this book.

We denote by $\mathcal{X}$ the set of all possible *examples* or *instances*. $\mathcal{X}$ is also sometimes referred to as the *input space*. The set of all possible *labels* or *target values* is denoted by $\mathcal{Y}$. For the purpose of this introductory chapter, we will limit ourselves to the case where $\mathcal{Y}$ is reduced to two labels, $\mathcal{Y} = \{0, 1\}$, so-called *binary classification*. Later chapters will extend these results to more general settings.

A *concept* c: $\mathcal{X} \to \mathcal{Y}$ is a mapping from $\mathcal{X}$ to $\mathcal{Y}$. Since $\mathcal{Y} = \{0, 1\}$, we can identify c with the subset of $\mathcal{X}$ over which it takes the value 1. Thus, in the following, we equivalently refer to a concept to learn as a mapping from $\mathcal{X}$ to $\{0, 1\}$, or to a subset of $\mathcal{X}$. As an example, a concept may be the set of points inside a triangle or the indicator function of these points. In such cases, we will say in short that the concept to learn is a triangle. A *concept class* is a set of concepts we may wish to learn and is denoted by C. This could, for example, be the set of all triangles in the

plane.

We assume that examples are independently and identically distributed (i.i.d.) according to some fixed but unknown distribution D. The learning problem is then formulated as follows. The learner considers a fixed set of possible concepts H, called a *hypothesis set*, which may not coincide with C. He receives a sample $S = (x_1, \ldots, x_m)$ drawn i.i.d. according to D as well as the labels $(c(x_1), \ldots, c(x_m))$, which are based on a specific target concept $c \in C$ to learn. His task is to use the labeled sample S to select a hypothesis $h_S \in H$ that has a small *generalization error* with respect to the concept c. The generalization error of a hypothesis $h \in H$, also referred to as the *true error* or just *error* of h is denoted by $R(h)$ and defined as follows.[1]

Definition 2.1 **Generalization error**
Given a hypothesis $h \in H$, a target concept $c \in C$, and an underlying distribution D, the generalization error *or* risk *of h is defined by*

$$R(h) = \Pr_{x \sim D}[h(x) \neq c(x)] = \mathop{\mathrm{E}}_{x \sim D}\left[1_{h(x) \neq c(x)}\right], \tag{2.1}$$

where 1_ω is the indicator function of the event ω.[2]

The generalization error of a hypothesis is not directly accessible to the learner since both the distribution D and the target concept c are unknown. However, the learner can measure the *empirical error* of a hypothesis on the labeled sample S.

Definition 2.2 **Empirical error**
Given a hypothesis $h \in H$, a target concept $c \in C$, and a sample $S = (x_1, \ldots, x_m)$, the empirical error *or* empirical risk *of h is defined by*

$$\widehat{R}(h) = \frac{1}{m} \sum_{i=1}^{m} 1_{h(x_i) \neq c(x_i)}. \tag{2.2}$$

Thus, the empirical error of $h \in H$ is its average error over the sample S, while the generalization error is its expected error based on the distribution D. We will see in this chapter and the following chapters a number of guarantees relating to these two quantities with high probability, under some general assumptions. We can already note that for a fixed $h \in H$, the expectation of the empirical error based on an i.i.d.

1. The choice of R instead of E to denote an error avoids possible confusions with the notation for expectations and is further justified by the fact that the term *risk* is also used in machine learning and statistics to refer to an error.
2. For this and other related definitions, the family of functions H and the target concept c must be measurable. The function classes we consider in this book all have this property.

sample S is equal to the generalization error:

$$\mathrm{E}[\widehat{R}(h)] = R(h). \tag{2.3}$$

Indeed, by the linearity of the expectation and the fact that the sample is drawn i.i.d., we can write

$$\mathop{\mathrm{E}}_{S\sim D^m}[\widehat{R}(h)] = \frac{1}{m}\sum_{i=1}^{m} \mathop{\mathrm{E}}_{S\sim D^m}[1_{h(x_i)\neq c(x_i)}] = \frac{1}{m}\sum_{i=1}^{m} \mathop{\mathrm{E}}_{S\sim D^m}[1_{h(x)\neq c(x)}],$$

for any x in sample S. Thus,

$$\mathop{\mathrm{E}}_{S\sim D^m}[\widehat{R}(h)] = \mathop{\mathrm{E}}_{S\sim D^m}[1_{\{h(x)\neq c(x)\}}] = \mathop{\mathrm{E}}_{x\sim D}[1_{\{h(x)\neq c(x)\}}] = R(h).$$

The following introduces the *Probably Approximately Correct* (PAC) learning framework. We denote by $O(n)$ an upper bound on the cost of the computational representation of any element $x \in \mathcal{X}$ and by size(c) the maximal cost of the computational representation of $c \in C$. For example, x may be a vector in $\mathbb{R}^n$, for which the cost of an array-based representation would be in $O(n)$.

Definition 2.3 **PAC-learning**
A concept class C is said to be PAC-learnable *if there exists an algorithm $\mathcal{A}$ and a polynomial function poly$(\cdot,\cdot,\cdot,\cdot)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions D on $\mathcal{X}$ and for any target concept $c \in C$, the following holds for any sample size $m \geq$ poly$(1/\epsilon, 1/\delta, n,$ size$(c))$:*

$$\Pr_{S\sim D^m}[R(h_S) \leq \epsilon] \geq 1 - \delta. \tag{2.4}$$

If $\mathcal{A}$ further runs in poly$(1/\epsilon, 1/\delta, n,$ size$(c))$, then C is said to be efficiently PAC-learnable. *When such an algorithm $\mathcal{A}$ exists, it is called a* PAC-learning algorithm *for C.*

A concept class C is thus PAC-learnable if the hypothesis returned by the algorithm after observing a number of points polynomial in $1/\epsilon$ and $1/\delta$ is *approximately correct* (error at most ϵ) with high *probability* (at least $1-\delta$), which justifies the PAC terminology. $\delta > 0$ is used to define the *confidence* $1-\delta$ and $\epsilon > 0$ the *accuracy* $1-\epsilon$. Note that if the running time of the algorithm is polynomial in $1/\epsilon$ and $1/\delta$, then the sample size m must also be polynomial if the full sample is received by the algorithm.

Several key points of the PAC definition are worth emphasizing. First, the PAC framework is a *distribution-free model* : no particular assumption is made about the distribution D from which examples are drawn. Second, the training sample and the test examples used to define the error are drawn according to the same distribution D. This is a necessary assumption for generalization to be possible in most cases.

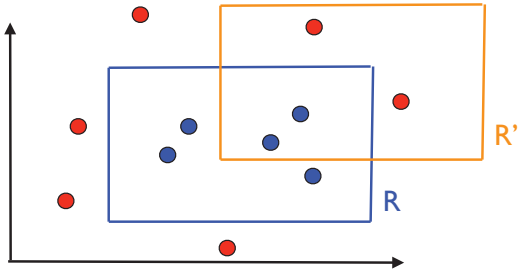


Figure 2.1 Target concept R and possible hypothesis R′. Circles represent training instances. A blue circle is a point labeled with 1, since it falls within the rectangle R. Others are red and labeled with 0.

Finally, the PAC framework deals with the question of learnability for a concept class C and not a particular concept. Note that the concept class C is known to the algorithm, but of course target concept $c \in C$ is unknown.

In many cases, in particular when the computational representation of the concepts is not explicitly discussed or is straightforward, we may omit the polynomial dependency on n and size(c) in the PAC definition and focus only on the sample complexity.

We now illustrate PAC-learning with a specific learning problem.

Example 2.1 Learning axis-aligned rectangles

Consider the case where the set of instances are points in the plane, $\mathcal{X} = \mathbb{R}^2$, and the concept class C is the set of all axis-aligned rectangles lying in $\mathbb{R}^2$. Thus, each concept c is the set of points inside a particular axis-aligned rectangle. The learning problem consists of determining with small error a target axis-aligned rectangle using the labeled training sample. We will show that the concept class of axis-aligned rectangles is PAC-learnable.

Figure 2.1 illustrates the problem. R represents a target axis-aligned rectangle and R′ a hypothesis. As can be seen from the figure, the error regions of R′ are formed by the area within the rectangle R but outside the rectangle R′ and the area within R′ but outside the rectangle R. The first area corresponds to *false negatives*, that is, points that are labeled as 0 or *negatively* by R′, which are in fact *positive* or labeled with 1. The second area corresponds to *false positives*, that is, points labeled positively by R′ which are in fact negatively labeled.

To show that the concept class is PAC-learnable, we describe a simple PAC-learning algorithm $\mathcal{A}$. Given a labeled sample S, the algorithm consists of returning the tightest axis-aligned rectangle $\mathsf{R}' = \mathsf{R}_S$ containing the points labeled with 1. Figure 2.2 illustrates the hypothesis returned by the algorithm. By definition, R_S does not produce any false positive, since its points must be included in the target concept R. Thus, the error region of R_S is included in R.

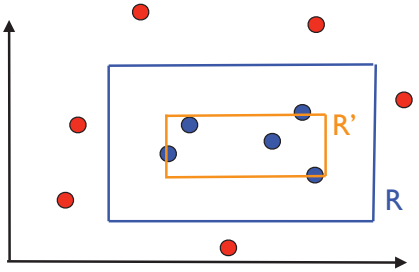


Figure 2.2 Illustration of the hypothesis $\mathsf{R}' = \mathsf{R}_\mathsf{S}$ returned by the algorithm.

Let $\mathsf{R} \in C$ be a target concept. Fix $\epsilon > 0$. Let $\Pr[\mathsf{R}_\mathsf{S}]$ denote the probability mass of the region defined by R_S, that is the probability that a point randomly drawn according to D falls within R_S. Since errors made by our algorithm can be due only to points falling inside R_S, we can assume that $\Pr[\mathsf{R}_\mathsf{S}] > \epsilon$; otherwise, the error of R_S is less than or equal to ϵ regardless of the training sample S received.

Now, since $\Pr[\mathsf{R}_\mathsf{S}] > \epsilon$, we can define four rectangular regions r_1, r_2, r_3, and r_4 along the sides of R_S, each with probability at least $\epsilon/4$. These regions can be constructed by starting with the empty rectangle along a side and increasing its size until its distribution mass is at least $\epsilon/4$. Figure 2.3 illustrates the definition of these regions.

Observe that if R_S meets all of these four regions, then, because it is a rectangle, it will have one side in each of these four regions (geometric argument). Its error area, which is the part of R that it does not cover, is thus included in these regions and cannot have probability mass more than ϵ. By contraposition, if $R(\mathsf{R}_\mathsf{S}) > \epsilon$, then R_S must miss at least one of the regions r_i, $i \in [1, 4]$. As a result, we can write

$$\begin{aligned}
\Pr_{S\sim D^m}[R(\mathsf{R}_\mathsf{S}) > \epsilon] &\leq \Pr_{S\sim D^m}[\cup_{i=1}^4 \{\mathsf{R}_\mathsf{S} \cap r_i = \emptyset\}] && (2.5)\\
&\leq \sum_{i=1}^4 \Pr_{S\sim D^m}[\{\mathsf{R}_\mathsf{S} \cap r_i = \emptyset\}] && \text{(by the union bound)}\\
&\leq 4(1-\epsilon/4)^m && (\text{since } \Pr[r_i] > \epsilon/4)\\
&\leq 4\exp(-m\epsilon/4),
\end{aligned}$$

where for the last step we used the general identity $1 - x \leq e^{-x}$ valid for all $x \in \mathbb{R}$. For any $\delta > 0$, to ensure that $\Pr_{S\sim D^m}[R(\mathsf{R}_\mathsf{S}) > \epsilon] \leq \delta$, we can impose

$$4\exp(-\epsilon m/4) \leq \delta \Leftrightarrow m \geq \frac{4}{\epsilon}\log\frac{4}{\delta}. \tag{2.6}$$

Thus, for any $\epsilon > 0$ and $\delta > 0$, if the sample size m is greater than $\frac{4}{\epsilon}\log\frac{4}{\delta}$, then $\Pr_{S\sim D^m}[R(\mathsf{R}_\mathsf{S}) > \epsilon] \leq 1 - \delta$. Furthermore, the computational cost of the

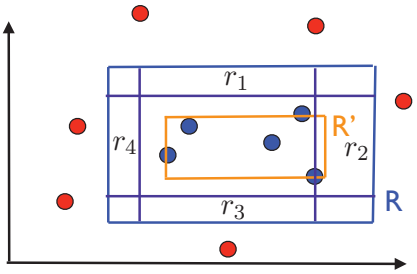


Figure 2.3 Illustration of the regions $r_1, \ldots, r_4$.

representation of points in $\mathbb{R}^2$ and axis-aligned rectangles, which can be defined by their four corners, is constant. This proves that the concept class of axis-aligned rectangles is PAC-learnable and that the sample complexity of PAC-learning axis-aligned rectangles is in $O(\frac{1}{\epsilon} \log \frac{1}{\delta})$.

An equivalent way to present sample complexity results like (2.6), which we will often see throughout this book, is to give a *generalization bound*. It states that with probability at least $1 - \delta$, $R(\mathsf{R_S})$ is upper bounded by some quantity that depends on the sample size m and δ. To obtain this, if suffices to set δ to be equal to the upper bound derived in (2.5), that is $\delta = 4 \exp(-m\epsilon/4)$ and solve for ϵ. This yields that with probability at least $1 - \delta$, the error of the algorithm is bounded as:

$$R(\mathsf{R_S}) \leq \frac{4}{m} \log \frac{4}{\delta}. \tag{2.7}$$

Other PAC-learning algorithms could be considered for this example. One alternative is to return the largest axis-aligned rectangle not containing the negative points, for example. The proof of PAC-learning just presented for the tightest axis-aligned rectangle can be easily adapted to the analysis of other such algorithms.

Note that the hypothesis set H we considered in this example coincided with the concept class C and that its cardinality was infinite. Nevertheless, the problem admitted a simple proof of PAC-learning. We may then ask if a similar proof can readily apply to other similar concept classes. This is not as straightforward because the specific geometric argument used in the proof is key. It is non-trivial to extend the proof to other concept classes such as that of non-concentric circles (see exercise 2.4). Thus, we need a more general proof technique and more general results. The next two sections provide us with such tools in the case of a finite hypothesis set.

2.2 Guarantees for finite hypothesis sets — consistent case

In the example of axis-aligned rectangles that we examined, the hypothesis h_S returned by the algorithm was always *consistent*, that is, it admitted no error on the training sample S. In this section, we present a general sample complexity bound, or equivalently, a generalization bound, for consistent hypotheses, in the case where the cardinality $|H|$ of the hypothesis set is finite. Since we consider consistent hypotheses, we will assume that the target concept c is in H.

Theorem 2.1 **Learning bounds — finite H, consistent case**
Let H be a finite set of functions mapping from $\mathcal{X}$ to $\mathcal{Y}$. Let $\mathcal{A}$ be an algorithm that for any target concept $c \in H$ and i.i.d. sample S returns a consistent hypothesis h_S: $\widehat{R}(h_S) = 0$. Then, for any $\epsilon, \delta > 0$, the inequality $\Pr_{S\sim D^m}[R(h_S) \leq \epsilon] \geq 1 - \delta$ holds if

$$m \geq \frac{1}{\epsilon}\Big(\log |H| + \log \frac{1}{\delta}\Big). \tag{2.8}$$

This sample complexity result admits the following equivalent statement as a generalization bound: for any $\epsilon, \delta > 0$, with probability at least $1 - \delta$,

$$R(h_S) \leq \frac{1}{m}\Big(\log |H| + \log \frac{1}{\delta}\Big). \tag{2.9}$$

Proof Fix $\epsilon > 0$. We do not know which consistent hypothesis $h_S \in H$ is selected by the algorithm $\mathcal{A}$. This hypothesis further depends on the training sample S. Therefore, we need to give a *uniform convergence bound*, that is, a bound that holds for the set of all consistent hypotheses, which a fortiori includes h_S. Thus, we will bound the probability that some $h \in H$ would be consistent and have error more than ϵ:

$$\begin{aligned}
&\Pr[\exists h \in H \colon \widehat{R}(h) = 0 \wedge R(h) > \epsilon] \\
&= \Pr[(h_1 \in H,\, \widehat{R}(h_1) = 0 \wedge R(h_1) > \epsilon) \vee (h_2 \in H,\, \widehat{R}(h_2) = 0 \wedge R(h_2) > \epsilon) \vee \cdots] \\
&\leq \sum_{h\in H} \Pr[\widehat{R}(h) = 0 \wedge R(h) > \epsilon] && \text{(union bound)} \\
&\leq \sum_{h\in H} \Pr[\widehat{R}(h) = 0 \mid R(h) > \epsilon]. && \text{(definition of conditional probability)}
\end{aligned}$$

Now, consider any hypothesis $h \in H$ with $R(h) > \epsilon$. Then, the probability that h would be consistent on a training sample S drawn i.i.d., that is, that it would have no error on any point in S, can be bounded as:

$$\Pr[\widehat{R}(h) = 0 \mid R(h) > \epsilon] \leq (1 - \epsilon)^m.$$

The previous inequality implies

$$\Pr[\exists h \in H \colon \widehat{R}(h) = 0 \wedge R(h) > \epsilon] \leq |H|(1-\epsilon)^m.$$

Setting the right-hand side to be equal to δ and solving for ϵ concludes the proof. ■

The theorem shows that when the hypothesis set H is finite, a consistent algorithm $\mathcal{A}$ is a PAC-learning algorithm, since the sample complexity given by (2.8) is dominated by a polynomial in $1/\epsilon$ and $1/\delta$. As shown by (2.9), the generalization error of consistent hypotheses is upper bounded by a term that decreases as a function of the sample size m. This is a general fact: as expected, learning algorithms benefit from larger labeled training samples. The decrease rate of $O(1/m)$ guaranteed by this theorem, however, is particularly favorable.

The price to pay for coming up with a consistent algorithm is the use of a larger hypothesis set H containing target concepts. Of course, the upper bound (2.9) increases with $|H|$. However, that dependency is only logarithmic. Note that the term $\log |H|$, or the related term $\log_2 |H|$ from which it differs by a constant factor, can be interpreted as the number of bits needed to represent H. Thus, the generalization guarantee of the theorem is controlled by the ratio of this number of bits, $\log_2 |H|$, and the sample size m.

We now use theorem 2.1 to analyze PAC-learning with various concept classes.

Example 2.2 Conjunction of Boolean literals

Consider learning the concept class C_n of conjunctions of at most n Boolean literals $x_1, \ldots, x_n$. A Boolean literal is either a variable x_i, $i \in [1, n]$, or its negation $\overline{x}_i$. For $n = 4$, an example is the conjunction: $x_1 \wedge \overline{x}_2 \wedge x_4$, where $\overline{x}_2$ denotes the negation of the Boolean literal x_2. $(1, 0, 0, 1)$ is a positive example for this concept while $(1, 0, 0, 0)$ is a negative example.

Observe that for $n = 4$, a positive example $(1, 0, 1, 0)$ implies that the target concept cannot contain the literals $\overline{x}_1$ and $\overline{x}_3$ and that it cannot contain the literals x_2 and x_4. In contrast, a negative example is not as informative since it is not known which of its n bits are incorrect. A simple algorithm for finding a consistent hypothesis is thus based on positive examples and consists of the following: for each positive example $(b_1, \ldots, b_n)$ and $i \in [1, n]$, if $b_i = 1$ then $\overline{x}_i$ is ruled out as a possible literal in the concept class and if $b_i = 0$ then x_i is ruled out. The conjunction of all the literals not ruled out is thus a hypothesis consistent with the target. Figure 2.4 shows an example training sample as well as a consistent hypothesis for the case $n = 6$.

We have $|H| = |C_n| = 3^n$, since each literal can be included positively, with negation, or not included. Plugging this into the sample complexity bound for consistent hypotheses yields the following sample complexity bound for any $\epsilon > 0$

0	1	1	0	1	1	+
0	1	1	1	1	1	+
0	0	1	1	0	1	-
0	1	1	1	1	1	+
1	0	0	1	1	0	-
0	1	0	0	1	1	+
0	1	?	?	1	1	

Figure 2.4 Each of the first six rows of the table represents a training example with its label, + or −, indicated in the last column. The last row contains 0 (respectively 1) in column $i \in [1, 6]$ if the ith entry is 0 (respectively 1) for all the positive examples. It contains "?" if both 0 and 1 appear as an ith entry for some positive example. Thus, for this training sample, the hypothesis returned by the consistent algorithm described in the text is $\overline{x}_1 \wedge x_2 \wedge x_5 \wedge x_6$.

and $\delta > 0$:

$$m \geq \frac{1}{\epsilon}\left((\log 3)n + \log\frac{1}{\delta}\right). \tag{2.10}$$

Thus, the class of conjunctions of at most n Boolean literals is PAC-learnable. Note that the computational complexity is also polynomial, since the training cost per example is in $O(n)$. For $\delta = 0.02$, $\epsilon = 0.1$, and $n = 10$, the bound becomes $m \geq 149$. Thus, for a labeled sample of at least 149 examples, the bound guarantees 99% accuracy with a confidence of at least 98%.

Example 2.3 Universal concept class

Consider the set $\mathcal{X} = \{0, 1\}^n$ of all Boolean vectors with n components, and let U_n be the concept class formed by all subsets of $\mathcal{X}$. Is this concept class PAC-learnable? To guarantee a consistent hypothesis the hypothesis class must include the concept class, thus $|H| \geq |U_n| = 2^{(2^n)}$. Theorem 2.1 gives the following sample complexity bound:

$$m \geq \frac{1}{\epsilon}\left((\log 2)2^n + \log\frac{1}{\delta}\right). \tag{2.11}$$

Here, the number of training samples required is exponential in n, which is the cost of the representation of a point in $\mathcal{X}$. Thus, PAC-learning is not guaranteed by the theorem. In fact, it is not hard to show that this universal concept class is not PAC-learnable.

Example 2.4 k-term DNF formulae

A disjunctive normal form (DNF) formula is a formula written as the disjunction of several terms, each term being a conjunction of Boolean literals. A k-term DNF is a DNF formula defined by the disjunction of k terms, each term being a conjunction of at most n Boolean literals. Thus, for $k = 2$ and $n = 3$, an example of a k-term DNF is $(x_1 \wedge \overline{x}_2 \wedge x_3) \vee (\overline{x}_1 \wedge x_3)$.

Is the class C of k-term DNF formulae is PAC-learnable? The cardinality of the class is 3^{nk}, since each term is a conjunction of at most n variables and there are 3^n such conjunctions, as seen previously. The hypothesis set H must contain C for consistency to be possible, thus $|H| \geq 3^{nk}$. Theorem 2.1 gives the following sample complexity bound:

$$m \geq \frac{1}{\epsilon}\Big((\log 3)nk + \log\frac{1}{\delta}\Big), \tag{2.12}$$

which is polynomial. However, it can be shown that the problem of learning k-term DNF is in RP, the complexity class of problems that admit a randomized polynomial-time decision solution. The problem is therefore computationally intractable unless RP = NP, which is commonly conjectured not to be the case. Thus, while the sample size needed for learning k-term DNF formulae is only polynomial, efficient PAC-learning of this class is not possible unless RP = NP.

Example 2.5 k-CNF formulae

A conjunctive normal form (CNF) formula is a conjunction of disjunctions. A k-CNF formula is an expression of the form $T_1 \wedge \ldots \wedge T_j$ with arbitrary length $j \in \mathbb{N}$ and with each term T_i being a disjunction of at most k Boolean attributes.

The problem of learning k-CNF formulae can be reduced to that of learning conjunctions of Boolean literals, which, as seen previously, is a PAC-learnable concept class. To do so, it suffices to associate to each term T_i a new variable. Then, this can be done with the following bijection:

$$a_i(x_1) \vee \cdots \vee a_i(x_n) \rightarrow Y_{a_i(x_1),\ldots,a_i(x_n)}, \tag{2.13}$$

where $a_i(x_j)$ denotes the assignment to x_j in term T_i. This reduction to PAC-learning of conjunctions of Boolean literals may affect the original distribution, but this is not an issue since in the PAC framework no assumption is made about the distribution. Thus, the PAC-learnability of conjunctions of Boolean literals implies that of k-CNF formulae.

This is a surprising result, however, since any k-term DNF formula can be written as a k-CNF formula. Indeed, using associativity, a k-term DNF can be rewritten as

a k-CNF formula via

$$\bigvee_{i=1}^{k} a_i(x_1) \wedge \cdots \wedge a_i(x_n) = \bigwedge_{i_1,\ldots,i_k=1}^{n} a_1(x_{i_1}) \vee \cdots \vee a_k(x_{i_k}).$$

To illustrate this rewriting in a specific case, observe, for example, that

$$(u_1 \wedge u_2 \wedge u_3) \vee (v_1 \wedge v_2 \wedge v_3) = \bigwedge_{i,j=1}^{3} (u_i \wedge v_j).$$

But, as we previously saw, k-term DNF formulae are not efficiently PAC-learnable! What can explain this apparent inconsistency? Observe that the number of new variables needed to write a k-term DNF as a k-CNF formula via the transformation just described is exponential in k, it is in $O(n^k)$. The discrepancy comes from the size of the representation of a concept. A k-term DNF formula can be an exponentially more compact representation, and efficient PAC-learning is intractable if a time-complexity polynomial in that size is required. Thus, this apparent paradox deals with key aspects of PAC-learning, which include the cost of the representation of a concept and the choice of the hypothesis set.

2.3 Guarantees for finite hypothesis sets — inconsistent case

In the most general case, there may be no hypothesis in H consistent with the labeled training sample. This, in fact, is the typical case in practice, where the learning problems may be somewhat difficult or the concept classes more complex than the hypothesis set used by the learning algorithm. However, inconsistent hypotheses with a small number of errors on the training sample can be useful and, as we shall see, can benefit from favorable guarantees under some assumptions. This section presents learning guarantees precisely for this inconsistent case and finite hypothesis sets.

To derive learning guarantees in this more general setting, we will use Hoeffding's inequality (theorem D.1) or the following corollary, which relates the generalization error and empirical error of a single hypothesis.

Corollary 2.1
Fix $\epsilon > 0$ and let S denote an i.i.d. sample of size m. Then, for any hypothesis $h\colon X \to \{0,1\}$, the following inequalities hold:

$$\Pr_{S\sim D^m}[\widehat{R}(h) - R(h) \geq \epsilon] \leq \exp(-2m\epsilon^2) \tag{2.14}$$

$$\Pr_{S\sim D^m}[\widehat{R}(h) - R(h) \leq -\epsilon] \leq \exp(-2m\epsilon^2). \tag{2.15}$$

By the union bound, this implies the following two-sided inequality:

$$\Pr_{S\sim D^m}\left[|\widehat{R}(h) - R(h)| \geq \epsilon\right] \leq 2\exp(-2m\epsilon^2). \tag{2.16}$$

Proof The result follows immediately theorem D.1. ■

Setting the right-hand side of (2.16) to be equal to δ and solving for ϵ yields immediately the following bound for a single hypothesis.

Corollary 2.2 **Generalization bound — single hypothesis**
Fix a hypothesis $h\colon \mathcal{X} \to \{0,1\}$. Then, for any $\delta > 0$, the following inequality holds with probability at least $1-\delta$:

$$R(h) \leq \widehat{R}(h) + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{2.17}$$

The following example illustrates this corollary in a simple case.

Example 2.6 Tossing a coin
Imagine tossing a biased coin that lands heads with probability p, and let our hypothesis be the one that always guesses heads. Then the true error rate is $R(h) = p$ and the empirical error rate $\widehat{R}(h) = \widehat{p}$, where $\widehat{p}$ is the empirical probability of heads based on the training sample drawn i.i.d. Thus, corollary 2.2 guarantees with probability at least $1-\delta$ that

$$|p - \widehat{p}| \leq \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{2.18}$$

Therefore, if we choose $\delta = 0.02$ and use a sample of size 500, with probability at least 98%, the following approximation quality is guaranteed for $\widehat{p}$:

$$|p - \widehat{p}| \leq \sqrt{\frac{\log(10)}{1000}} \approx 0.048. \tag{2.19}$$

Can we readily apply corollary 2.2 to bound the generalization error of the hypothesis h_S returned by a learning algorithm when training on a sample S? No, since h_S is not a fixed hypothesis, but a random variable depending on the training sample S drawn. Note also that unlike the case of a fixed hypothesis for which

the expectation of the empirical error is the generalization error (equation 2.3), the generalization error $R(h_S)$ is a random variable and in general distinct from the expectation $\mathrm{E}[\widehat{R}(h_S)]$, which is a constant.

Thus, as in the proof for the consistent case, we need to derive a uniform convergence bound, that is a bound that holds with high probability for all hypotheses $h \in H$.

Theorem 2.2 **Learning bound — finite H, inconsistent case**
Let H be a finite hypothesis set. Then, for any $\delta > 0$, with probability at least $1-\delta$, the following inequality holds:

$$\forall h \in H, \quad R(h) \leq \widehat{R}(h) + \sqrt{\frac{\log |H| + \log \frac{2}{\delta}}{2m}}. \tag{2.20}$$

Proof Let $h_1, \ldots, h_{|H|}$ be the elements of H. Using the union bound and applying corollary 2.2 to each hypothesis yield:

$$\begin{aligned}
&\Pr\left[\exists h \in H \middle| \widehat{R}(h) - R(h)\middle| > \epsilon\right] \\
&\qquad = \Pr\left[\left(\middle|\widehat{R}(h_1) - R(h_1)\middle| > \epsilon\right) \vee \ldots \vee \left(\middle|\widehat{R}(h_{|H|}) - R(h_{|H|})\middle| > \epsilon\right)\right] \\
&\qquad \leq \sum_{h \in H} \Pr\left[\middle|\widehat{R}(h) - R(h)\middle| > \epsilon\right] \\
&\qquad \leq 2|H| \exp(-2m\epsilon^2).
\end{aligned}$$

Setting the right-hand side to be equal to δ completes the proof. ■

Thus, for a finite hypothesis set H,

$$R(h) \leq \widehat{R}(h) + O\left(\sqrt{\frac{\log_2 |H|}{m}}\right).$$

As already pointed out, $\log_2 |H|$ can be interpreted as the number of bits needed to represent H. Several other remarks similar to those made on the generalization bound in the consistent case can be made here: a larger sample size m guarantees better generalization, and the bound increases with $|H|$, but only logarithmically. But, here, the bound is a less favorable function of $\frac{\log_2 |H|}{m}$; it varies as the square root of this term. This is not a minor price to pay: for a fixed $|H|$, to attain the same guarantee as in the consistent case, a quadratically larger labeled sample is needed.

Note that the bound suggests seeking a trade-off between reducing the empirical error versus controlling the size of the hypothesis set: a larger hypothesis set is penalized by the second term but could help reduce the empirical error, that is the first term. But, for a similar empirical error, it suggests using a smaller hypothesis

set. This can be viewed as an instance of the so-called *Occam's Razor principle* named after the theologian William of Occam: *Plurality should not be posited without necessity*, also rephrased as, *the simplest explanation is best*. In this context, it could be expressed as follows: All other things being equal, a simpler (smaller) hypothesis set is better.

2.4 Generalities

In this section we will consider several important questions related to the learning scenario, which we left out of the discussion of the earlier sections for simplicity.

2.4.1 Deterministic versus stochastic scenarios

In the most general scenario of supervised learning, the distribution D is defined over $\mathcal{X} \times \mathcal{Y}$, and the training data is a labeled sample S drawn i.i.d. according to D:

$$S = ((x_1, y_1), \ldots, (x_m, y_m)).$$

The learning problem is to find a hypothesis $h \in H$ with small generalization error

$$R(h) = \Pr_{(x,y)\sim D}[h(x) \neq y] = \mathop{\mathrm{E}}_{(x,y)\sim D}[1_{h(x)\neq y}].$$

This more general scenario is referred to as the *stochastic scenario*. Within this setting, the output label is a probabilistic function of the input. The stochastic scenario captures many real-world problems where the label of an input point is not unique. For example, if we seek to predict gender based on input pairs formed by the height and weight of a person, then the label will typically not be unique. For most pairs, both male and female are possible genders. For each fixed pair, there would be a probability distribution of the label being male.

The natural extension of the PAC-learning framework to this setting is known as the *agnostic PAC-learning*.

Definition 2.4 **Agnostic PAC-learning**
Let H be a hypothesis set. $\mathcal{A}$ is an agnostic PAC-learning *algorithm if there exists a polynomial function $poly(\cdot,\cdot,\cdot,\cdot)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions D over $\mathcal{X} \times \mathcal{Y}$, the following holds for any sample size $m \geq poly(1/\epsilon, 1/\delta, n, size(c))$:*

$$\Pr_{S\sim D^m}[R(h_S) - \min_{h\in H} R(h) \leq \epsilon] \geq 1 - \delta. \tag{2.21}$$

If $\mathcal{A}$ further runs in poly$(1/\epsilon, 1/\delta, n, size(c))$, then it is said to be an efficient agnostic PAC-learning algorithm.

When the label of a point can be uniquely determined by some measurable function $f\colon \mathcal{X} \to \mathcal{Y}$ (with probability one), then the scenario is said to be *deterministic*. In that case, it suffices to consider a distribution D over the input space. The training sample is obtained by drawing $(x_1, \ldots, x_m)$ according to D and the labels are obtained via f: $y_i = f(x_i)$ for all $i \in [1, m]$. Many learning problems can be formulated within this deterministic scenario.

In the previous sections, as well as in most of the material presented in this book, we have restricted our presentation to the deterministic scenario in the interest of simplicity. However, for all of this material, the extension to the stochastic scenario should be straightforward for the reader.

2.4.2 Bayes error and noise

In the deterministic case, by definition, there exists a target function f with no generalization error: $R(h) = 0$. In the stochastic case, there is a minimal non-zero error for any hypothesis.

Definition 2.5 **Bayes error**
Given a distribution D over $\mathcal{X} \times \mathcal{Y}$, the Bayes error R^* *is defined as the infimum of the errors achieved by measurable functions $h\colon \mathcal{X} \to \mathcal{Y}$:*

$$R^\star = \inf_{\substack{h \\ h\ measurable}} R(h). \tag{2.22}$$

A hypothesis h with $R(h) = R^*$ is called a Bayes hypothesis *or* Bayes classifier.

By definition, in the deterministic case, we have $R^* = 0$, but, in the stochastic case, $R^* \neq 0$. Clearly, the Bayes classifier h_{Bayes} can be defined in terms of the conditional probabilities as:

$$\forall x \in \mathcal{X}, \quad h_{\text{Bayes}}(x) = \operatorname*{argmax}_{y \in \{0,1\}} \Pr[y|x]. \tag{2.23}$$

The average error made by h_{Bayes} on $x \in \mathcal{X}$ is thus $\min\{\Pr[0|x], \Pr[1|x]\}$, and this is the minimum possible error. This leads to the following definition of *noise*.

Definition 2.6 **Noise**
Given a distribution D over $\mathcal{X} \times \mathcal{Y}$, the noise *at point $x \in \mathcal{X}$ is defined by*

$$noise(x) = \min\{\Pr[1|x], \Pr[0|x]\}. \tag{2.24}$$

The average noise *or the* noise *associated to D is* $\mathrm{E}[noise(x)]$.

Thus, the average noise is precisely the Bayes error: noise $= \mathrm{E}[\mathrm{noise}(x)] = R^*$. The noise is a characteristic of the learning task indicative of its level of difficulty. A point $x \in \mathcal{X}$, for which $\mathrm{noise}(x)$ is close to $1/2$, is sometimes referred to as *noisy* and is of course a challenge for accurate prediction.

2.4.3 Estimation and approximation errors

The difference between the error of a hypothesis $h \in H$ and the Bayes error can be decomposed as:

$$R(h) - R^* = \underbrace{(R(h) - R(h^*))}_{\text{estimation}} + \underbrace{(R(h^*) - R^*)}_{\text{approximation}}, \tag{2.25}$$

where h^* is a hypothesis in H with minimal error, or a *best-in-class hypothesis*.[3]

The second term is referred to as the *approximation error*, since it measures how well the Bayes error can be approximated using H. It is a property of the hypothesis set H, a measure of its richness. The approximation error is not accessible, since in general the underlying distribution D is not known. Even with various noise assumptions, estimating the approximation error is difficult.

The first term is the *estimation error*, and it depends on the hypothesis h selected. It measures the quality of the hypothesis h with respect to the best-in-class hypothesis. The definition of agnostic PAC-learning is also based on the estimation error. The *estimation error of an algorithm* $\mathcal{A}$, that is, the estimation error of the hypothesis h_S returned after training on a sample S, can sometimes be bounded in terms of the generalization error.

For example, let h_S^{ERM} denote the hypothesis returned by the empirical risk minimization algorithm, that is the algorithm that returns a hypothesis h_S^{ERM} with the smallest empirical error. Then, the generalization bound given by theorem 2.2, or any other bound on $\sup_{h \in H} |R(h) - \widehat{R}(h)|$, can be used to bound the estimation error of the empirical risk minimization algorithm. Indeed, rewriting the estimation error to make $\widehat{R}(h_S^{\mathrm{ERM}})$ appear and using $\widehat{R}(h_S^{\mathrm{ERM}}) \leq \widehat{R}(h^*)$, which holds by the definition of the algorithm, we can write

$$\begin{aligned} R(h_S^{\mathrm{ERM}}) - R(h^*) &= R(h_S^{\mathrm{ERM}}) - \widehat{R}(h_S^{\mathrm{ERM}}) + \widehat{R}(h_S^{\mathrm{ERM}}) - R(h^*) \\ &\leq R(h_S^{\mathrm{ERM}}) - \widehat{R}(h_S^{\mathrm{ERM}}) + \widehat{R}(h^*) - R(h^*) \\ &\leq 2 \sup_{h \in H} |R(h) - \widehat{R}(h)|. \end{aligned} \tag{2.26}$$

3. When H is a finite hypothesis set, h^* necessarily exists; otherwise, in this discussion $R(h^*)$ can be replaced by $\inf_{h \in H} R(h)$.

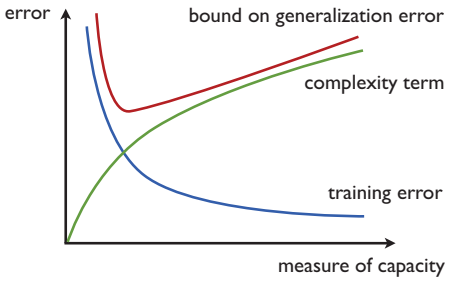


Figure 2.5 Illustration of structural risk minimization. The plots of three errors are shown as a function of a measure of capacity. Clearly, as the size or capacity of the hypothesis set increases, the training error decreases, while the complexity term increases. SRM selects the hypothesis minimizing a bound on the generalization error, which is a sum of the empirical error, and the complexity term is shown in red.

The right-hand side of (2.26) can be bounded by theorem 2.2 and increases with the size of the hypothesis set, while $R(h^*)$ decreases with $|H|$.

2.4.4 Model selection

Here, we discuss some broad model selection and algorithmic ideas based on the theoretical results presented in the previous sections. We assume an i.i.d. labeled training sample S of size m and denote the error of a hypothesis h on S by $\widehat{R}_S(h)$ to explicitly indicate its dependency on S.

While the guarantee of theorem 2.2 holds only for finite hypothesis sets, it already provides us with some useful insights for the design of algorithms and, as we will see in the next chapters, similar guarantees hold in the case of infinite hypothesis sets. Such results invite us to consider two terms: the empirical error and a complexity term, which here is a function of $|H|$ and the sample size m.

In view of that, the ERM algorithm , which only seeks to minimize the error on the training sample

$$h_S^{\mathrm{ERM}} = \operatorname*{argmin}_{h \in H} \widehat{R}_S(h), \tag{2.27}$$

might not be successful, since it disregards the complexity term. In fact, the performance of the ERM algorithm is typically very poor in practice. Additionally, in many cases, determining the ERM solution is computationally intractable. For example, finding a linear hypothesis with the smallest error on the training sample is NP-hard (as a function of the dimension of the space).

Another method known as *structural risk minimization* (SRM) consists of con-

sidering instead an infinite sequence of hypothesis sets with increasing sizes

$$H_0 \subset H_1 \subset \cdots \subset H_n \cdots \tag{2.28}$$

and to find the ERM solution h_n^{ERM} for each H_n. The hypothesis selected is the one among the h_n^{ERM} solutions with the smallest sum of the empirical error and a complexity term complexity(H_n, m) that depends on the size (or more generally the *capacity*, that is, another measure of the richness of H) of H_n, and the sample size m:

$$h_S^{\mathrm{SRM}} = \operatorname*{argmin}_{\substack{h \in H_n \\ n \in \mathbb{N}}} \widehat{R}_S(h) + \text{complexity}(H_n, m). \tag{2.29}$$

Figure 2.5 illustrates the SRM method. While SRM benefits from strong theoretical guarantees, it is typically computationally very expensive, since it requires determining the solution of multiple ERM problems. Note that the number of ERM problems is not infinite if for some n the minimum empirical error is zero: The objective function can only be larger for $n' \geq n$.

An alternative family of algorithms is based on a more straightforward optimization that consists of minimizing the sum of the empirical error and a *regularization* term that penalizes more complex hypotheses. The regularization term is typically defined as $\|h\|^2$ for some norm $\|\cdot\|$ when H is a vector space:

$$h_S^{\mathrm{REG}} = \operatorname*{argmin}_{h \in H} \widehat{R}_S(h) + \lambda \|h\|^2. \tag{2.30}$$

$\lambda \geq 0$ is a *regularization parameter*, which can be used to determine the trade-off between empirical error minimization and control of the complexity. In practice, λ is typically selected using n-fold cross-validation. In the next chapters, we will see a number of different instances of such regularization-based algorithms.

2.5 Chapter notes

The PAC learning framework was introduced by Valiant [1984]. The book of Kearns and Vazirani [1994] is an excellent reference dealing with most aspects of PAC-learning and several other foundational questions in machine learning. Our example of learning axis-aligned rectangles is based on that reference.

The PAC learning framework is a computational framework since it takes into account the cost of the computational representations and the time complexity of the learning algorithm. If we omit the computational aspects, it is similar to the learning framework considered earlier by Vapnik and Chervonenkis [see Vapnik, 2000].

Occam's razor principle is invoked in a variety of contexts, such as in linguistics to justify the superiority of a set of rules or syntax. The Kolmogorov complexity can be viewed as the corresponding framework in information theory. In the context of the learning guarantees presented in this chapter, the principle suggests selecting the most parsimonious explanation (the hypothesis set with the smallest cardinality). We will see in the next sections other applications of this principle with different notions of simplicity or complexity. The idea of structural risk minimization (SRM) is due to Vapnik [1998].

2.6 Exercises

2.1 Two-oracle variant of the PAC model. Assume that positive and negative examples are now drawn from two separate distributions D_+ and D_-. For an accuracy $(1-\epsilon)$, the learning algorithm must find a hypothesis h such that:

$$\Pr_{x\sim D_+}[h(x)=0] \leq \epsilon \text{ and } \Pr_{x\sim D_-}[h(x)=1] \leq \epsilon\,. \tag{2.31}$$

Thus, the hypothesis must have a small error on both distributions. Let C be any concept class and H be any hypothesis space. Let h_0 and h_1 represent the identically 0 and identically 1 functions, respectively. Prove that C is efficiently PAC-learnable using H in the standard (one-oracle) PAC model if and only if it is efficiently PAC-learnable using $H \cup \{h_0, h_1\}$ in this two-oracle PAC model.

2.2 PAC learning of hyper-rectangles. An axis-aligned hyper-rectangle in $\mathbb{R}^n$ is a set of the form $[a_1, b_1] \times \ldots \times [a_n, b_n]$. Show that axis-aligned hyper-rectangles are PAC-learnable by extending the proof given in Example 2.1 for the case $n = 2$.

2.3 Concentric circles. Let $X = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{(x, y)\colon x^2 + y^2 \leq r^2\}$ for some real number r. Show that this class can be (ϵ, δ)-PAC-learned from training data of size $m \geq (1/\epsilon)\log(1/\delta)$.

2.4 Non-concentric circles. Let $X = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{x \in \mathbb{R}^2\colon ||x - x_0|| \leq r\}$ for some point $x_0 \in \mathbb{R}^2$ and real number r. Gertrude, an aspiring machine learning researcher, attempts to show that this class of concepts may be (ϵ, δ)-PAC-learned with sample complexity $m \geq (3/\epsilon)\log(3/\delta)$, but she is having trouble with her proof. Her idea is that the learning algorithm would select the smallest circle consistent with the training data. She has drawn three regions r_1, r_2, r_3 around the edge of concept c, with each region having probability $\epsilon/3$ (see figure 2.6). She wants to argue that if the generalization error is greater than or equal to ϵ, then one of these regions must have been missed by the training data,

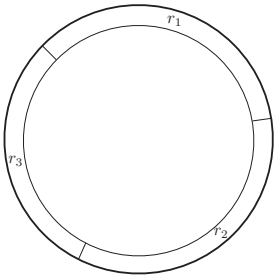


Figure 2.6 Gertrude's regions r_1, r_2, r_3.

and hence this event will occur with probability at most δ. Can you tell Gertrude if her approach works?

2.5 Triangles. Let $X = \mathbb{R}^2$ with orthonormal basis $(\mathbf{e}_1, \mathbf{e}_2)$, and consider the set of concepts defined by the area inside a right triangle ABC with two sides parallel to the axes, with $\overrightarrow{AB}/\|\overrightarrow{AB}\| = \mathbf{e}_1$ and $\overrightarrow{AC}/\|\overrightarrow{AC}\| = \mathbf{e}_2$, and $\|\overrightarrow{AB}\|/\|\overrightarrow{AC}\| = \alpha$ for some positive real $\alpha \in \mathbb{R}_+$. Show, using similar methods to those used in the chapter for the axis-aligned rectangles, that this class can be (ϵ, δ)-PAC-learned from training data of size $m \geq (3/\epsilon)\log(3/\delta)$.

2.6 Learning in the presence of noise — rectangles. In example 2.1, we showed that the concept class of axis-aligned rectangles is PAC-learnable. Consider now the case where the training points received by the learner are subject to the following noise: points negatively labeled are unaffected by noise but the label of a positive training point is randomly flipped to negative with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$. Show that the algorithm described in class returning the tightest rectangle containing positive points can still PAC-learn axis-aligned rectangles in the presence of this noise. To do so, you can proceed using the following steps:

(a) Using the same notation as in example 2.1, assume that $\Pr[\mathsf{R}] > \epsilon$. Suppose that $R(\mathsf{R}') > \epsilon$. Give an upper bound on the probability that R' misses a region r_j, $j \in [1, 4]$ in terms of ϵ and η'?

(b) Use that to give an upper bound on $\Pr[R(\mathsf{R}') > \epsilon]$ in terms of ϵ and η' and conclude by giving a sample complexity bound.

2.7 Learning in the presence of noise — general case. In this question, we will seek a result that is more general than in the previous question. We consider a finite hypothesis set H, assume that the target concept is in H, and adopt the following

noise model: the label of a training point received by the learner is randomly changed with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$.

(a) For any $h \in H$, let $d(h)$ denote the probability that the label of a training point received by the learner disagrees with the one given by h. Let h^* be the target hypothesis, show that $d(h^*) = \eta$.

(b) More generally, show that for any $h \in H$, $d(h) = \eta + (1 - 2\eta)\, R(h)$, where $R(h)$ denotes the generalization error of h.

(c) Fix $\epsilon > 0$ for this and all the following questions. Use the previous questions to show that if $R(h) > \epsilon$, then $d(h) - d(h^*) \geq \epsilon'$, where $\epsilon' = \epsilon(1 - 2\eta')$.

(d) For any hypothesis $h \in H$ and sample S of size m, let $\widehat{d}(h)$ denote the fraction of the points in S whose labels disagree with those given by h. We will consider the algorithm L which, after receiving S, returns the hypothesis h_S with the smallest number of disagreements (thus $\widehat{d}(h_S)$ is minimal). To show PAC-learning for L, we will show that for any h, if $R(h) > \epsilon$, then with high probability $\widehat{d}(h) \geq \widehat{d}(h^*)$. First, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2} \log \frac{2}{\delta}$, the following holds:

$$\widehat{d}(h^*) - d(h^*) \leq \epsilon'/2$$

(e) Second, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$:

$$d(h) - \widehat{d}(h) \leq \epsilon'/2$$

(f) Finally, show that for any $\delta > 0$, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon^2(1-2\eta')^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$ with $R(h) > \epsilon$:

$$\widehat{d}(h) - \widehat{d}(h^*) \geq 0.$$

(*Hint*: use $\widehat{d}(h) - \widehat{d}(h^*) = [\widehat{d}(h) - d(h)] + [d(h) - d(h^*)] + [d(h^*) - \widehat{d}(h^*)]$ and use previous questions to lower bound each of these three terms).

2.8 Learning union of intervals. Let $[a, b]$ and $[c, d]$ be two intervals of the real line with $a \leq b \leq c \leq d$. Let $\epsilon > 0$, and assume that $\Pr_D((b, c)) > \epsilon$, where D is the distribution according to which points are drawn.

(a) Show that the probability that m points are drawn i.i.d. without any of them falling in the interval (b, c) is at most $e^{-m\epsilon}$.

(b) Show that the concept class formed by the union of two closed intervals

in $\mathbb{R}$, e.g., $[a, b] \cup [c, d]$, is PAC-learnable by giving a proof similar to the one given in Example 2.1 for axis-aligned rectangles. (*Hint*: your algorithm might not return a hypothesis consistent with future negative points in this case.)

2.9 Consistent hypotheses. In this chapter, we showed that for a finite hypothesis set H, a consistent learning algorithm $\mathcal{A}$ is a PAC-learning algorithm. Here, we consider a converse question. Let Z be a finite set of m labeled points. Suppose that you are given a PAC-learning algorithm $\mathcal{A}$. Show that you can use $\mathcal{A}$ and a finite training sample S to find in polynomial time a hypothesis $h \in H$ that is consistent with Z, with high probability. (*Hint*: you can select an appropriate distribution D over Z and give a condition on $R(h)$ for h to be consistent.)

2.10 Senate laws. For important questions, President Mouth relies on expert advice. He selects an appropriate advisor from a collection of $H = 2{,}800$ experts.

(a) Assume that laws are proposed in a random fashion independently and identically according to some distribution D determined by an unknown group of senators. Assume that President Mouth can find and select an expert senator out of H who has consistently voted with the majority for the last $m = 200$ laws. Give a bound on the probability that such a senator incorrectly predicts the global vote for a future law. What is the value of the bound with 95% confidence?

(b) Assume now that President Mouth can find and select an expert senator out of H who has consistently voted with the majority for all but $m' = 20$ of the last $m = 200$ laws. What is the value of the new bound?

3 Rademacher Complexity and VC-Dimension

The hypothesis sets typically used in machine learning are infinite. But the sample complexity bounds of the previous chapter are uninformative when dealing with infinite hypothesis sets. One could ask whether efficient learning from a finite sample is even possible when the hypothesis set H is infinite. Our analysis of the family of axis-aligned rectangles (Example 2.1) indicates that this is indeed possible at least in some cases, since we proved that that infinite concept class was PAC-learnable. Our goal in this chapter will be to generalize that result and derive general learning guarantees for infinite hypothesis sets.

A general idea for doing so consists of reducing the infinite case to the analysis of finite sets of hypotheses and then proceed as in the previous chapter. There are different techniques for that reduction, each relying on a different notion of complexity for the family of hypotheses. The first complexity notion we will use is that of *Rademacher complexity*. This will help us derive learning guarantees using relatively simple proofs based on McDiarmid's inequality, while obtaining high-quality bounds, including data-dependent ones, which we will frequently make use of in future chapters. However, the computation of the empirical Rademacher complexity is NP-hard for some hypothesis sets. Thus, we subsequently introduce two other purely combinatorial notions, the *growth function* and the *VC-dimension*. We first relate the Rademacher complexity to the growth function and then bound the growth function in terms of the VC-dimension. The VC-dimension is often easier to bound or estimate. We will review a series of examples showing how to compute or bound it, then relate the growth function and the VC-dimensions. This leads to generalization bounds based on the VC-dimension. Finally, we present lower bounds based on the VC-dimension both in the realizable and non-realizable cases, which will demonstrate the critical role of this notion in learning.

3.1 Rademacher complexity

We will continue to use H to denote a hypothesis set as in the previous chapters, and h an element of H. Many of the results of this section are general and hold for an arbitrary loss function $L\colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$. To each $h\colon \mathcal{X} \to \mathcal{Y}$, we can associate a function g that maps $(x, y) \in \mathcal{X} \times \mathcal{Y}$ to $L(h(x), y)$ without explicitly describing the specific loss L used. In what follows G will generally be interpreted as *the family of loss functions associated to* H.

The Rademacher complexity captures the richness of a family of functions by measuring the degree to which a hypothesis set can fit random noise. The following states the formal definitions of the empirical and average Rademacher complexity.

Definition 3.1 **Empirical Rademacher complexity**
Let G be a family of functions mapping from Z to $[a, b]$ and $S = (z_1, \ldots, z_m)$ a fixed sample of size m with elements in Z. Then, the empirical Rademacher complexity *of G with respect to the sample S is defined as:*

$$\widehat{\mathfrak{R}}_S(G) = \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^{m} \sigma_i g(z_i)\right], \tag{3.1}$$

where $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_m)^\top$, with σ_is independent uniform random variables taking values in $\{-1, +1\}$.[1] *The random variables σ_i are called* Rademacher variables.

Let $\mathbf{g}_S$ denote the vector of values taken by function g over the sample S: $\mathbf{g}_S = (g(z_1), \ldots, g(z_m))^\top$. Then, the empirical Rademacher complexity can be rewritten as

$$\widehat{\mathfrak{R}}_S(G) = \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{g \in G} \frac{\boldsymbol{\sigma} \cdot \mathbf{g}_S}{m}\right].$$

The inner product $\boldsymbol{\sigma} \cdot \mathbf{g}_S$ measures the correlation of $\mathbf{g}_S$ with the vector of random noise $\boldsymbol{\sigma}$. The supremum $\sup_{g \in G} \frac{\boldsymbol{\sigma} \cdot \mathbf{g}_S}{m}$ is a measure of how well the function class G correlates with $\boldsymbol{\sigma}$ over the sample S. Thus, the empirical Rademacher complexity measures on average how well the function class G correlates with random noise on S. This describes the richness of the family G: richer or more complex families G can generate more vectors $\mathbf{g}_S$ and thus better correlate with random noise, on average.

1. We assume implicitly that the supremum over the family G in this definition is measurable and in general will adopt the same assumption throughout this book for other suprema over a class of functions. This assumption does not hold for arbitrary function classes but it is valid for the hypotheses sets typically considered in practice in machine learning, and the instances discussed in this book.

Definition 3.2 **Rademacher complexity**
Let D denote the distribution according to which samples are drawn. For any integer $m \geq 1$, the Rademacher complexity *of G is the expectation of the empirical Rademacher complexity over all samples of size m drawn according to D:*

$$\mathfrak{R}_m(G) = \mathop{\mathrm{E}}_{S\sim D^m}[\widehat{\mathfrak{R}}_S(G)]. \tag{3.2}$$

We are now ready to present our first generalization bounds based on Rademacher complexity.

Theorem 3.1
Let G be a family of functions mapping from Z to $[0,1]$. Then, for any $\delta > 0$, with probability at least $1-\delta$, each of the following holds for all $g \in G$:

$$\mathrm{E}[g(z)] \leq \frac{1}{m}\sum_{i=1}^{m} g(z_i) + 2\mathfrak{R}_m(G) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{3.3}$$

$$\textit{and} \quad \mathrm{E}[g(z)] \leq \frac{1}{m}\sum_{i=1}^{m} g(z_i) + 2\widehat{\mathfrak{R}}_S(G) + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{3.4}$$

Proof For any sample $S = (z_1, \ldots, z_m)$ and any $g \in G$, we denote by $\widehat{\mathrm{E}}_S[g]$ the empirical average of g over S: $\widehat{\mathrm{E}}_S[g] = \frac{1}{m}\sum_{i=1}^{m} g(z_i)$. The proof consists of applying McDiarmid's inequality to function Φ defined for any sample S by

$$\Phi(S) = \sup_{g\in G} \mathrm{E}[g] - \widehat{\mathrm{E}}_S[g]. \tag{3.5}$$

Let S and S' be two samples differing by exactly one point, say z_m in S and z'_m in S'. Then, since the difference of suprema does not exceed the supremum of the difference, we have

$$\Phi(S') - \Phi(S) \leq \sup_{g\in G} \widehat{\mathrm{E}}_S[g] - \widehat{\mathrm{E}}_{S'}[g] = \sup_{g\in G} \frac{g(z_m) - g(z'_m)}{m} \leq \frac{1}{m}. \tag{3.6}$$

Similarly, we can obtain $\Phi(S) - \Phi(S') \leq 1/m$, thus $|\Phi(S) - \Phi(S')| \leq 1/m$. Then, by McDiarmid's inequality, for any $\delta > 0$, with probability at least $1 - \delta/2$, the following holds:

$$\Phi(S) \leq \mathop{\mathrm{E}}_{S}[\Phi(S)] + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{3.7}$$

We next bound the expectation of the right-hand side as follows:

$$\begin{aligned}
\mathop{\mathrm{E}}_{S}[\Phi(S)] &= \mathop{\mathrm{E}}_{S}\Big[\sup_{g\in H} \mathrm{E}[g] - \widehat{\mathrm{E}}_S(g)\Big] \\
&= \mathop{\mathrm{E}}_{S}\Big[\sup_{g\in H} \mathop{\mathrm{E}}_{S'}\big[\widehat{\mathrm{E}}_{S'}(g) - \widehat{\mathrm{E}}_S(g)\big]\Big] && (3.8)\\
&\leq \mathop{\mathrm{E}}_{S,S'}\Big[\sup_{g\in H} \widehat{\mathrm{E}}_{S'}(g) - \widehat{\mathrm{E}}_S(g)\Big] && (3.9)\\
&= \mathop{\mathrm{E}}_{S,S'}\Big[\sup_{g\in H} \frac{1}{m}\sum_{i=1}^{m}(g(z'_i) - g(z_i))\Big] && (3.10)\\
&= \mathop{\mathrm{E}}_{\boldsymbol{\sigma},S,S'}\Big[\sup_{g\in H} \frac{1}{m}\sum_{i=1}^{m}\sigma_i(g(z'_i) - g(z_i))\Big] && (3.11)\\
&\leq \mathop{\mathrm{E}}_{\boldsymbol{\sigma},S'}\Big[\sup_{g\in H} \frac{1}{m}\sum_{i=1}^{m}\sigma_i g(z'_i)\Big] + \mathop{\mathrm{E}}_{\boldsymbol{\sigma},S}\Big[\sup_{g\in H} \frac{1}{m}\sum_{i=1}^{m}-\sigma_i g(z_i)\Big] && (3.12)\\
&= 2\mathop{\mathrm{E}}_{\boldsymbol{\sigma},S}\Big[\sup_{g\in H} \frac{1}{m}\sum_{i=1}^{m}\sigma_i g(z_i)\Big] = 2\mathfrak{R}_m(G). && (3.13)
\end{aligned}$$

Equation 3.8 uses the fact that points in S' are sampled in an i.i.d. fashion and thus $\mathrm{E}[g] = \mathrm{E}_{S'}[\widehat{\mathrm{E}}_{S'}(g)]$, as in (2.3). Inequality 3.9 holds by Jensen's inequality and the convexity of the supremum function. In equation 3.11, we introduce Rademacher variables σ_is, that is uniformly distributed independent random variables taking values in $\{-1, +1\}$ as in definition 3.2. This does not change the expectation appearing in (3.10): when $\sigma_i = 1$, the associated summand remains unchanged; when $\sigma_i = -1$, the associated summand flips signs, which is equivalent to swapping z_i and z'_i between S and S'. Since we are taking the expectation over all possible S and S', this swap does not affect the overall expectation. We are simply changing the order of the summands within the expectation. (3.12) holds by the sub-additivity of the supremum function, that is the identity $\sup(U+V) \leq \sup(U)+\sup(V)$. Finally, (3.13) stems from the definition of Rademacher complexity and the fact that the variables σ_i and $-\sigma_i$ are distributed in the same way.

The reduction to $\mathfrak{R}_m(G)$ in equation 3.13 yields the bound in equation 3.3, using δ instead of $\delta/2$. To derive a bound in terms of $\widehat{\mathfrak{R}}_S(G)$, we observe that, by definition 3.2, changing one point in S changes $\widehat{\mathfrak{R}}_S(G)$ by at most $1/m$. Then, using again McDiarmid's inequality, with probability $1 - \delta/2$ the following holds:

$$\mathfrak{R}_m(G) \leq \widehat{\mathfrak{R}}_S(G) + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{3.14}$$

Finally, we use the union bound to combine inequalities 3.7 and 3.14, which yields

with probability at least $1-\delta$:

$$\Phi(S) \leq 2\widehat{\mathfrak{R}}_S(G) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}, \tag{3.15}$$

which matches (3.4). ■

The following result relates the empirical Rademacher complexities of a hypothesis set H and to the family of loss functions G associated to H in the case of binary loss (zero-one loss).

Lemma 3.1
Let H be a family of functions taking values in $\{-1,+1\}$ and let G be the family of loss functions associated to H for the zero-one loss: $G = \{(x,y) \mapsto 1_{h(x)\neq y} \colon h \in H\}$. For any sample $S = ((x_1,y_1),\ldots,(x_m,y_m))$ of elements in $\mathcal{X} \times \{-1,+1\}$, let $S_{\mathcal{X}}$ denote its projection over $\mathcal{X}$: $S_{\mathcal{X}} = (x_1,\ldots,x_m)$. Then, the following relation holds between the empirical Rademacher complexities of G and H:

$$\widehat{\mathfrak{R}}_S(G) = \frac{1}{2}\widehat{\mathfrak{R}}_{S_{\mathcal{X}}}(H). \tag{3.16}$$

Proof For any sample $S = ((x_1,y_1),\ldots,(x_m,y_m))$ of elements in $\mathcal{X} \times \{-1,+1\}$, by definition, the empirical Rademacher complexity of G can be written as:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(G) &= \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in H} \frac{1}{m}\sum_{i=1}^{m} \sigma_i 1_{h(x_i)\neq y_i}\Big] \\
&= \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in H} \frac{1}{m}\sum_{i=1}^{m} \sigma_i \tfrac{1-y_i h(x_i)}{2}\Big] \\
&= \frac{1}{2}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in H} \frac{1}{m}\sum_{i=1}^{m} -\sigma_i y_i h(x_i)\Big] \\
&= \frac{1}{2}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in H} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)\Big] = \frac{1}{2}\mathfrak{R}_{S_{\mathcal{X}}}(H),
\end{aligned}$$

where we used the fact that $1_{h(x_i)\neq y_i} = (1-y_i h(x_i))/2$ and the fact that for a fixed $y_i \in \{-1,+1\}$, σ_i and $-y_i\sigma_i$ are distributed in the same way. ■

Note that the lemma implies, by taking expectations, that for any $m \geq 1$, $\mathfrak{R}_m(G) = \frac{1}{2}\mathfrak{R}_m(H)$. These connections between the empirical and average Rademacher complexities can be used to derive generalization bounds for binary classification in terms of the Rademacher complexity of the hypothesis set H.

Theorem 3.2 **Rademacher complexity bounds – binary classification**
Let H be a family of functions taking values in $\{-1,+1\}$ and let D be the distribution over the input space $\mathcal{X}$. Then, for any $\delta > 0$, with probability at least $1-\delta$ over

a sample S of size m drawn according to D, each of the following holds for any $h \in H$:

$$R(h) \leq \widehat{R}(h) + \mathfrak{R}_m(H) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} \tag{3.17}$$

$$\textit{and} \quad R(h) \leq \widehat{R}(h) + \widehat{\mathfrak{R}}_S(H) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \tag{3.18}$$

Proof The result follows immediately by theorem 3.1 and lemma 3.1. ■

The theorem provides two generalization bounds for binary classification based on the Rademacher complexity. Note that the second bound, (3.18), is data-dependent: the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(H)$ is a function of the specific sample S drawn. Thus, this bound could be particularly informative if we could compute $\widehat{\mathfrak{R}}_S(H)$. But, how can we compute the empirical Rademacher complexity? Using again the fact that σ_i and $-\sigma_i$ are distributed in the same way, we can write

$$\widehat{\mathfrak{R}}_S(H) = \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in H} \frac{1}{m}\sum_{i=1}^{m} -\sigma_i h(x_i)\Big] = -\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\inf_{h\in H} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)\Big].$$

Now, for a fixed value of $\boldsymbol{\sigma}$, computing $\inf_{h\in H} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)$ is equivalent to an *empirical risk minimization* problem, which is known to be computationally hard for some hypothesis sets. Thus, in some cases, computing $\widehat{\mathfrak{R}}_S(H)$ could be computationally hard. In the next sections, we will relate the Rademacher complexity to combinatorial measures that are easier to compute.

3.2 Growth function

Here we will show how the Rademacher complexity can be bounded in terms of the *growth function*.

Definition 3.3 **Growth function**
The growth function $\Pi_H : \mathbb{N} \to \mathbb{N}$ *for a hypothesis set H is defined by:*

$$\forall m \in \mathbb{N},\ \Pi_H(m) = \max_{\{x_1,\ldots,x_m\}\subseteq X} \Big|\big\{\big(h(x_1),\ldots,h(x_m)\big) : h \in H\big\}\Big|. \tag{3.19}$$

Thus, $\Pi_H(m)$ is the maximum number of distinct ways in which m points can be classified using hypotheses in H. This provides another measure of the richness of the hypothesis set H. However, unlike the Rademacher complexity, this measure does not depend on the distribution, it is purely combinatorial.

To relate the Rademacher complexity to the growth function, we will use Massart's lemma.

Theorem 3.3 **Massart's lemma**
Let $A \subseteq \mathbb{R}^m$ be a finite set, with $r = \max_{\mathbf{x}\in A} \|\mathbf{x}\|_2$, then the following holds:

$$\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\frac{1}{m}\sup_{\mathbf{x}\in A}\sum_{i=1}^{m}\sigma_i x_i\right] \leq \frac{r\sqrt{2\log|A|}}{m}, \tag{3.20}$$

where σ_is are independent uniform random variables taking values in $\{-1,+1\}$ and $x_1,\ldots,x_m$ are the components of vector $\mathbf{x}$.

Proof For any $t>0$, using Jensen's inequality, rearranging terms, and bounding the supremum by a sum, we obtain:

$$\begin{aligned}\exp\left(t\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{x\in A}\sum_{i=1}^{m}\sigma_i x_i\right]\right) &\leq \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left(\exp\left[t\sup_{x\in A}\sum_{i=1}^{m}\sigma_i x_i\right]\right)\\ &= \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left(\sup_{x\in A}\exp\left[t\sum_{i=1}^{m}\sigma_i x_i\right]\right) \leq \sum_{x\in A}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left(\exp\left[t\sum_{i=1}^{m}\sigma_i x_i\right]\right).\end{aligned}$$

We next use the independence of the σ_is, then apply Hoeffding's lemma (lemma D.1), and use the definition of r to write:

$$\begin{aligned}\exp\left(t\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{x\in A}\sum_{i=1}^{m}\sigma_i x_i\right]\right) &\leq \sum_{x\in A}\Pi_{i=1}^{m}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}_i}\left(\exp\left[t\sigma_i x_i\right]\right)\\ &\leq \sum_{x\in A}\Pi_{i=1}^{m}\exp\left[\frac{t^2(2x_i)^2}{8}\right]\\ &= \sum_{x\in A}\exp\left[\frac{t^2}{2}\sum_{i=1}^{m}x_i^2\right] \leq \sum_{x\in A}\exp\left[\frac{t^2r^2}{2}\right] = |A|e^{\frac{t^2R^2}{2}}.\end{aligned}$$

Taking the log of both sides and dividing by t gives us:

$$\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{x\in A}\sum_{i=1}^{m}\sigma_i x_i\right] \leq \frac{\log|A|}{t}+\frac{tr^2}{2}. \tag{3.21}$$

If we choose $t=\frac{\sqrt{2\log|A|}}{r}$, which minimizes this upper bound, we get:

$$\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{x\in A}\sum_{i=1}^{m}\sigma_i x_i\right] \leq r\sqrt{2\log|A|}. \tag{3.22}$$

Dividing both sides by m leads to the statement of the lemma. ■

Using this result, we can now bound the Rademacher complexity in terms of the growth function.

Corollary 3.1
Let G be a family of functions taking values in $\{-1, +1\}$. Then the following holds:

$$\mathfrak{R}_m(G) \leq \sqrt{\frac{2 \log \Pi_G(m)}{m}}. \tag{3.23}$$

Proof For a fixed sample $S = (x_1, \ldots, x_m)$, we denote by $G_{|S}$ the set of vectors of function values $(g(x_1), \ldots, g(x_m))^\top$ where g is in G. Since $g \in G$ takes values in $\{-1, +1\}$, the norm of these vectors is bounded by $\sqrt{m}$. We can then apply Massart's lemma as follows:

$$\mathfrak{R}_m(G) = \mathop{\mathrm{E}}_{S}\left[\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{u \in G_{|S}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i u_i\right]\right] \leq \mathop{\mathrm{E}}_{S}\left[\frac{\sqrt{m}\sqrt{2 \log |G_{|S}|}}{m}\right].$$

By definition, $|G_{|S}|$ is bounded by the growth function, thus,

$$\mathfrak{R}_m(G) \leq \mathop{\mathrm{E}}_{S}\left[\frac{\sqrt{m}\sqrt{2 \log \Pi_G(m)}}{m}\right] = \sqrt{\frac{2 \log \Pi_G(m)}{m}},$$

which concludes the proof. ■

Combining the generalization bound (3.17) of theorem 3.2 with corollary 3.1 yields immediately the following generalization bound in terms of the growth function.

Corollary 3.2 **Growth function generalization bound**
Let H be a family of functions taking values in $\{-1, +1\}$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, for any $h \in H$,

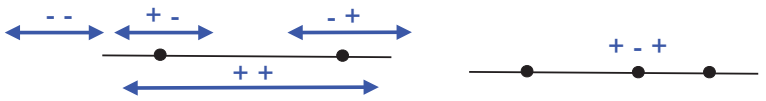
$$R(h) \leq \widehat{R}(h) + \sqrt{\frac{2 \log \Pi_H(m)}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{3.24}$$

Growth function bounds can be also derived directly (without using Rademacher complexity bounds first). The resulting bound is then the following:

$$\Pr\left[\left|R(h) - \widehat{R}(h)\right| > \epsilon\right] \leq 4\Pi_H(2m) \exp\left(-\frac{m\epsilon^2}{8}\right), \tag{3.25}$$

which only differs from (3.24) by constants.

The computation of the growth function may not be always convenient since, by definition, it requires computing $\Pi_H(m)$ for all $m \geq 1$. The next section introduces an alternative measure of the complexity of a hypothesis set H that is based instead on a single scalar, which will turn out to be in fact deeply related to the behavior of the growth function.



(a) (b)

Figure 3.1 VC-dimension of intervals on the real line. (a) Any two points can be shattered. (b) No sample of three points can be shattered as the $(+,-,+)$ labeling cannot be realized.

3.3 VC-dimension

Here, we introduce the notion of *VC-dimension* (Vapnik-Chervonenkis dimension). The VC-dimension is also a purely combinatorial notion but it is often easier to compute than the growth function (or the Rademacher Complexity). As we shall see, the VC-dimension is a key quantity in learning and is directly related to the growth function.

To define the VC-dimension of a hypothesis set H, we first introduce the concepts of *dichotomy* and that of *shattering*. Given a hypothesis set H, a dichotomy of a set S is one of the possible ways of labeling the points of S using a hypothesis in H. A set S of $m \geq 1$ points is said to be shattered by a hypothesis set H when H realizes all possible dichotomies of S, that is when $\Pi_H(m) = 2^m$.

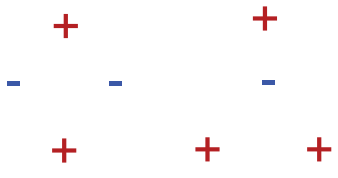
Definition 3.4 **VC-dimension**
The VC-dimension of a hypothesis set H is the size of the largest set that can be fully shattered by H:

$$VCdim(H) = \max\{m \colon \Pi_H(m) = 2^m\}. \tag{3.26}$$

Note that, by definition, if $\text{VCdim}(H) = d$, there exists a set of size d that can be fully shattered. But, this does not imply that all sets of size d or less are fully shattered, in fact, this is typically not the case.

To further illustrate this notion, we will examine a series of examples of hypothesis sets and will determine the VC-dimension in each case. To compute the VC-dimension we will typically show a lower bound for its value and then a matching upper bound. To give a lower bound d for $\text{VCdim}(H)$, it suffices to show that a set S of cardinality d can be shattered by H. To give an upper bound, we need to prove that no set S of cardinality $d+1$ can be shattered by H, which is typically more difficult.

Example 3.1 Intervals on the real line
Our first example involves the hypothesis class of intervals on the real line. It is clear that the VC-dimension is at least two, since all four dichotomies



(a) (b)

Figure 3.2 Unrealizable dichotomies for four points using hyperplanes in $\mathbb{R}^2$. (a) All four points lie on the convex hull. (b) Three points lie on the convex hull while the remaining point is interior.

$(+,+),(-,-),(+,-),(-,+)$ can be realized, as illustrated in figure 3.1(a). In contrast, by the definition of intervals, no set of three points can be shattered since the $(+,-,+)$ labeling cannot be realized. Hence, VCdim(intervals in $\mathbb{R}$) = 2.

Example 3.2 Hyperplanes

Consider the set of hyperplanes in $\mathbb{R}^2$. We first observe that any three non-collinear points in $\mathbb{R}^2$ can be shattered. To obtain the first three dichotomies, we choose a hyperplane that has two points on one side and the third point on the opposite side. To obtain the fourth dichotomy we have all three points on the same side of the hyperplane. The remaining four dichotomies are realized by simply switching signs. Next, we show that four points cannot be shattered by considering two cases: (i) the four points lie on the convex hull defined by the four points, and (ii) three of the four points lie on the convex hull and the remaining point is internal. In the first case, a positive labeling for one diagonal pair and a negative labeling for the other diagonal pair cannot be realized, as illustrated in figure 3.2(a). In the second case, a labeling which is positive for the points on the convex hull and negative for the interior point cannot be realized, as illustrated in figure 3.2(b). Hence, VCdim(hyperplanes in $\mathbb{R}^2$) = 3.

More generally in $\mathbb{R}^d$, we derive a lower bound by starting with a set of $d+1$ points in $\mathbb{R}^d$, setting x_0 to be the origin and defining x_i, for $i \in \{1,\ldots,d\}$, as the point whose ith coordinate is 1 and all others are 0. Let $y_0, y_1, \ldots, y_d \in \{-1,+1\}$ be an arbitrary set of labels for $x_0, x_1, \ldots, x_d$. Let w be the vector whose ith coordinate is y_i. Then the classifier defined by the hyperplane of equation $w \cdot x + \frac{y_0}{2} = 0$ shatters $x_0, x_1, \ldots, x_d$ since for any $i \in [0,d]$,

$$\operatorname{sgn}\left(w \cdot x_i + \frac{y_0}{2}\right) = \operatorname{sgn}\left(y_i + \frac{y_0}{2}\right) = y_i. \tag{3.27}$$

To obtain an upper bound, it suffices to show that no set of $d+2$ points can be shattered by halfspaces. To prove this, we will use the following general theorem.

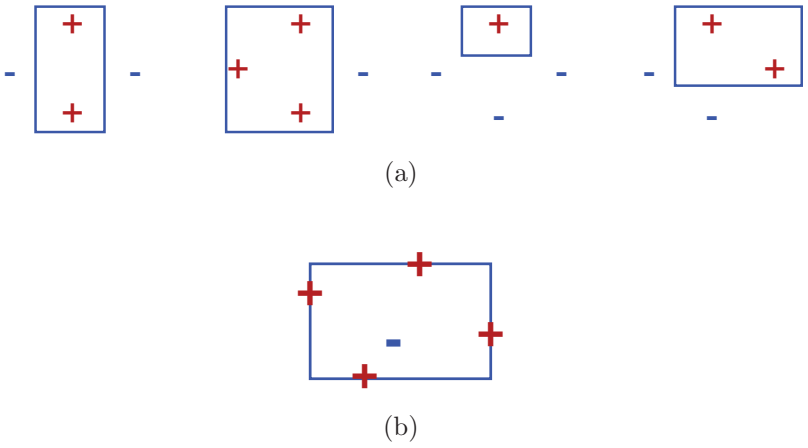


Figure 3.3 VC-dimension of axis-aligned rectangles. (a) Examples of realizable dichotomies for four points in a diamond pattern. (b) No sample of five points can be realized if the interior point and the remaining points have opposite labels.

Theorem 3.4 **Radon's theorem**
Any set X of $d+2$ points in $\mathbb{R}^d$ can be partitioned into two subsets X_1 and X_2 such that the convex hulls of X_1 and X_2 intersect.

Proof Let $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_{d+2}\} \subset \mathbb{R}^d$. The following is a system of $d+1$ linear equations in $\alpha_1, \ldots, \alpha_{d+2}$:

$$\sum_{i=1}^{d+2} \alpha_i \mathbf{x}_i = 0 \qquad \text{and} \qquad \sum_{i=1}^{d+2} \alpha_i = 0, \tag{3.28}$$

since the first equality leads to d equations, one for each component. The number of unknowns, $d+2$, is larger than the number of equations, $d+1$, therefore the system admits a non-zero solution $\beta_1, \ldots, \beta_{d+2}$. Since $\sum_{i=1}^{d+2} \beta_i = 0$, both $I_1 = \{i \in [1, d+2] \colon \beta_i > 0\}$ and $I_2 = \{i \in [1, d+2] \colon \beta_i < 0\}$ are non-empty sets and $X_1 = \{\mathbf{x}_i \colon i \in I_1\}$ and $X_2 = \{\mathbf{x}_i \colon i \in I_2\}$ form a partition of X. By the last equation of (3.28), $\sum_{i \in I_1} \beta_i = -\sum_{i \in I_2} \beta_i$. Let $\beta = \sum_{i \in I_1} \beta_i$. Then, the first part of (3.28) implies

$$\sum_{i \in I_1} \frac{\beta_i}{\beta} \mathbf{x}_i = \sum_{i \in I_2} \frac{-\beta_i}{\beta} \mathbf{x}_i,$$

with $\sum_{i \in I_1} \frac{\beta_i}{\beta} = \sum_{i \in I_2} \frac{-\beta_i}{\beta} = 1$, and $\frac{\beta_i}{\beta} \geq 0$ for $i \in I_1$ and $\frac{-\beta_i}{\beta} \geq 0$ for $i \in I_2$. By definition of the convex hulls (B.4), this implies that $\sum_{i \in I_1} \frac{\beta_i}{\beta} \mathbf{x}_i$ belongs both to

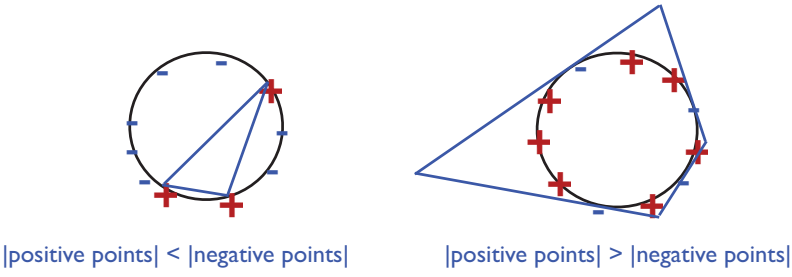


Figure 3.4 Convex d-gons in the plane can shatter $2d + 1$ points. (a) d-gon construction when there are more negative labels. (b) d-gon construction when there are more positive labels.

the convex hull of X_1 and to that of X_2. ■

Now, let X be a set of $d + 2$ points. By Radon's theorem, it can be partitioned into two sets X_1 and X_2 such that their convex hulls intersect. Observe that when two sets of points X_1 and X_2 are separated by a hyperplane, their convex hulls are also separated by that hyperplane. Thus, X_1 and X_2 cannot be separated by a hyperplane and X is not shattered. Combining our lower and upper bounds, we have proven that VCdim(hyperplanes in $\mathbb{R}^d$) $= d + 1$.

Example 3.3 Axis-aligned Rectangles

We first show that the VC-dimension is at least four, by considering four points in a diamond pattern. Then, it is clear that all 16 dichotomies can be realized, some of which are illustrated in figure 3.2(a). In contrast, for any set of five distinct points, if we construct the minimal axis-aligned rectangle containing these points, one of the five points is in the interior of this rectangle. Imagine that we assign a negative label to this interior point and a positive label to each of the remaining four points, as illustrated in figure 3.2(b). There is no axis-aligned rectangle that can realize this labeling. Hence, no set of five distinct points can be shattered and VCdim(axis-aligned rectangles) $= 4$.

Example 3.4 Convex Polygons

We focus on the class of convex d-gons in the plane. To get a lower bound, we show that any set of $2d + 1$ points can be fully shattered. To do this, we select $2d + 1$ points that lie on a circle, and for a particular labeling, if there are more negative than positive labels, then the points with the positive labels are used as the polygon's vertices, as in figure 3.4(a). Otherwise, the tangents of the negative points serve as the edges of the polygon, as shown in (3.4)(b). To derive an upper

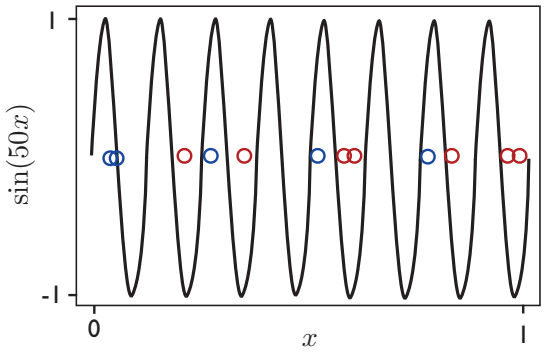


Figure 3.5 An example of a sine function (with $\omega = 50$) used for classification.

bound, it can be shown that choosing points on the circle maximizes the number of possible dichotomies, and thus VCdim(convex d-gons) $= 2d + 1$. Note also that VCdim(convex polygons) $= +\infty$.

Example 3.5 Sine Functions
The previous examples could suggest that the VC-dimension of H coincides with the number of free parameters defining H. For example, the number of parameters defining hyperplanes matches their VC-dimension. However, this does not hold in general. Several of the exercises in this chapter illustrate this fact. The following provides a striking example from this point of view. Consider the following family of sine functions: $\{t \mapsto \sin(\omega t) \colon \omega \in \mathbb{R}\}$. One instance of this function class is shown in figure 3.5. These sine functions can be used to classify the points on the real line: a point is labeled positively if it is above the curve, negatively otherwise. Although this family of sine function is defined via a single parameter, ω, it can be shown that VCdim(sine functions) $= +\infty$ (exercise 3.12).

The VC-dimension of many other hypothesis sets can be determined or upper-bounded in a similar way (see this chapter's exercises). In particular, the VC-dimension of any vector space of dimension $r < \infty$ can be shown to be at most r (exercise 3.11). The next result known as *Sauer's lemma* clarifies the connection between the notions of growth function and VC-dimension.

***Theorem 3.5* Sauer's lemma**
Let H be a hypothesis set with VCdim$(H) = d$. Then, for all $m \in \mathbb{N}$, the following inequality holds:

$$\Pi_H(m) \leq \sum_{i=0}^{d} \binom{m}{i}. \tag{3.29}$$

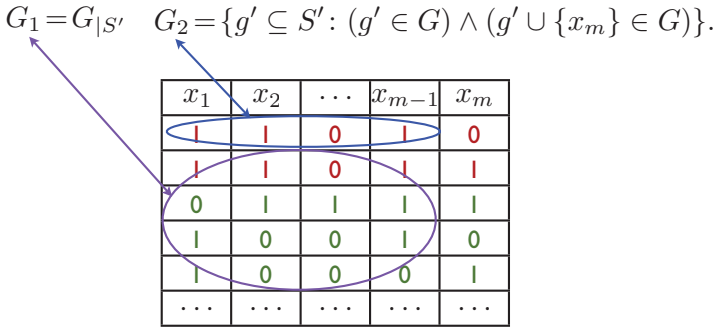


Figure 3.6 Illustration of how G_1 and G_2 are constructed in the proof of Sauer's lemma.

Proof The proof is by induction on $m + d$. The statement clearly holds for $m = 1$ and $d = 0$ or $d = 1$. Now, assume that it holds for $(m-1, d-1)$ and $(m-1, d)$. Fix a set $S = \{x_1, \ldots, x_m\}$ with $\Pi_H(m)$ dichotomies and let $G = H_{|S}$ be the set of concepts H induces by restriction to S.

Now consider the following families over $S' = \{x_1, \ldots, x_{m-1}\}$. We define $G_1 = G_{|S'}$ as the set of concepts H includes by restriction to S'. Next, by identifying each concept as the set of points (in S' or S) for which it is non-zero, we can define G_2 as

$$G_2 = \{g' \subseteq S' : (g' \in G) \wedge (g' \cup \{x_m\} \in G)\}.$$

Since $g' \subseteq S'$, $g' \in G$ means that without adding x_m it is a concept of G. Further, the constraint $g' \cup \{x_m\} \in G$ means that adding x_m to g' also makes it a concept of G. The construction of G_1 and G_2 is illustrated pictorially in figure 3.6. Given our definitions of G_1 and G_2, observe that $|G_1| + |G_2| = |G|$.

Since $\mathrm{VCdim}(G_1) \leq \mathrm{VCdim}(G) \leq d$, then by definition of the growth function and using the induction hypothesis,

$$|G_1| \leq \Pi_{G_1}(m-1) \leq \sum_{i=0}^{d} \binom{m-1}{i}.$$

Further, by definition of G_2, if a set $Z \subseteq S'$ is shattered by G_2, then the set $Z \cup \{x_m\}$ is shattered by G. Hence,

$$\mathrm{VCdim}(G_2) \leq \mathrm{VCdim}(G) - 1 = d - 1,$$

and by definition of the growth function and using the induction hypothesis,

$$|G_2| \leq \Pi_{G_2}(m-1) \leq \sum_{i=0}^{d-1} \binom{m-1}{i}.$$

Thus,

$$|G| = |G_1| + |G_2| \leq \sum_{i=0}^{d} \tbinom{m-1}{i} + \sum_{i=0}^{d-1} \tbinom{m-1}{i} = \sum_{i=0}^{d} \tbinom{m-1}{i} + \tbinom{m-1}{i-1} = \sum_{i=0}^{d} \tbinom{m}{i},$$

which completes the inductive proof. ■

The significance of Sauer's lemma can be seen by corollary 3.3, which remarkably shows that growth function only exhibits two types of behavior: either $\text{VCdim}(H) = d < +\infty$, in which case $\Pi_H(m) = O(m^d)$, or $\text{VCdim}(H) = +\infty$, in which case $\Pi_H(m) = 2^m$.

Corollary 3.3
Let H be a hypothesis set with VCdim$(H) = d$. Then for all $m \geq d$,

$$\Pi_H(m) \leq \left(\frac{em}{d}\right)^d = O(m^d). \tag{3.30}$$

Proof The proof begins by using Sauer's lemma. The first inequality multiplies each summand by a factor that is greater than or equal to one since $m \geq d$, while the second inequality adds non-negative summands to the summation.

$$\begin{aligned}
\Pi_H(m) &\leq \sum_{i=0}^{d} \binom{m}{i} \\
&\leq \sum_{i=0}^{d} \binom{m}{i} \left(\frac{m}{d}\right)^{d-i} \\
&\leq \sum_{i=0}^{m} \binom{m}{i} \left(\frac{m}{d}\right)^{d-i} \\
&= \left(\frac{m}{d}\right)^d \sum_{i=0}^{m} \binom{m}{i} \left(\frac{d}{m}\right)^i \\
&= \left(\frac{m}{d}\right)^d \left(1 + \frac{d}{m}\right)^m \leq \left(\frac{m}{d}\right)^d e^d.
\end{aligned}$$

After simplifying the expression using the binomial theorem, the final inequality follows using the general identity $(1 - x) \leq e^{-x}$. ■

The explicit relationship just formulated between VC-dimension and the growth function combined with corollary 3.2 leads immediately to the following generaliza-

tion bounds based on the VC-dimension.

Corollary 3.4 **VC-dimension generalization bounds**
Let H be a family of functions taking values in $\{-1, +1\}$ with VC-dimension d. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for all $h \in H$:

$$R(h) \leq \widehat{R}(h) + \sqrt{\frac{2d \log \frac{em}{d}}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{3.31}$$

Thus, the form of this generalization bound is

$$R(h) \leq \widehat{R}(h) + O\left(\sqrt{\frac{\log(m/d)}{(m/d)}}\right), \tag{3.32}$$

which emphasizes the importance of the ratio m/d for generalization. The theorem provides another instance of Occam's razor principle where simplicity is measured in terms of smaller VC-dimension.

VC-dimension bounds can be derived directly without using an intermediate Rademacher complexity bound, as for (3.25): combining Sauer's lemma with (3.25) leads to the following high-probability bound

$$R(h) \leq \widehat{R}(h) + \sqrt{\frac{8d \log \frac{2em}{d} + 8 \log \frac{4}{\delta}}{m}},$$

which has the general form of (3.32). The log factor plays only a minor role in these bounds. A finer analysis can be used in fact to eliminate that factor.

3.4 Lower bounds

In the previous section, we presented several upper bounds on the generalization error. In contrast, this section provides lower bounds on the generalization error of any learning algorithm in terms of the VC-dimension of the hypothesis set used.

These lower bounds are shown by finding for any algorithm a 'bad' distribution. Since the learning algorithm is arbitrary, it will be difficult to specify that particular distribution. Instead, it suffices to prove its existence non-constructively. At a high level, the proof technique used to achieve this is the *probabilistic method* of Paul Erdös. In the context of the following proofs, first a lower bound is given on the expected error over the parameters defining the distributions. From that, the lower bound is shown to hold for at least one set of parameters, that is one distribution.

Theorem 3.6 **Lower bound, realizable case**
Let H be a hypothesis set with VC-dimension $d > 1$. Then, for any learning algorithm $\mathcal{A}$, there exist a distribution D over $\mathcal{X}$ and a target function $f \in H$ such that

$$\Pr_{S \sim D^m}\left[R_D(h_S, f) > \frac{d-1}{32m}\right] \geq 1/100. \tag{3.33}$$

Proof Let $X = \{x_0, x_1, \ldots, x_{d-1}\} \subseteq \mathcal{X}$ be a set that is fully shattered by H. For any $\epsilon > 0$, we choose D such that its support is reduced to X and so that one point (x_0) has very high probability ($1 - \epsilon$), with the rest of the probability mass distributed uniformly among the other points:

$$\Pr_D[x_0] = 1 - 8\epsilon \quad \text{and} \quad \forall i \in [1, d-1], \Pr_D[x_i] = \frac{8\epsilon}{d-1}. \tag{3.34}$$

With this definition, most samples would contain x_0 and, since X is fully shattered, $\mathcal{A}$ can essentially do no better than tossing a coin when determining the label of a point x_i not falling in the training set.

We assume without loss of generality that $\mathcal{A}$ makes no error on x_0. For a sample S, we let $\overline{S}$ denote the set of its elements falling in $\{x_1, \ldots, x_{d-1}\}$, and let $\mathcal{S}$ be the set of samples S of size m such that $|\overline{S}| \leq (d-1)/2$. Now, fix a sample $S \in \mathcal{S}$, and consider the uniform distribution U over all labelings $f\colon X \to \{0, 1\}$, which are all in H since the set is shattered. Then, the following lower bound holds:

$$\begin{aligned}
\mathop{\mathrm{E}}_{f \sim U}[R_D(h_S, f)] &= \sum_f \sum_{x \in X} 1_{h(x) \neq f(x)} \Pr[x] \Pr[f] \\
&\geq \sum_f \sum_{x \notin \overline{S}} 1_{h(x) \neq f(x)} \Pr[x] \Pr[f] \\
&= \sum_{x \notin \overline{S}} \Big(\sum_f 1_{h(x) \neq f(x)} \Pr[f] \Big) \Pr[x] \\
&= \frac{1}{2} \sum_{x \notin \overline{S}} \Pr[x] \geq \frac{1}{2} \frac{d-1}{2} \frac{8\epsilon}{d-1} = 2\epsilon.
\end{aligned} \tag{3.35}$$

The first lower bound holds because we remove non-negative terms from the summation when we only consider $x \notin \overline{S}$ instead of all x in X. After rearranging terms, the subsequent equality holds since we are taking an expectation over $f \in H$ with uniform weight on each f and H shatters X. The final lower bound holds due to the definitions of D and $\overline{S}$, the latter which implies that $|X - \overline{S}| \geq (d-1)/2$.

Since (3.35) holds for all $S \in \mathcal{S}$, it also holds in expectation over all $S \in \mathcal{S}$: $\mathrm{E}_{S \in \mathcal{S}}\left[\mathrm{E}_{f \sim U}[R_D(h_S, f)]\right] \geq 2\epsilon$. By Fubini's theorem, the expectations can be

permuted, thus,

$$\mathop{\mathrm{E}}_{f\sim U}\Big[\mathop{\mathrm{E}}_{S\in\mathcal{S}}[R_D(h_S,f)]\Big] \geq 2\epsilon. \tag{3.36}$$

This implies that $\mathrm{E}_{S\in\mathcal{S}}[R_D(h_S,f_0)] \geq 2\epsilon$ for at least one labeling $f_0 \in H$. Decomposing this expectation into two parts and using $R_D(h_S,f_0) \leq \Pr_D[X - \{x_0\}]$, we obtain:

$$\begin{aligned}\mathop{\mathrm{E}}_{S\in\mathcal{S}}[R_D(h_S,f_0)] &= \sum_{S:R_D(h_S,f_0)\geq\epsilon} R_D(h_S,f_0)\Pr[R_D(h_S,f_0)] + \sum_{S:R_D(h_S,f_0)<\epsilon} R_D(h_S,f_0)\Pr[R_D(h_S,f_0)]\\ &\leq \Pr_D[X-\{x_0\}]\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon] + \epsilon\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)<\epsilon]\\ &\leq 8\epsilon\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon] + \epsilon\big(1-\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon]\big).\end{aligned}$$

Collecting terms in $\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon]$ yields

$$\Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon] \geq \frac{1}{7\epsilon}(2\epsilon-\epsilon) = \frac{1}{7}. \tag{3.37}$$

Thus, the probability over all samples S (not necessarily in $\mathcal{S}$) can be lower bounded as

$$\Pr_S[R_D(h_S,f_0)\geq\epsilon] \geq \Pr_{S\in\mathcal{S}}[R_D(h_S,f_0)\geq\epsilon]\Pr[\mathcal{S}] \geq \frac{1}{7}\Pr[\mathcal{S}]. \tag{3.38}$$

This leads us to find a lower bound for $\Pr[\mathcal{S}]$. The probability that more than $(d-1)/2$ points are drawn in a sample of size m verifies the Chernoff bound for any $\gamma > 0$:

$$1-\Pr[\mathcal{S}] = \Pr[S_m \geq 8\epsilon m(1+\gamma)] \leq e^{-8\epsilon m\frac{\gamma^2}{3}}. \tag{3.39}$$

Therefore, for $\epsilon = (d-1)/(32m)$ and $\gamma = 1$,

$$\Pr[S_m \geq \tfrac{d-1}{2}] \leq e^{-(d-1)/12} \leq e^{-1/12} \leq 1-7\delta, \tag{3.40}$$

for $\delta \leq .01$. Thus $\Pr[\mathcal{S}] \geq 7\delta$ and $\Pr_S[R_D(h_S,f_0)\geq\epsilon] \geq \delta$. ■

The theorem shows that for any algorithm $\mathcal{A}$, there exists a 'bad' distribution over $\mathcal{X}$ and a target function f for which the error of the hypothesis returned by $\mathcal{A}$ is $\Omega(\frac{d}{m})$ with some constant probability. This further demonstrates the key role played by the VC-dimension in learning. The result implies in particular that PAC-learning in the non-realizable case is not possible when the VC-dimension is infinite.

Note that the proof shows a stronger result than the statement of the theorem: the distribution D is selected independently of the algorithm $\mathcal{A}$. We now present a theorem giving a lower bound in the non-realizable case. The following two lemmas will be needed for the proof.

Lemma 3.2
Let α be a uniformly distributed random variable taking values in $\{\alpha_-, \alpha_+\}$, where $\alpha_- = \frac{1}{2} - \frac{\epsilon}{2}$ and $\alpha_+ = \frac{1}{2} + \frac{\epsilon}{2}$, and let S be a sample of $m \geq 1$ random variables $X_1, \ldots, X_m$ taking values in $\{0,1\}$ and drawn i.i.d. according to the distribution D_α defined by $\Pr_{D_\alpha}[X = 1] = \alpha$. Let h be a function from $\mathcal{X}^m$ to $\{\alpha_-, \alpha_+\}$, then the following holds:

$$\mathop{\mathrm{E}}_{\alpha}\Big[\Pr_{S\sim D_\alpha^m}[h(S) \neq \alpha]\Big] \geq \Phi(2\lceil m/2\rceil, \epsilon), \tag{3.41}$$

where $\Phi(m,\epsilon) = \frac{1}{4}\Big(1 - \sqrt{1 - \exp\big(-\frac{m\epsilon^2}{1-\epsilon^2}\big)}\Big)$ for all m and ϵ.

Proof The lemma can be interpreted in terms of an experiment with two coins with biases α_- and α_+. It implies that for a discriminant rule $h(S)$ based on a sample S drawn from D_{α_-} or D_{α_+}, to determine which coin was tossed, the sample size m must be at least $\Omega(1/\epsilon^2)$. The proof is left as an exercise (exercise 3.19). ■

We will make use of the fact that for any fixed ϵ the function $m \mapsto \Phi(m,x)$ is convex, which is not hard to establish.

Lemma 3.3
Let Z be a random variable taking values in $[0,1]$. Then, for any $\gamma \in [0,1)$,

$$\Pr[z > \gamma] \geq \frac{\mathrm{E}[Z] - \gamma}{1 - \gamma} > \mathrm{E}[Z] - \gamma. \tag{3.42}$$

Proof Since the values taken by Z are in $[0,1]$,

$$\begin{aligned}
\mathrm{E}[Z] &= \sum_{z\leq\gamma} \Pr[Z = z]z + \sum_{z>\gamma} \Pr[Z = z]z \\
&\leq \sum_{z\leq\gamma} \Pr[Z = z]\gamma + \sum_{z>\gamma} \Pr[Z = z] \\
&= \gamma \Pr[Z \leq \gamma] + \Pr[Z > \gamma] \\
&= \gamma(1 - \Pr[Z > \gamma]) + \Pr[Z > \gamma] \\
&= (1-\gamma)\Pr[Z > \gamma] + \gamma,
\end{aligned}$$

which concludes the proof. ■

Theorem 3.7 **Lower bound, non-realizable case**
Let H be a hypothesis set with VC-dimension $d > 1$. Then, for any learning algorithm $\mathcal{A}$, there exists a distribution D over $\mathcal{X} \times \{0,1\}$ such that:

$$\Pr_{S\sim D^m}\left[R_D(h_S) - \inf_{h\in H} R_D(h) > \sqrt{\frac{d}{320m}}\right] \geq 1/64. \tag{3.43}$$

Equivalently, for any learning algorithm, the sample complexity verifies

$$m \geq \frac{d}{320\epsilon^2}. \tag{3.44}$$

Proof Let $X = \{x_1, x_1, \ldots, x_d\} \subseteq \mathcal{X}$ be a set fully shattered by H. For any $\alpha \in [0,1]$ and any vector $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_d)^\top \in \{-1,+1\}^d$, we define a distribution $D_{\boldsymbol{\sigma}}$ with support $X \times \{0,1\}$ as follows:

$$\forall i \in [1,d], \quad \Pr_{D_{\boldsymbol{\sigma}}}[(x_i,1)] = \frac{1}{d}\Big(\frac{1}{2} + \frac{\sigma_i \alpha}{2}\Big). \tag{3.45}$$

Thus, the label of each point x_i, $i \in [1,d]$, follows the distribution $\Pr_{D_{\boldsymbol{\sigma}}}[\cdot|x_i]$, that of a biased coin where the bias is determined by the sign of σ_i and the magnitude of α. To determine the most likely label of each point x_i, the learning algorithm will therefore need to estimate $\Pr_{D_{\boldsymbol{\sigma}}}[1|x_i]$ with an accuracy better than α. To make this further difficult, α and $\boldsymbol{\sigma}$ will be selected based on the algorithm, requiring, as in lemma 3.2, $\Omega(1/\alpha^2)$ instances of each point x_i in the training sample.

Clearly, the Bayes classifier $h^*_{D_{\boldsymbol{\sigma}}}$ is defined by $h^*_{D_{\boldsymbol{\sigma}}}(x_i) = \operatorname{argmax}_{y \in \{0,1\}} \Pr[y|x_i] = 1_{\sigma_i > 0}$ for all $i \in [1,d]$. $h^*_{D_{\boldsymbol{\sigma}}}$ is in H since X is fully shattered. For all $h \in H$,

$$R_{D_{\boldsymbol{\sigma}}}(h) - R_{D_{\boldsymbol{\sigma}}}(h^*_{D_{\boldsymbol{\sigma}}}) = \frac{1}{d}\sum_{x \in X}\Big(\frac{\alpha}{2} + \frac{\alpha}{2}\Big) 1_{h(x) \neq h^*_{D_{\boldsymbol{\sigma}}}(x)} = \frac{\alpha}{d}\sum_{x \in X} 1_{h(x) \neq h^*_{D_{\boldsymbol{\sigma}}}(x)}. \tag{3.46}$$

Let h_S denote the hypothesis returned by the learning algorithm $\mathcal{A}$ after receiving a labeled sample S drawn according to $D_{\boldsymbol{\sigma}}$. We will denote by $|S|_x$ the number of occurrences of a point x in S. Let U denote the uniform distribution over $\{-1,+1\}^d$.

Then, in view of (3.46), the following holds:

$$
\begin{aligned}
&\mathop{\mathrm{E}}_{\substack{\boldsymbol{\sigma}\sim U\\ S\sim D^m_{\boldsymbol{\sigma}}}}\left[\frac{1}{\alpha}\big[R_{D_{\boldsymbol{\sigma}}}(h_S)-R_{D_{\boldsymbol{\sigma}}}(h^*_{D_{\boldsymbol{\sigma}}})\big]\right]\\
&\quad=\frac{1}{d}\sum_{x\in X}\mathop{\mathrm{E}}_{\substack{\boldsymbol{\sigma}\sim U\\ S\sim D^m_{\boldsymbol{\sigma}}}}\left[1_{h_S(x)\neq h^*_{D_{\boldsymbol{\sigma}}}(x)}\right]\\
&\quad=\frac{1}{d}\sum_{x\in X}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}\sim U}\left[\Pr_{S\sim D^m_{\boldsymbol{\sigma}}}\big[h_S(x)\neq h^*_{D_{\boldsymbol{\sigma}}}(x)\big]\right]\\
&\quad=\frac{1}{d}\sum_{x\in X}\sum_{n=0}^{m}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}\sim U}\left[\Pr_{S\sim D^m_{\boldsymbol{\sigma}}}\big[h_S(x)\neq h^*_{D_{\boldsymbol{\sigma}}}(x)\,\big|\,|S|_x=n\big]\Pr[|S|_x=n]\right]\\
&\quad\geq\frac{1}{d}\sum_{x\in X}\sum_{n=0}^{m}\Phi(n+1,\alpha)\Pr[|S|_x=n] && \text{(lemma 3.2)}\\
&\quad\geq\frac{1}{d}\sum_{x\in X}\Phi(m/d+1,\alpha) && \text{(convexity of } \Phi(\cdot,\alpha) \text{ and Jensen's ineq.)}\\
&\quad=\Phi(m/d+1,\alpha).
\end{aligned}
$$

Since the expectation over $\boldsymbol{\sigma}$ is lower-bounded by $\Phi(m/d+1,\alpha)$, there must exist some $\boldsymbol{\sigma}\in\{-1,+1\}^d$ for which

$$
\mathop{\mathrm{E}}_{S\sim D^m_{\boldsymbol{\sigma}}}\left[\frac{1}{\alpha}\big[R_{D_{\boldsymbol{\sigma}}}(h_S)-R_{D_{\boldsymbol{\sigma}}}(h^*_{D_{\boldsymbol{\sigma}}})\big]\right]>\Phi(m/d+1,\alpha). \tag{3.47}
$$

Then, by lemma 3.3, for that $\boldsymbol{\sigma}$, for any $\gamma\in[0,1]$,

$$
\Pr_{S\sim D^m_{\boldsymbol{\sigma}}}\left[\frac{1}{\alpha}\big[R_{D_{\boldsymbol{\sigma}}}(h_S)-R_{D_{\boldsymbol{\sigma}}}(h^*_{D_{\boldsymbol{\sigma}}})\big]>\gamma u\right]>(1-\gamma)u, \tag{3.48}
$$

where $u=\Phi(m/d+1,\alpha)$. Selecting δ and ϵ such that $\delta\leq(1-\gamma)u$ and $\epsilon\leq\gamma\alpha u$ gives

$$
\Pr_{S\sim D^m_{\boldsymbol{\sigma}}}\big[R_{D_{\boldsymbol{\sigma}}}(h_S)-R_{D_{\boldsymbol{\sigma}}}(h^*_{D_{\boldsymbol{\sigma}}})>\epsilon\big]>\delta. \tag{3.49}
$$

To satisfy the inequalities defining ϵ and δ, let $\gamma = 1 - 8\delta$. Then,

$$\delta \leq (1-\gamma)u \iff u \geq \frac{1}{8} \tag{3.50}$$

$$\iff \frac{1}{4}\left(1 - \sqrt{1 - \exp\left(-\frac{(m/d+1)\alpha^2}{1-\alpha^2}\right)}\right) \geq \frac{1}{8} \tag{3.51}$$

$$\iff \frac{(m/d+1)\alpha^2}{1-\alpha^2} \leq \log\frac{4}{3} \tag{3.52}$$

$$\iff \frac{m}{d} \leq (\frac{1}{\alpha^2} - 1)\log\frac{4}{3} - 1. \tag{3.53}$$

Selecting $\alpha = 8\epsilon/(1-8\delta)$ gives $\epsilon = \gamma\alpha/8$ and the condition

$$\frac{m}{d} \leq \left(\frac{(1-8\delta)^2}{64\epsilon^2} - 1\right)\log\frac{4}{3} - 1. \tag{3.54}$$

Let $f(1/\epsilon^2)$ denote the right-hand side. We are seeking a sufficient condition of the form $m/d \leq \omega/\epsilon^2$. Since $\epsilon \leq 1/64$, to ensure that $\omega/\epsilon^2 \leq f(1/\epsilon^2)$, it suffices to impose $\omega/(1/64)^2 = f(1/(1/64)^2)$. This condition gives

$$\omega = (7/64)^2\log(4/3) - (1/64)^2(\log(4/3)+1) \approx .003127 \geq 1/320 = .003125.$$

Thus, $\epsilon^2 \leq \frac{1}{320(m/d)}$ is sufficient to ensure the inequalities. ■

The theorem shows that for any algorithm $\mathcal{A}$, in the non-realizable case, there exists a 'bad' distribution over $\mathcal{X} \times \{0,1\}$ such that the error of the hypothesis returned by $\mathcal{A}$ is $\Omega\left(\sqrt{\frac{d}{m}}\right)$ with some constant probability. The VC-dimension appears as a critical quantity in learning in this general setting as well. In particular, with an infinite VC-dimension, agnostic PAC-learning is not possible.

3.5 Chapter notes

The use of Rademacher complexity for deriving generalization bounds in learning was first advocated by Koltchinskii [2001], Koltchinskii and Panchenko [2000], and Bartlett, Boucheron, and Lugosi [2002a], see also [Koltchinskii and Panchenko, 2002, Bartlett and Mendelson, 2002]. Bartlett, Bousquet, and Mendelson [2002b] introduced the notion of *local Rademacher complexity*, that is the Rademacher complexity restricted to a subset of the hypothesis set limited by a bound on the variance. This can be used to derive better guarantees under some regularity assumptions about the noise.

Theorem 3.3 is due to Massart [2000]. The notion of VC-dimension was introduced by Vapnik and Chervonenkis [1971] and has been since extensively studied [Vapnik,

2006, Vapnik and Chervonenkis, 1974, Blumer et al., 1989, Assouad, 1983, Dudley, 1999]. In addition to the key role it plays in machine learning, the VC-dimension is also widely used in a variety of other areas of computer science and mathematics (e.g., see Shelah [1972], Chazelle [2000]). Theorem 3.5 is known as *Sauer's lemma* in the learning community, however the result was first given by Vapnik and Chervonenkis [1971] (in a somewhat different version) and later independently by Sauer [1972] and Shelah [1972].

In the realizable case, lower bounds for the expected error in terms of the VC-dimension were given by Vapnik and Chervonenkis [1974] and Haussler et al. [1988]. Later, a lower bound for the probability of error such as that of theorem 3.6 was given by Blumer et al. [1989]. Theorem 3.6 and its proof, which improves upon this previous result, are due to Ehrenfeucht, Haussler, Kearns, and Valiant [1988]. Devroye and Lugosi [1995] gave slightly tighter bounds for the same problem with a more complex expression. Theorem 3.7 giving a lower bound in the non-realizable case and the proof presented are due to Anthony and Bartlett [1999]. For other examples of application of the probabilistic method demonstrating its full power, consult the reference book of Alon and Spencer [1992].

There are several other measures of the complexity of a family of functions used in machine learning, including *covering numbers*, *packing numbers*, and some other complexity measures discussed in chapter 10. A covering number $\mathcal{N}_p(G, \epsilon)$ is the minimal number of L_p balls of radius $\epsilon > 0$ needed to cover a family of loss functions G. A packing number $\mathcal{M}_p(G, \epsilon)$ is the maximum number of non-overlapping L_p balls of radius ϵ centered in G. The two notions are closely related, in particular it can be shown straightfowardly that $\mathcal{M}_p(G, 2\epsilon) \leq \mathcal{N}_p(G, \epsilon) \leq \mathcal{M}_p(G, \epsilon)$ for G and $\epsilon > 0$. Each complexity measure naturally induces a different reduction of infinite hypothesis sets to finite ones, thereby resulting in generalization bounds for infinite hypothesis sets. Exercise 3.22 illustrates the use of covering numbers for deriving generalization bounds using a very simple proof. There are also close relationships between these complexity measures: for example, by Dudley's theorem, the empirical Rademacher complexity can be bounded in terms of $\mathcal{N}_2(G, \epsilon)$ [Dudley, 1967, 1987] and the covering and packing numbers can be bounded in terms of the VC-dimension [Haussler, 1995]. See also [Ledoux and Talagrand, 1991, Alon et al., 1997, Anthony and Bartlett, 1999, Cucker and Smale, 2001, Vidyasagar, 1997] for a number of upper bounds on the covering number in terms of other complexity measures.

3.6 Exercises

3.1 Growth function of intervals in $\mathbb{R}$. Let H be the set of intervals in $\mathbb{R}$. The VC-dimension of H is 2. Compute its shattering coefficient $\Pi_H(m)$, $m \geq 0$. Compare

your result with the general bound for growth functions.

3.2 Lower bound on growth function. Prove that Sauer's lemma (theorem 3.5) is tight, i.e., for any set X of $m > d$ elements, show that there exists a hypothesis class H of VC-dimension d such that $\Pi_H(m) = \sum_{i=0}^{d} \binom{m}{i}$.

3.3 Singleton hypothesis class. Consider the trivial hypothesis set $H = \{h_0\}$.

(a) Show that $\mathfrak{R}_m(H) = 0$ for any $m > 0$.

(b) Use a similar construction to show that Massart's lemma (theorem 3.3) is tight.

3.4 Rademacher identities. Fix $m \geq 1$. Prove the following identities for any $\alpha \in \mathbb{R}$ and any two hypothesis sets H and H' of functions mapping from $\mathcal{X}$ to $\mathbb{R}$:

(a) $\mathfrak{R}_m(\alpha H) = |\alpha| \mathfrak{R}_m(H)$.

(b) $\mathfrak{R}_m(H + H') = \mathfrak{R}_m(H) + \mathfrak{R}_m(H')$.

(c) $\mathfrak{R}_m(\{\max(h, h') \colon h \in H, h' \in H'\})$,
where $\max(h, h')$ denotes the function $x \mapsto \max_{x \in \mathcal{X}}(h(x), h'(x))$ (*Hint*: you could use the identity $\max(a, b) = \frac{1}{2}[a + b + |a - b|]$ valid for all $a, b \in \mathbb{R}$ and Talagrand's contraction lemma (see lemma 4.2)).

3.5 Rademacher complexity. Professor Jesetoo claims to have found a better bound on the Rademacher complexity of any hypothesis set H of functions taking values in $\{-1, +1\}$, in terms of its VC-dimension $\mathrm{VCdim}(H)$. His bound is of the form $\mathfrak{R}_m(H) \leq O\big(\frac{\mathrm{VCdim}(H)}{m}\big)$. Can you show that Professor Jesetoo's claim cannot be correct? (*Hint*: consider a hypothesis set H reduced to just two simple functions.)

3.6 VC-dimension of union of k intervals. What is the VC-dimension of subsets of the real line formed by the union of k intervals?

3.7 VC-dimension of finite hypothesis sets. Show that the VC-dimension of a finite hypothesis set H is at most $\log_2 |H|$.

3.8 VC-dimension of subsets. What is the VC-dimension of the set of subsets I_α of the real line parameterized by a single parameter α: $I_\alpha = [\alpha, \alpha + 1] \cup [\alpha + 2, +\infty)$?

3.9 VC-dimension of closed balls in $\mathbb{R}^n$. Show that the VC-dimension of the set of all closed balls in $\mathbb{R}^n$, i.e., sets of the form $\{x \in \mathbb{R}^n \colon \|x - x_0\|^2 \leq r\}$ for some $x_0 \in \mathbb{R}^n$ and $r \geq 0$, is less than or equal to $n + 2$.

3.10 VC-dimension of ellipsoids. What is the VC-dimension of the set of all ellipsoids in $\mathbb{R}^n$?

3.11 VC-dimension of a vector space of real functions. Let F be a finite-dimensional vector space of real functions on $\mathbb{R}^n$, $\dim(F) = r < \infty$. Let H be the set of hypotheses:

$$H = \{\{x\colon f(x) \geq 0\}\colon f \in F\}.$$

Show that d, the VC-dimension of H, is finite and that $d \leq r$. (*Hint*: select an arbitrary set of $m = r + 1$ points and consider linear mapping $u\colon F \to \mathbb{R}^m$ defined by: $u(f) = (f(x_1), \ldots, f(x_m))$.)

3.12 VC-dimension of sine functions. Consider the hypothesis family of sine functions (Example 3.5): $\{x \to \sin(\omega x)\colon \omega \in \mathbb{R}\}$.

(a) Show that for any $x \in \mathbb{R}$ the points $x, 2x, 3x$ and $4x$ cannot be shattered by this family of sine functions.

(b) Show that the VC-dimension of the family of sine functions is infinite. (*Hint*: show that $\{2^{-m}\colon m \in \mathbb{N}\}$ can be fully shattered for any $m > 0$.)

3.13 VC-dimension of union of halfspaces. Determine the VC-dimension of the subsets of the real line formed by the union of k intervals.

3.14 VC-dimension of intersection of halfspaces. Consider the class C_k of convex intersections of k halfspaces. Give lower and upper bound estimates for $\mathrm{VCdim}(C_k)$.

3.15 VC-dimension of intersection concepts.

(a) Let C_1 and C_2 be two concept classes. Show that for any concept class $C = \{c_1 \cap c_2\colon c_1 \in C_1, c_2 \in C_2\}$,

$$\Pi_C(m) \leq \Pi_{C_1}(m)\,\Pi_{C_2}(m). \tag{3.55}$$

(b) Let C be a concept class with VC-dimension d and let C_s be the concept class formed by all intersections of s concepts from C, $s \geq 1$. Show that the VC-dimension of C_s is bounded by $2ds\log_2(3s)$. (*Hint*: show that $\log_2(3x) < 9x/(2e)$ for any $x \geq 2$.)

3.16 VC-dimension of union of concepts. Let A and B be two sets of functions mapping from X into $\{0, 1\}$, and assume that both A and B have finite VC-dimension, with $\mathrm{VCdim}(A) = d_A$ and $\mathrm{VCdim}(B) = d_B$. Let $C = A \cup B$ be the

union of A and B.

(a) Prove that for all m, $\Pi_C(m) \leq \Pi_A(m) + \Pi_B(m)$.

(b) Use Sauer's lemma to show that for $m \geq d_A + d_B + 2$, $\Pi_C(m) < 2^m$, and give a bound on the VC-dimension of C.

3.17 VC-dimension of symmetric difference of concepts. For two sets A and B, let $A \Delta B$ denote the symmetric difference of A and B, i.e., $A \Delta B = (A \cup B) - (A \cap B)$. Let H be a non-empty family of subsets of X with finite VC-dimension. Let A be an element of H and define $H \Delta A = \{X \Delta A \colon X \in H\}$. Show that

$$\mathrm{VCdim}(H \Delta A) = \mathrm{VCdim}(H).$$

3.18 Symmetric functions. A function $h \colon \{0,1\}^n \to \{0,1\}$ is *symmetric* if its value is uniquely determined by the number of 1's in the input. Let C denote the set of all symmetric functions.

(a) Determine the VC-dimension of C.

(b) Give lower and upper bounds on the sample complexity of any consistent PAC learning algorithm for C.

(c) Note that any hypothesis $h \in C$ can be represented by a vector $(y_0, y_1, ..., y_n) \in \{0,1\}^{n+1}$, where y_i is the value of h on examples having precisely i 1's. Devise a consistent learning algorithm for C based on this representation.

3.19 Biased coins. Professor Moent has two coins in his pocket, coin x_A and coin x_B. Both coins are slightly biased, i.e., $\Pr[x_A = 0] = 1/2 - \epsilon/2$ and $\Pr[x_B = 0] = 1/2 + \epsilon/2$, where $0 < \epsilon < 1$ is a small positive number, 0 denotes heads and 1 denotes tails. He likes to play the following game with his students. He picks a coin $x \in \{x_A, x_B\}$ from his pocket uniformly at random, tosses it m times, reveals the sequence of 0s and 1s he obtained and asks which coin was tossed. Determine how large m needs to be for a student's coin prediction error to be at most $\delta > 0$.

(a) Let S be a sample of size m. Professor Moent's best student, Oskar, plays according to the decision rule $f_o \colon \{0,1\}^m \to \{x_A, x_B\}$ defined by $f_o(S) = x_A$ iff $N(S) < m/2$, where $N(S)$ is the number of 0's in sample S.
Suppose m is even, then show that

$$error(f_o) \geq \frac{1}{2} \Pr\left[N(S) \geq \frac{m}{2} \middle| x = x_A\right]. \tag{3.56}$$

(b) Assuming m even, use the inequalities given in the appendix (section D.3)

to show that

$$error(f_o) > \frac{1}{4}\Big[1 - \Big[1 - e^{-\frac{m\epsilon^2}{1-\epsilon^2}}\Big]^{\frac{1}{2}}\Big]. \tag{3.57}$$

(c) Argue that if m is odd, the probability can be lower bounded by using $m+1$ in the bound in (a) and conclude that for both odd and even m,

$$error(f_o) > \frac{1}{4}\Big[1 - \Big[1 - e^{-\frac{2\lceil m/2\rceil\epsilon^2}{1-\epsilon^2}}\Big]^{\frac{1}{2}}\Big]. \tag{3.58}$$

(d) Using this bound, how large must m be if Oskar's error is at most δ, where $0 < \delta < 1/4$. What is the asymptotic behavior of this lower bound as a function of ϵ?

(e) Show that no decision rule $f\colon \{0,1\}^m \to \{x_a, x_B\}$ can do better than Oskar's rule f_o. Conclude that the lower bound of the previous question applies to all rules.

3.20 Infinite VC-dimension.

(a) Show that if a concept class C has infinite VC-dimension, then it is not PAC-learnable.

(b) In the standard PAC-learning scenario, the learning algorithm receives all examples first and then computes its hypothesis. Within that setting, PAC-learning of concept classes with infinite VC-dimension is not possible as seen in the previous question.
Imagine now a different scenario where the learning algorithm can alternate between drawing more examples and computation. The objective of this problem is to prove that PAC-learning can then be possible for some concept classes with infinite VC-dimension.
Consider for example the special case of the concept class C of all subsets of natural numbers. Professor Vitres has an idea for the first stage of a learning algorithm L PAC-learning C. In the first stage, L draws a sufficient number of points m such that the probability of drawing a point beyond the maximum value M observed be small with high confidence. Can you complete Professor Vitres' idea by describing the second stage of the algorithm so that it PAC-learns C? The description should be augmented with the proof that L can PAC-learn C.

3.21 VC-dimension generalization bound – realizable case. In this exercise we show that the bound given in corollary 3.4 can be improved to $O(\frac{d\log(m/d)}{m})$ in the realizable setting. Assume we are in the realizable scenario, i.e. the target concept is included in our hypothesis class H. We will show that if a hypothesis h is consistent

with a sample $S \sim D^m$ then for any $\epsilon > 0$ such that $m\epsilon \geq 8$

$$\Pr[R(h) > \epsilon] \leq 2\Big[\frac{2em}{d}\Big]^d 2^{-m\epsilon/2}\,. \tag{3.59}$$

(a) Let $H_S \subseteq H$ be the subset of hypotheses consistent with the sample S, let $\widehat{R}_S(h)$ denote the empirical error with respect to the sample S and define S' as a another independent sample drawn from D^m. Show that the following inequality holds for any $h_0 \in H_S$:

$$\Pr\Big[\sup_{h\in H_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\Big] \geq \Pr\Big[B[m,\epsilon] > \frac{m\epsilon}{2}\Big]\Pr[R(h_0) > \epsilon]\,,$$

where $B[m,\epsilon]$ is a binomial random variable with parameters $[m,\epsilon]$. (*Hint*: prove and use the fact that $\Pr[\widehat{R}(h) \geq \frac{\epsilon}{2}] \geq \Pr[\widehat{R}(h) > \frac{\epsilon}{2} \wedge R(h) > \epsilon]$.)

(b) Prove that $\Pr\Big[B(m,\epsilon) > \frac{m\epsilon}{2}\Big] \geq \frac{1}{2}$. Use this inequality along with the result from (a) to show that for any $h_0 \in H_S$

$$\Pr\Big[R(h_0) > \epsilon\Big] \leq 2\Pr\Big[\sup_{h\in H_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\Big]\,.$$

(c) Instead of drawing two samples, we can draw one sample T of size $2m$ then uniformly at random split it into S and S'. The right hand side of part (b) can then be rewritten as:

$$\Pr\Big[\sup_{h\in H_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\Big] = \Pr_{\substack{T\sim D^{2m}:\\ T\to[S,S']}}\Big[\exists h \in H : \widehat{R}_S(h) = 0 \wedge \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\Big]\,.$$

Let h_0 be a hypothesis such that $\widehat{R}_T(h_0) > \frac{\epsilon}{2}$ and let $l > \frac{m\epsilon}{2}$ be the total number of errors h_0 makes on T. Show that the probability of all l errors falling into S' is upper bounded by 2^{-l}.

(d) Part (b) implies that for any $h \in H$

$$\Pr_{\substack{T\sim D^{2m}:\\ T\to(S,S')}}\Big[\widehat{R}_S(h) = 0 \ \wedge\ \widehat{R}_{S'}(h) > \frac{\epsilon}{2} \ \Big|\ \widehat{R}_T(h_0) > \frac{\epsilon}{2}\Big] \leq 2^{-l}\,.$$

Use this bound to show that for any $h \in H$

$$\Pr_{\substack{T\sim D^{2m}:\\ T\to(S,S')}}\Big[\widehat{R}_S(h) = 0 \ \wedge\ \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\Big] \leq 2^{-\frac{\epsilon m}{2}}\,.$$

(e) Complete the proof of inequality (3.59) by using the union bound to upper bound $\Pr_{\substack{T\sim D^{2m}:\\ T\to(S,S')}}\Big[\exists h \in H : \widehat{R}_S(h) = 0 \wedge \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\Big]$. Show that we can achieve a high probability generalization bound that is of the order $O(\frac{d\log(m/d)}{m})$.

3.22 Generalization bound based on covering numbers. Let H be a family of functions mapping $\mathcal{X}$ to a subset of real numbers $\mathcal{Y} \subseteq \mathbb{R}$. For any $\epsilon > 0$, the *covering number* $\mathcal{N}(H, \epsilon)$ of H for the L_∞ norm is the minimal $k \in \mathbb{N}$ such that H can be covered with k balls of radius ϵ, that is, there exists $\{h_1, \ldots, h_k\} \subseteq H$ such that, for all $h \in H$, there exists $i \leq k$ with $\|h - h_i\|_\infty = \max_{x \in \mathcal{X}} |h(x) - h_i(x)| \leq \epsilon$. In particular, when H is a compact set, a finite covering can be extracted from a covering of H with balls of radius ϵ and thus $\mathcal{N}(H, \epsilon)$ is finite.

Covering numbers provide a measure of the complexity of a class of functions: the larger the covering number, the richer is the family of functions. The objective of this problem is to illustrate this by proving a learning bound in the case of the squared loss. Let D denote a distribution over $\mathcal{X} \times \mathcal{Y}$ according to which labeled examples are drawn. Then, the generalization error of $h \in H$ for the squared loss is defined by $R(h) = \mathrm{E}_{(x,y)\sim D}[(h(x) - y)^2]$ and its empirical error for a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$ by $\widehat{R}(h) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2$. We will assume that H is bounded, that is there exists $M > 0$ such that $|h(x) - y| \leq M$ for all $(x, y) \in \mathcal{X} \times \mathcal{Y}$. The following is the generalization bound proven in this problem:

$$\Pr_{S\sim D^m}\left[\sup_{h\in H} |R(h) - \widehat{R}(h)| \geq \epsilon\right] \leq \mathcal{N}\Big(H, \frac{\epsilon}{8M}\Big) 2 \exp\Big(\frac{-m\epsilon^2}{2M^4}\Big). \tag{3.60}$$

The proof is based on the following steps.

(a) Let $L_S = R(h) - \widehat{R}(h)$, then show that for all $h_1, h_2 \in H$ and any labeled sample S, the following inequality holds:

$$|L_S(h_1) - L_S(h_2)| \leq 4M \|h_1 - h_2\|_\infty .$$

(b) Assume that H can be covered by k subsets $B_1, \ldots, B_k$, that is $H = B_1 \cup \ldots \cup B_k$. Then, show that, for any $\epsilon > 0$, the following upper bound holds:

$$\Pr_{S\sim D^m}\left[\sup_{h\in H} |L_S(h)| \geq \epsilon\right] \leq \sum_{i=1}^k \Pr_{S\sim D^m}\left[\sup_{h\in B_i} |L_S(h)| \geq \epsilon\right].$$

(c) Finally, let $k = \mathcal{N}(H, \frac{\epsilon}{8M})$ and let $B_1, \ldots, B_k$ be balls of radius $\epsilon/(8M)$ centered at $h_1, \ldots, h_k$ covering H. Use part (a) to show that for all $i \in [1, k]$,

$$\Pr_{S\sim D^m}\left[\sup_{h\in B_i} |L_S(h)| \geq \epsilon\right] \leq \Pr_{S\sim D^m}\left[|L_S(h_i)| \geq \frac{\epsilon}{2}\right],$$

and apply Hoeffding's inequality (theorem D.1) to prove (3.60).

4 Support Vector Machines

This chapter presents one of the most theoretically well motivated and practically most effective classification algorithms in modern machine learning: Support Vector Machines (SVMs). We first introduce the algorithm for separable datasets, then present its general version designed for non-separable datasets, and finally provide a theoretical foundation for SVMs based on the notion of margin. We start with the description of the problem of linear classification.

4.1 Linear classification

Consider an input space $\mathcal{X}$ that is a subset of $\mathbb{R}^N$ with $N \geq 1$, and the output or target space $\mathcal{Y} = \{-1, +1\}$, and let $f\colon \mathcal{X} \to \mathcal{Y}$ be the target function. Given a hypothesis set H of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, the binary classification task is formulated as follows. The learner receives a training sample S of size m drawn i.i.d. from $\mathcal{X}$ according to some unknown distribution D, $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$, with $y_i = f(x_i)$ for all $i \in [1, m]$. The problem consists of determining a hypothesis $h \in H$, a *binary classifier*, with small generalization error:

$$R_D(h) = \Pr_{x \sim D}[h(x) \neq f(x)]. \tag{4.1}$$

Different hypothesis sets H can be selected for this task. In view of the results presented in the previous section, which formalized Occam's razor principle, hypothesis sets with smaller complexity — e.g., smaller VC-dimension or Rademacher complexity — provide better learning guarantees, everything else being equal. A natural hypothesis set with relatively small complexity is that of *linear classifiers*, or hyperplanes, which can be defined as follows:

$$H = \{\mathbf{x} \mapsto \operatorname{sign}(\mathbf{w} \cdot \mathbf{x} + b)\colon \mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}\}. \tag{4.2}$$

A hypothesis of the form $\mathbf{x} \mapsto \operatorname{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ thus labels positively all points falling on one side of the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ and negatively all others. The problem is referred to as a *linear classification problem*.

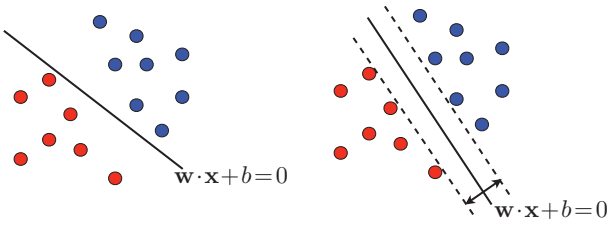


Figure 4.1 Two possible separating hyperplanes. The right-hand side figure shows a hyperplane that maximizes the margin.

4.2 SVMs — separable case

In this section, we assume that the training sample S can be linearly separated, that is, we assume the existence of a hyperplane that perfectly separates the training sample into two populations of positively and negatively labeled points, as illustrated by the left panel of figure 4.1. But there are then infinitely many such separating hyperplanes. Which hyperplane should a learning algorithm select? The solution returned by the SVM algorithm is the hyperplane with the maximum *margin*, or distance to the closest points, and is thus known as the *maximum-margin hyperplane*. The right panel of figure 4.1 illustrates that choice.

We will present later in this chapter a margin theory that provides a strong justification for this solution. We can observe already, however, that the SVM solution can also be viewed as the "safest" choice in the following sense: a test point is classified correctly by a separating hyperplane with margin ρ even when it falls within a distance ρ of the training samples sharing the same label; for the SVM solution, ρ is the maximum margin and thus the "safest" value.

4.2.1 Primal optimization problem

We now derive the equations and optimization problem that define the SVM solution. The general equation of a hyperplane in $\mathbb{R}^N$ is

$$\mathbf{w} \cdot \mathbf{x} + b = 0, \tag{4.3}$$

where $\mathbf{w} \in \mathbb{R}^N$ is a non-zero vector normal to the hyperplane and $b \in \mathbb{R}$ a scalar. Note that this definition of a hyperplane is invariant to non-zero scalar multiplication. Hence, for a hyperplane that does not pass through any sample point, we can scale $\mathbf{w}$ and b appropriately such that $\min_{(\mathbf{x},y)\in S} |\mathbf{w} \cdot \mathbf{x} + b| = 1$.

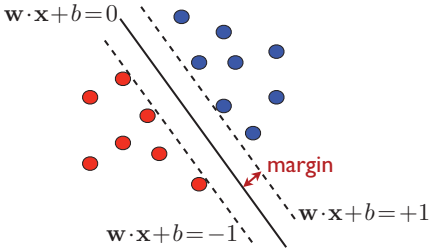


Figure 4.2 Margin and equations of the hyperplanes for a canonical maximum-margin hyperplane. The marginal hyperplanes are represented by dashed lines on the figure.

We define this representation of the hyperplane, i.e., the corresponding pair $(\mathbf{w}, b)$, as the *canonical hyperplane*. The distance of any point $\mathbf{x}_0 \in \mathbb{R}^N$ to a hyperplane defined by (4.3) is given by

$$\frac{|\mathbf{w} \cdot \mathbf{x}_0 + b|}{\|\mathbf{w}\|}. \tag{4.4}$$

Thus, for a canonical hyperplane, the margin ρ is given by

$$\rho = \min_{(\mathbf{x},y)\in S} \frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}. \tag{4.5}$$

Figure 4.2 illustrates the margin for a maximum-margin hyperplane with a canonical representation $(\mathbf{w}, b)$. It also shows the *marginal hyperplanes*, which are the hyperplanes parallel to the separating hyperplane and passing through the closest points on the negative or positive sides. Since they are parallel to the separating hyperplane, they admit the same normal vector $\mathbf{w}$. Furthermore, by definition of a canonical representation, for a point $\mathbf{x}$ on a marginal hyperplane, $|\mathbf{w} \cdot \mathbf{x} + b| = 1$, and thus the equations of the marginal hyperplanes are $\mathbf{w} \cdot \mathbf{x} + b = \pm 1$.

A hyperplane defined by $(\mathbf{w}, b)$ correctly classifies a training point $\mathbf{x}_i$, $i \in [1, m]$ when $\mathbf{w} \cdot \mathbf{x}_i + b$ has the same sign as y_i. For a canonical hyperplane, by definition, we have $|\mathbf{w} \cdot \mathbf{x}_i + b| \geq 1$ for all $i \in [1, m]$; thus, $\mathbf{x}_i$ is correctly classified when $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$. In view of (4.5), maximizing the margin of a canonical hyperplane is equivalent to minimizing $\|\mathbf{w}\|$ or $\frac{1}{2}\|\mathbf{w}\|^2$. Thus, in the separable case, the SVM solution, which is a hyperplane maximizing the margin while correctly classifying all training points, can be expressed as the solution to the following convex optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \tag{4.6}$$
$$\text{subject to: } y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1,\ \forall i \in [1,m]\,.$$

The objective function $F\colon \mathbf{w} \mapsto \frac{1}{2}\|\mathbf{w}\|^2$ is infinitely differentiable. Its gradient is $\nabla_{\mathbf{w}}(F) = \mathbf{w}$ and its Hessian the identity matrix $\nabla^2 F(\mathbf{w}) = \mathbf{I}$, whose eigenvalues are strictly positive. Therefore, $\nabla^2 F(\mathbf{w}) \succ \mathbf{0}$ and F is strictly convex. The constraints are all defined by affine functions $g_i\colon (\mathbf{w}, b) \mapsto 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)$ and are thus qualified. Thus, in view of the results known for convex optimization (see appendix B for details), the optimization problem of (4.6) admits a unique solution, an important and favorable property that does not hold for all learning algorithms.

Moreover, since the objective function is quadratic and the constraints affine, the optimization problem of (4.6) is in fact a specific instance of *quadratic programming* (QP), a family of problems extensively studied in optimization. A variety of commercial and open-source solvers are available for solving convex QP problems. Additionally, motivated by the empirical success of SVMs along with its rich theoretical underpinnings, specialized methods have been developed to more efficiently solve this particular convex QP problem, notably the block coordinate descent algorithms with blocks of just two coordinates.

4.2.2 Support vectors

The constraints are affine and thus qualified. The objective function as well as the affine constraints are convex and differentiable. Thus, the hypotheses of theorem B.8 hold and the KKT conditions apply at the optimum. We shall use these conditions to both analyze the algorithm and demonstrate several of its crucial properties, and subsequently derive the dual optimization problem associated to SVMs in section 4.2.3.

We introduce Lagrange variables $\alpha_i \geq 0$, $i \in [1,m]$, associated to the m constraints and denote by $\boldsymbol{\alpha}$ the vector $(\alpha_1, \ldots, \alpha_m)^\top$. The Lagrangian can then be defined for all $\mathbf{w} \in \mathbb{R}^N$, $b \in \mathbb{R}$, and $\boldsymbol{\alpha} \in \mathbb{R}^m_+$, by

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i [y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1]\,. \tag{4.7}$$

The KKT conditions are obtained by setting the gradient of the Lagrangian with respect to the primal variables $\mathbf{w}$ and b to zero and by writing the complementarity

conditions:

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0 \qquad \Longrightarrow \qquad \mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \tag{4.8}$$

$$\nabla_b \mathcal{L} = -\sum_{i=1}^{m} \alpha_i y_i = 0 \qquad \Longrightarrow \qquad \sum_{i=1}^{m} \alpha_i y_i = 0 \tag{4.9}$$

$$\forall i, \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] = 0 \qquad \Longrightarrow \qquad \alpha_i = 0 \vee y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1. \tag{4.10}$$

By equation 4.8, the weight vector $\mathbf{w}$ solution of the SVM problem is a linear combination of the training set vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$. A vector $\mathbf{x}_i$ appears in that expansion iff $\alpha_i \neq 0$. Such vectors are called *support vectors*. By the complementarity conditions (4.10), if $\alpha_i \neq 0$, then $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$. Thus, support vectors lie on the marginal hyperplanes $\mathbf{w} \cdot \mathbf{x}_i + b = \pm 1$.

Support vectors fully define the maximum-margin hyperplane or SVM solution, which justifies the name of the algorithm. By definition, vectors not lying on the marginal hyperplanes do not affect the definition of these hyperplanes — in their absence, the solution to the SVM problem remains unchanged. Note that while the solution $\mathbf{w}$ of the SVM problem is unique, the support vectors are not. In dimension N, $N+1$ points are sufficient to define a hyperplane. Thus, when more than $N+1$ points lie on a marginal hyperplane, different choices are possible for the $N+1$ support vectors.

4.2.3 Dual optimization problem

To derive the dual form of the constrained optimization problem (4.6), we plug into the Lagrangian the definition of $\mathbf{w}$ in terms of the dual variables as expressed in (4.8) and apply the constraint (4.9). This yields

$$\mathcal{L} = \underbrace{\frac{1}{2}\|\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i\|^2 - \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)}_{-\frac{1}{2}\sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)} - \underbrace{\sum_{i=1}^{m} \alpha_i y_i b}_{0} + \sum_{i=1}^{m} \alpha_i \,, \tag{4.11}$$

which simplifies to

$$\mathcal{L} = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \,. \tag{4.12}$$

This leads to the following dual optimization problem for SVMs in the separable case:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \tag{4.13}$$
$$\text{subject to: } \alpha_i \geq 0 \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, \ \forall i \in [1, m] \,.$$

The objective function $G \colon \boldsymbol{\alpha} \mapsto \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$ is infinitely differentiable. Its Hessian is given by $\nabla^2 G = -\mathbf{A}$, with $\mathbf{A} = \big(y_i \mathbf{x}_i \cdot y_j \mathbf{x}_j\big)_{ij}$. $\mathbf{A}$ is the Gram matrix associated to the vectors $y_1 \mathbf{x}_1, \ldots, y_m \mathbf{x}_m$ and is therefore positive semidefinite, which shows that $\nabla^2 G \preceq \mathbf{0}$ and that G is a concave function. Since the constraints are affine and convex, the maximization problem (4.13) is equivalent to a convex optimization problem. Since G is a quadratic function of $\boldsymbol{\alpha}$, this dual optimization problem is also a QP problem, as in the case of the primal optimization and once again both general-purpose and specialized QP solvers can be used to obtain the solution (see exercise 4.4 for details on the SMO algorithm, which is often used to solve the dual form of the SVM problem in the more general non-separable setting).

Moreover, since the constraints are affine, they are qualified and strong duality holds (see appendix B). Thus, the primal and dual problems are equivalent, i.e., the solution $\boldsymbol{\alpha}$ of the dual problem (4.13) can be used directly to determine the hypothesis returned by SVMs, using equation (4.8):

$$h(\mathbf{x}) = \operatorname{sgn}(\mathbf{w} \cdot \mathbf{x} + b) = \operatorname{sgn}\big(\sum_{i=1}^{m} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}) + b\big). \tag{4.14}$$

Since support vectors lie on the marginal hyperplanes, for any support vector $\mathbf{x}_i$, $\mathbf{w} \cdot \mathbf{x}_i + b = y_i$, and thus b can be obtained via

$$b = y_i - \sum_{j=1}^{m} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i) \,. \tag{4.15}$$

The dual optimization problem (4.13) and the expressions (4.14) and (4.15) reveal an important property of SVMs: the hypothesis solution depends only on inner products between vectors and not directly on the vectors themselves.

Equation (4.15) can now be used to derive a simple expression of the margin ρ in terms of $\boldsymbol{\alpha}$. Since (4.15) holds for all i with $\alpha_i \neq 0$, multiplying both sides by $\alpha_i y_i$ and taking the sum leads to

$$\sum_{i=1}^{m} \alpha_i y_i b = \sum_{i=1}^{m} \alpha_i y_i^2 - \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \,. \tag{4.16}$$

Using the fact that $y_i^2 = 1$ along with equation 4.8 then yields

$$0 = \sum_{i=1}^{m} \alpha_i - \|\mathbf{w}\|^2. \tag{4.17}$$

Noting that $\alpha_i \geq 0$, we obtain the following expression of the margin ρ in terms of the L_1 norm of $\boldsymbol{\alpha}$:

$$\rho^2 = \frac{1}{\|\mathbf{w}\|_2^2} = \frac{1}{\sum_{i=1}^{m} \alpha_i} = \frac{1}{\|\boldsymbol{\alpha}\|_1}. \tag{4.18}$$

4.2.4 Leave-one-out analysis

We now use the notion of *leave-one-out error* to derive a first learning guarantee for SVMs based on the fraction of support vectors in the training set.

Definition 4.1 **Leave-one-out error**
Let h_S denote the hypothesis returned by a learning algorithm $\mathcal{A}$, when trained on a fixed sample S. Then, the leave-one-out error *of $\mathcal{A}$ on a sample S of size m is defined by*

$$\widehat{R}_{LOO}(\mathcal{A}) = \frac{1}{m} \sum_{i=1}^{m} 1_{h_{S-\{x_i\}}(x_i) \neq y_i}.$$

Thus, for each $i \in [1, m]$, $\mathcal{A}$ is trained on all the points in S except for x_i, i.e., $S - \{x_i\}$, and its error is then computed using x_i. The leave-one-out error is the average of these errors. We will use an important property of the leave-one-out error stated in the following lemma.

Lemma 4.1
The average leave-one-out error for samples of size $m \geq 2$ is an unbiased estimate of the average generalization error for samples of size $m - 1$:

$$\mathop{\mathrm{E}}_{S \sim D^m}[\widehat{R}_{LOO}(\mathcal{A})] = \mathop{\mathrm{E}}_{S' \sim D^{m-1}}[R(h_{S'})], \tag{4.19}$$

where D denotes the distribution according to which points are drawn.

Proof By the linearity of expectation, we can write

$$\begin{aligned}
\mathop{\mathrm{E}}_{S\sim D^m}[\widehat{R}_{\mathrm{LOO}}(\mathcal{A})] &= \frac{1}{m}\sum_{i=1}^{m}\mathop{\mathrm{E}}_{S\sim D^m}[1_{h_{S-\{x_i\}}(x_i)\neq y_i}] \\
&= \mathop{\mathrm{E}}_{S\sim D^m}[1_{h_{S-\{x_1\}}(x_1)\neq y_1}] \\
&= \mathop{\mathrm{E}}_{S'\sim D^{m-1},x_1\sim D}[1_{h_{S'}(x_1)\neq y_1}] \\
&= \mathop{\mathrm{E}}_{S'\sim D^{m-1}}[\mathop{\mathrm{E}}_{x_1\sim D}[1_{h_{S'}(x_1)\neq y_1}]] \\
&= \mathop{\mathrm{E}}_{S'\sim D^{m-1}}[R(h_{S'})].
\end{aligned}$$

For the second equality, we used the fact that, since the points of S are drawn in an i.i.d. fashion, the expectation $\mathrm{E}_{S\sim D^m}[1_{h_{S-\{x_i\}}(x_i)\neq y_i}]$ does not depend on the choice of $i \in [1, m]$ and is thus equal to $\mathrm{E}_{S\sim D^m}[1_{h_{S-\{x_1\}}(x_1)\neq y_1}]$. ■

In general, computing the leave-one-out error may be costly since it requires training m times on samples of size $m-1$. In some situations however, it is possible to derive the expression of $\widehat{R}_{\mathrm{loo}}(\mathcal{A})$ much more efficiently (see exercise 10.9).

Theorem 4.1
Let h_S be the hypothesis returned by SVMs for a sample S, and let $N_{SV}(S)$ be the number of support vectors that define h_S. Then,

$$\mathop{\mathrm{E}}_{S\sim D^m}[R(h_S)] \leq \mathop{\mathrm{E}}_{S\sim D^{m+1}}\left[\frac{N_{\mathrm{SV}}(S)}{m+1}\right].$$

Proof Let S be a linearly separable sample of $m+1$. If x is not a support vector for h_S, removing it does not change the SVM solution. Thus, $h_{S-\{x\}} = h_S$ and $h_{S-\{x\}}$ correctly classifies x. By contraposition, if $h_{S-\{x\}}$ misclassifies x, x must be a support vector, which implies

$$\widehat{R}_{\mathrm{loo}}(\mathrm{SVM}) \leq \frac{N_{\mathrm{SV}}(S)}{m+1}. \tag{4.20}$$

Taking the expectation of both sides and using lemma 4.1 yields the result. ■

Theorem 4.1 gives a sparsity argument in favor of SVMs: the average error of the algorithm is upper bounded by the average fraction of support vectors. One may hope that for many distributions seen in practice, a relatively small number of the training points will lie on the marginal hyperplanes. The solution will then be sparse in the sense that a small fraction of the dual variables α_i will be non-zero. Note, however, that this bound is relatively weak since it applies only to the average generalization error of the algorithm over all samples of size m. It provides no information about the variance of the generalization error. In section 4.4, we present stronger high-probability bounds using a different argument based on the

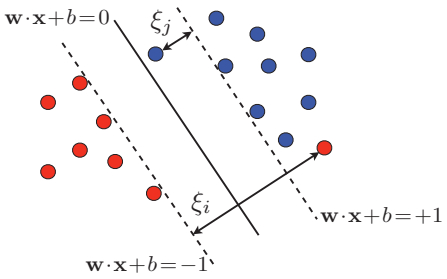


Figure 4.3 A separating hyperplane with point x_i classified incorrectly and point x_j correctly classified, but with margin less than 1.

notion of margin.

4.3 SVMs — non-separable case

In most practical settings, the training data is not linearly separable, i.e., for any hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$, there exists $x_i \in S$ such that

$$y_i \left[\mathbf{w} \cdot \mathbf{x}_i + b\right] \ngeq 1 \,. \tag{4.21}$$

Thus, the constraints imposed in the linearly separable case discussed in section 4.2 cannot all hold simultaneously. However, a relaxed version of these constraints can indeed hold, that is, for each $i \in [1, m]$, there exist $\xi_i \geq 0$ such that

$$y_i \left[\mathbf{w} \cdot \mathbf{x}_i + b\right] \geq 1 - \xi_i \,. \tag{4.22}$$

The variables ξ_i are known as *slack variables* and are commonly used in optimization to define relaxed versions of some constraints. Here, a slack variable ξ_i measures the distance by which vector $\mathbf{x}_i$ violates the desired inequality, $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$. Figure 4.3 illustrates the situation. For a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$, a vector $\mathbf{x}_i$ with $\xi_i > 0$ can be viewed as an *outlier*. Each $\mathbf{x}_i$ must be positioned on the correct side of the appropriate marginal hyperplane to not be considered an outlier. As a consequence, a vector $\mathbf{x}_i$ with $0 < y_i(\mathbf{w} \cdot \mathbf{x}_i + b) < 1$ is correctly classified by the hyperplane $\mathbf{w}\cdot\mathbf{x}+b = 0$ but is nonetheless considered to be an outlier, that is, $\xi_i > 0$. If we omit the outliers, the training data is correctly separated by $\mathbf{w} \cdot \mathbf{x} + b = 0$ with a margin $\rho = 1/\|\mathbf{w}\|$ that we refer to as the *soft margin*, as opposed to the *hard margin* in the separable case.

How should we select the hyperplane in the non-separable case? One idea consists of selecting the hyperplane that minimizes the empirical error. But, that solution

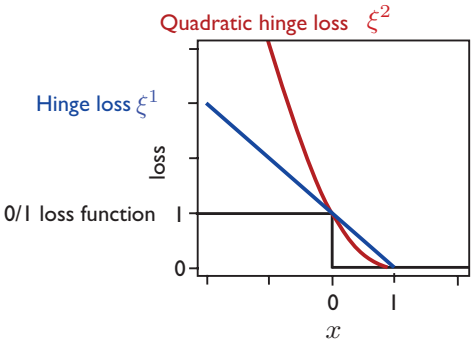


Figure 4.4 Both the hinge loss and the quadratic hinge loss provide convex upper bounds on the binary zero-one loss.

will not benefit from the large-margin guarantees we will present in section 4.4. Furthermore, the problem of determining a hyperplane with the smallest zero-one loss, that is the smallest number of misclassifications, is NP-hard as a function of the dimension N of the space.

Here, there are two conflicting objectives: on one hand, we wish to limit the total amount of slack due to outliers, which can be measured by $\sum_{i=1}^m \xi_i$, or, more generally by $\sum_{i=1}^m \xi_i^p$ for some $p \geq 1$; on the other hand, we seek a hyperplane with a large margin, though a larger margin can lead to more outliers and thus larger amounts of slack.

4.3.1 Primal optimization problem

This leads to the following general optimization problem defining SVMs in the non-separable case where the parameter $C \geq 0$ determines the trade-off between margin-maximization (or minimization of $\|\mathbf{w}\|^2$) and the minimization of the slack penalty $\sum_{i=1}^m \xi_i^p$:

$$\begin{aligned} \min_{\mathbf{w},b,\boldsymbol{\xi}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^m \xi_i^p \\ \text{subject to} \quad & y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i \ \wedge \ \xi_i \geq 0, i \in [1,m]\,, \end{aligned} \tag{4.23}$$

where $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_m)^\top$. The parameter C is typically determined via n-fold cross-validation (see section 1.3).

As in the separable case, (4.23) is a convex optimization problem since the constraints are affine and thus convex and since the objective function is convex for any $p \geq 1$. In particular, $\boldsymbol{\xi} \mapsto \sum_{i=1}^m \xi_i^p = \|\boldsymbol{\xi}\|_p^p$ is convex in view of the convexity of the norm $\|\cdot\|_p$.

There are many possible choices for p leading to more or less aggressive penalizations of the slack terms (see exercise 4.1). The choices $p = 1$ and $p = 2$ lead to the most straightforward solutions and analyses. The loss functions associated with $p = 1$ and $p = 2$ are called the *hinge loss* and the *quadratic hinge loss*, respectively. Figure 4.4 shows the plots of these loss functions as well as that of the standard zero-one loss function. Both hinge losses are convex upper bounds on the zero-one loss, thus making them well suited for optimization. In what follows, the analysis is presented in the case of the hinge loss ($p = 1$), which is the most widely used loss function for SVMs.

4.3.2 Support vectors

As in the separable case, the constraints are affine and thus qualified. The objective function as well as the affine constraints are convex and differentiable. Thus, the hypotheses of theorem B.8 hold and the KKT conditions apply at the optimum. We use these conditions to both analyze the algorithm and demonstrate several of its crucial properties, and subsequently derive the dual optimization problem associated to SVMs in section 4.3.3.

We introduce Lagrange variables $\alpha_i \geq 0$, $i \in [1, m]$, associated to the first m constraints and $\beta_i \geq 0$, $i \in [1, m]$ associated to the non-negativity constraints of the slack variables. We denote by $\boldsymbol{\alpha}$ the vector $(\alpha_1, \ldots, \alpha_m)^\top$ and by $\boldsymbol{\beta}$ the vector $(\beta_1, \ldots, \beta_m)^\top$. The Lagrangian can then be defined for all $\mathbf{w} \in \mathbb{R}^N$, $b \in \mathbb{R}$, and $\boldsymbol{\alpha} \in \mathbb{R}^m_+$, by

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} \xi_i - \sum_{i=1}^{m} \alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^{m} \beta_i \xi_i \,. \quad (4.24)$$

The KKT conditions are obtained by setting the gradient of the Lagrangian with respect to the primal variables $\mathbf{w}$, b, and ξ_is to zero and by writing the complementarity conditions:

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0 \quad \Longrightarrow \quad \mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \quad (4.25)$$

$$\nabla_b \mathcal{L} = -\sum_{i=1}^{m} \alpha_i y_i = 0 \quad \Longrightarrow \quad \sum_{i=1}^{m} \alpha_i y_i = 0 \quad (4.26)$$

$$\nabla_{\xi_i}\mathcal{L} = C - \alpha_i - \beta_i = 0 \quad \Longrightarrow \quad \alpha_i + \beta_i = C \quad (4.27)$$

$$\forall i, \alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i] = 0 \quad \Longrightarrow \quad \alpha_i = 0 \vee y_i(\mathbf{w}\cdot\mathbf{x}_i + b) = 1 - \xi_i \quad (4.28)$$

$$\forall i, \beta_i \xi_i = 0 \quad \Longrightarrow \quad \beta_i = 0 \vee \xi_i = 0\,. \quad (4.29)$$

By equation 4.25, as in the separable case, the weight vector $\mathbf{w}$ solution of the SVMproblem is a linear combination of the training set vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$. A vector

$\mathbf{x}_i$ appears in that expansion iff $\alpha_i \neq 0$. Such vectors are called *support vectors*. Here, there are two types of support vectors. By the complementarity condition (4.28), if $\alpha_i \neq 0$, then $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1 - \xi_i$. If $\xi_i = 0$, then $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$ and $\mathbf{x}_i$ lies on a marginal hyperplane, as in the separable case. Otherwise, $\xi_i \neq 0$ and $\mathbf{x}_i$ is an outlier. In this case, (4.29) implies $\beta_i = 0$ and (4.27) then requires $\alpha_i = C$. Thus, support vectors $\mathbf{x}_i$ are either outliers, in which case $\alpha_i = C$, or vectors lying on the marginal hyperplanes. As in the separable case, note that while the weight vector $\mathbf{w}$ solution is unique, the support vectors are not.

4.3.3 Dual optimization problem

To derive the dual form of the constrained optimization problem (4.23), we plug into the Lagrangian the definition of $\mathbf{w}$ in terms of the dual variables (4.25) and apply the constraint (4.26). This yields

$$\mathcal{L} = \underbrace{\frac{1}{2}\|\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i\|^2 - \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)}_{-\frac{1}{2}\sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)} - \underbrace{\sum_{i=1}^{m} \alpha_i y_i b}_{0} + \sum_{i=1}^{m} \alpha_i \,. \tag{4.30}$$

Remarkably, we find that the objective function is no different than in the separable case:

$$\mathcal{L} = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \,. \tag{4.31}$$

However, here, in addition to $\alpha_i \geq 0$, we must impose the constraint on the Lagrange variables $\beta_i \geq 0$. In view of (4.27), this is equivalent to $\alpha_i \leq C$. This leads to the following dual optimization problem for SVMs in the non-separable case, which only differs from that of the separable case (4.13) by the constraints $\alpha_i \leq C$:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \tag{4.32}$$

$$\text{subject to: } 0 \leq \alpha_i \leq C \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [1, m].$$

Thus, our previous comments about the optimization problem (4.13) apply to (4.32) as well. In particular, the objective function is concave and infinitely differentiable and (4.32) is equivalent to a convex QP. The problem is equivalent to the primal problem (4.23).

The solution $\boldsymbol{\alpha}$ of the dual problem (4.32) can be used directly to determine the

hypothesis returned by SVMs, using equation (4.25):

$$h(\mathbf{x}) = \operatorname{sgn}(\mathbf{w} \cdot \mathbf{x} + b) = \operatorname{sgn}\big(\sum_{i=1}^{m} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}) + b\big). \tag{4.33}$$

Moreover, b can be obtained from any support vector $\mathbf{x}_i$ lying on a marginal hyperplane, that is any vector $\mathbf{x}_i$ with $0 < \alpha_i < C$. For such support vectors, $\mathbf{w} \cdot \mathbf{x}_i + b = y_i$ and thus

$$b = y_i - \sum_{j=1}^{m} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i)\,. \tag{4.34}$$

As in the separable case, the dual optimization problem (4.32) and the expressions (4.33) and (4.34) show an important property of SVMs: the hypothesis solution depends only on inner products between vectors and not directly on the vectors themselves. This fact can be used to extend SVMs to define non-linear decision boundaries, as we shall see in chapter 5.

4.4 Margin theory

This section presents generalization bounds based on the notion of margin, which provide a strong theoretical justification for the SVM algorithm. We first give the definitions of some basic margin concepts.

Definition 4.2 **Margin**
The geometric margin $\rho(x)$ *of a point* $\mathbf{x}$ *with label* y *with respect to a linear classifier* $h\colon \mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} + b$ *is its distance to the hyperplane* $\mathbf{w} \cdot \mathbf{x} + b = 0$*:*

$$\rho(x) = \frac{y\,(\mathbf{w} \cdot \mathbf{x} + b)}{\|\mathbf{w}\|}\,. \tag{4.35}$$

The margin *of a linear classifier* h *for a sample* $S = (\mathbf{x}_1, \ldots, \mathbf{x}_m)$ *is the minimum margin over the points in the sample:*

$$\rho = \min_{1 \le i \le m} \frac{y_i\,(\mathbf{w} \cdot \mathbf{x}_i + b)}{\|\mathbf{w}\|}\,. \tag{4.36}$$

Recall that the VC-dimension of the family of hyperplanes or linear hypotheses in $\mathbb{R}^N$ is $N+1$. Thus, the application of the VC-dimension bound (3.31) of corollary 3.4 to this hypothesis set yields the following: for any $\delta > 0$, with probability at least

$1-\delta$, for any $h \in H$,

$$R(h) \leq \widehat{R}(h) + \sqrt{\frac{2(N+1)\log\frac{em}{N+1}}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}. \tag{4.37}$$

When the dimension of the feature space N is large compared to the sample size, this bound is uninformative. The following theorem presents instead a bound on the VC-dimension of canonical hyperplanes that does not depend on the dimension of feature space N, but only on the margin and the radius r of the sphere containing the data.

Theorem 4.2
Let $S \subseteq \{\mathbf{x}\colon \|\mathbf{x}\| \leq r\}$. Then, the VC-dimension d of the set of canonical hyperplanes $\{x \mapsto \operatorname{sgn}(\mathbf{w}\cdot\mathbf{x})\colon \min_{x\in S}|\mathbf{w}\cdot\mathbf{x}| = 1 \wedge \|\mathbf{w}\| \leq \Lambda\}$ verifies

$$d \leq r^2\Lambda^2\,.$$

Proof Assume $\{\mathbf{x}_1,\ldots,\mathbf{x}_d\}$ is a set that can be fully shattered. Then, for all $\mathbf{y} = (y_1,\ldots,y_d) \in \{-1,+1\}^d$, there exists $\mathbf{w}$ such that,

$$\forall i \in [1,d], 1 \leq y_i(\mathbf{w}\cdot\mathbf{x}_i)\,.$$

Summing up these inequalities yields

$$d \leq \mathbf{w}\cdot\sum_{i=1}^{d} y_i\mathbf{x}_i \leq \|\mathbf{w}\|\|\sum_{i=1}^{d} y_i\mathbf{x}_i\| \leq \Lambda\|\sum_{i=1}^{d} y_i\mathbf{x}_i\|\,.$$

Since this inequality holds for all $\mathbf{y} \in \{-1,+1\}^d$, it also holds on expectation over $y_1,\ldots,y_d$ drawn i.i.d. according to a uniform distribution over $\{-1,+1\}$. In view of the independence assumption, for $i \neq j$ we have $\mathrm{E}[y_iy_j] = \mathrm{E}[y_i]\,\mathrm{E}[y_j]$. Thus, since the distribution is uniform, $\mathrm{E}[y_iy_j] = 0$ if $i \neq j$, $\mathrm{E}[y_iy_j] = 1$ otherwise. This gives

$$\begin{aligned}
d &\leq \Lambda \mathop{\mathrm{E}}_{\mathbf{y}}[\|\sum_{i=1}^{d} y_i\mathbf{x}_i\|] && \text{(taking expectations)}\\
&\leq \Lambda\Big[\mathop{\mathrm{E}}_{\mathbf{y}}[\|\sum_{i=1}^{d} y_i\mathbf{x}_i\|^2]\Big]^{1/2} && \text{(Jensen's inequality)}\\
&= \Lambda\Big[\sum_{i,j=1}^{d} \mathop{\mathrm{E}}_{\mathbf{y}}[y_iy_j](\mathbf{x}_i\cdot\mathbf{x}_j)\Big]^{1/2}\\
&= \Lambda\big[\sum_{i=1}^{d}(\mathbf{x}_i\cdot\mathbf{x}_i)\big]^{1/2} \leq \Lambda\big[dr^2\big]^{1/2} = \Lambda r\sqrt{d}.
\end{aligned}$$

Thus, $\sqrt{d} \leq \Lambda r$, which completes the proof. ■

When the training data is linearly separable, by the results of section 4.2, the maximum-margin canonical hyperplane with $\|\mathbf{w}\| = 1/\rho$ can be plugged into theorem 4.2. In this case, Λ can be set to $1/\rho$, and the upper bound can be rewritten as r^2/ρ^2. Note that the choice of Λ must be made before receiving the sample S.

It is also possible to bound the Rademacher complexity of linear hypotheses with bounded weight vector in a similar way, as shown by the following theorem.

Theorem 4.3
Let $S \subseteq \{x\colon \|\mathbf{x}\| \leq R\}$ be a sample of size m and let $H = \{\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x}\colon \|\mathbf{w}\| \leq \Lambda\}$. Then, the empirical Rademacher complexity of H can be bounded as follows:

$$\widehat{\mathfrak{R}}_S(H) \leq \sqrt{\frac{r^2\Lambda^2}{m}}\,.$$

Proof The proof follows through a series of inequalities similar to those of theorem 4.2:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(H) &= \frac{1}{m}\operatorname*{E}_{\sigma}\Big[\sum_{i=1}^{m}\sigma_i\mathbf{w}\cdot\mathbf{x}_i\Big] = \frac{1}{m}\operatorname*{E}_{\sigma}\Big[\mathbf{w}\cdot\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\Big] \leq \frac{\Lambda}{m}\operatorname*{E}_{\sigma}\Big[\Big\|\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\Big\|\Big] \\
&\leq \frac{\Lambda}{m}\left[\operatorname*{E}_{\sigma}\Big[\Big\|\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\Big\|^2\Big]\right]^{1/2} = \frac{\Lambda}{m}\left[\operatorname*{E}_{\sigma}\Big[\sum_{i,j=1}^{m}\sigma_i\sigma_j(\mathbf{x}_i\cdot\mathbf{x}_j)\Big]\right]^{1/2} \\
&\leq \frac{\Lambda}{m}\left[\sum_{i=1}^{m}\|\mathbf{x}_i\|^2\right]^{1/2} \leq \frac{\Lambda\sqrt{mr^2}}{m} = \sqrt{\frac{r^2\Lambda^2}{m}}\,,
\end{aligned}$$

The first inequality makes use of the Cauchy-Schwarz inequality and the bound on $\|\mathbf{w}\|$, the second follows by Jensen's inequality, the third by $\operatorname{E}[\sigma_i\sigma_j] = \operatorname{E}[\sigma_i]\operatorname{E}[\sigma_j] = 0$ for $i \neq j$, and the last one by $\|\mathbf{x}_i\| \leq R$. ■

To present the main margin-based generalization bounds of this section, we need to introduce a *margin loss function*. Here, the training data is not assumed to be separable. The quantity $\rho > 0$ should thus be interpreted as the margin we wish to achieve.

Definition 4.3 **Margin loss function**
For any $\rho > 0$, the ρ-margin loss is the function $L_\rho\colon \mathbb{R}\times\mathbb{R} \to \mathbb{R}_+$ defined for all $y, y' \in \mathbb{R}$ by $L_\rho(y, y') = \Phi_\rho(yy')$ with,

$$\Phi_\rho(x) = \begin{cases} 0 & \text{if } \rho \leq x \\ 1 - x/\rho & \text{if } 0 \leq x \leq \rho \\ 1 & \text{if } x \leq 0\,. \end{cases}$$

This loss function is illustrated in figure 4.5. The empirical margin loss is then defined as the margin loss over the training sample.

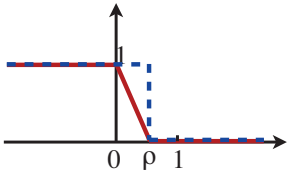


Figure 4.5 The margin loss, defined with respect to margin parameter ρ.

Definition 4.4 **Empirical margin loss**
Given a sample $S = (x_1, \ldots, x_m)$ and a hypothesis h, the empirical margin loss is defined by

$$\widehat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^{m} \Phi_\rho(y_i h(x_i)) \,. \tag{4.38}$$

Note that for any $i \in [1, m]$, $\Phi_\rho(y_i h(x_i)) \leq 1_{y_i h(x_i) \leq \rho}$. Thus, the empirical margin loss can be upper-bounded as follows:

$$\widehat{R}_\rho(h) \leq \frac{1}{m} \sum_{i=1}^{m} 1_{y_i h(x_i) \leq \rho} \,. \tag{4.39}$$

In all the results that follow, the empirical margin loss can be replaced by this upper bound, which admits a simple interpretation: it is the fraction of the points in the training sample S that have been misclassified or classified with confidence less than ρ. When h is a linear function defined by a weight vector $\mathbf{w}$ with $\|\mathbf{w}\| = 1$, $y_i h(x_i)$ is the margin of point x_i. Thus, the upper bound is then the fraction of the points in the training data with margin less than ρ. This corresponds to the loss function indicated by the blue dotted line in figure 4.5.

The slope of the function Φ_ρ defining the margin loss is at most $1/\rho$, thus Φ_ρ is $1/\rho$-Lipschitz. The following lemma bounds the empirical Rademacher complexity of a hypothesis set H after composition with such a Lipschitz function in terms of the empirical Rademacher complexity of H. It will be needed for the proof of the margin-based generalization bound.

Lemma 4.2 **Talagrand's lemma**
Let $\Phi\colon \mathbb{R} \to \mathbb{R}$ be an l-Lipschitz. Then, for any hypothesis set H of real-valued functions, the following inequality holds:

$$\widehat{\mathfrak{R}}_S(\Phi \circ H) \leq l\, \widehat{\mathfrak{R}}_S(H) \,.$$

Proof First we fix a sample $S = (x_1, \ldots, x_m)$, then, by definition,

$$\begin{aligned}\widehat{\mathfrak{R}}_S(\Phi \circ H) &= \frac{1}{m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h \in H} \sum_{i=1}^{m} \sigma_i (\Phi \circ h)(x_i)\Big] \\ &= \frac{1}{m} \mathop{\mathrm{E}}_{\sigma_1, \ldots, \sigma_{m-1}}\Big[\mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m (\Phi \circ h)(x_m)\Big]\Big],\end{aligned}$$

where $u_{m-1}(h) = \sum_{i=1}^{m-1} \sigma_i (\Phi \circ h)(x_i)$. By definition of the supremum, for any $\epsilon > 0$, there exist $h_1, h_2 \in H$ such that

$$\begin{aligned} u_{m-1}(h_1) + (\Phi \circ h_1)(x_m) &\geq (1-\epsilon)\Big[\sup_{h \in H} u_{m-1}(h) + (\Phi \circ h)(x_m)\Big] \\ \text{and} \quad u_{m-1}(h_2) - (\Phi \circ h_2)(x_m) &\geq (1-\epsilon)\Big[\sup_{h \in H} u_{m-1}(h) - (\Phi \circ h)(x_m)\Big].\end{aligned}$$

Thus, for any $\epsilon > 0$, by definition of E_{σ_m},

$$\begin{aligned}&(1-\epsilon) \mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m (\Phi \circ h)(x_m)\Big] \\ &\quad = (1-\epsilon)\Big[\frac{1}{2} \sup_{h \in H} u_{m-1}(h) + (\Phi \circ h)(x_m) + \frac{1}{2} \sup_{h \in H} u_{m-1}(h) - (\Phi \circ h)(x_m)\Big] \\ &\quad \leq \frac{1}{2}[u_{m-1}(h_1) + (\Phi \circ h_1)(x_m)] + \frac{1}{2}[u_{m-1}(h_2) - (\Phi \circ h_2)(x_m)].\end{aligned}$$

Let $s = \operatorname{sgn}(h_1(x_m) - h_2(x_m))$. Then, the previous inequality implies

$$\begin{aligned}&(1-\epsilon) \mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m (\Phi \circ h)(x_m)\Big] \\ &\leq \frac{1}{2}[u_{m-1}(h_1) + u_{m-1}(h_2) + sl(h_1(x_m) - h_2(x_m))] && \text{(Lipschitz property)} \\ &= \frac{1}{2}[u_{m-1}(h_1) + slh_1(x_m)] + \frac{1}{2}[u_{m-1}(h_2) - slh_2(x_m)] && \text{(rearranging)} \\ &\leq \frac{1}{2} \sup_{h \in H}[u_{m-1}(h) + slh(x_m)] + \frac{1}{2} \sup_{h \in H}[u_{m-1}(h) - slh(x_m)] && \text{(definition of sup)} \\ &= \mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m lh(x_m)\Big]. && \text{(definition of } \mathop{\mathrm{E}}_{\sigma_m}\text{)}\end{aligned}$$

Since the inequality holds for all $\epsilon > 0$, we have

$$\mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m (\Phi \circ h)(x_m)\Big] \leq \mathop{\mathrm{E}}_{\sigma_m}\Big[\sup_{h \in H} u_{m-1}(h) + \sigma_m lh(x_m)\Big].$$

Proceeding in the same way for all other σ_is ($i \neq m$) proves the lemma. ■

The following is a general margin-based generalization bound that will be used in the analysis of several algorithms.

Theorem 4.4 **Margin bound for binary classification**
Let H be a set of real-valued functions. Fix $\rho > 0$, then, for any $\delta > 0$, with probability at least $1 - \delta$, each of the following holds for all $h \in H$:

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2}{\rho}\mathfrak{R}_m(H) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} \tag{4.40}$$

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2}{\rho}\widehat{\mathfrak{R}}_S(H) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}\,. \tag{4.41}$$

Proof Let $\widetilde{H} = \{z = (x, y) \mapsto yh(x) \colon h \in H\}$. Consider the family of functions taking values in $[0, 1]$:

$$\widetilde{\mathcal{H}} = \{\Phi_\rho \circ f \colon f \in \widetilde{H}\}\,.$$

By theorem 3.1, with probability at least $1 - \delta$, for all $g \in \widetilde{\mathcal{H}}$,

$$\mathrm{E}[g(z)] \leq \frac{1}{m}\sum_{i=1}^{m} g(z_i) + 2\mathfrak{R}_m(\widetilde{\mathcal{H}}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}\,,$$

and thus, for all $h \in H$,

$$\mathrm{E}[\Phi_\rho(yh(x))] \leq \widehat{R}_\rho(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}\,.$$

Since $1_{u \leq 0} \leq \Phi_\rho(u)$ for all $u \in \mathbb{R}$, we have $R(h) = \mathrm{E}[1_{yh(x) \leq 0}] \leq \mathrm{E}[\Phi_\rho(yh(x))]$, thus

$$R(h) \leq \widehat{R}_\rho(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}\,.$$

$\mathfrak{R}_m$ is invariant to a constant shift, therefore we have

$$\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) = \mathfrak{R}_m\big((\Phi_\rho - 1) \circ \widetilde{H}\big).$$

Since $(\Phi_\rho - 1)(0) = 0$ and since $(\Phi_\rho - 1)$ is $1/\rho$-Lipschitz as with Φ_ρ, by lemma 4.2, we have $\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) \leq \frac{1}{\rho}\mathfrak{R}_m(\widetilde{H})$ and $\mathfrak{R}_m(\widetilde{H})$ can be rewritten as follows:

$$\mathfrak{R}_m(\widetilde{H}) = \frac{1}{m}\operatorname*{E}_{S,\sigma}\Big[\sup_{h \in H}\sum_{i=1}^{m}\sigma_i y_i h(x_i)\Big] = \frac{1}{m}\operatorname*{E}_{S,\sigma}\Big[\sup_{h \in H}\sum_{i=1}^{m}\sigma_i h(x_i)\Big] = \mathfrak{R}_m\big(H\big)\,.$$

This proves (4.40). The second inequality, (4.41), can be derived in the same way by using the second inequality of theorem 3.1, (3.4), instead of (3.3). ■

The generalization bounds of theorem 4.4 shows the conflict between two terms: the larger the desired margin ρ, the smaller the middle term; however, the first

term, the empirical margin loss $\widehat{R}_\rho$, increases as a function of ρ. The bounds of this theorem can be generalized to hold uniformly for all $\rho > 0$ at the cost of an additional term $\sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}}$, as shown in the following theorem (a version of this theorem with better constants can be derived, see exercise 4.2).

Theorem 4.5
Let H be a set of real-valued functions. Then, for any $\delta > 0$, with probability at least $1 - \delta$, each of the following holds for all $h \in H$ and $\rho \in (0, 1)$:

$$R(h) \leq \widehat{R}_\rho(h) + \frac{4}{\rho}\mathfrak{R}_m(H) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}} \tag{4.42}$$

$$R(h) \leq \widehat{R}_\rho(h) + \frac{4}{\rho}\widehat{\mathfrak{R}}_S(H) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + 3\sqrt{\frac{\log \frac{4}{\delta}}{2m}}\,. \tag{4.43}$$

Proof Consider two sequences $(\rho_k)_{k\geq 1}$ and $(\epsilon_k)_{k\geq 1}$, with $\epsilon_k \in (0, 1)$. By theorem 4.4, for any fixed $k \geq 1$,

$$\Pr\left[R(h) - \widehat{R}_{\rho_k}(h) > \frac{2}{\rho_k}\mathfrak{R}_m(H) + \epsilon_k\right] \leq \exp(-2m\epsilon_k^2). \tag{4.44}$$

Choose $\epsilon_k = \epsilon + \sqrt{\frac{\log k}{m}}$, then, by the union bound,

$$\begin{aligned}
\Pr\left[\exists k\colon R(h) - \widehat{R}_{\rho_k}(h) > \frac{2}{\rho_k}\mathfrak{R}_m(H) + \epsilon_k\right] &\leq \sum_{k\geq 1} \exp(-2m\epsilon_k^2) \\
&= \sum_{k\geq 1} \exp\left[-2m(\epsilon + \sqrt{(\log k)/m})^2\right] \\
&\leq \sum_{k\geq 1} \exp(-2m\epsilon^2)\exp(-2\log k) \\
&= \Big(\sum_{k\geq 1} 1/k^2\Big)\exp(-2m\epsilon^2) \\
&= \frac{\pi^2}{6}\exp(-2m\epsilon^2) \leq 2\exp(-2m\epsilon^2).
\end{aligned}$$

We can choose $\rho_k = 1/2^k$. For any $\rho \in (0, 1)$, there exists $k \geq 1$ such that $\rho \in (\rho_k, \rho_{k-1}]$, with $\rho_0 = 1$. For that k, $\rho \leq \rho_{k-1} = 2\rho_k$, thus $1/\rho_k \leq 2/\rho$ and $\log k = \sqrt{\log\log_2(1/\rho_k)} \leq \sqrt{\log\log_2(2/\rho)}$. Furthermore, for any $h \in H$, $\widehat{R}_{\rho_k}(h) \leq \widehat{R}_\rho(h)$. Thus,

$$\Pr\left[\exists k\colon R(h) - \widehat{R}_\rho(h) > \frac{4}{\rho}\mathfrak{R}_m(H) + \sqrt{\frac{\log\log_2(2/\rho)}{m}} + \epsilon\right] \leq 2\exp(-2m\epsilon^2),$$

which proves the first statement. The second statement can be proven in a similar

way. ■

Combining theorem 4.3 and theorem 4.4 gives directly the following general margin bound for linear hypotheses with bounded weight vectors, presented in corollary 4.1.

Corollary 4.1
Let $H = \{\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} \colon \|\mathbf{w}\| \leq \Lambda\}$ *and assume that* $X \subseteq \{\mathbf{x} \colon \|\mathbf{x}\| \leq r\}$. *Fix* $\rho > 0$, *then, for any* $\delta > 0$, *with probability at least* $1 - \delta$, *for any* $h \in H$,

$$R(h) \leq \widehat{R}_\rho(h) + 2\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}\,. \tag{4.45}$$

As with theorem 4.4, the bound of this corollary can be generalized to hold uniformly for all $\rho > 0$ at the cost of an additional term $\sqrt{\frac{\log \log_2 \frac{2}{\rho}}{m}}$ by combining theorems 4.3 and 4.5. This generalization bound for linear hypotheses is remarkable, since it does not depend directly on the dimension of the feature space, but only on the margin. It suggests that a small generalization error can be achieved when ρ/r is large (small second term) while the empirical margin loss is relatively small (first term). The latter occurs when few points are either classified incorrectly or correctly, but with margin less than ρ.

The fact that the guarantee does not explicitly depend on the dimension of the feature space may seem surprising and appear to contradict the VC-dimension lower bounds of theorems 3.6 and 3.7. Those lower bounds show that for any learning algorithm $\mathcal{A}$ there exists a *bad* distribution for which the error of the hypothesis returned by the algorithm is $\Omega(\sqrt{d/m})$ with a non-zero probability. The bound of the corollary does not rule out such *bad* cases, however: for such bad distributions, the empirical margin loss would be large even for a relatively small margin ρ, and thus the bound of the corollary would be loose in that case.

Thus, in some sense, the learning guarantee of the corollary hinges upon the hope of a good margin value ρ: if there exists a relatively large margin value $\rho > 0$ for which the empirical margin loss is small, then a small generalization error is guaranteed by the corollary. This favorable margin situation depends on the distribution: while the learning bound is distribution-independent, the existence of a good margin is in fact distribution-dependent. A favorable margin seems to appear relatively often in applications.

The bound of the corollary gives a strong justification for margin-maximization algorithms such as SVMs. First, note that for $\rho = 1$, the margin loss can be upper bounded by the hinge loss:

$$\forall x \in \mathbb{R}, \Phi_1(x) \leq \max(1 - x, 0). \tag{4.46}$$

Using this fact, the bound of the corollary implies that with probability at least $1-\delta$, for all $h \in H = \{\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} \colon \|\mathbf{w}\| \leq \Lambda\}$,

$$R(h) \leq \frac{1}{m}\sum_{i=1}^{m} \xi_i + 2\sqrt{\frac{r^2\Lambda^2}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}\,, \tag{4.47}$$

where $\xi_i = \max(1 - y_i(\mathbf{w} \cdot \mathbf{x}_i), 0)$. The objective function minimized by the SVM algorithm has precisely the form of this upper bound: the first term corresponds to the slack penalty over the training set and the second to the minimization of the $\|\mathbf{w}\|$ which is equivalent to that of $\|\mathbf{w}\|^2$. Note that an alternative objective function would be based on the empirical margin loss instead of the hinge loss. However, the advantage of the hinge loss is that it is convex, while the margin loss is not.

As already pointed out, the bounds just discussed do not directly depend on the dimension of the feature space and guarantee good generalization with a favorable margin. Thus, they suggest seeking large-margin separating hyperplanes in a very high-dimensional space. In view of the form of the dual optimization problems for SVMs, determining the solution of the optimization and using it for prediction both require computing many inner products in that space. For very high-dimensional spaces, the computation of these inner products could become very costly. The next chapter provides a solution to this problem which further generalizes SVMs to non-linear separation.

4.5 Chapter notes

The maximum-margin or *optimal hyperplane* solution described in section 4.2 was introduced by Vapnik and Chervonenkis [1964]. The algorithm had limited applications, since in most tasks in practice the data is not linearly separable. In contrast, the SVM algorithm of section 4.3 for the general non-separable case, introduced by Cortes and Vapnik [1995] under the name *support-vector networks*, has been widely adopted and been shown to be effective in practice. The algorithm and its theory have had a profound impact on theoretical and applied machine learning and inspired research on a variety of topics. Several specialized algorithms have been suggested for solving the specific QP that arises when solving the SVM problem, for example the SMO algorithm of Platt [1999] (see exercise 4.4) and a variety of other decomposition methods such as those used in the LibLinear software library [Hsieh et al., 2008], and [Allauzen et al., 2010] for solving the problem when using rational kernels (see chapter 5).

Much of the theory supporting the SVM algorithm ([Cortes and Vapnik, 1995, Vapnik, 1998]), in particular the margin theory presented in section 4.4, has been adopted in the learning theory and statistics communities and applied to a variety

of other problems. The margin bound on the VC-dimension of canonical hyperplanes (theorem 4.2) is by Vapnik [1998], the proof is very similar to Novikoff's margin bound on the number of updates made by the Perceptron algorithm in the separable case. Our presentation of margin guarantees based on the Rademacher complexity follows the elegant analysis of Koltchinskii and Panchenko [2002] (see also Bartlett and Mendelson [2002], Shawe-Taylor et al. [1998]). Our proof of Talagrand's lemma 4.2 is a simpler and more concise version of a more general result given by Ledoux and Talagrand [1991, pp. 112–114]. See Höffgen et al. [1995] for hardness results related to the problem of finding a hyperplane with the minimal number of errors on a training sample.

4.6 Exercises

4.1 Soft margin hyperplanes. The function of the slack variables used in the optimization problem for soft margin hyperplanes has the form: $\xi \mapsto \sum_{i=1}^{m} \xi_i$. Instead, we could use $\xi \mapsto \sum_{i=1}^{m} \xi_i^p$, with $p > 1$.

(a) Give the dual formulation of the problem in this general case.

(b) How does this more general formulation ($p > 1$) compare to the standard setting ($p = 1$)? In the case $p = 2$ is the optimization still convex?

Sparse SVM. One can give two types of arguments in favor of the SVM algorithm: one based on the sparsity of the support vectors, another based on the notion of margin. Suppose that instead of maximizing the margin, we choose instead to maximize sparsity by minimizing the L_p norm of the vector $\boldsymbol{\alpha}$ that defines the weight vector $\mathbf{w}$, for some $p \geq 1$. First, consider the case $p = 2$. This gives the following optimization problem:

$$\begin{aligned} \min_{\boldsymbol{\alpha},b} \ & \frac{1}{2}\sum_{i=1}^{m} \alpha_i^2 + C\sum_{i=1}^{m} \xi_i \\ \text{subject to } & y_i\Big(\sum_{j=1}^{m} \alpha_j y_j \mathbf{x}_i \cdot \mathbf{x}_j + b\Big) \geq 1 - \xi_i, i \in [1,m] \\ & \xi_i, \alpha_i \geq 0, i \in [1,m]. \end{aligned} \tag{4.48}$$

(a) Show that modulo the non-negativity constraint on $\boldsymbol{\alpha}$, the problem coincides with an instance of the primal optimization problem of SVM.

(b) Derive the dual optimization of problem of (4.48).

(c) Setting $p = 1$ will induce a more sparse $\boldsymbol{\alpha}$. Derive the dual optimization in

this case.

4.2 Tighter Rademacher Bound. Derive the following tighter version of the bound of theorem 4.5: for any $\delta > 0$, with probability at least $1 - \delta$, for all $h \in H$ and $\rho \in (0, 1)$ the following holds:

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2\gamma}{\rho}\mathfrak{R}_m(H) + \sqrt{\frac{\log\log_\gamma \frac{\gamma}{\rho}}{m}} + \sqrt{\frac{\log\frac{2}{\delta}}{2m}} \tag{4.49}$$

for any $\gamma > 1$.

4.3 Importance weighted SVM. Suppose you wish to use SVMs to solve a learning problem where some training data points are more important than others. More formally, assume that each training point consists of a triplet (x_i, y_i, p_i), where $0 \leq p_i \leq 1$ is the importance of the ith point. Rewrite the primal SVM constrained optimization problem so that the penalty for mis-labeling a point x_i is scaled by the priority p_i. Then carry this modification through the derivation of the dual solution.

4.4 Sequential minimal optimization (SMO). The SMO algorithm is an optimization algorithm introduced to speed up the training of SVMs. SMO reduces a (potentially) large quadratic programming (QP) optimization problem into a series of small optimizations involving only two Lagrange multipliers. SMO reduces memory requirements, bypasses the need for numerical QP optimization and is easy to implement. In this question, we will derive the update rule for the SMO algorithm in the context of the dual formulation of the SVM problem.

(a) Assume that we want to optimize equation 4.32 only over α_1 and α_2. Show that the optimization problem reduces to

$$\max_{\alpha_1,\alpha_2} \underbrace{\alpha_1 + \alpha_2 - \frac{1}{2}K_{11}\alpha_1^2 - \frac{1}{2}K_{22}\alpha_2^2 - sK_{12}\alpha_1\alpha_2 - y_1\alpha_1 v_1 - y_2\alpha_2 v_2}_{\Psi_1(\alpha_1,\alpha_2)}$$
$$\text{subject to: } 0 \leq \alpha_1, \alpha_2 \leq C \wedge \alpha_1 + s\alpha_2 = \gamma\,,$$

where $\gamma = y_1 \sum_{i=3}^m y_i\alpha_i$, $s = y_1 y_2 \in \{-1, +1\}$, $K_{ij} = (\mathbf{x}_i \cdot \mathbf{x}_j)$ and $v_i = \sum_{j=3}^m \alpha_j y_j K_{ij}$ for $i = 1, 2$.

(b) Substitute the linear constraint $\alpha_1 = \gamma - s\alpha_2$ into Ψ_1 to obtain a new objective function Ψ_2 that depends only on α_2. Show that the α_2 that minimizes Ψ_2 (without the constraints $0 \leq \alpha_1, \alpha_2 \leq C$) can be expressed as

$$\alpha_2 = \frac{s(K_{11} - K_{12})\gamma + y_2(v_1 - v_2) - s + 1}{\eta}\,,$$

where $\eta = K_{11} + K_{22} - 2K_{12}$.

(c) Show that

$$v_1 - v_2 = f(\mathbf{x}_1) - f(\mathbf{x}_2) + \alpha_2^* y_2 \eta - s y_2 \gamma (K_{11} - K_{12})$$

where $f(\mathbf{x}) = \sum_{i=1}^m \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}) + b^*$ and α_i^* are values for the Lagrange multipliers prior to optimization over α_1 and α_2 (similarly, b^* is the previous value for the offset).

(d) Show that

$$\alpha_2 = \alpha_2^* + y_2 \frac{(y_2 - f(\mathbf{x}_2)) - (y_1 - f(\mathbf{x}_1))}{\eta} .$$

(e) For $s = +1$, define $L = \max\{0, \gamma - C\}$ and $H = \min\{C, \gamma\}$ as the lower and upper bounds on α_2. Similarly, for $s = -1$, define $L = \max\{0, -\gamma\}$ and $H = \min\{C, C - \gamma\}$. The update rule for SMO involves "clipping" the value of α_2, i.e.,

$$\alpha_2^{clip} = \begin{cases} \alpha_2 & \text{if } L < \alpha_2 < H \\ L & \text{if } \alpha_2 \leq L \\ H & \text{if } \alpha_2 \geq H \end{cases} .$$

We subsequently solve for α_1 such that we satisfy the equality constraint, resulting in $\alpha_1 = \alpha_1^* + s(\alpha_2^* - \alpha_2^{clip})$. Why is "clipping" is required? How are L and H derived for the case $s = +1$?

4.5 SVMs hands-on.

(a) Download and install the `libsvm` software library from:

`http://www.csie.ntu.edu.tw/~cjlin/libsvm/`.

(b) Download the `satimage` data set found at:

`http://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/`.

Merge the training and validation sets into one. We will refer to the resulting set as the training set from now on. Normalize both the training and test vectors.

(c) Consider the binary classification that consists of distinguishing class 6 from the rest of the data points. Use SVMs combined with polynomial kernels (see chapter 5) to solve this classification problem. To do so, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c, be equal to their default values 1). Report the best

value of the trade-off constant C measured on the validation set.

(d) Let (C^*, d^*) be the best pair found previously. Fix C to be C^*. Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d. Plot the average number of support vectors obtained as a function of d.

(e) How many of the support vectors lie on the margin hyperplanes?

(f) In the standard two-group classification, errors on positive or negative points are treated in the same manner. Suppose, however, that we wish to penalize an error on a negative point (false positive error) $k > 0$ times more than an error on a positive point. Give the dual optimization problem corresponding to SVMs modified in this way.

(g) Assume that k is an integer. Show how you can use `libsvm` without writing any additional code to find the solution of the modified SVMs just described.

(h) Apply the modified SVMs to the classification task previously examined and compare with your previous SVMs results for $k = 2, 4, 8, 16$.

5 Kernel Methods

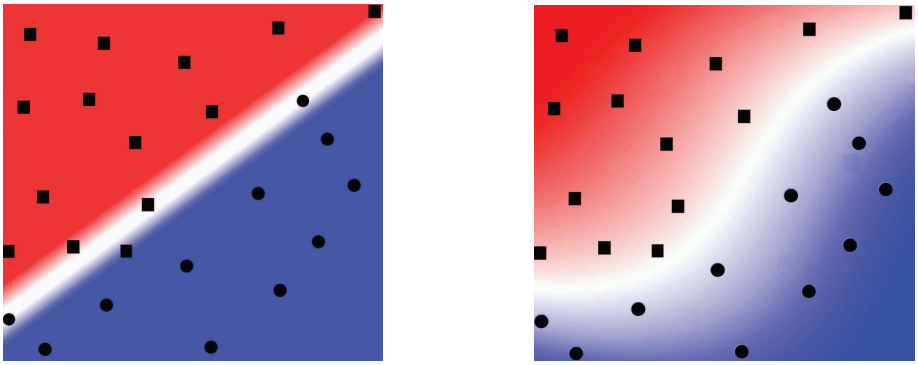
Kernel methods are widely used in machine learning. They are flexible techniques that can be used to extend algorithms such as SVMs to define non-linear decision boundaries. Other algorithms that only depend on inner products between sample points can be extended similarly, many of which will be studied in future chapters.

The main idea behind these methods is based on so-called *kernels* or *kernel functions*, which, under some technical conditions of symmetry and *positive-definiteness*, implicitly define an inner product in a high-dimensional space. Replacing the original inner product in the input space with positive definite kernels immediately extends algorithms such as SVMs to a linear separation in that high-dimensional space, or, equivalently, to a non-linear separation in the input space.

In this chapter, we present the main definitions and key properties of positive definite symmetric kernels, including the proof of the fact that they define an inner product in a Hilbert space, as well as their closure properties. We then extend the SVM algorithm using these kernels and present several theoretical results including general margin-based learning guarantees for hypothesis sets based on kernels. We also introduce *negative definite symmetric kernels* and point out their relevance to the construction of positive definite kernels, in particular from distances or metrics. Finally, we illustrate the design of kernels for non-vectorial discrete structures by introducing a general family of kernels for sequences, *rational kernels*. We describe an efficient algorithm for the computation of these kernels and illustrate them with several examples.

5.1 Introduction

In the previous chapter, we presented an algorithm for linear classification, SVMs, which is both effective in applications and benefits from a strong theoretical justification. In practice, linear separation is often not possible. Figure 5.1a shows an example where any hyperplane crosses both populations. However, one can use more complex functions to separate the two sets as in figure 5.1b. One way to define such a non-linear decision boundary is to use a non-linear mapping Φ from the input



(a) (b)

Figure 5.1 Non-linearly separable case. The classification task consists of discriminating between solid squares and solid circles. (a) No hyperplane can separate the two populations. (b) A non-linear mapping can be used instead.

space $\mathcal{X}$ to a higher-dimensional space $\mathbb{H}$, where linear separation is possible.

The dimension of $\mathbb{H}$ can truly be very large in practice. For example, in the case of document classification, one may wish to use as features sequences of three consecutive words, i.e., *trigrams*. Thus, with a vocabulary of just 100,000 words, the dimension of the feature space $\mathbb{H}$ reaches 10^{15}. On the positive side, the margin bounds presented in section 4.4 show that, remarkably, the generalization ability of large-margin classification algorithms such as SVMs do not depend on the dimension of the feature space, but only on the margin ρ and the number of training examples m. Thus, with a favorable margin ρ, such algorithms could succeed even in very high-dimensional space. However, determining the hyperplane solution requires multiple inner product computations in high-dimensional spaces, which can become be very costly.

A solution to this problem is to use *kernel methods*, which are based on *kernels* or *kernel functions*.

Definition 5.1 **Kernels**
A function $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is called a* kernel over $\mathcal{X}$.

The idea is to define a kernel K such that for any two points $x, x' \in \mathcal{X}$, $K(x, x')$ be

equal to an inner product of vectors $\Phi(x)$ and $\Phi(y)$:[1]

$$\forall x, x' \in \mathcal{X}, \quad K(x, x') = \langle \Phi(x), \Phi(x') \rangle , \tag{5.1}$$

for some mapping $\Phi \colon \mathcal{X} \to \mathbb{H}$ to a Hilbert space $\mathbb{H}$ called a *feature space*. Since an inner product is a measure of the similarity of two vectors, K is often interpreted as a similarity measure between elements of the input space $\mathcal{X}$.

An important advantage of such a kernel K is efficiency: K is often significantly more efficient to compute than Φ and an inner product in $\mathbb{H}$. We will see several common examples where the computation of $K(x, x')$ can be achieved in $O(N)$ while that of $\langle \Phi(x), \Phi(x') \rangle$ typically requires $O(\dim(\mathbb{H}))$ work, with $\dim(\mathbb{H}) \gg N$. Furthermore, in some cases, the dimension of $\mathbb{H}$ is infinite.

Perhaps an even more crucial benefit of such a kernel function K is flexibility: there is no need to explicitly define or compute a mapping Φ. The kernel K can be arbitrarily chosen so long as the existence of Φ is guaranteed, i.e. K satisfies *Mercer's condition* (see theorem 5.1).

Theorem 5.1 **Mercer's condition**
Let $\mathcal{X} \subset \mathbb{R}^N$ be a compact set and let $K \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a continuous and symmetric function. Then, K admits a uniformly convergent expansion of the form

$$K(x, x') = \sum_{n=0}^{\infty} a_n \phi_n(x) \phi_n(x'),$$

with $a_n > 0$ iff for any square integrable function c ($c \in L_2(\mathcal{X})$), the following condition holds:

$$\int \int_{\mathcal{X} \times \mathcal{X}} c(x) c(x') K(x, x') dx dx' \geq 0.$$

This condition is important to guarantee the convexity of the optimization problem for algorithms such as SVMs and thus convergence guarantees. A condition that is equivalent to Mercer's condition under the assumptions of the theorem is that the kernel K be *positive definite symmetric* (PDS). This property is in fact more general since in particular it does not require any assumption about $\mathcal{X}$. In the next section, we give the definition of this property and present several commonly used examples of PDS kernels, then show that PDS kernels induce an inner product in a Hilbert space, and prove several general closure properties for PDS kernels.

1. To differentiate that inner product from the one of the input space, we will typically denote it by $\langle \cdot, \cdot \rangle$.

5.2 Positive definite symmetric kernels

5.2.1 Definitions

Definition 5.2 **Positive definite symmetric kernels**
A kernel $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is said to be* positive definite symmetric *(PDS) if for any* $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$*, the matrix* $\mathbf{K} = [K(x_i, x_j)]_{ij} \in \mathbb{R}^{m \times m}$ *is symmetric positive semidefinite (SPSD).*

$\mathbf{K}$ is SPSD if it is symmetric and one of the following two equivalent conditions holds:

- the eigenvalues of $\mathbf{K}$ are non-negative;
- for any column vector $\mathbf{c} = (c_1, \ldots, c_m)^\top \in \mathbb{R}^{m \times 1}$,

$$\mathbf{c}^\top \mathbf{K} \mathbf{c} = \sum_{i,j=1}^{n} c_i c_j K(x_i, x_j) \geq 0. \tag{5.2}$$

For a sample $S = (x_1, \ldots, x_m)$, $\mathbf{K} = [K(x_i, x_j)]_{ij} \in \mathbb{R}^{m \times m}$ is called the *kernel matrix* or the *Gram matrix* associated to K and the sample S.

Let us insist on the terminology: the kernel matrix associated to a *positive definite kernel* is *positive semidefinite* . This is the correct mathematical terminology. Nevertheless, the reader should be aware that in the context of machine learning, some authors have chosen to use instead the term *positive definite kernel* to imply a *positive definite* kernel matrix or used new terms such as *positive semidefinite kernel*.

The following are some standard examples of PDS kernels commonly used in applications.

Example 5.1 Polynomial kernels
For any constant $c > 0$, a *polynomial kernel of degree* $d \in \mathbb{N}$ is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d. \tag{5.3}$$

Polynomial kernels map the input space to a higher-dimensional space of dimension $\binom{N+d}{d}$ (see exercise 5.9). As an example, for an input space of dimension $N = 2$, a second-degree polynomial ($d = 2$) corresponds to the following inner product in

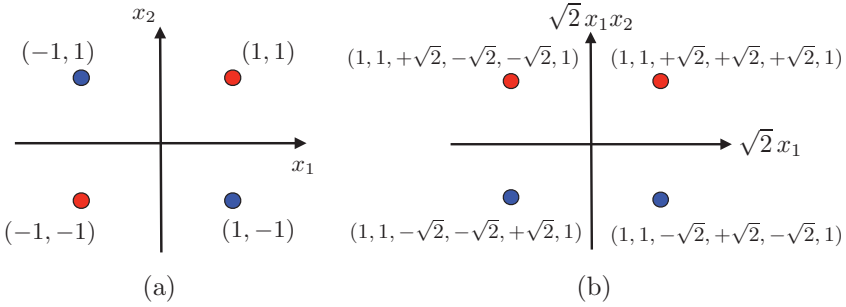


Figure 5.2 Illustration of the XOR classification problem and the use of polynomial kernels. (a) XOR problem linearly non-separable in the input space. (b) Linearly separable using second-degree polynomial kernel.

dimension 6:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^2, \quad K(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + c)^2 = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \\ \sqrt{2c}\, x_1 \\ \sqrt{2c}\, x_2 \\ c \end{bmatrix} \cdot \begin{bmatrix} {x'}_1^2 \\ {x'}_2^2 \\ \sqrt{2}\, x_1' x_2' \\ \sqrt{2c}\, x_1' \\ \sqrt{2c}\, x_2' \\ c \end{bmatrix}. \tag{5.4}$$

Thus, the features corresponding to a second-degree polynomial are the original features (x_1 and x_2), as well as products of these features, and the constant feature. More generally, the features associated to a polynomial kernel of degree d are all the monomials of degree at most d based on the original features. The explicit expression of polynomial kernels as inner products, as in (5.4), proves directly that they are PDS kernels.

To illustrate the application of polynomial kernels, consider the example of figure 5.2a which shows a simple data set in dimension two that is not linearly separable. This is known as the XOR problem due to its interpretation in terms of the exclusive OR (XOR) function: the label of a point is blue iff exactly one of its coordinates is 1. However, if we map these points to the six-dimensional space defined by a second-degree polynomial as described in (5.4), then the problem becomes separable by the hyperplane of equation $x_1 x_2 = 0$. Figure 5.2b illustrates that by showing the projection of these points on the two-dimensional space defined by their third and fourth coordinates.

Example 5.2 Gaussian kernels

For any constant $\sigma > 0$, a *Gaussian kernel* or *radial basis function (RBF)* is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x}' - \mathbf{x}\|^2}{2\sigma^2}\right). \tag{5.5}$$

Gaussians kernels are among the most frequently used kernels in applications. We will prove in section 5.2.3 that they are PDS kernels and that they can be derived by *normalization* from the kernels $K' \colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x} \cdot \mathbf{x}'}{\sigma^2}\right)$. Using the power series expansion of the function exponential, we can rewrite the expression of K' as follows:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K'(\mathbf{x}, \mathbf{x}') = \sum_{n=0}^{+\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^n}{\sigma^n \, n!},$$

which shows that the kernels K', and thus Gaussian kernels, are positive linear combinations of polynomial kernels of all degrees $n \geq 0$.

Example 5.3 Sigmoid kernels
For any real constants $a, b \geq 0$, a *sigmoid kernel* is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = \tanh\big(a(\mathbf{x} \cdot \mathbf{x}') + b\big). \tag{5.6}$$

Using sigmoid kernels with SVMs leads to an algorithm that is closely related to learning algorithms based on simple neural networks, which are also often defined via a sigmoid function. When $a < 0$ or $b < 0$, the kernel is not PDS and the corresponding neural network does not benefit from the convergence guarantees of convex optimization (see exercise 5.15).

5.2.2 Reproducing kernel Hilbert space

Here, we prove the crucial property of PDS kernels, which is to induce an inner product in a Hilbert space. The proof will make use of the following lemma.

Lemma 5.1 **Cauchy-Schwarz inequality for PDS kernels**
Let K be a PDS kernel. Then, for any $x, x' \in \mathcal{X}$,

$$K(x, x')^2 \leq K(x, x) K(x', x'). \tag{5.7}$$

Proof Consider the matrix $\mathbf{K} = \begin{pmatrix} K(x,x) & K(x,x') \\ K(x',x) & K(x',x') \end{pmatrix}$. By definition, if K is PDS, then $\mathbf{K}$ is SPSD for all $x, x' \in \mathcal{X}$. In particular, the product of the eigenvalues of $\mathbf{K}$, $\det(\mathbf{K})$, must be non-negative, thus, using $K(x', x) = K(x, x')$, we have

$$\det(\mathbf{K}) = K(x, x) K(x', x') - K(x, x')^2 \geq 0,$$

which concludes the proof. ■

The following is the main result of this section.

Theorem 5.2 **Reproducing kernel Hilbert space (RKHS)**
Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel. Then, there exists a Hilbert space $\mathbb{H}$ and a mapping Φ from $\mathcal{X}$ to $\mathbb{H}$ such that:

$$\forall x, x' \in \mathcal{X}, \quad K(x, x') = \langle \Phi(x), \Phi(x') \rangle . \tag{5.8}$$

Furthermore, $\mathbb{H}$ has the following property known as the reproducing property*:*

$$\forall h \in \mathbb{H}, \forall x \in \mathcal{X}, \quad h(x) = \langle h, K(x, \cdot) \rangle . \tag{5.9}$$

$\mathbb{H}$ is called a reproducing kernel Hilbert space *(RKHS) associated to K.*

Proof For any $x \in \mathcal{X}$, define $\Phi(x)\colon \mathcal{X} \to \mathbb{R}$ as follows:

$$\forall x' \in \mathcal{X},\ \Phi(x)(x') = K(x, x').$$

We define $\mathbb{H}_0$ as the set of finite linear combinations of such functions $\Phi(x)$:

$$\mathbb{H}_0 = \left\{ \sum_{i \in I} a_i \Phi(x_i)\colon a_i \in \mathbb{R}, x_i \in \mathcal{X}, \mathrm{card}(I) < \infty \right\}.$$

Now, we introduce an operation $\langle \cdot, \cdot \rangle$ on $\mathbb{H}_0 \times \mathbb{H}_0$ defined for all $f, g \in \mathbb{H}_0$ with $f = \sum_{i \in I} a_i \Phi(x_i)$ and $g = \sum_{j \in J} b_j \Phi(x_j)$ by

$$\langle f, g \rangle = \sum_{i \in I, j \in J} a_i b_j K(x_i, x'_j) = \sum_{j \in J} b_j f(x'_j) = \sum_{i \in I} a_i g(x_i).$$

By definition, $\langle \cdot, \cdot \rangle$ is symmetric. The last two equations show that $\langle f, g \rangle$ does not depend on the particular representations of f and g, and also show that $\langle \cdot, \cdot \rangle$ is bilinear. Further, for any $f = \sum_{i \in I} a_i \Phi(x_i) \in \mathbb{H}_0$, since K is PDS, we have

$$\langle f, f \rangle = \sum_{i,j \in I} a_i a_j K(x_i, x_j) \geq 0.$$

Thus, $\langle \cdot, \cdot \rangle$ is positive semidefinite bilinear form. This inequality implies more generally using the bilinearity of $\langle \cdot, \cdot \rangle$ that for any $f_1, \ldots, f_m$ and $c_1, \ldots, c_m \in \mathbb{R}$,

$$\sum_{i,j=1}^{m} c_i c_j \langle f_i, f_j \rangle = \left\langle \sum_{i=1}^{m} c_i f_i, \sum_{j=1}^{m} c_j f_j \right\rangle \geq 0.$$

Hence, $\langle \cdot, \cdot \rangle$ is a PDS kernel on $\mathbb{H}_0$. Thus, for any $f \in \mathbb{H}_0$ and any $x \in \mathcal{X}$, by

lemma 5.1, we can write

$$\langle f, \Phi(x)\rangle^2 \leq \langle f, f\rangle \langle \Phi(x), \Phi(x)\rangle.$$

Further, we observe the reproducing property of $\langle \cdot, \cdot \rangle$: for any $f = \sum_{i \in I} a_i \Phi(x_i) \in \mathbb{H}_0$, by definition of $\langle \cdot, \cdot \rangle$,

$$\forall x \in \mathcal{X}, \quad f(x) = \sum_{i \in I} a_i K(x_i, x) = \langle f, \Phi(x)\rangle \,. \tag{5.10}$$

Thus, $[f(x)]^2 \leq \langle f, f\rangle K(x, x)$ for all $x \in \mathcal{X}$, which shows the definiteness of $\langle \cdot, \cdot \rangle$. This implies that $\langle \cdot, \cdot \rangle$ defines an inner product on $\mathbb{H}_0$, which thereby becomes a pre-Hilbert space. $\mathbb{H}_0$ can be completed to form a Hilbert space $\mathbb{H}$ in which it is dense, following a standard construction. By the Cauchy-Schwarz inequality , for any $x \in \mathcal{X}$, $f \mapsto \langle f, \Phi(x)\rangle$ is Lipschitz, therefore continuous. Thus, since $\mathbb{H}_0$ is dense in $\mathbb{H}$, the reproducing property (5.10) also holds over $\mathbb{H}$. ■

The Hilbert space $\mathbb{H}$ defined in the proof of the theorem for a PDS kernel K is called *the reproducing kernel Hilbert space (RKHS) associated to* K. Any Hilbert space $\mathbb{H}$ such that there exists $\Phi \colon \mathcal{X} \to \mathbb{H}$ with $K(x, x') = \langle \Phi(x), \Phi(x')\rangle$ for all $x, x' \in \mathcal{X}$ is called a *feature space* associated to K and Φ is called a *feature mapping*. We will denote by $\|\cdot\|_{\mathbb{H}}$ the norm induced by the inner product in feature space $\mathbb{H}$: $\|\mathbf{w}\|_{\mathbb{H}} = \sqrt{\langle \mathbf{w}, \mathbf{w}\rangle}$ for all $\mathbf{w} \in \mathbb{H}$. Note that the feature spaces associated to K are in general not unique and may have different dimensions. In practice, when referring to the *dimension of the feature space* associated to K, we either refer to the dimension of the feature space based on a feature mapping described explicitly, or to that of the RKHS associated to K.

Theorem 5.2 implies that PDS kernels can be used to implicitly define a feature space or feature vectors. As already underlined in previous chapters, the role played by the features in the success of learning algorithms is crucial: with poor features, uncorrelated with the target labels, learning could become very challenging or even impossible; in contrast, good features could provide invaluable clues to the algorithm. Therefore, in the context of learning with PDS kernels and for a fixed input space, the problem of seeking useful features is replaced by that of finding useful PDS kernels. While features represented the user's prior knowledge about the task in the standard learning problems, here PDS kernels will play this role. Thus, in practice, an appropriate choice of PDS kernel for a task will be crucial.

5.2.3 Properties

This section highlights several important properties of PDS kernels. We first show that PDS kernels can be *normalized* and that the resulting normalized kernels are also PDS. We also introduce the definition of *empirical kernel maps* and describe

their properties and extension. We then prove several important closure properties of PDS kernels, which can be used to construct complex PDS kernels from simpler ones.

To any kernel K, we can associate a *normalized kernel* K' defined by

$$\forall x, x' \in \mathcal{X}, \quad K'(x,x') = \begin{cases} 0 & \text{if } (K(x,x)=0) \wedge (K(x',x')=0) \\ \frac{K(x,x')}{\sqrt{K(x,x)K(x',x')}} & \text{otherwise.} \end{cases} \tag{5.11}$$

By definition, for a normalized kernel K', $K'(x,x) = 1$ for all $x \in \mathcal{X}$ such that $K(x,x) \neq 0$. An example of normalized kernel is the Gaussian kernel with parameter $\sigma > 0$, which is the normalized kernel associated to $K' \colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}\right)$:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \frac{K'(\mathbf{x},\mathbf{x}')}{\sqrt{K'(\mathbf{x},\mathbf{x})K'(\mathbf{x}',\mathbf{x}')}} = \frac{e^{\frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}}}{e^{\frac{\|\mathbf{x}\|^2}{2\sigma^2}} e^{\frac{\|\mathbf{x}'\|^2}{2\sigma^2}}} = \exp\left(-\frac{\|\mathbf{x}'-\mathbf{x}'\|^2}{2\sigma^2}\right). \tag{5.12}$$

Lemma 5.2 **Normalized PDS kernels**
Let K be a PDS kernel. Then, the normalized kernel K' associated to K is PDS.

Proof Let $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and let $\mathbf{c}$ be an arbitrary vector in $\mathbb{R}^m$. We will show that the sum $\sum_{i,j=1}^m c_i c_j K'(x_i, x_j)$ is non-negative. By lemma 5.1, if $K(x_i, x_i) = 0$ then $K(x_i, x_j) = 0$ and thus $K'(x_i, x_j) = 0$ for all $j \in [1, m]$. Thus, we can assume that $K(x_i, x_i) > 0$ for all $i \in [1, m]$. Then, the sum can be rewritten as follows:

$$\sum_{i,j=1}^m \frac{c_i c_j K(x_i,x_j)}{\sqrt{K(x_i,x_i)K(x_j,x_j)}} = \sum_{i,j=1}^m \frac{c_i c_j \langle \Phi(x_i), \Phi(x_j)\rangle}{\|\Phi(x_i)\|_{\mathbb{H}} \|\Phi(x_j)\|_{\mathbb{H}}} = \left\| \sum_{i=1}^m \frac{c_i \Phi(x_i)}{\|\Phi(x_i)\|_{\mathbb{H}}} \right\|_{\mathbb{H}}^2 \geq 0,$$

where Φ is a feature mapping associated to K, which exists by theorem 5.2. ■

As indicated earlier, PDS kernels can be interpreted as a similarity measure since they induce an inner product in some Hilbert space $\mathbb{H}$. This is more evident for a normalized kernel K since $K(x,x')$ is then exactly the cosine of the angle between the feature vectors $\Phi(x)$ and $\Phi(x')$, provided that none of them is zero: $\Phi(x)$ and $\Phi(x')$ are then unit vectors since $\|\Phi(x)\|_{\mathbb{H}} = \|\Phi(x')\|_{\mathbb{H}} = \sqrt{K(x,x)} = 1$.

While one of the advantages of PDS kernels is an implicit definition of a feature mapping, in some instances, it may be desirable to define an explicit feature mapping based on a PDS kernel. This may be to work in the primal for various optimization and computational reasons, to derive an approximation based on an explicit mapping, or as part of a theoretical analysis where an explicit mapping is more convenient. The *empirical kernel map* Φ associated to a PDS kernel K is a feature mapping that can be used precisely in such contexts. Given a training

sample containing points $x_1, \ldots, x_m \in \mathcal{X}$, $\Phi\colon \mathcal{X} \to \mathbb{R}^m$ is defined for all $x \in \mathcal{X}$ by

$$\Phi(x) = \begin{bmatrix} K(x, x_1) \\ \vdots \\ K(x, x_m) \end{bmatrix}.$$

Thus, $\Phi(x)$ is the vector of the K-similarity measures of x with each of the training points. Let $\mathbf{K}$ be the kernel matrix associated to K and $\mathbf{e}_i$ the ith unit vector. Note that for any $i \in [1, m]$, $\Phi(x_i)$ is the ith column of $\mathbf{K}$, that is $\Phi(x_i) = \mathbf{K}\mathbf{e}_i$. In particular, for all $i, j \in [1, m]$,

$$\langle \Phi(x_i), \Phi(x_j) \rangle = (\mathbf{K}\mathbf{e}_i)^\top (\mathbf{K}\mathbf{e}_j) = \mathbf{e}_i^\top \mathbf{K}^2 \mathbf{e}_j.$$

Thus, the kernel matrix $\mathbf{K}'$ associated to Φ is $\mathbf{K}^2$. It may desirable in some cases to define a feature mapping whose kernel matrix coincides with $\mathbf{K}$. Let $\mathbf{K}^{\dagger\frac{1}{2}}$ denote the SPSD matrix whose square is $\mathbf{K}^\dagger$, the pseudo-inverse of $\mathbf{K}$. $\mathbf{K}^{\dagger\frac{1}{2}}$ can be derived from $\mathbf{K}^\dagger$ via singular value decomposition and if the matrix $\mathbf{K}$ is invertible, $\mathbf{K}^{\dagger\frac{1}{2}}$ coincides with $\mathbf{K}^{-1/2}$ (see appendix A for properties of the pseudo-inverse). Then, Ψ can be defined as follows using the empirical kernel map Φ:

$$\forall x \in \mathcal{X}, \quad \Psi(x) = \mathbf{K}^{\dagger\frac{1}{2}} \Phi(x).$$

Using the identity $\mathbf{K}\mathbf{K}^\dagger\mathbf{K} = \mathbf{K}$ valid for any symmetric matrix $\mathbf{K}$, for all $i, j \in [1, m]$, the following holds:

$$\langle \Psi(x_i), \Psi(x_j) \rangle = (\mathbf{K}^{\dagger\frac{1}{2}} \mathbf{K}\mathbf{e}_i)^\top (\mathbf{K}^{\dagger\frac{1}{2}} \mathbf{K}\mathbf{e}_j) = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger\mathbf{K}\mathbf{e}_j = \mathbf{e}_i^\top \mathbf{K}\mathbf{e}_j.$$

Thus, the kernel matrix associated to Ψ is $\mathbf{K}$. Finally, note that for the feature mapping $\Omega\colon \mathcal{X} \to \mathbb{R}^m$ defined by

$$\forall x \in \mathcal{X}, \quad \Omega(x) = \mathbf{K}^\dagger \Phi(x),$$

for all $i, j \in [1, m]$, we have $\langle \Omega(x_i), \Omega(x_j) \rangle = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger\mathbf{K}^\dagger\mathbf{K}\mathbf{e}_j = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger\mathbf{e}_j$, using the identity $\mathbf{K}^\dagger\mathbf{K}^\dagger\mathbf{K} = \mathbf{K}^\dagger$ valid for any symmetric matrix $\mathbf{K}$. Thus, the kernel matrix associated to Ω is $\mathbf{K}\mathbf{K}^\dagger$, which reduces to the identity matrix $\mathbf{I} \in \mathbb{R}^{m \times m}$ when $\mathbf{K}$ is invertible, since $\mathbf{K}^\dagger = \mathbf{K}^{-1}$ in that case.

As pointed out in the previous section, kernels represent the user's prior knowledge about a task. In some cases, a user may come up with appropriate similarity measures or PDS kernels for some subtasks — for example, for different subcategories of proteins or text documents to classify. But how can he combine these PDS kernels to form a PDS kernel for the entire class? Is the resulting combined kernel guaranteed to be PDS? In the following, we will show that PDS kernels are closed under several useful operations which can be used to design complex PDS

kernels. These operations are the sum and the product of kernels, as well as the *tensor product* of two kernels K and K', denoted by $K \otimes K'$ and defined by

$$\forall x_1, x_2, x_1', x_2' \in \mathcal{X}, \quad (K \otimes K')(x_1, x_1', x_2, x_2') = K(x_1, x_2)K'(x_1', x_2').$$

They also include the pointwise limit: given a sequence of kernels $(K_n)_{n\in\mathbb{N}}$ such that for all $x, x' \in \mathcal{X}$ $(K_n(x, x'))_{n\in\mathbb{N}}$ admits a limit, the pointwise limit of $(K_n)_{n\in\mathbb{N}}$ is the kernel K defined for all $x, x' \in \mathcal{X}$ by $K(x, x') = \lim_{n\to+\infty}(K_n)(x, x')$. Similarly, if $\sum_{n=0}^{\infty} a_n x^n$ is a power series with radius of convergence $\rho > 0$ and K a kernel taking values in $(-\rho, +\rho)$, then $\sum_{n=0}^{\infty} a_n K^n$ is the kernel obtained by composition of K with that power series. The following theorem provides closure guarantees for all of these operations.

Theorem 5.3 **PDS kernels — closure properties**
PDS kernels are closed under sum, product, tensor product, pointwise limit, and composition with a power series $\sum_{n=0}^{\infty} a_n x^n$ *with* $a_n \geq 0$ *for all* $n \in \mathbb{N}$.

Proof We start with two kernel matrices, $\mathbf{K}$ and $\mathbf{K}'$, generated from PDS kernels K and K' for an arbitrary set of m points. By assumption, these kernel matrices are SPSD. Observe that for any $\mathbf{c} \in \mathbb{R}^{m\times 1}$,

$$(\mathbf{c}^\top \mathbf{K}\mathbf{c} \geq 0) \wedge (\mathbf{c}^\top \mathbf{K}'\mathbf{c} \geq 0) \Rightarrow \mathbf{c}^\top(\mathbf{K} + \mathbf{K}')\mathbf{c} \geq 0.$$

By (5.2), this shows that $\mathbf{K} + \mathbf{K}'$ is SPSD and thus that $K + K'$ is PDS. To show closure under product, we will use the fact that for any SPSD matrix $\mathbf{K}$ there exists $\mathbf{M}$ such that $\mathbf{K} = \mathbf{M}\mathbf{M}^\top$. The existence of $\mathbf{M}$ is guaranteed as it can be generated via, for instance, singular value decomposition of $\mathbf{K}$, or by Cholesky decomposition. The kernel matrix associated to KK' is $(\mathbf{K}_{ij}\mathbf{K}'_{ij})_{ij}$. For any $\mathbf{c} \in \mathbb{R}^{m\times 1}$, expressing $\mathbf{K}_{ij}$ in terms of the entries of $\mathbf{M}$, we can write

$$\begin{aligned}
\sum_{i,j=1}^{m} c_i c_j (\mathbf{K}_{ij}\mathbf{K}'_{ij}) &= \sum_{i,j=1}^{m} c_i c_j \Bigg(\Big[\sum_{k=1}^{m} \mathbf{M}_{ik}\mathbf{M}_{jk} \Big] \mathbf{K}'_{ij} \Bigg) \\
&= \sum_{k=1}^{m} \Bigg[\sum_{i,j=1}^{m} c_i c_j \mathbf{M}_{ik}\mathbf{M}_{jk}\mathbf{K}'_{ij} \Bigg] \\
&= \sum_{k=1}^{m} \mathbf{z}_k^\top \mathbf{K}' \mathbf{z}_k \geq 0,
\end{aligned}$$

with $\mathbf{z}_k = \begin{bmatrix} c_1\mathbf{M}_{1k} \\ \vdots \\ c_m\mathbf{M}_{mk} \end{bmatrix}$. This shows that PDS kernels are closed under product. The tensor product of K and K' is PDS as the product of the two PDS kernels $(x_1, x_1', x_2, x_2') \mapsto K(x_1, x_2)$ and $(x_1, x_1', x_2, x_2') \mapsto K'(y_1, y_2)$. Next, let $(K_n)_{n\in\mathbb{N}}$ be a sequence of PDS kernels with pointwise limit K. Let $\mathbf{K}$ be the kernel matrix

associated to K and $\mathbf{K}_n$ the one associated to K_n for any $n \in \mathbb{N}$. Observe that

$$(\forall n, \mathbf{c}^\top \mathbf{K}_n \mathbf{c} \geq 0) \Rightarrow \lim_{n \to \infty} \mathbf{c}^\top \mathbf{K}_n \mathbf{c} = \mathbf{c}^\top \mathbf{K} \mathbf{c} \geq 0.$$

This shows the closure under pointwise limit. Finally, assume that K is a PDS kernel with $|K(x, x')| < \rho$ for all $x, x' \in \mathcal{X}$ and let $f\colon x \mapsto \sum_{n=0}^{\infty} a_n x^n, a_n \geq 0$ be a power series with radius of convergence ρ. Then, for any $n \in \mathbb{N}$, K^n and thus $a_n K_n$ are PDS by closure under product. For any $N \in \mathbb{N}$, $\sum_{n=0}^{N} a_n K^n$ is PDS by closure under sum of $a_n K_n$s and $f \circ K$ is PDS by closure under the limit of $\sum_{n=0}^{N} a_n K^n$ as N tends to infinity. ■

The theorem implies in particular that for any PDS kernel matrix K, $\exp(K)$ is PDS, since the radius of convergence of exp is infinite. In particular, the kernel $K'\colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x} \cdot \mathbf{x}'}{\sigma^2}\right)$ is PDS since $(\mathbf{x}, \mathbf{x}') \mapsto \frac{\mathbf{x} \cdot \mathbf{x}'}{\sigma^2}$ is PDS. Thus, by lemma 5.2, this shows that a Gaussian kernel, which is the normalized kernel associated to K', is PDS.

5.3 Kernel-based algorithms

In this section we discuss how SVMs can be used with kernels and analyze the impact that kernels have on generalization.

5.3.1 SVMs with PDS kernels

In chapter 4, we noted that the dual optimization problem for SVMs as well as the form of the solution did not directly depend on the input vectors but only on inner products. Since a PDS kernel implicitly defines an inner product (theorem 5.2), we can extend SVMs and combine it with an arbitrary PDS kernel K by replacing each instance of an inner product $x \cdot x'$ with $K(x, x')$. This leads to the following general form of the SVM optimization problem and solution with PDS kernels extending (4.32):

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \tag{5.13}$$

$$\text{subject to: } 0 \leq \alpha_i \leq C \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [1, m].$$

In view of (4.33), the hypothesis h solution can be written as:

$$h(x) = \operatorname{sgn}\Big(\sum_{i=1}^{m} \alpha_i y_i K(x_i, x) + b\Big), \tag{5.14}$$

with $b = y_i - \sum_{j=1}^{m} \alpha_j y_j K(x_j, x_i)$ for any x_i with $0 < \alpha_i < C$. We can rewrite the optimization problem (5.13) in a vector form, by using the kernel matrix $\mathbf{K}$ associated to K for the training sample $(x_1, \ldots, x_m)$ as follows:

$$\max_{\boldsymbol{\alpha}} 2\, \mathbf{1}^\top \boldsymbol{\alpha} - (\boldsymbol{\alpha} \circ \mathbf{y})^\top \mathbf{K} (\boldsymbol{\alpha} \circ \mathbf{y}) \quad \text{subject to: } \mathbf{0} \leq \boldsymbol{\alpha} \leq \mathbf{C} \wedge \boldsymbol{\alpha}^\top \mathbf{y} = 0. \tag{5.15}$$

In this formulation, $\boldsymbol{\alpha} \circ \mathbf{y}$ is the Hadamard product or entry-wise product of the vectors $\boldsymbol{\alpha}$ and $\mathbf{y}$. Thus, it is the column vector in $\mathbb{R}^{m \times 1}$ whose ith component equals $\alpha_i y_i$. The solution in vector form is the same as in (5.14), but with $b = y_i - (\boldsymbol{\alpha} \circ \mathbf{y})^\top \mathbf{K} \mathbf{e}_i$ for any x_i with $0 < \alpha_i < C$.

This version of SVMs used with PDS kernels is the general form of SVMs we will consider in all that follows. The extension is important, since it enables an implicit non-linear mapping of the input points to a high-dimensional space where large-margin separation is sought.

Many other algorithms in areas including regression, ranking, dimensionality reduction or clustering can be extended using PDS kernels following the same scheme (see in particular chapters 8, 9, 10, 12).

5.3.2 Representer theorem

Observe that modulo the offset b, the hypothesis solution of SVMs can be written as a linear combination of the functions $K(x_i, \cdot)$, where x_i is a sample point. The following theorem known as the *representer theorem* shows that this is in fact a general property that holds for a broad class of optimization problems, including that of SVMs with no offset.

Theorem 5.4 **Representer theorem**
Let $K \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and $\mathbb{H}$ its corresponding RKHS. Then, for any non-decreasing function $G \colon \mathbb{R} \to \mathbb{R}$ and any loss function $L \colon \mathbb{R}^m \to \mathbb{R} \cup \{+\infty\}$, the optimization problem

$$\operatorname*{argmin}_{h \in \mathbb{H}} F(h) = \operatorname*{argmin}_{h \in \mathbb{H}} G(\|h\|_{\mathbb{H}}) + L\big(h(x_1), \ldots, h(x_m)\big)$$

admits a solution of the form $h^* = \sum_{i=1}^{m} \alpha_i K(x_i, \cdot)$. If G is further assumed to be increasing, then any solution has this form.

Proof Let $\mathbb{H}_1 = \operatorname{span}(\{K(x_i, \cdot) \colon i \in [1, m]\})$. Any $h \in \mathbb{H}$ admits the decomposition $h = h_1 + h^\perp$ according to $\mathbb{H} = \mathbb{H}_1 \oplus \mathbb{H}_1^\perp$, where $\oplus$ is the direct sum. Since G is non-decreasing, $G(\|h_1\|_{\mathbb{H}}) \leq G(\sqrt{\|h_1\|_{\mathbb{H}}^2 + \|h^\perp\|_{\mathbb{H}}^2}) = G(\|h\|_{\mathbb{H}})$. By the reproducing property, for all $i \in [1, m]$, $h(x_i) = \langle h, K(x_i, \cdot) \rangle = \langle h_1, K(x_i, \cdot) \rangle = h_1(x_i)$. Thus, $L\big(h(x_1), \ldots, h(x_m)\big) = L\big(h_1(x_1), \ldots, h_1(x_m)\big)$ and $F(h_1) \leq F(h)$. This proves the

first part of the theorem. If G is further increasing, then $F(h_1) < F(h)$ when $\|h^\perp\|_{\mathbb{H}} > 0$ and any solution of the optimization problem must be in $\mathbb{H}_1$. ■

5.3.3 Learning guarantees

Here, we present general learning guarantees for hypothesis sets based on PDS kernels, which hold in particular for SVMs combined with PDS kernels.

The following theorem gives a general bound on the empirical Rademacher complexity of kernel-based hypotheses with bounded norm, that is a hypothesis set of the form $H = \{h \in \mathbb{H} : \|h\|_{\mathbb{H}} \leq \Lambda\}$, for some $\Lambda \geq 0$, where $\mathbb{H}$ is the RKHS associated to a kernel K. By the reproducing property, any $h \in H$ is of the form $x \mapsto \langle h, K(x, \cdot)\rangle = \langle h, \Phi(x)\rangle$ with $\|h\|_{\mathbb{H}} \leq \Lambda$, where Φ is a feature mapping associated to K, that is of the form $x \mapsto \langle \mathbf{w}, \Phi(x)\rangle$ with $\|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda$.

Theorem 5.5 **Rademacher complexity of kernel-based hypotheses**
Let $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and let $\Phi : \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K. Let $S \subseteq \{x : K(x, x) \leq r^2\}$ be a sample of size m, and let $H = \{\mathbf{x} \mapsto \mathbf{w} \cdot \Phi(x) : \|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda\}$ for some $\Lambda \geq 0$. Then

$$\widehat{\mathfrak{R}}_S(H) \leq \frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m} \leq \sqrt{\frac{r^2\Lambda^2}{m}}. \tag{5.16}$$

Proof The proof steps are as follows:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(H) &= \frac{1}{m}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sup_{\|\mathbf{w}\|\leq\Lambda}\left\langle \mathbf{w}, \sum_{i=1}^m \sigma_i \Phi(x_i)\right\rangle\right] \\
&= \frac{\Lambda}{m}\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\left\|\sum_{i=1}^m \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}\right] && \text{(Cauchy-Schwarz , eq. case)} \\
&\leq \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\left\|\sum_{i=1}^m \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}^2\right]\right]^{1/2} && \text{(Jensen's ineq.)} \\
&= \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sum_{i=1}^m \|\Phi(x_i)\|_{\mathbb{H}}^2\right]\right]^{1/2} && (i \neq j \Rightarrow \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j] = 0) \\
&= \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{\boldsymbol{\sigma}}\left[\sum_{i=1}^m K(x_i, x_i)\right]\right]^{1/2} \\
&= \frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m} \leq \sqrt{\frac{r^2\Lambda^2}{m}}.
\end{aligned}$$

The initial equality holds by definition of the empirical Rademacher complexity (definition 3.2). The first inequality is due to the Cauchy-Schwarz inequality and $\|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda$. The following inequality results from Jensen's inequality (theorem B.4) applied to the concave function $\sqrt{\cdot}$. The subsequent equality is a consequence of

$\mathrm{E}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j] = \mathrm{E}_{\boldsymbol{\sigma}}[\sigma_i]\,\mathrm{E}_{\boldsymbol{\sigma}}[\sigma_j] = 0$ for $i \neq j$, since the Rademacher variables σ_i and σ_j are independent. The statement of the theorem then follows by noting that $\mathrm{Tr}[\mathbf{K}] \leq mr^2$. ■

The theorem indicates that the trace of the kernel matrix is an important quantity for controlling the complexity of hypothesis sets based on kernels. Observe that by the Khintchine-Kahane inequality (D.22), the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(H) = \frac{\Lambda}{m}\,\mathrm{E}_{\boldsymbol{\sigma}}[\|\sum_{i=1}^m \sigma_i\Phi(x_i)\|_{\mathbb{H}}]$ can also be lower bounded by $\frac{1}{\sqrt{2}}\frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m}$, which only differs from the upper bound found by the constant $\frac{1}{\sqrt{2}}$. Also, note that if $K(x,x) \leq r^2$ for all $x \in \mathcal{X}$, then the inequalities 5.16 hold for all samples S.

The bound of theorem 5.5 or the inequalities 5.16 can be plugged into any of the Rademacher complexity generalization bounds presented in the previous chapters. In particular, in combination with theorem 4.4, they lead directly to the following margin bound similar to that of corollary 4.1.

Corollary 5.1 **Margin bounds for kernel-based hypotheses**
Let $K\colon \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ be a PDS kernel with $r = \sup_{x\in\mathcal{X}} K(x,x)$. Let $\Phi\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K and let $H = \{\mathbf{x} \mapsto \mathbf{w}\cdot\Phi(x)\colon \|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda\}$ for some $\Lambda \geq 0$. Fix $\rho > 0$. Then, for any $\delta > 0$, each of the following statements holds with probability at least $1-\delta$ for any $h \in H$:

$$R(h) \leq \widehat{R}_\rho(h) + 2\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{5.17}$$

$$R(h) \leq \widehat{R}_\rho(h) + 2\frac{\sqrt{\mathrm{Tr}[\mathbf{K}]\Lambda^2/\rho^2}}{m} + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{5.18}$$

5.4 Negative definite symmetric kernels

Often in practice, a natural distance or metric is available for the learning task considered. This metric could be used to define a similarity measure. As an example, Gaussian kernels have the form $\exp(-d^2)$, where d is a metric for the input vector space. Several natural questions arise such as: what other PDS kernels can we construct from a metric in a Hilbert space? What technical condition should d satisfy to guarantee that $\exp(-d^2)$ is PDS? A natural mathematical definition that helps address these questions is that of *negative definite symmetric (NDS) kernels*.

Definition 5.3 **Negative definite symmetric (NDS) kernels**
A kernel $K\colon \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ is said to be negative-definite symmetric (NDS) *if it is symmetric and if for all $\{x_1,\ldots,x_m\} \subseteq \mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^{m\times 1}$ with $\mathbf{1}^\top\mathbf{c} = 0$, the*

following holds:

$$\mathbf{c}^\top \mathbf{K}\mathbf{c} \leq 0.$$

Clearly, if K is PDS, then $-K$ is NDS, but the converse does not hold in general. The following gives a standard example of an NDS kernel.

Example 5.4 Squared distance — NDS kernel
The squared distance $(x, x') \mapsto \|x' - x\|^2$ in $\mathbb{R}^N$ defines an NDS kernel. Indeed, let $\mathbf{c} \in \mathbb{R}^{m\times 1}$ with $\sum_{i=1}^m c_i = 0$. Then, for any $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$, we can write

$$\begin{aligned}
\sum_{i,j=1}^m c_i c_j ||\mathbf{x}_i - \mathbf{x}_j||^2 &= \sum_{i,j=1}^m c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - 2\mathbf{x}_i \cdot \mathbf{x}_j) \\
&= \sum_{i,j=1}^m c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) - 2\sum_{i=1}^m c_i \mathbf{x}_i \cdot \sum_{j=1}^m c_j \mathbf{x}_j \\
&= \sum_{i,j=1}^m c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) - 2\Big\|\sum_{i=1}^m c_i \mathbf{x}_i\Big\|^2 \\
&\leq \sum_{i,j=1}^m c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) \\
&= \Big(\sum_{j=1}^m c_j\Big)\Big(\sum_{i=1}^m c_i (\|\mathbf{x}_i\|^2\Big) + \Big(\sum_{i=1}^m c_i\Big)\Big(\sum_{j=1}^m c_j \|\mathbf{x}_j\|^2\Big) = 0.
\end{aligned}$$

The next theorems show connections between NDS and PDS kernels. These results provide another series of tools for designing PDS kernels.

Theorem 5.6
Let K' be defined for any x_0 by

$$K'(x, x') = K(x, x_0) + K(x', x_0) - K(x, x') - K(x_0, x_0)$$

for all $x, x' \in \mathcal{X}$. Then K is NDS iff K' is PDS.

Proof Assume that K' is PDS and define K such that for any x_0 we have $K(x, x') = K(x, x_0) + K(x_0, x') - K(x_0, x_0) - K'(x, x')$. Then for any $\mathbf{c} \in \mathbb{R}^m$ such that $\mathbf{c}^\top \mathbf{1} = 0$ and any set of points $(x_1, \ldots, x_m) \in \mathcal{X}^m$ we have

$$\begin{aligned}
\sum_{i,j=1}^m c_i c_j K(x_i, x_j) &= \Big(\sum_{i=1}^m c_i K(x_i, x_0)\Big)\Big(\sum_{j=1}^m c_j\Big) + \Big(\sum_{i=1}^m c_i\Big)\Big(\sum_{j=1}^m c_j K(x_0, x_j)\Big) \\
&\quad - \Big(\sum_{i=1}^m c_i\Big)^2 K(x_0, x_0) - \sum_{i,j=1}^m c_i c_j K'(x_i, x_j) = -\sum_{i,j=1}^m c_i c_j K'(x_i, x_j) \leq 0\,.
\end{aligned}$$

which proves K is NDS.

Now, assume K is NDS and define K' for any x_0 as above. Then, for any $\mathbf{c} \in \mathbb{R}^m$, we can define $c_0 = -\mathbf{c}^\top \mathbf{1}$ and the following holds by the NDS property for any points $(x_1, \ldots, x_m) \in \mathcal{X}^m$ as well as x_0 defined previously: $\sum_{i,j=0}^m c_i c_j K(x_i, x_j) \leq 0$. This implies that

$$\Big(\sum_{i=0}^m c_i K(x_i, x_0)\Big)\Big(\sum_{j=0}^m c_j\Big) + \Big(\sum_{i=0}^m c_i\Big)\Big(\sum_{j=0}^m c_j K(x_0, x_j)\Big) - \Big(\sum_{i=0}^m c_i\Big)^2 K(x_0, x_0) - \sum_{i,j=0}^m c_i c_j K'(x_i, x_j) = -\sum_{i,j=0}^m c_i c_j K'(x_i, x_j) \leq 0\,,$$

which implies $2\sum_{i,j=1}^m c_i c_j K'(x_i, x_j) \geq -2c_0 \sum_{i=0}^m c_i K'(x_i, x_0) + c_0^2 K'(x_0, x_0) = 0$. The equality holds since $\forall x \in \mathcal{X}, K'(x, x_0) = 0$. ■

This theorem is useful in showing other connections, such the following theorems, which are left as exercises (see exercises 5.14 and 5.15).

Theorem 5.7

Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a symmetric kernel. Then, K is NDS iff $\exp(-tK)$ is a PDS kernel for all $t > 0$.

The theorem provides another proof that Gaussian kernels are PDS: as seen earlier (Example 5.4), the squared distance $(x, x') \mapsto \|x - x'\|^2$ in $\mathbb{R}^N$ is NDS, thus $(x, x') \mapsto \exp(-t\|x - x'\|^2)$ is PDS for all $t > 0$.

Theorem 5.8

Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be an NDS kernel such that for all $x, x' \in \mathcal{X}, K(x, x') = 0$ iff $x = x'$. Then, there exists a Hilbert space $\mathbb{H}$ and a mapping $\Phi\colon \mathcal{X} \to \mathbb{H}$ such that for all $x, x' \in \mathcal{X}$,

$$K(x, x') = \|\Phi(x) - \Phi(x')\|^2.$$

Thus, under the hypothesis of the theorem, $\sqrt{K}$ defines a metric.

This theorem can be used to show that the kernel $(x, x') \mapsto \exp(-|x - x'|^p)$ in $\mathbb{R}$ is not PDS for $p > 2$. Otherwise, for any $t > 0$, $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^{m \times 1}$, we would have:

$$\sum_{i,j=1}^m c_i c_j e^{-t|x_i - x_j|^p} = \sum_{i,j=1}^m c_i c_j e^{-|t^{1/p} x_i - t^{1/p} x_j|^p} \geq 0.$$

This would imply that $(x, x') \mapsto |x - x'|^p$ is NDS for $p > 2$, which can be proven (via theorem 5.8) not to be valid.

5.5 Sequence kernels

The examples given in the previous sections, including the commonly used polynomial or Gaussian kernels, were all for PDS kernels over vector spaces. In many learning tasks found in practice, the input space $\mathcal{X}$ is not a vector space. The examples to classify in practice could be protein sequences, images, graphs, parse trees, finite automata, or other discrete structures which may not be directly given as vectors. PDS kernels provide a method for extending algorithms such as SVMs originally designed for a vectorial space to the classification of such objects. But, how can we define PDS kernels for these structures?

This section will focus on the specific case of *sequence kernels*, that is, kernels for sequences or strings. PDS kernels can be defined for other discrete structures in somewhat similar ways. Sequence kernels are particularly relevant to learning algorithms applied to computational biology or natural language processing, which are both important applications.

How can we define PDS kernels for sequences, which are similarity measures for sequences? One idea consists of declaring two sequences, e.g., two documents or two biosequences, as similar when they share common substrings or subsequences. One example could be the kernel between two sequences defined by the sum of the product of the counts of their common substrings. But which substrings should be used in that definition? Most likely, we would need some flexibility in the definition of the matching substrings. For computational biology applications, for example, the match could be imperfect. Thus, we may need to consider some number of mismatches, possibly gaps, or wildcards. More generally, we might need to allow various substitutions and might wish to assign different weights to common substrings to emphasize some matching substrings and deemphasize others.

As can be seen from this discussion, there are many different possibilities and we need a general framework for defining such kernels. In the following, we will introduce a general framework for sequence kernels, *rational kernels*, which will include all the kernels considered in this discussion. We will also describe a general and efficient algorithm for their computation and will illustrate them with some examples.

The definition of these kernels relies on that of *weighted transducers*. Thus, we start with the definition of these devices as well as some relevant algorithms.

5.5.1 Weighted transducers

Sequence kernels can be effectively represented and computed using *weighted transducers*. In the following definition, let Σ denote a finite input alphabet, Δ a finite output alphabet, and ϵ the *empty string* or null label, whose concatenation with

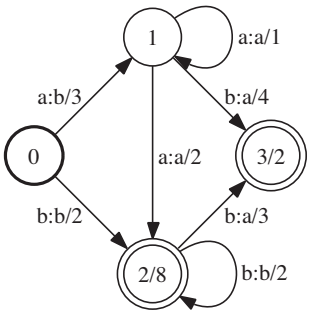


Figure 5.3 Example of weighted transducer.

any string leaves it unchanged.

Definition 5.4
A weighted transducer T *is a 7-tuple* $T = (\Sigma, \Delta, Q, I, F, E, \rho)$ *where* Σ *is a finite input alphabet,* Δ *a finite output alphabet,* Q *is a finite set of states,* $I \subseteq Q$ *the set of initial states,* $F \subseteq Q$ *the set of final states,* E *a finite multiset of transitions elements of* $Q \times (\Sigma \cup \{\epsilon\}) \times (\Delta \cup \{\epsilon\}) \times \mathbb{R} \times Q$*, and* $\rho : F \to \mathbb{R}$ *a final weight function mapping* F *to* $\mathbb{R}$*. The* size *of transducer* T *is the sum of its number of states and transitions and is denoted by* $|T|$*.*[2]

Thus, weighted transducers are finite automata in which each transition is labeled with both an input and an output label and carries some real-valued weight. Figure 5.3 shows an example of a weighted finite-state transducer. In this figure, the input and output labels of a transition are separated by a colon delimiter, and the weight is indicated after the slash separator. The initial states are represented by a bold circle and final states by double circles. The final weight $\rho[q]$ at a final state q is displayed after the slash.

The input label of a path π is a string element of Σ^* obtained by concatenating input labels along π. Similarly, the output label of a path π is obtained by concatenating output labels along π. A path from an initial state to a final state is an *accepting path*. The weight of an accepting path is obtained by multiplying the weights of its constituent transitions and the weight of the final state of the path.

A weighted transducer defines a mapping from $\Sigma^* \times \Delta^*$ to $\mathbb{R}$. The weight associated by a weighted transducer T to a pair of strings $(x, y) \in \Sigma^* \times \Delta^*$ is denoted by $T(x, y)$ and is obtained by summing the weights of all accepting paths

2. A multiset in the definition of the transitions is used to allow for the presence of several transitions from a state p to a state q with the same input and output label, and even the same weight, which may occur as a result of various operations.

with input label x and output label y. For example, the transducer of figure 5.3 associates to the pair (aab, baa) the weight $3 \times 1 \times 4 \times 2 + 3 \times 2 \times 3 \times 2$, since there is a path with input label aab and output label baa and weight $3 \times 1 \times 4 \times 2$, and another one with weight $3 \times 2 \times 3 \times 2$.

The sum of the weights of all accepting paths of an acyclic transducer, that is a transducer T with no cycle, can be computed in linear time, that is $O(|T|)$, using a general *shortest-distance* or forward-backward algorithm. These are simple algorithms, but a detailed description would require too much of a digression from the main topic of this chapter.

Composition An important operation for weighted transducers is *composition*, which can be used to combine two or more weighted transducers to form more complex weighted transducers. As we shall see, this operation is useful for the creation and computation of sequence kernels. Its definition follows that of composition of relations. Given two weighted transducers $T_1 = (\Sigma, \Delta, Q_1, I_1, F_1, E_1, \rho_1)$ and $T_2 = (\Delta, \Omega, Q_2, I_2, F_2, E_2, \rho_2)$, the result of the composition of T_1 and T_2 is a weighted transducer denoted by $T_1 \circ T_2$ and defined for all $x \in \Sigma^*$ and $y \in \Omega^*$ by

$$(T_1 \circ T_2)(x, y) = \sum_{z \in \Delta^*} T_1(x, z) \cdot T_2(z, y), \tag{5.19}$$

where the sum runs over all strings z over the alphabet Δ. Thus, composition is similar to matrix multiplication with infinite matrices.

There exists a general and efficient algorithm to compute the composition of two weighted transducers. In the absence of ϵs on the input side of T_1 or the output side of T_2, the states of $T_1 \circ T_2 = (\Sigma, \Delta, Q, I, F, E, \rho)$ can be identified with pairs made of a state of T_1 and a state of T_2, $Q \subseteq Q_1 \times Q_2$. Initial states are those obtained by pairing initial states of the original transducers, $I = I_1 \times I_2$, and similarly final states are defined by $F = Q \cap (F_1 \times F_2)$. The final weight at a state $(q_1, q_2) \in F_1 \times F_2$ is $\rho(q) = \rho_1(q_1)\rho_2(q_2)$, that is the product of the final weights at q_1 and q_2. Transitions are obtained by matching a transition of T_1 with one of T_2 from appropriate transitions of T_1 and T_2:

$$E = \biguplus_{\substack{(q_1, a, b, w_1, q_2) \in E_1 \\ (q_1', b, c, w_2, q_2') \in E_2}} \left\{ \Big((q_1, q_1'), a, c, w_1 \otimes w_2, (q_2, q_2') \Big) \right\}.$$

Here, $\uplus$ denotes the standard join operation of multisets as in $\{1, 2\} \uplus \{1, 3\} = \{1, 1, 2, 3\}$, to preserve the multiplicity of the transitions.

In the worst case, all transitions of T_1 leaving a state q_1 match all those of T_2 leaving state q_1', thus the space and time complexity of composition is quadratic: $O(|T_1||T_2|)$. In practice, such cases are rare and composition is very efficient. Figure 5.4 illustrates the algorithm in a particular case.

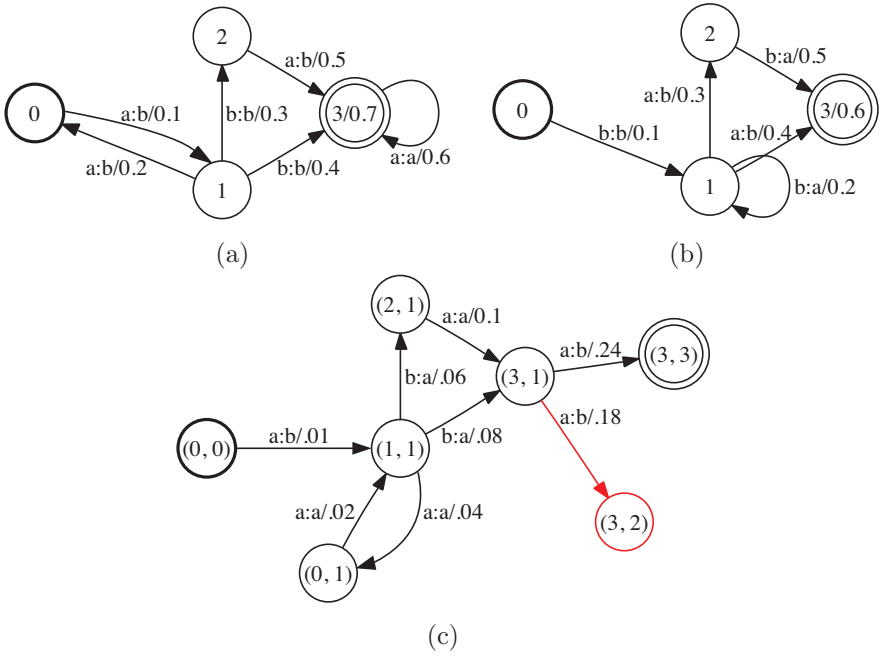


Figure 5.4 (a) Weighted transducer T_1. (b) Weighted transducer T_2. (c) Result of composition of T_1 and T_2, $T_1 \circ T_2$. Some states might be constructed during the execution of the algorithm that are not *co-accessible*, that is, they do not admit a path to a final state, e.g., $(3, 2)$. Such states and the related transitions (in red) can be removed by a trimming (or connection) algorithm in linear time.

As illustrated by figure 5.5, when T_1 admits output ϵ labels or T_2 input ϵ labels, the algorithm just described may create redundant ϵ-paths, which would lead to an incorrect result. The weight of the matching paths of the original transducers would be counted p times, where p is the number of redundant paths in the result of composition. To avoid with this problem, all but one ϵ-path must be filtered out of the composite transducer. Figure 5.5 indicates in boldface one possible choice for that path, which in this case is the shortest. Remarkably, that filtering mechanism itself can be encoded as a finite-state transducer F (figure 5.5b).

To apply that filter, we need to first augment T_1 and T_2 with auxiliary symbols that make the semantics of ϵ explicit: let $\tilde{T}_1$ ($\tilde{T}_2$) be the weighted transducer obtained from T_1 (respectively T_2) by replacing the output (respectively input) ϵ labels with ϵ_2 (respectively ϵ_1) as illustrated by figure 5.5. Thus, matching with the symbol ϵ_1 corresponds to remaining at the same state of T_1 and taking a transition of T_2 with input ϵ. ϵ_2 can be described in a symmetric way. The filter transducer F disallows a matching (ϵ_2, ϵ_2) immediately after (ϵ_1, ϵ_1) since this can be done instead via (ϵ_2, ϵ_1).

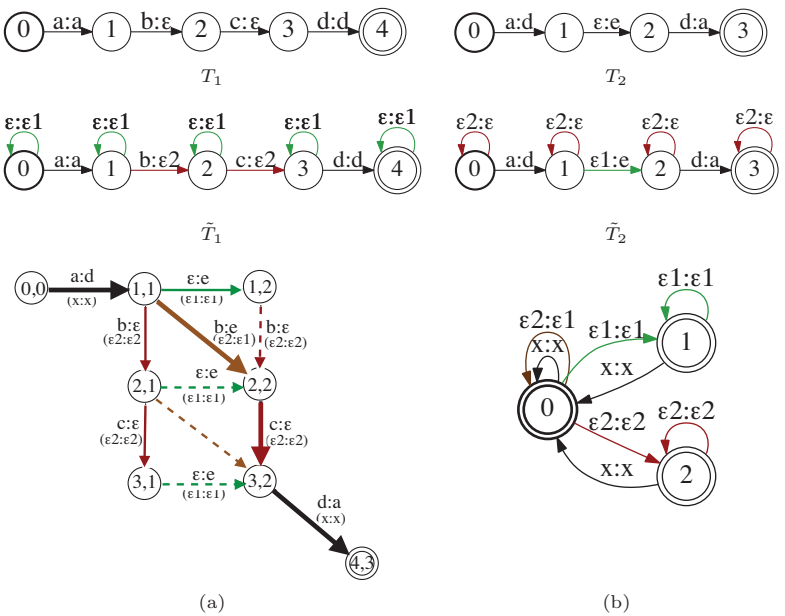


Figure 5.5 Redundant ϵ-paths in composition. All transition and final weights are equal to one. (a) A straightforward generalization of the ϵ-free case would generate all the paths from $(1, 1)$ to $(3, 2)$ when composing T_1 and T_2 and produce an incorrect results in non-idempotent semirings. (b) Filter transducer F. The shorthand x is used to represent an element of Σ.

By symmetry, it also disallows a matching (ϵ_1, ϵ_1) immediately after (ϵ_2, ϵ_2). In the same way, a matching (ϵ_1, ϵ_1) immediately followed by (ϵ_2, ϵ_1) is not permitted by the filter F since a path via the matchings $(\epsilon_2, \epsilon_1)(\epsilon_1, \epsilon_1)$ is possible. Similarly, $(\epsilon_2, \epsilon_2)(\epsilon_2, \epsilon_1)$ is ruled out. It is not hard to verify that the filter transducer F is precisely a finite automaton over pairs accepting the complement of the language

$$L = \sigma^*((\epsilon_1, \epsilon_1)(\epsilon_2, \epsilon_2) + (\epsilon_2, \epsilon_2)(\epsilon_1, \epsilon_1) + (\epsilon_1, \epsilon_1)(\epsilon_2, \epsilon_1) + (\epsilon_2, \epsilon_2)(\epsilon_2, \epsilon_1))\sigma^*,$$

where $\sigma = \{(\epsilon_1, \epsilon_1), (\epsilon_2, \epsilon_2), (\epsilon_2, \epsilon_1), x\}$. Thus, the filter F guarantees that exactly one ϵ-path is allowed in the composition of each ϵ sequences. To obtain the correct result of composition, it suffices then to use the ϵ-free composition algorithm already described and compute

$$\tilde{T}_1 \circ F \circ \tilde{T}_2. \tag{5.20}$$

Indeed, the two compositions in $\tilde{T}_1 \circ F \circ \tilde{T}_2$ no longer involve ϵs. Since the size of the filter transducer F is constant, the complexity of general composition is the

same as that of ϵ-free composition, that is $O(|T_1||T_2|)$. In practice, the augmented transducers $\tilde{T}_1$ and $\tilde{T}_2$ are not explicitly constructed, instead the presence of the auxiliary symbols is simulated. Further filter optimizations help limit the number of non-coaccessible states created, for example, by examining more carefully the case of states with only outgoing non-ϵ-transitions or only outgoing ϵ-transitions.

5.5.2 Rational kernels

The following establishes a general framework for the definition of sequence kernels.

Definition 5.5 **Rational kernels**
A kernel $K\colon \Sigma^* \times \Sigma^* \to \mathbb{R}$ *is said to be* rational *if it coincides with the mapping defined by some weighted transducer* U*:* $\forall x, y \in \Sigma^*, K(x, y) = U(x, y)$.

Note that we could have instead adopted a more general definition: instead of using weighted transducers, we could have used more powerful sequence mappings such as *algebraic transductions*, which are the functional counterparts of context-free languages, or even more powerful ones. However, an essential need for kernels is an efficient computation, and more complex definitions would lead to substantially more costly computational complexities for kernel computation. For rational kernels, there exists a general and efficient computation algorithm.

Computation We will assume that the transducer U defining a rational kernel K does not admit any ϵ-cycle with non-zero weight, otherwise the kernel value is infinite for all pairs. For any sequence x, let T_x denote a weighted transducer with just one accepting path whose input and output labels are both x and its weight equal to one. T_x can be straightforwardly constructed from x in linear time $O(|x|)$. Then, for any $x, y \in \Sigma^*$, $U(x, y)$ can be computed by the following two steps:

1. Compute $V = T_x \circ U \circ T_y$ using the composition algorithm in time $O(|U||T_x||T_y|)$.
2. Compute the sum of the weights of all accepting paths of V using a general shortest-distance algorithm in time $O(|V|)$.

By definition of composition, V is a weighted transducer whose accepting paths are precisely those accepting paths of U that have input label x and output label y. The second step computes the sum of the weights of these paths, that is, exactly $U(x, y)$. Since U admits no ϵ-cycle, V is acyclic, and this step can be performed in linear time. The overall complexity of the algorithm for computing $U(x, y)$ is then in $O(|U||T_x||T_y|)$. Since U is fixed for a rational kernel K and $|T_x| = O(|x|)$ for any x, this shows that the kernel values can be obtained in quadratic time $O(|x||y|)$. For some specific weighted transducers U, the computation can be more efficient, for example in $O(|x| + |y|)$ (see exercise 5.17).

PDS rational kernels For any transducer T, let T^{-1} denote the *inverse* of T, that is the transducer obtained from T by swapping the input and output labels of every transition. For all x, y, we have $T^{-1}(x, y) = T(y, x)$. The following theorem gives a general method for constructing a PDS rational kernel from an arbitrary weighted transducer.

Theorem 5.9
For any weighted transducer $T = (\Sigma, \Delta, Q, I, F, E, \rho)$, the function $K = T \circ T^{-1}$ is a PDS rational kernel.

Proof By definition of composition and the inverse operation, for all $x, y \in \Sigma^*$,

$$K(x, y) = \sum_{z \in \Delta^*} T(x, z)\, T(y, z).$$

K is the pointwise limit of the kernel sequence $(K_n)_{n \geq 0}$ defined by:

$$\forall n \in \mathbb{N}, \forall x, y \in \Sigma^*, \quad K_n(x, y) = \sum_{|z| \leq n} T(x, z)\, T(y, z),$$

where the sum runs over all sequences in Δ^* of length at most n. K_n is PDS since its corresponding kernel matrix $\mathbf{K}_n$ for any sample $(x_1, \ldots, x_m)$ is SPSD. This can be see form the fact that $\mathbf{K}_n$ can be written as $\mathbf{K}_n = \mathbf{A}\mathbf{A}^\top$ with $\mathbf{A} = (K_n(x_i, z_j))_{i \in [1,m], j \in [1,N]}$, where $z_1, \ldots, z_N$ is some arbitrary enumeration of the set of strings in Σ^* with length at most n. Thus, K is PDS as the pointwise limit of the sequence of PDS kernels $(K_n)_{n \in \mathbb{N}}$. ■

The sequence kernels commonly used in computational biology, natural language processing, computer vision, and other applications are all special instances of rational kernels of the form $T \circ T^{-1}$. All of these kernels can be computed efficiently using the same general algorithm for the computational of rational kernels presented in the previous paragraph. Since the transducer $U = T \circ T^{-1}$ defining such PDS rational kernels has a specific form, there are different options for the computation of the composition $T_x \circ U \circ T_y$:

- compute $U = T \circ T^{-1}$ first, then $V = T_x \circ U \circ T_y$;
- compute $V_1 = T_x \circ T$ and $V_2 = T_y \circ T$ first, then $V = V_1 \circ V_2^{-1}$;
- compute first $V_1 = T_x \circ T$, then $V_2 = V_1 \circ T^{-1}$, then $V = V_2 \circ T_y$, or the similar series of operations with x and y permuted.

All of these methods lead to the same result after computation of the sum of the weights of all accepting paths, and they all have the same worst-case complexity. However, in practice, due to the sparsity of intermediate compositions, there may be substantial differences between their time and space computational costs. An

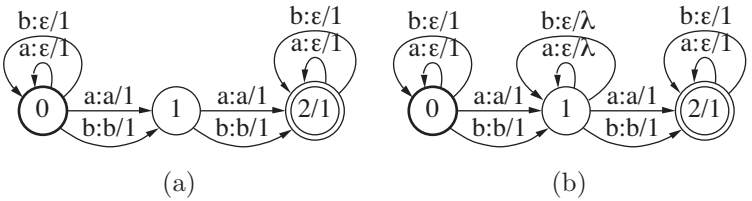


Figure 5.6 (a) Transducer T_{bigram} defining the bigram kernel $T_{\text{bigram}} \circ T_{\text{bigram}}^{-1}$ for $\Sigma = \{a, b\}$. (b) Transducer $T_{\text{gappy\_bigram}}$ defining the gappy bigram kernel $T_{\text{gappy\_bigram}} \circ T_{\text{gappy\_bigram}}^{-1}$ with gap penalty $\lambda \in (0, 1)$.

alternative method based on an *n-way composition* can further lead to significantly more efficient computations.

Example 5.5 Bigram and gappy bigram sequence kernels

Figure 5.6a shows a weighted transducer T_{bigram} defining a common sequence kernel, the *bigram sequence kernel*, for the specific case of an alphabet reduced to $\Sigma = \{a, b\}$. The bigram kernel associates to any two sequences x and y the sum of the product of the counts of all bigrams in x and y. For any sequence $x \in \Sigma^*$ and any bigram $z \in \{aa, ab, ba, bb\}$, $T_{\text{bigram}}(x, z)$ is exactly the number of occurrences of the bigram z in x. Thus, by definition of composition and the inverse operation, $T_{\text{bigram}} \circ T_{\text{bigram}}^{-1}$ computes exactly the bigram kernel.

Figure 5.6b shows a weighted transducer $T_{\text{gappy\_bigram}}$ defining the so-called *gappy bigram kernel*. The gappy bigram kernel associates to any two sequences x and y the sum of the product of the counts of all gappy bigrams in x and y penalized by the length of their *gaps*. Gappy bigrams are sequences of the form aua, aub, bua, or bub, where $u \in \Sigma^*$ is called the gap. The count of a gappy bigram is multiplied by $|u|^\lambda$ for some fixed $\lambda \in (0, 1)$ so that gappy bigrams with longer gaps contribute less to the definition of the similarity measure. While this definition could appear to be somewhat complex, figure 5.6 shows that $T_{\text{gappy\_bigram}}$ can be straightforwardly derived from T_{bigram}. The graphical representation of rational kernels helps understanding or modifying their definition.

Counting transducers The definition of most sequence kernels is based on the counts of some common patterns appearing in the sequences. In the examples just examined, these were bigrams or gappy bigrams. There exists a simple and general method for constructing a weighted transducer counting the number of occurrences of patterns and using them to define PDS rational kernels. Let X be a finite automaton representing the set of patterns to count. In the case of bigram kernels with $\Sigma = \{a, b\}$, X would be an automaton accepting exactly the set of strings $\{aa, ab, ba, bb\}$. Then, the weighted transducer of figure 5.7 can be used to compute exactly the number of occurrences of each pattern accepted by X.

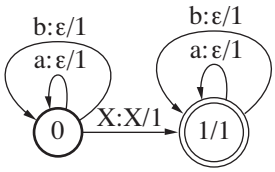


Figure 5.7 Counting transducer T_{count} for $\Sigma = \{a, b\}$. The "transition" $X : X/1$ stands for the weighted transducer created from the automaton X by adding to each transition an output label identical to the existing label, and by making all transition and final weights equal to one.

Theorem 5.10
For any $x \in \Sigma^*$ and any sequence z accepted by X, $T_{count}(x, z)$ is the number of occurrences of z in x.

Proof Let $x \in \Sigma^*$ be an arbitrary sequence and let z be a sequence accepted by X. Since all accepting paths of T_{count} have weight one, $T_{\text{count}}(x, z)$ is equal to the number of accepting paths in T_{count} with input label x and output z.

Now, an accepting path π in T_{count} with input x and output z can be decomposed as $\pi = \pi_0\, \pi_{01}\, \pi_1$, where π_0 is a path through the loops of state 0 with input label some prefix x_0 of x and output label ϵ, π_{01} an accepting path from 0 to 1 with input and output labels equal to z, and π_1 a path through the self-loops of state 1 with input label a suffix x_1 of x and output ϵ. Thus, the number of such paths is exactly the number of distinct ways in which we can write sequence x as $x = x_0 z x_1$, which is exactly the number of occurrences of z in x. ■

The theorem provides a very general method for constructing PDS rational kernels $T_{\text{count}} \circ T_{\text{count}}^{-1}$ that are based on counts of some patterns that can be defined via a finite automaton, or equivalently a regular expression. Figure 5.7 shows the transducer for the case of an input alphabet reduced to $\Sigma = \{a, b\}$. The general case can be obtained straightforwardly by augmenting states 0 and 1 with other self-loops using other symbols than a and b. In practice, a lazy evaluation can be used to avoid the explicit creation of these transitions for all alphabet symbols and instead creating them on-demand based on the symbols found in the input sequence x. Finally, one can assign different weights to the patterns counted to emphasize or deemphasize some, as in the case of gappy bigrams. This can be done simply by changing the transitions weight or final weights of the automaton X used in the definition of T_{count}.

5.6 Chapter notes

The mathematical theory of PDS kernels in a general setting originated with the fundamental work of Mercer [1909] who also proved the equivalence of a condition similar to that of theorem 5.1 for continuous kernels with the PDS property. The connection between PDS and NDS kernels, in particular theorems 5.8 and 5.7, are due to Schoenberg [1938]. A systematic treatment of the theory of reproducing kernel Hilbert spaces was presented in a long and elegant paper by Aronszajn [1950]. For an excellent mathematical presentation of PDS kernels and positive definite functions we refer the reader to Berg, Christensen, and Ressel [1984], which is also the source of several of the exercises given in this chapter.

The fact that SVMs could be extended by using PDS kernels was pointed out by Boser, Guyon, and Vapnik [1992]. The idea of kernel methods has been since then widely adopted in machine learning and applied in a variety of different tasks and settings. The following two books are in fact specifically devoted to the study of kernel methods: Schölkopf and Smola [2002] and Shawe-Taylor and Cristianini [2004]. The classical representer theorem is due to Kimeldorf and Wahba [1971]. A generalization to non-quadratic cost functions was stated by Wahba [1990]. The general form presented in this chapter was given by Schölkopf, Herbrich, Smola, and Williamson [2000].

Rational kernels were introduced by Cortes, Haffner, and Mohri [2004]. A general class of kernels, *convolution kernels*, was earlier introduced by Haussler [1999]. The convolution kernels for sequences described by Haussler [1999], as well as the pair-HMM string kernels described by Watkins [1999], are special instances of rational kernels. Rational kernels can be straightforwardly extended to define kernels for finite automata and even weighted automata [Cortes et al., 2004]. Cortes, Mohri, and Rostamizadeh [2008b] study the problem of *learning* rational kernels such as those based on counting transducers.

The composition of weighted transducers and the filter transducers in the presence of ϵ-paths are described in Pereira and Riley [1997], Mohri, Pereira, and Riley [2005], and Mohri [2009]. Composition can be further generalized to the *N-way composition* of weighted transducers [Allauzen and Mohri, 2009]. N-way composition of three or more transducers can substantially speed up computation, in particular for PDS rational kernels of the form $T \circ T^{-1}$. A generic *shortest-distance algorithm* which can be used with a large class of semirings and arbitrary queue disciplines is described by Mohri [2002]. A specific instance of that algorithm can be used to compute the sum of the weights of all paths as needed for the computation of rational kernels after composition. For a study of the class of languages linearly separable with rational kernels , see Cortes, Kontorovich, and Mohri [2007a].

5.7 Exercises

5.1 Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel, and let $\alpha\colon \mathcal{X} \to \mathbb{R}$ be a positive function. Show that the kernel K' defined for all $x, y \in \mathcal{X}$ by $K'(x, y) = \frac{K(x,y)}{\alpha(x)\alpha(y)}$ is a PDS kernel.

5.2 Show that the following kernels K are PDS:

(a) $K(x, y) = \cos(x - y)$ over $\mathbb{R} \times \mathbb{R}$.

(b) $K(x, y) = \cos(x^2 - y^2)$ over $\mathbb{R} \times \mathbb{R}$.

(c) $K(x, y) = (x + y)^{-1}$ over $(0, +\infty) \times (0, +\infty)$.

(d) $K(\mathbf{x}, \mathbf{x}') = \cos \angle(\mathbf{x}, \mathbf{x}')$ over $\mathbb{R}^n \times \mathbb{R}^n$, where $\angle(\mathbf{x}, \mathbf{x}')$ is the angle between $\mathbf{x}$ and $\mathbf{x}'$.

(e) $\forall \lambda > 0,\ K(x, x') = \exp\big(-\lambda[\sin(x' - x)]^2\big)$ over $\mathbb{R} \times \mathbb{R}$. (*Hint*: rewrite $[\sin(x' - x)]^2$ as the square of the norm of the difference of two vectors.)

5.3 Show that the following kernels K are NDS:

(a) $K(x, y) = [\sin(x - y)]^2$ over $\mathbb{R} \times \mathbb{R}$.

(b) $K(x, y) = \log(x + y)$ over $(0, +\infty) \times (0, +\infty)$.

5.4 Define a *difference kernel* as $K(x, x') = |x - x'|$ for $x, x' \in \mathbb{R}$. Show that this kernel is not positive definite symmetric (PDS).

5.5 Is the kernel K defined over $\mathbb{R}^n \times \mathbb{R}^n$ by $K(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|^{3/2}$ PDS? Is it NDS?

5.6 Let H be a Hilbert space with the corresponding dot product $\langle \cdot, \cdot \rangle$. Show that the kernel K defined over $H \times H$ by $K(x, y) = 1 - \langle x, y \rangle$ is negative definite.

5.7 For any $p > 0$, let K_p be the kernel defined over $\mathbb{R}_+ \times \mathbb{R}_+$ by

$$K_p(x, y) = e^{-(x+y)^p}. \tag{5.21}$$

Show that K_p is positive definite symmetric (PDS) iff $p \leq 1$. (*Hint*: you can use the fact that if K is NDS, then for any $0 < \alpha \leq 1$, K^α is also NDS.)

5.8 Explicit mappings.

(a) Denote a data set $x_1, \ldots, x_m$ and a kernel $K(x_i, x_j)$ with a Gram matrix $\mathbf{K}$. Assuming $\mathbf{K}$ is positive semidefinite, then give a map $\Phi(\cdot)$ such that

$K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$.

(b) Show the converse of the previous statement, i.e., if there exists a mapping $\Phi(x)$ from input space to some Hilbert space, then the corresponding matrix $\mathbf{K}$ is positive semidefinite.

5.9 Explicit polynomial kernel mapping. Let K be a polynomial kernel of degree d, i.e., $K\colon \mathbb{R}^N \times \mathbb{R}^N \to \mathbb{R}$, $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$, with $c > 0$, Show that the dimension of the feature space associated to K is

$$\binom{N+d}{d}. \tag{5.22}$$

Write K in terms of kernels $k_i\colon (\mathbf{x}, \mathbf{x}') \mapsto (\mathbf{x} \cdot \mathbf{x}')^i$, $i \in [0, d]$. What is the weight assigned to each k_i in that expression? How does it vary as a function of c?

5.10 High-dimensional mapping. Let $\Phi\colon \mathcal{X} \to H$ be a feature mapping such that the dimension N of H is very large and let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel defined by

$$K(x, x') = \mathop{\mathrm{E}}_{i \sim D} \big[[\Phi(x)]_i [\Phi(x')]_i \big], \tag{5.23}$$

where $[\Phi(x)]_i$ is the ith component of $\Phi(x)$ (and similarly for $\Phi'(x)$) and where D is a distribution over the indices i. We shall assume that $|[\Phi(x)]_i| \leq R$ for all $x \in \mathcal{X}$ and $i \in [1, N]$. Suppose that the only method available to compute $K(x, x')$ involved direct computation of the inner product (5.23), which would require $O(N)$ time. Alternatively, an approximation can be computed based on random selection of a subset I of the N components of $\Phi(x)$ and $\Phi(x')$ according to D, that is:

$$K'(x, x') = \frac{1}{n} \sum_{i \in I} D(i) [\Phi(x)]_i [\Phi(x')]_i, \tag{5.24}$$

where $|I| = n$.

(a) Fix x and x' in X. Prove that

$$\Pr_{I \sim D^n} [|K(x, x') - K'(x, x')| > \epsilon] \leq 2e^{\frac{-n\epsilon^2}{2r^2}}. \tag{5.25}$$

(*Hint*: use McDiarmid's inequality).

(b) Let $\mathbf{K}$ and $\mathbf{K}'$ be the kernel matrices associated to K and K'. Show that for any $\epsilon, \delta > 0$, for $n > \frac{r^2}{\epsilon^2} \log \frac{m(m+1)}{\delta}$, with probability at least $1 - \delta$, $|\mathbf{K}'_{ij} - \mathbf{K}_{ij}| \leq \epsilon$ for all $i, j \in [1, m]$.

5.11 Classifier based kernel. Let S be a training sample of size m. Assume that

S has been generated according to some probability distribution $D(x, y)$, where $(x, y) \in X \times \{-1, +1\}$.

(a) Define the Bayes classifier $h^*\colon X \to \{-1, +1\}$. Show that the kernel K^* defined by $K^*(x, x') = h^*(x)h^*(x')$ for any $x, x' \in X$ is positive definite symmetric. What is the dimension of the natural feature space associated to K^*?

(b) Give the expression of the solution obtained using SVMs with this kernel. What is the number of support vectors? What is the value of the margin? What is the generalization error of the solution obtained? Under what condition are the data linearly separable?

(c) Let $h : X \to \mathbb{R}$ be an arbitrary real-valued function. Under what condition on h is the kernel K defined by $K(x, x') = h(x)h(x')$, $x, x' \in X$, positive definite symmetric?

5.12 Image classification kernel. For $\alpha \geq 0$, the kernel

$$K_\alpha\colon (\mathbf{x}, \mathbf{x}') \mapsto \sum_{k=1}^{N} \min(|x_k|^\alpha, |x'_k|^\alpha) \tag{5.26}$$

over $\mathbb{R}^N \times \mathbb{R}^N$ is used in image classification. Show that K_α is PDS for all $\alpha \geq 0$. To do so, proceed as follows.

(a) Use the fact that $(f, g) \mapsto \int_{t=0}^{+\infty} f(t)g(t)dt$ is an inner product over the set of measurable functions over $[0, +\infty)$ to show that $(x, x') \mapsto \min(x, x')$ is a PDS kernel. (*Hint*: associate an indicator function to x and another one to x'.)

(b) Use the result from (a) to first show that K_1 is PDS and similarly that K_α with other values of α is also PDS.

5.13 Fraud detection. To prevent fraud, a credit-card company decides to contact Professor Villebanque and provides him with a random list of several thousand fraudulent and non-fraudulent *events*. There are many different types of events, e.g., transactions of various amounts, changes of address or card-holder information, or requests for a new card. Professor Villebanque decides to use SVMs with an appropriate kernel to help predict fraudulent events accurately. It is difficult for Professor Villebanque to define relevant features for such a diverse set of events. However, the risk department of his company has created a complicated method to estimate a probability $\Pr[U]$ for any event U. Thus, Professor Villebanque decides to make use of that information and comes up with the following kernel defined

over all pairs of events (U, V):

$$K(U, V) = \Pr[U \wedge V] - \Pr[U]\Pr[V]. \tag{5.27}$$

Help Professor Villebanque show that his kernel is positive definite symmetric.

5.14 Relationship between NDS and PDS kernels. Prove the statement of theorem 5.7. (*Hint*: Use the fact that if K is PDS then $\exp(K)$ is also PDS, along with theorem 5.6.)

5.15 Metrics and Kernels. Let $\mathcal{X}$ be a non-empty set and $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a negative definite symmetric kernel such that $K(x, x) = 0$ for all $x \in \mathcal{X}$.

(a) Show that there exists a Hilbert space $\mathbb{H}$ and a mapping $\Phi(x)$ from $\mathcal{X}$ to $\mathbb{H}$ such that:

$$K(x, y) = ||\Phi(x) - \Phi(x')||^2 .$$

Assume that $K(x, x') = 0 \Rightarrow x = x'$. Use theorem 5.6 to show that $\sqrt{K}$ defines a metric on $\mathcal{X}$.

(b) Use this result to prove that the kernel $K(x, y) = \exp(-|x-x'|^p)$, $x, x' \in \mathbb{R}$, is not positive definite for $p > 2$.

(c) The kernel $K(x, x') = \tanh(a(x \cdot x')+b)$ was shown to be equivalent to a two-layer neural network when combined with SVMs. Show that K is not positive definite if $a < 0$ or $b < 0$. What can you conclude about the corresponding neural network when $a < 0$ or $b < 0$?

5.16 Sequence kernels. Let $X = \{a, c, g, t\}$. To classify DNA sequences using SVMs, we wish to define a kernel between sequences defined over X. We are given a finite set $I \subset X^*$ of non-coding regions (introns). For $x \in X^*$, denote by $|x|$ the length of x and by $F(x)$ the set of factors of x, i.e., the set of subsequences of x with contiguous symbols. For any two strings $x, y \in X^*$ define $K(x, y)$ by

$$K(x, y) = \sum_{z \in (F(x) \cap F(y)) - I} \rho^{|z|}, \tag{5.28}$$

where $\rho \geq 1$ is a real number.

(a) Show that K is a rational kernel and that it is positive definite symmetric.

(b) Give the time and space complexity of the computation of $K(x, y)$ with respect to the size s of a minimal automaton representing $X^* - I$.

(c) Long common factors between x and y of length greater than or equal to

n are likely to be important coding regions (exons). Modify the kernel K to assign weight $\rho_2^{|z|}$ to z when $|z| \geq n$, $\rho_1^{|z|}$ otherwise, where $1 \leq \rho_1 \ll \rho_2$. Show that the resulting kernel is still positive definite symmetric.

5.17 n-gram kernel. Show that for all $n \geq 1$, and any n-gram kernel K_n, $K_n(x, y)$ can be computed in linear time $O(|x| + |y|)$, for all $x, y \in \Sigma^*$ assuming n and the alphabet size are constants.

5.18 Mercer's condition. Let $\mathcal{X} \subset \mathbb{R}^N$ be a compact set and $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a continuous kernel function. Prove that if K verifies Mercer's condition (theorem 5.1), then it is PDS. (*Hint*: assume that K is not PDS and consider a set $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and a column-vector $c \in \mathbb{R}^{m \times 1}$ such that $\sum_{i,j=1}^{m} c_i c_j K(x_i, x_j) < 0$.)

6 Boosting

Ensemble methods are general techniques in machine learning for combining several predictors to create a more accurate one. This chapter studies an important family of ensemble methods known as *boosting*, and more specifically the *AdaBoost* algorithm. This algorithm has been shown to be very effective in practice in some scenarios and is based on a rich theoretical analysis. We first introduce AdaBoost, show how it can rapidly reduce the empirical error as a function of the number of rounds of boosting, and point out its relationship with some known algorithms. Then we present a theoretical analysis of its generalization properties based on the VC-dimension of its hypothesis set and based on a notion of margin that we will introduce. Much of that margin theory can be applied to other similar ensemble algorithms. A game-theoretic interpretation of AdaBoost further helps analyzing its properties. We end with a discussion of AdaBoost's benefits and drawbacks.

6.1 Introduction

It is often difficult, for a non-trivial learning task, to directly devise an accurate algorithm satisfying the strong PAC-learning requirements of chapter 2. But, there can be more hope for finding simple predictors guaranteed only to perform slightly better than random. The following gives a formal definition of such *weak learners*.

Definition 6.1 **Weak learning**
A concept class C is said to be weakly PAC-learnable *if there exists an algorithm $\mathcal{A}$, $\gamma > 0$, and a polynomial function $poly(\cdot, \cdot, \cdot, \cdot)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions D on $\mathcal{X}$ and for any target concept $c \in C$, the following holds for any sample size $m \geq poly(1/\epsilon, 1/\delta, n, size(c))$:*

$$\Pr_{S \sim D^m}\left[R(h_S) \leq \frac{1}{2} - \gamma\right] \geq 1 - \delta. \tag{6.1}$$

When such an algorithm $\mathcal{A}$ exists, it is called a weak learning algorithm *for C or a* weak learner*. The hypotheses returned by a weak learning algorithm are called* base classifiers*.*

ADABOOST($S = ((x_1, y_1), \ldots, (x_m, y_m))$)

1 **for** $i \leftarrow 1$ **to** m **do**
2 $\quad D_1(i) \leftarrow \frac{1}{m}$
3 **for** $t \leftarrow 1$ **to** T **do**
4 $\quad h_t \leftarrow$ base classifier in H with small error $\epsilon_t = \Pr_{i \sim D_t}[h_t(x_i) \neq y_i]$
5 $\quad \alpha_t \leftarrow \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$
6 $\quad Z_t \leftarrow 2[\epsilon_t(1-\epsilon_t)]^{\frac{1}{2}}$ ▷ normalization factor
7 $\quad$ **for** $i \leftarrow 1$ **to** m **do**
8 $\quad\quad D_{t+1}(i) \leftarrow \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$
9 $g \leftarrow \sum_{t=1}^{T} \alpha_t h_t$
10 **return** $h = \text{sgn}(g)$

Figure 6.1 AdaBoost algorithm for $H \subseteq \{-1, +1\}^{\mathcal{X}}$.

The key idea behind boosting techniques is to use a weak learning algorithm to build a *strong learner*, that is, an accurate PAC-learning algorithm. To do so, boosting techniques use an ensemble method: they combine different base classifiers returned by a weak learner to create a more accurate predictor. But which base classifiers should be used and how should they be combined? The next section addresses these questions by describing in detail one of the most prevalent and successful boosting algorithms, AdaBoost.

6.2 AdaBoost

We denote by H the hypothesis set out of which the base classifiers are selected. Figure 6.1 gives the pseudocode of AdaBoost in the case where the base classifiers are functions mapping from $\mathcal{X}$ to $\{-1, +1\}$, thus $H \subseteq \{-1, +1\}^{\mathcal{X}}$.

The algorithm takes as input a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$, with $(x_i, y_i) \in \mathcal{X} \times \{-1, +1\}$ for all $i \in [1, m]$, and maintains a distribution over the indices $\{1, \ldots, m\}$. Initially (lines 1-2), the distribution is uniform (D_1). At each *round of boosting*, that is each iteration $t \in [1, T]$ of the loop 3–8, a new base classifier $h_t \in H$ is selected that minimizes the error on the training sample weighted by the

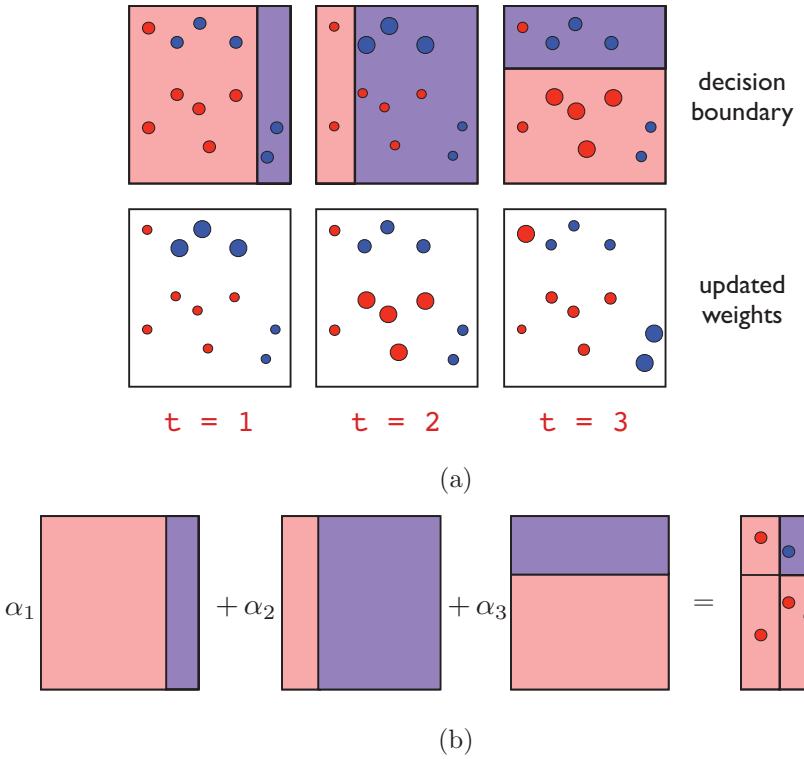


Figure 6.2 Example of AdaBoost with axis-aligned hyperplanes as base learners. (a) The top row shows decision boundaries at each boosting round. The bottom row shows how weights are updated at each round, with incorrectly (resp., correctly) points given increased (resp., decreased) weights. (b) Visualization of final classifier, constructed as a linear combination of base learners.

distribution D_t:

$$h_t \in \operatorname*{argmin}_{h \in H} \Pr_{i \sim D_t} [h_t(x_i) \neq y_i] = \operatorname*{argmin}_{h \in H} \sum_{i=1}^{m} D_t(i) 1_{h(x_i) \neq y_i}.$$

Z_t is simply a normalization factor to ensure that the weights $D_{t+1}(i)$ sum to one. The precise reason for the definition of the coefficient α_t will become clear later. For now, observe that if ϵ_t, the error of the base classifier, is less than $1/2$, then $\frac{1-\epsilon_t}{\epsilon_t} > 1$ and $\alpha_t > 0$. Thus, the new distribution D_{t+1} is defined from D_t by substantially increasing the weight on i if point x_i is incorrectly classified ($y_i h_t(x_i) < 0$), and, on the contrary, decreasing it if x_i is correctly classified. This has the effect of focusing more on the points incorrectly classified at the next round of boosting, less on those correctly classified by h_t.

After T rounds of boosting, the classifier returned by AdaBoost is based on the sign of function g, which is a linear combination of the base classifiers h_t. The weight α_t assigned to h_t in that sum is a logarithmic function of the ratio of the accuracy $1 - \epsilon_t$ and error ϵ_t of h_t. Thus, more accurate base classifiers are assigned a larger weight in that sum. Figure 6.2 illustrates the AdaBoost algorithm. The size of the points represents the distribution weight assigned to them at each boosting round.

For any $t \in [1, T]$, we will denote by g_t the linear combination of the base classifiers after t rounds of boosting: $f_t = \sum_{s=1}^{t} \alpha_t h_t$. In particular, we have $g_T = g$. The distribution D_{t+1} can be expressed in terms of g_t and the normalization factors Z_s, $s \in [1, t]$, as follows:

$$\forall i \in [1, m], \quad D_{t+1}(i) = \frac{e^{-y_i g_t(x_i)}}{m \prod_{s=1}^{t} Z_s}. \tag{6.2}$$

We will make use of this identity several times in the proofs of the following sections. It can be shown straightforwardly by repeatedly expanding the definition of the distribution over the point x_i:

$$\begin{aligned} D_{t+1}(i) = \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)}}{Z_t} &= \frac{D_{t-1}(i) e^{-\alpha_{t-1} y_i h_{t-1}(x_i)} e^{-\alpha_t y_i h_t(x_i)}}{Z_{t-1} Z_t} \\ &= \frac{e^{-y_i \sum_{s=1}^{t} \alpha_s h_s(x_i)}}{m \prod_{s=1}^{t} Z_s}. \end{aligned}$$

The AdaBoost algorithm can be generalized in several ways:

- instead of a hypothesis with minimal weighted error, h_t can be more generally the base classifier returned by a weak learning algorithm trained on D_t;
- the range of the base classifiers could be $[-1, +1]$, or more generally $\mathbb{R}$. The coefficients α_t can then be different and may not even admit a closed form. In general, they are chosen to minimize an upper bound on the empirical error, as discussed in the next section. Of course, in that general case, the hypothesis h_t are not binary *classifiers*, but the sign of their values could indicate the label, and their magnitude could be interpreted as a measure of confidence.

In the remainder of this section, we will further analyze the properties of AdaBoost and discuss its typical use in practice.

6.2.1 Bound on the empirical error

We first show that the empirical error of AdaBoost decreases exponentially fast as a function of the number of rounds of boosting.

Theorem 6.1
The empirical error of the classifier returned by AdaBoost verifies:

$$\widehat{R}(h) \leq \exp\Big[-2\sum_{t=1}^{T}\Big(\frac{1}{2}-\epsilon_t\Big)^2\Big]. \tag{6.3}$$

Furthermore, if for all $t \in [1,T]$, $\gamma \leq (\frac{1}{2}-\epsilon_t)$, *then*

$$\widehat{R}(h) \leq \exp(-2\gamma^2 T)\,. \tag{6.4}$$

Proof Using the general inequality $1_{u\leq 0} \leq \exp(-u)$ valid for all $u \in \mathbb{R}$ and identity 6.2, we can write:

$$\widehat{R}(h) = \frac{1}{m}\sum_{i=1}^{m} 1_{y_i g(x_i)\leq 0} \leq \frac{1}{m}\sum_{i=1}^{m} e^{-y_i g(x_i)} = \frac{1}{m}\sum_{i=1}^{m}\Big[m\prod_{t=1}^{T} Z_t\Big] D_{T+1}(i) = \prod_{t=1}^{T} Z_t.$$

Since, for all $t \in [1,T]$, Z_t is a normalization factor, it can be expressed in terms of ϵ_t by:

$$\begin{aligned} Z_t = \sum_{i=1}^{m} D_t(i)e^{-\alpha_t y_i h_t(x_i)} &= \sum_{i:y_i h_t(x_i)=+1} D_t(i)e^{-\alpha_t} + \sum_{i:y_i h_t(x_i)=-1} D_t(i)e^{\alpha_t} \\ &= (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t} \\ &= (1-\epsilon_t)\sqrt{\frac{\epsilon_t}{1-\epsilon_t}} + \epsilon_t\sqrt{\frac{1-\epsilon_t}{\epsilon_t}} = 2\sqrt{\epsilon_t(1-\epsilon_t)}\,. \end{aligned}$$

Thus, the product of the normalization factors can be expressed and upper bounded as follows:

$$\begin{aligned} \prod_{t=1}^{T} Z_t = \prod_{t=1}^{T} 2\sqrt{\epsilon_t(1-\epsilon_t)} = \prod_{t=1}^{T}\sqrt{1-4\big(\tfrac{1}{2}-\epsilon_t\big)^2} &\leq \prod_{t=1}^{T}\exp\Big[-2\big(\tfrac{1}{2}-\epsilon_t\big)^2\Big] \\ &= \exp\Big[-2\sum_{t=1}^{T}\big(\tfrac{1}{2}-\epsilon_t\big)^2\Big], \end{aligned}$$

where the inequality follows from the identity $1-x \leq e^{-x}$ valid for all $x \in \mathbb{R}$. ■

Note that the value of γ, which is known as the *edge*, and the accuracy of the base classifiers do not need to be known to the algorithm. The algorithm adapts to their accuracy and defines a solution based on these values. This is the source of the extended name of AdaBoost: *adaptive boosting*.

The proof of theorem 6.1 reveals several other important properties. First, observe that α_t is the minimizer of the function $g\colon \alpha \mapsto (1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}$. Indeed, g is

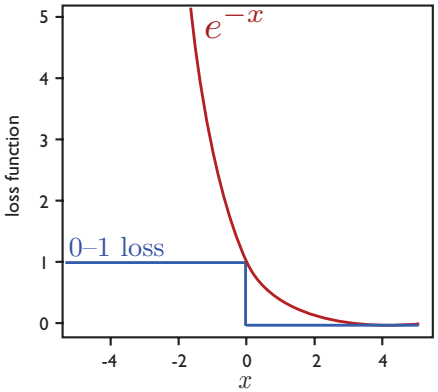


Figure 6.3 Visualization of the zero-one loss (blue) and the convex and differentiable upper bound on the zero-one loss (red) that is optimized by AdaBoost.

convex and differentiable, and setting its derivative to zero yields:

$$g'(\alpha) = -(1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha} = 0 \Leftrightarrow (1-\epsilon_t)e^{-\alpha} = \epsilon_t e^{\alpha} \Leftrightarrow \alpha = \frac{1}{2}\log\frac{1-\epsilon_t}{\epsilon_t}. \quad (6.5)$$

Thus, α_t is chosen to minimize $Z_t = g(\alpha_t)$, and in light of the bound $\widehat{R}(h) \leq \prod_{t=1}^{T} Z_t$ shown in the proof, these coefficients are selected to minimize an upper bound on the empirical error. In fact, for base classifiers whose range is $[-1,+1]$ or $\mathbb{R}$, α_t can be chosen in a similar fashion to minimize Z_t, and this is the way AdaBoost is extended to these more general cases.

Observe also that the equality $(1-\epsilon_t)e^{-\alpha_t} = \epsilon_t e^{\alpha_t}$ just shown in (6.5) implies that at each iteration, AdaBoost assigns equal distribution mass to correctly and incorrectly classified instances, since $(1-\epsilon_t)e^{-\alpha_t}$ is the total distribution assigned to correctly classified points and $\epsilon_t e^{\alpha_t}$ that of incorrectly classified ones. This may seem to contradict the fact that AdaBoost increases the weights of incorrectly classified points and decreases that of others, but there is in fact no inconsistency: the reason is that there are always fewer incorrectly classified points, since the base classifier's accuracy is better than random.

6.2.2 Relationship with coordinate descent

AdaBoost was designed to address a novel theoretical question, that of designing a strong learning algorithm using a weak learning algorithm. We will show, however, that it coincides in fact with a very simple and classical algorithm, which consists of applying a coordinate descent technique to a convex and differentiable objective function. The objective function F for AdaBoost is defined for all samples $S =$

$((x_1, y_1), \ldots, (x_m, y_m))$ and $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n$, $n \geq 1$, by

$$F(\boldsymbol{\alpha}) = \sum_{i=1}^{m} e^{-y_i g_n(x_i)} = \sum_{i=1}^{m} e^{-y_i \sum_{t=1}^{n} \alpha_t h_t(x_i)}, \tag{6.6}$$

where $g_n = \sum_{t=1}^{n} \alpha_t h_t$. This function is an upper bound on the zero-one loss function we wish to minimize, as shown in figure 6.3. Let $\mathbf{e}_t$ denote the unit vector corresponding to the tth coordinate in $\mathbb{R}^n$ and let $\boldsymbol{\alpha}_{t-1}$ denote the vector based on the $(t-1)$ first coefficients, i.e. $\boldsymbol{\alpha}_{t-1} = (\alpha_1, \ldots, \alpha_{t-1}, 0, \ldots, 0)^\top$ if $t - 1 > 0$, $\boldsymbol{\alpha}_{t-1} = \mathbf{0}$ otherwise. At each iteration $t \geq 1$, the direction $\mathbf{e}_t$ selected by coordinate descent is the one minimizing the directional derivative:

$$\mathbf{e}_t = \operatorname*{argmin}_t \left. \frac{dF(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t)}{d\eta} \right|_{\eta=0}.$$

Since $F(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t) = \sum_{i=1}^{m} e^{-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) - y_i \eta h_t(x_i)}$, the directional derivative along $\mathbf{e}_t$ can be expressed as follows:

$$\begin{aligned}
\left. \frac{dF(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t)}{d\eta} \right|_{\eta=0} &= -\sum_{i=1}^{m} y_i h_t(x_i) \exp\Big[- y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) \Big] \\
&= -\sum_{i=1}^{m} y_i h_t(x_i) D_t(i) \Big[m \prod_{s=1}^{t-1} Z_s \Big] \\
&= -\Big[\sum_{i: y_i h_t(x_i) = +1} D_t(i) - \sum_{i: y_i h_t(x_i) = -1} D_t(i) \Big] \Big[m \prod_{s=1}^{t-1} Z_s \Big] \\
&= -[(1 - \epsilon_t) - \epsilon_t] \Big[m \prod_{s=1}^{t-1} Z_s \Big] = [2\epsilon_t - 1] \Big[m \prod_{s=1}^{t-1} Z_s \Big].
\end{aligned}$$

The first equality holds by differentiation and evaluation at $\eta = 0$, and the second one follows from (6.2). The third equality divides the sample set into points correctly and incorrectly classified by h_t, and the fourth equality uses the definition of ϵ_t. In view of the final equality, since $m \prod_{s=1}^{t-1} Z_s$ is fixed, the direction $\mathbf{e}_t$ selected by coordinate descent is the one minimizing ϵ_t, which corresponds exactly to the base learner h_t selected by AdaBoost.

The step size η is identified by setting the derivative to zero in order to minimize the function in the chosen direction $\mathbf{e}_t$. Thus, using identity 6.2 and the definition

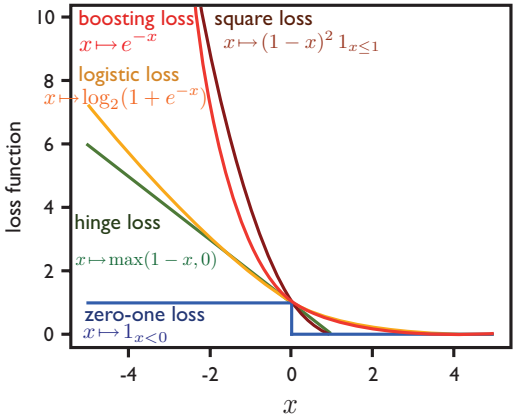


Figure 6.4 Examples of several convex upper bounds on the zero-one loss.

of ϵ_t, we can write:

$$\begin{aligned}
\frac{dF(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t)}{d\eta} = 0 &\Leftrightarrow -\sum_{i=1}^{m} y_i h_t(x_i) \exp\Big[-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i)\Big] e^{-\eta y_i h_t(x_i)} = 0 \\
&\Leftrightarrow -\sum_{i=1}^{m} y_i h_t(x_i) D_t(i) \Big[m \prod_{s=1}^{t-1} Z_s\Big] e^{-y_i h_t(x_i)\eta} = 0 \\
&\Leftrightarrow -\sum_{i=1}^{m} y_i h_t(x_i) D_t(i) e^{-y_i h_t(x_i)\eta} = 0 \\
&\Leftrightarrow -[(1-\epsilon_t)e^{-\eta} - \epsilon_t e^{\eta}] = 0 \\
&\Leftrightarrow \eta = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}.
\end{aligned}$$

This proves that the step size chosen by coordinate descent matches the base classifier weight α_t of AdaBoost. Thus, coordinate descent applied to F precisely coincides with the AdaBoost algorithm.

In light of this relationship, one may wish to consider similar applications of coordinate descent to other convex and differentiable functions of $\boldsymbol{\alpha}$ upper-bounding the zero-one loss. In particular, the *logistic loss* $x \mapsto \log_2(1 + e^{-x})$ is convex and differentiable and upper bounds the zero-one loss. Figure 6.4 shows other examples of alternative convex loss functions upper-bounding the zero-one loss. Using the logistic loss, instead of the exponential loss used by AdaBoost, leads to an algorithm that coincides with *logistic regression*.

6.2.3 Relationship with logistic regression

Logistic regression is a widely used binary classification algorithm, a specific instance of conditional maximum entropy models. For the purpose of this chapter, we first give a very brief description of the algorithm. Logistic regression returns a conditional probability distribution of the form

$$p_{\boldsymbol{\alpha}}[y|x] = \frac{1}{Z(x)} \exp(y\, \boldsymbol{\alpha} \cdot \boldsymbol{\Phi}(x)), \tag{6.7}$$

where $y \in \{-1, +1\}$ is the label predicted for $x \in \mathcal{X}$, $\boldsymbol{\Phi}(x) \in \mathbb{R}^N$ is a feature vector associated to x, $\boldsymbol{\alpha} \in \mathbb{R}^N$ a parameter weight vector, and $Z(x) = e^{+\boldsymbol{\alpha}\cdot\boldsymbol{\Phi}(x)} + e^{-\boldsymbol{\alpha}\cdot\boldsymbol{\Phi}(x)}$ a normalization factor. Dividing both the numerator and denominator by $e^{+\boldsymbol{\alpha}\cdot\boldsymbol{\Phi}(x)}$ and taking the log leads to:

$$\log(p_{\boldsymbol{\alpha}}[y|x]) = -\log(1 + e^{-2y\boldsymbol{\alpha}\cdot\Phi(x)}). \tag{6.8}$$

The parameter $\boldsymbol{\alpha}$ is learned via *maximum likelihood* by logistic regression, that is, by maximizing the probability of the sample $S = ((x_1, y_s), \ldots, (x_m, y_m))$. Since the points are sampled i.i.d., this can be written as follows: $\operatorname{argmax}_{\boldsymbol{\alpha}} \prod_{i=1}^{m} p_{\boldsymbol{\alpha}}[y_i|x_i]$. Taking the negative log of the probabilities shows that the objective function minimized by logistic regression is

$$G(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \log(1 + e^{-2y_i \boldsymbol{\alpha}\cdot\Phi(x_i)})\,. \tag{6.9}$$

Thus, modulo constants, which do not affect the solution sought, the objective function coincides with the one based on the logistic loss. AdaBoost and Logistic regression have in fact many other relationships that we will not discuss in detail here. In particular, it can be shown that both algorithms solve exactly the same optimization problem, except for a normalization constraint required for logistic regression not imposed in the case of AdaBoost.

6.2.4 Standard use in practice

Here we briefly describe the practical use of AdaBoost. An important requirement for the algorithm is the choice of the base classifiers or that of the weak learner. The family of base classifiers typically used with AdaBoost in practice is that of *decision trees*, which are equivalent to hierarchical partitions of the space (see chapter 8, section 8.3.3). In fact, more precisely, decision trees of depth one, also known as *stumps* or *boosting stumps* are by far the most frequently used base classifiers.

Boosting stumps are threshold functions associated to a single feature. Thus, a stump corresponds to a single axis-aligned partition of the space, as illustrated

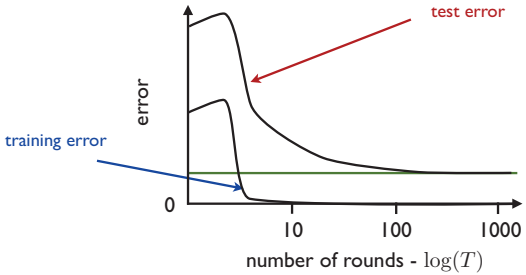


Figure 6.5 An empirical result using AdaBoost with C4.5 decision trees as base learners. In this example, the training error goes to zero after about 5 rounds of boosting ($T \approx 5$), yet the test error continues to decrease for larger values of T.

in figure 6.2. If the data is in $\mathbb{R}^N$, we can associate a stump to each of the N components. Thus, to determine the stump with the minimal weighted error at each of round of boosting, the best component and the best threshold for each component must be computed.

To do so, we can first presort each component in $O(m \log m)$ time with a total computational cost of $O(mN \log m)$. For a given component, there are only $m + 1$ possible distinct thresholds, since two thresholds between the same consecutive component values are equivalent. To find the best threshold at each round of boosting, all of these possible $m + 1$ values can be compared, which can be done in $O(m)$ time. Thus, the total computational complexity of the algorithm for T rounds of boosting is $O(mN \log m + mNT)$.

Observe, however, that while boosting stumps are widely used in combination with AdaBoost and can perform well in practice, the algorithm that returns the stump with the minimal empirical error is *not a weak learner* (see definition 6.1)! Consider, for example, the simple XOR example with four data points lying in $\mathbb{R}^2$ (see figure 5.2a), where points in the second and fourth quadrants are labeled positively and those in the first and third quadrants negatively. Then, no decision stump can achieve an accuracy better than 1/2.

6.3 Theoretical results

In this section we present a theoretical analysis of the generalization properties of AdaBoost.

6.3.1 VC-dimension-based analysis

We start with an analysis of AdaBoost based on the VC-dimension of its hypothesis set. For T rounds of boosting, its hypothesis set is

$$\mathcal{F}_T = \left\{ \operatorname{sgn}\left(\sum_{t=1}^{T} \alpha_t h_t \right) : \alpha_t \in \mathbb{R}, h_t \in H, t \in [1,T] \right\}. \tag{6.10}$$

The VC-dimension of $\mathcal{F}_T$ can be bounded as follows in terms of the VC-dimension d of the family of base hypothesis H (exercise 6.1):

$$\operatorname{VCdim}(\mathcal{F}_T) \leq 2(d+1)(T+1)\log_2((T+1)e)\,. \tag{6.11}$$

The upper bound grows as $O(dT \log T)$, thus the bound suggests that AdaBoost could overfit for large values of T, and indeed this can occur. However, in many cases, it has been observed empirically that the generalization error of AdaBoost decreases as a function of the number of rounds of boosting T, as illustrated in figure 6.5! How can these empirical results be explained? The following sections present an analysis based on a concept of margin, similar to the one presented for SVMs.

6.3.2 Margin-based analysis

In chapter 4 we gave a definition of margin for linear classifiers such as SVMs (definition 4.2). Here we will need a somewhat different but related definition of margin for linear combinations of base classifiers, as in the case of AdaBoost.

First note that a linear combination of base classifiers $g = \sum_{t=1}^{T} \alpha_t h_t$ can be defined equivalently via $g(x) = \boldsymbol{\alpha} \cdot \mathbf{h}(x)$ for all $x \in \mathcal{X}$, with $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_T)^\top$ and $\mathbf{h}(x) = [h_1(x), \ldots, h_T(x)]^\top$. This makes their similarity with the linear hypotheses considered in chapter 4 and chapter 5 evident: $\mathbf{h}(x)$ is the feature vector associated to x, which was previously denoted by $\Phi(x)$, and $\boldsymbol{\alpha}$ is the weight vector that was denoted by $\mathbf{w}$. The base classifiers values $h_t(x)$ are the components of the feature vector associated to x. For AdaBoost, additionally, the weight vector is non-negative: $\boldsymbol{\alpha} \geq 0$.

We will use the same notation to introduce the following definition.

Definition 6.2 **L_1-margin**
The L_1-margin $\rho(x)$ *of a point* $x \in \mathcal{X}$ *with label* $y \in \{-1, +1\}$ *for a linear combination of base classifiers* $g = \sum_{t=1}^{T} \alpha_t h_t$ *with* $\boldsymbol{\alpha} \neq 0$ *and* $h_t \in H$ *for all* $t \in [1,T]$ *is defined as*

$$\rho(x) = \frac{yg(x)}{\sum_{t=T}^{m} |\alpha_t|} = \frac{y \sum_{t=1}^{T} \alpha_t h_t(x)}{\|\boldsymbol{\alpha}\|_1} = y \frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x)}{\|\boldsymbol{\alpha}\|_1}. \tag{6.12}$$

The L_1-margin *of a linear combination classifier* g *with respect to a sample* $S = (x_1, \ldots, x_m)$ *is the minimum margin of the points within the sample:*

$$\rho = \min_{i \in [1,m]} y_i \frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)}{\|\boldsymbol{\alpha}\|_1}. \tag{6.13}$$

When the coefficients α_t are non-negative, as in the case of AdaBoost, $\rho(x)$ is a convex combination of the base classifier values $h_t(x)$. In particular, if the base classifiers h_t take values in $[-1, +1]$, then $\rho(x)$ is in $[-1, +1]$. The absolute value $|\rho(x)|$ can be interpreted as the confidence of the classifier g in that label.

This definition of margin differs from definition 4.2 given for linear classifiers only by the norm used for the weight vector: L_1 norm here, L_2 norm in definition 4.2. Indeed, in the case of a linear hypothesis $x \mapsto \mathbf{w} \cdot \Phi(x)$, the margin for point x with label y was defined as follows:

$$\rho(x) = y \frac{\mathbf{w} \cdot \phi(x)}{\|\mathbf{w}\|_2}$$

and was based on the L_2 norm of $\mathbf{w}$. When the prediction is correct, that is $y(\boldsymbol{\alpha} \cdot \mathbf{h}(x)) \geq 0$, the L_1-margin and L_2 margin of definition 4.2 can be rewritten as

$$\rho_1(x) = \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_1} \quad \text{and} \quad \rho_2(x) = \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_2}.$$

It is known that for $p, q \geq 1$, p and q *conjugate*, i.e. $1/p + 1/q = 1$, that $|\boldsymbol{\alpha} \cdot \mathbf{x}|/\|\boldsymbol{\alpha}\|_p$ is the L_q distance of $\mathbf{x}$ to the hyperplane of equation $\boldsymbol{\alpha} \cdot \mathbf{x} = 0$. Thus, both $\rho_1(x)$ and $\rho_2(x)$ measure the distance of the feature vector $\mathbf{h}(x)$ to the hyperplane $\boldsymbol{\alpha} \cdot \mathbf{x} = 0$ in $\mathbb{R}^T$, $\rho_1(x)$ its $\|\cdot\|_\infty$ distance, $\rho_2(x)$ its $\|\cdot\|_2$ or Euclidean distance (see figure 6.6).

To examine the generalization properties of AdaBoost, we first analyze the Rademacher complexity of convex combinations of hypotheses such as those defined by AdaBoost. Next, we use the margin-based analysis from chapter 4 to derive a margin-based generalization bound for boosting with the definition of margin just introduced.

For any hypothesis set H of real-valued functions, we denote by $\text{conv}(H)$ its convex hull defined by

$$\text{conv}(H) = \left\{ \sum_{k=1}^{p} \mu_k h_k \colon p \geq 1, \forall k \in [1,p], \mu_k \geq 0, h_k \in H, \sum_{k=1}^{p} \mu_k \leq 1 \right\}. \tag{6.14}$$

The following theorem shows that, remarkably, the empirical Rademacher complexity of $\text{conv}(H)$, which in general is a strictly larger set including H, coincides with that of H.

Theorem 6.2
Let H be a set of functions mapping from $\mathcal{X}$ to $\mathbb{R}$. Then, for any sample S, we have

$$\widehat{\mathfrak{R}}_S(\text{conv}(H)) = \widehat{\mathfrak{R}}_S(H)\,.$$

Proof The proof follows from a straightforward series of equalities:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\text{conv}(H)) &= \frac{1}{m}\,\mathop{\mathrm{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in H,\boldsymbol{\mu}\geq 0,\|\boldsymbol{\mu}\|_1\leq 1}\sum_{i=1}^{m}\sigma_i\sum_{k=1}^{p}\mu_k h_k(x_i)\right]\\
&= \frac{1}{m}\,\mathop{\mathrm{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in H}\sup_{\boldsymbol{\mu}\geq 0,\|\boldsymbol{\mu}\|_1\leq 1}\sum_{k=1}^{p}\mu_k\Big(\sum_{i=1}^{m}\sigma_i h_k(x_i)\Big)\right]\\
&= \frac{1}{m}\,\mathop{\mathrm{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in H}\max_{k\in[1,p]}\Big(\sum_{i=1}^{m}\sigma_i h_k(x_i)\Big)\right]\\
&= \frac{1}{m}\,\mathop{\mathrm{E}}_{\sigma}\left[\sup_{h\in H}\sum_{i=1}^{m}\sigma_i h(x_i)\right] = \widehat{\mathfrak{R}}_S(H).
\end{aligned}$$

The main equality to recognize is the third one, which is based on the observation that the maximizing vector $\boldsymbol{\mu}$ for the convex combination of p terms is the one placing all the weight on the largest term of the sum. ■

This theorem can be used directly in combination with theorem 4.4 to derive the following Rademacher complexity generalization bound for convex combination ensembles of hypotheses.

Corollary 6.1 **Ensemble Rademacher margin bound**
Let H denote a set of real-valued functions. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1-\delta$, each of the following holds for all $h \in \text{conv}(H)$:

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2}{\rho}\mathfrak{R}_m\big(H\big) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{6.15}$$

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2}{\rho}\widehat{\mathfrak{R}}_S\big(H\big) + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{6.16}$$

Using corollary 3.1 and corollary 3.3 to bound the Rademacher complexity in terms of the VC-dimension yields immediately the following VC-dimension-based generalization bounds for convex combination ensembles of hypotheses.

Corollary 6.2 **Ensemble VC-Dimension margin bound**
Let H be a family of functions taking values in $\{+1,-1\}$ with VC-dimension d. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1-\delta$, the following holds for

all $h \in \operatorname{conv}(H)$*:*

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2}{\rho}\sqrt{\frac{2d\log\frac{em}{d}}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}\,. \tag{6.17}$$

These bounds can be generalized to hold uniformly for all $\rho > 0$, instead of a fixed ρ, at the price of an additional term of the form $\sqrt{(\log\log_2\frac{2}{\delta})/m}$ as in theorem 4.5. They cannot be directly applied to the linear combination g generated by AdaBoost, since it is not a convex combination of base hypotheses, but they can be applied to the following normalized version of g:

$$x \mapsto \frac{g(x)}{\|\boldsymbol{\alpha}\|_1} = \frac{\sum_{t=1}^T \alpha_t h_t(x)}{\|\boldsymbol{\alpha}\|_1} \in \operatorname{conv}(H)\,. \tag{6.18}$$

Note that from the point of view of binary classification, g and $g/\|\boldsymbol{\alpha}\|_1$ are equivalent since $\operatorname{sgn}(g) = \operatorname{sgn}(g/\|\boldsymbol{\alpha}\|_1)$, thus $R(g) = R(g/\|\boldsymbol{\alpha}\|_1)$, but their empirical margin loss are distinct. Let $g = \sum_{t=1}^T \alpha_t h_t$ denote the function defining the classifier returned by AdaBoost after T rounds of boosting when trained on sample S. Then, in view of (6.15), for any $\delta > 0$, the following holds with probability at least $1 - \delta$:

$$R(g) \leq \widehat{R}_\rho(g/\|\boldsymbol{\alpha}\|_1) + \frac{2}{\rho}\mathfrak{R}_m\big(H\big) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}. \tag{6.19}$$

Similar bounds can be derived from (6.16) and (6.17). Remarkably, the number of rounds of boosting T does not appear in the generalization bound (6.19). The bound depends only on the margin ρ, the sample size m, and the Rademacher complexity of the family of base classifiers H. Thus, the bound guarantees an effective generalization if the margin loss $\widehat{R}_\rho(g/\|\boldsymbol{\alpha}\|_1)$ is small for a relatively large ρ. Recall that the margin loss can be upper bounded by the fraction of the points x in the training sample with $g(x)/\|\boldsymbol{\alpha}\|_1 \geq \rho$ (see (4.39)). Thus, with our definition of L_1-margin, it can be bounded by the fraction of the points in S with L_1-margin more than ρ:

$$\widehat{R}_\rho(g/\|\boldsymbol{\alpha}\|_1) \leq \frac{|\{i \in [1,m] \colon \rho(x_i) \geq \rho\}|}{m}. \tag{6.20}$$

Additionally, the following theorem provides a bound on the empirical margin loss, which decreases with T under conditions discussed later.

Theorem 6.3

Let $g = \sum_{t=1}^T \alpha_t h_t$ *denote the function defining the classifier returned by AdaBoost after* T *rounds of boosting and assume for all* $t \in [1,T]$ *that* $\epsilon_t < \frac{1}{2}$*, which implies*

$a_t > 0$. *Then, for any* $\rho > 0$, *the following holds:*

$$\widehat{R}_\rho\left(\frac{g}{\|\boldsymbol{\alpha}\|_1}\right) \leq 2^T \prod_{t=1}^{T} \sqrt{\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}}\,.$$

Proof Using the general inequality $1_{u\leq 0} \leq \exp(-u)$ valid for all $u \in \mathbb{R}$, identity 6.2, that is $D_{t+1}(i) = \frac{e^{-y_i g(x_i)}}{m\prod_{t=1}^{T} Z_t}$, the equality $Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)}$ from the proof of theorem 6.1, and the definition of $\boldsymbol{\alpha}$ in AdaBoost, we can write:

$$\begin{aligned}
\frac{1}{m}\sum_{i=1}^{m} 1_{y_i g(x_i)-\rho\|\boldsymbol{\alpha}\|_1\leq 0} &\leq \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i g(x_i) + \rho\|\boldsymbol{\alpha}\|_1) \\
&= \frac{1}{m}\sum_{i=1}^{m} e^{\rho\|\boldsymbol{\alpha}\|_1}\left[m\prod_{t=1}^{T} Z_t\right] D_{T+1}(i) \\
&= e^{\rho\|\boldsymbol{\alpha}\|_1}\prod_{t=1}^{T} Z_t = e^{\rho\sum_i \alpha_i}\prod_{t=1}^{T} Z_t \\
&= 2^T \prod_{t=1}^{T}\left[\sqrt{\tfrac{1-\epsilon_t}{\epsilon_t}}\right]^{\rho}\sqrt{\epsilon_t(1-\epsilon_t)}\,,
\end{aligned}$$

which concludes the proof. ■

Moreover, if for all $t \in [1, T]$ we have $\gamma \leq (\frac{1}{2} - \epsilon_t)$ and $\rho \leq 2\gamma$, then the expression $4\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}$ is maximized at $\epsilon_t = \frac{1}{2}-\gamma$.[1] Thus, the upper bound on the empirical margin loss can then be bounded by

$$\widehat{R}_\rho\left(\frac{g}{\|\boldsymbol{\alpha}\|_1}\right) \leq \left[(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho}\right]^{T/2}. \tag{6.21}$$

Observe that $(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} = (1-4\gamma^2)\left(\frac{1+2\gamma}{1-2\gamma}\right)^{\rho}$. This is an increasing function of ρ since we have $\left(\frac{1+2\gamma}{1-2\gamma}\right) > 1$ as a consequence of $\gamma > 0$. Thus, if $\rho < \gamma$, it can be strictly upper bounded as follows

$$(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} < (1-2\gamma)^{1-\gamma}(1+2\gamma)^{1+\gamma}.$$

The function $\gamma \mapsto (1-2\gamma)^{1-\gamma}(1+2\gamma)^{1+\gamma}$ is strictly upper bounded by 1 over the interval $(0, 1/2)$, thus, if $\rho < \gamma$, then $(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} < 1$ and the right-hand side of (6.21) decreases exponentially with T. Since the condition $\rho \gg O(1/\sqrt{m})$ is necessary in order for the given margin bounds to converge, this places a condition

1. The differential of $f\colon \epsilon \mapsto \log[\epsilon^{1-\rho}(1-\epsilon)^{1+\rho}] = (1-\rho)\log\epsilon + (1+\rho)\log(1-\epsilon)$ over the interval $(0,1)$ is given by $f'(\epsilon) = \frac{1-\rho}{\epsilon} - \frac{1+\rho}{1-\epsilon} = 2\frac{(\frac{1}{2}-\frac{\rho}{2})-\epsilon}{\epsilon(1-e)}$. Thus, f is an increasing function over $(0, \frac{1}{2} - \frac{\rho}{2})$, which implies that it is increasing over $(0, \frac{1}{2} - \gamma)$ when $\gamma \geq \frac{\rho}{2}$.

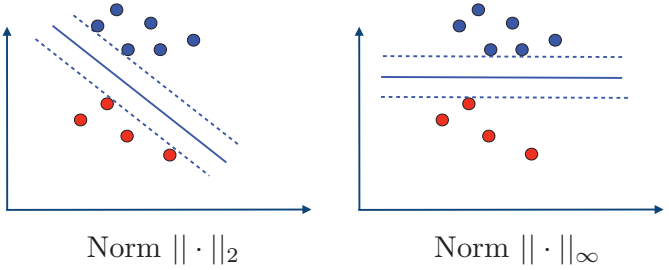


Figure 6.6 Maximum margins with respect to both the L_2 and L_∞ norm.

of $\gamma \gg O(1/\sqrt{m})$ on the edge value. In practice, the error ϵ_t of the base classifier at round t may increase as a function of t. Informally, this is because boosting presses the weak learner to concentrate on instances that are harder and harder to classify, for which even the best base classifier could not achieve an error significantly better than random. If ϵ_t becomes close to $1/2$ relatively fast as a function of t, then the bound of theorem 6.3 becomes uninformative.

The margin bounds of corollary 6.1 and corollary 6.2, combined with the bound on the empirical margin loss of theorem 6.3, suggest that under some conditions, AdaBoost can achieve a large margin on the training sample. They could also serve as a theoretical explanation of the empirical observation that in some tasks the generalization error increases as a function of T even after the error on the training sample is zero: the margin would continue to increase. But does AdaBoost maximize the L_1-margin?

No. It has been shown that AdaBoost may converge to a margin that is significantly smaller than the maximum margin (e.g., $1/3$ instead of $3/8$). However, under some general assumptions, when the data is separable and the base learners satisfy particular conditions, it has been proven that AdaBoost can asymptotically achieve a margin that is at least half the maximum margin, $\rho_{\max}/2$.

6.3.3 Margin maximization

In view of these results, several algorithms have been devised with the explicit goal of maximizing the L_1-margin. These algorithms correspond to different methods for solving a linear program (LP).

By definition of the L_1-margin, the maximum margin for a sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$ is given by

$$\rho = \max_{\boldsymbol{\alpha}} \min_{i \in [1,m]} y_i \frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)}{\|\boldsymbol{\alpha}\|_1} \,. \tag{6.22}$$

By definition of the maximization, the optimization problem can be written as:

$$\max_{\boldsymbol{\alpha}} \rho$$
$$\text{subject to : } y_i \frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)}{\|\boldsymbol{\alpha}\|_1} \geq \rho, \ \forall i \in [1, m].$$

Since $\frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)}{\|\boldsymbol{\alpha}\|_1}$ is invariant to the scaling of $\boldsymbol{\alpha}$, we can restrict ourselves to $\|\boldsymbol{\alpha}\|_1 = 1$. Further seeking a non-negative $\boldsymbol{\alpha}$ as in the case of AdaBoost leads to the following optimization:

$$\max_{\boldsymbol{\alpha}} \rho$$
$$\text{subject to : } y_i(\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)) \geq \rho, \ \forall i \in [1, m]$$
$$\Big(\sum_{t=1}^{T} \alpha_t = 1\Big) \wedge (\alpha_t \geq 0, \forall t \in [1, T]).$$

This is a linear program (LP), that is, an optimization problem with a linear objective function and linear constraints. There are several different methods for solving relative large LPs in practice, using the simplex method, interior-point methods, or a variety of special-purpose solutions.

Note that the solution of this algorithm differs from the margin-maximization defining SVMs in the separable case only by the definition of the margin used (L_1 versus L_2) and the non-negativity constraint on the weight vector. Figure 6.6 illustrates the margin-maximizing hyperplanes found using these two distinct margin definitions in a simple case. The left figure shows the SVM solution, where the distance to the closest points to the hyperplane is measured with respect to the norm $\|\cdot\|_2$. The right figure shows the solution for the L_1-margin, where the distance to the closest points to the hyperplane is measured with respect to the norm $\|\cdot\|_\infty$.

By definition, the solution of the LP just described admits an L_1-margin that is larger or equal to that of the AdaBoost solution. However, empirical results do not show a systematic benefit for the solution of the LP. In fact, it appears that in many cases, AdaBoost outperforms that algorithm. The margin theory described does not seem sufficient to explain that performance.

6.3.4 Game-theoretic interpretation

In this section, we first show that AdaBoost admits a natural game-theoretic interpretation. The application of von Neumann's theorem then helps us relate the maximum margin and the optimal edge and clarify the connection of AdaBoost's weak-learning assumption with the notion of L_1-margin. We first introduce the definition of the edge for a specific classifier and a particular distribution.

	rock	paper	scissors
rock	0	+1	-1
paper	-1	0	+1
scissors	+1	-1	0

Table 6.1 The loss matrix for the standard rock-paper-scissors game.

Definition 6.3
The edge of a base classifier h_t for a distribution D *over the training sample is defined by*

$$\gamma_t(D) = \frac{1}{2} - \epsilon_t = \frac{1}{2}\sum_{i=1}^{m} y_i h_t(x_i) D(i). \tag{6.23}$$

AdaBoost's weak learning condition can now be formulated as: there exists $\gamma > 0$ such that for any distribution D over the training sample and any base classifier h_t, the following holds:

$$\gamma_t(D) \geq \gamma. \tag{6.24}$$

This condition is required for the analysis of theorem 6.1 and the non-negativity of the coefficients α_t. We will frame boosting as a two-person zero-sum game.

Definition 6.4 **Zero-sum game**
A two-person zero-sum game *consists of a* loss matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$*, where* m *is the number of possible* actions *(or* pure strategies*) for the row player and* n *the number of possible actions for the column player. The entry* M_{ij} *is the loss for the row player (or equivalently the payoff for the column payer) when the row player takes action* i *and the column player takes action* j*.*[2]

An example of a loss matrix for the familiar "rock-paper-scissors" game is shown in table 6.1.

Definition 6.5 **Mixed strategy**
A mixed strategy *for the row player is a distribution* p *over the* m *possible row actions, a distribution* q *over the* n *possible column actions for the column player. The* expected loss *for the row player (expected payoff for the column player) with*

2. To be consistent with the results discussed in other chapters, we consider the loss matrix as opposed to the payoff matrix (its opposite).

respect to the mixed strategies p and q is

$$\mathrm{E}[loss] = \mathbf{p}^\top \mathbf{M}\mathbf{q} = \sum_{i=1}^{m}\sum_{j=1}^{n} p_i M_{ij} q_j \,.$$

The following is a fundamental result in game theory proven in chapter 7.

Theorem 6.4 **Von Neumann's minimax theorem**
For any two-person zero-sum game defined by matrix $\mathbf{M}$*,*

$$\min_{\mathbf{p}} \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M}\mathbf{q} = \max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M}\mathbf{q}\,. \tag{6.25}$$

The common value in (6.25) is called the *value of the game*. The theorem states that for any two-person zero-sum game, there exists a mixed strategy for each player such that the expected loss for one is the same as the expected payoff for the other, both of which are equal to the value of the game. Note that, given the row player's strategy, the column player can choose an optimal pure strategy, that is, the column player can choose the single strategy corresponding the smallest coordinate of the vector $\mathbf{p}^\top \mathbf{M}$. A similar comment applies to the reverse. Thus, an alternative and equivalent form of the minimax theorem is

$$\max_{\mathbf{p}} \min_{j\in[1,n]} \mathbf{p}^\top \mathbf{M}\mathbf{e}_j = \min_{\mathbf{q}} \max_{i\in[1,m]} \mathbf{e}_i^\top \mathbf{M}\mathbf{q}, \tag{6.26}$$

where $\mathbf{e}_i$ denotes the ith unit vector. We can now view AdaBoost as a zero-sum game, where an action of the row player is the selection of a training instance x_i, $i \in [1, m]$, and an action of the column player the selection of a base learner h_t, $t \in [1, T]$. A mixed strategy for the row player is thus a distribution D over the training points' indices $[1, m]$. A mixed strategy for the column player is a distribution over the based classifiers' indices $[1, T]$. This can be defined from a non-negative vector $\boldsymbol{\alpha} \geq 0$: the weight assigned to $t \in [1, T]$ is $\alpha_t / \|\boldsymbol{\alpha}\|_1$. The loss matrix $\mathbf{M} \in \{-1, +1\}^{m\times T}$ for AdaBoost is defined by $M_{it} = y_i h_t(x_i)$ for all $(i, t) \in [1, m] \times [1, T]$. By von Neumann's theorem (6.26), the following holds:

$$\min_{D\in\mathcal{D}} \max_{t\in[1,T]} \sum_{i=1}^{m} D(i) y_i h_t(x_i) = \max_{\boldsymbol{\alpha}\geq 0} \min_{i\in[1,m]} \sum_{t=1}^{T} \frac{\alpha_t}{\|\boldsymbol{\alpha}\|_1} y_i h_t(x_i), \tag{6.27}$$

where $\mathcal{D}$ denotes the set of all distributions over the training sample. Let $\rho_{\boldsymbol{\alpha}}(x)$ denote the margin of point x for the classifier defined by $g = \sum_{t=1}^{T} \alpha_t h_t$. The result can be rewritten as follows in terms of the margins and edges:

$$2\gamma^* = 2 \min_{D} \max_{t\in[1,T]} \gamma_t(D) = \max_{\boldsymbol{\alpha}} \min_{i\in[1,m]} \rho_{\boldsymbol{\alpha}}(x_i) = \rho^*, \tag{6.28}$$

where ρ^* is the maximum margin of a classifier and γ^* the best possible edge. This result has several implications. First, it shows that the weak learning condition ($\gamma^* > 0$) implies $\rho^* > 0$ and thus the existence of a classifier with positive margin, which motivates the search for a non-zero margin. AdaBoost can be viewed as an algorithm seeking to achieve such a non-zero margin, though, as discussed earlier, AdaBoost does not always achieve an optimal margin and is thus suboptimal in that respect. Furthermore, we see that the "weak learning" assumption, which originally appeared to be the weakest condition one could require for an algorithm (that of performing better than random), is in fact a strong condition: it implies that the training sample is linearly separable with margin $2\gamma^* > 0$. Linear separability often does not hold for the data sets found in practice.

6.4 Discussion

AdaBoost offers several advantages: it is simple, its implementation is straightforward, and the time complexity of each round of boosting as a function of the sample size is rather favorable. As already discussed, when using decision stumps, the time complexity of each round of boosting is in $O(mN)$. Of course, if the dimension of the feature space N is very large, then the algorithm could become in fact quite slow.

AdaBoost additionally benefits from a rich theoretical analysis. Nevertheless, there are still many theoretical questions. For example, as we saw, the algorithm in fact does not maximize the margin, and yet algorithms that do maximize the margin do not always outperform it. This suggests that perhaps a finer analysis based on a notion different from that of margin could shed more light on the properties of the algorithm.

The main drawbacks of the algorithm are the need to select the parameter T and the base classifiers, and its poor performance in the presence of noise. The choice of the number of rounds of boosting T (stopping criterion) is crucial to the performance of the algorithm. As suggested by the VC-dimension analysis, larger values of T can lead to overfitting. In practice, T is typically determined via cross-validation.

The choice of the base classifiers is also crucial. The complexity of the family of base classifiers H appeared in all the bounds presented and it is important to control it in order to guarantee generalization. On the other hand, insufficiently complex hypothesis sets could lead to low margins.

Probably the most serious disadvantage of AdaBoost is its performance in the presence of noise; it has been shown empirically that noise severely damages its accuracy. The distribution weight assigned to examples that are harder to classify substantially increases with the number of rounds of boosting, by the nature of the

algorithm. These examples end up dominating the selection of the base classifiers, which, with a large enough number of rounds, will play a detrimental role in the definition of the linear combination defined by AdaBoost.

Several solutions have been proposed to address these issues. One consists of using a "less aggressive" objective function than the exponential function of AdaBoost, such as the logistic loss, to penalize less incorrectly classified points. Another solution is based on a regularization, e.g., an L_1-regularization, which consists of adding a term to the objective function to penalize larger weights. This could be viewed as a soft margin approach for boosting. However, recent theoretical results show that boosting algorithms based on convex potentials do not tolerate even low levels of random noise, even with L_1-regularization or early stopping.

The behavior of AdaBoost in the presence of noise can be used, however, as a useful feature for detecting *outliers*, that is, examples that are incorrectly labeled or that are hard to classify. Examples with large weights after a certain number of rounds of boosting can be identified as outliers.

6.5 Chapter notes

The question of whether a weak learning algorithm could be *boosted* to derive a strong learning algorithm was first posed by Kearns and Valiant [1988, 1994], who also gave a negative proof of this result for a distribution-dependent setting. The first positive proof of this result in a distribution-independent setting was given by Schapire [1990], and later by Freund [1990].

These early boosting algorithms, boosting by filtering [Schapire, 1990] or boosting by majority [Freund, 1990, 1995] were not practical. The AdaBoost algorithm introduced by Freund and Schapire [1997] solved several of these practical issues. Freund and Schapire [1997] further gave a detailed presentation and analysis of the algorithm including the bound on its empirical error, a VC-dimension analysis, and its applications to multi-class classification and regression.

Early experiments with AdaBoost were carried out by Drucker, Schapire, and Simard [1993], who gave the first implementation in OCR with weak learners based on neural networks and Drucker and Cortes [1995], who reported the empirical performance of AdaBoost combined with decision trees, in particular decision stumps.

The fact that AdaBoost coincides with coordinate descent applied to an exponential objective function was later shown by Duffy and Helmbold [1999], Mason et al. [1999], and Friedman [2000]. Friedman, Hastie, and Tibshirani [2000] also gave an interpretation of boosting in terms of additive models. They also pointed out the close connections between AdaBoost and logistic regression, in particular

the fact that their objective functions have a similar behavior near zero or the fact that their expectation admit the same minimizer, and derived an alternative boosting algorithm, LogitBoost, based on the logistic loss. Lafferty [1999] showed how an incremental family of algorithms, including LogitBoost, can be derived from Bregman divergences and designed to closely approximate AdaBoost when varying a parameter. Kivinen and Warmuth [1999] observed that boosting can be viewed as a type of entropy projection. Collins, Schapire, and Singer [2002] later showed that boosting and logistic regression were special instances of a common framework based on Bregman divergences and used that to give the first convergence proof of AdaBoost. Probably the most direct relationship between AdaBoost and logistic regression is the proof by Lebanon and Lafferty [2001] that the two algorithms minimize the same extended relative entropy objective function subject to the same feature constraints, except from an additional normalization constraint for logistic regression.

A margin-based analysis of AdaBoost was first presented by Schapire, Freund, Bartlett, and Lee [1997], including theorem 6.3 which gives a bound on the empirical margin loss. Our presentation is based on the elegant derivation of margin bounds by Koltchinskii and Panchenko [2002] using the notion of Rademacher complexity. Rudin et al. [2004] gave an example showing that, in general, AdaBoost does not maximize the L_1-margin. Rätsch and Warmuth [2002] provided asymptotic lower bounds for the margin achieved by AdaBoost under some conditions. The L_1-margin maximization based on a LP is due to Grove and Schuurmans [1998]. The game-theoretic interpretation of boosting and the application of von Neumann's minimax theorem [von Neumann, 1928] in that context were pointed out by Freund and Schapire [1996, 1999b]; see also Grove and Schuurmans [1998], Breiman [1999].

Dietterich [2000] provided extensive empirical evidence for the fact that noise can severely damage the accuracy of AdaBoost. This has been reported by a number of other authors since then. Rätsch, Onoda, and Müller [2001] suggested the use of a soft margin for AdaBoost based on a regularization of the objective function and pointed out its connections with SVMs. Long and Servedio [2010] recently showed the failure of boosting algorithms based on convex potentials to tolerate random noise, even with L_1-regularization or early stopping.

There are several excellent surveys and tutorials related to boosting [Schapire, 2003, Meir and Rätsch, 2002, Meir and Rätsch, 2003].

6.6 Exercises

6.1 VC-dimension of the hypothesis set of AdaBoost.

Prove the upper bound on the VC-dimension of the hypothesis set $\mathcal{F}_T$ of AdaBoost

after T rounds of boosting, as stated in equation 6.11.

6.2 Alternative objective functions.

This problem studies boosting-type algorithms defined with objective functions different from that of AdaBoost. We assume that the training data are given as m labeled examples $(x_1, y_1), \ldots, (x_m, y_m) \in X \times \{-1, +1\}$. We further assume that Φ is a strictly increasing convex and differentiable function over $\mathbb{R}$ such that: $\forall x \geq 0, \Phi(x) \geq 1$ and $\forall x < 0, \Phi(x) > 0$.

(a) Consider the loss function $L(\alpha) = \sum_{i=1}^{m} \Phi(-y_i g(x_i))$ where g is a linear combination of base classifiers, i.e., $g = \sum_{t=1}^{T} \alpha_t h_t$ (as in AdaBoost). Derive a new boosting algorithm using the objective function L. In particular, characterize the best base classifier h_u to select at each round of boosting if we use coordinate descent.

(b) Consider the following functions: (1) zero-one loss $\Phi_1(-u) = 1_{u \leq 0}$; (2) least squared loss $\Phi_2(-u) = (1-u)^2$; (3) SVM loss $\Phi_3(-u) = \max\{0, 1-u\}$; and (4) logistic loss $\Phi_4(-u) = \log(1 + e^{-u})$. Which functions satisfy the assumptions on Φ stated earlier in this problem?

(c) For each loss function satisfying these assumptions, derive the corresponding boosting algorithm. How do the algorithm(s) differ from AdaBoost?

6.3 Update guarantee. Assume that the main weak learner assumption of AdaBoost holds. Let h_t be the base learner selected at round t. Show that the base learner h_{t+1} selected at round $t+1$ must be different from h_t.

6.4 Weighted instances. Let the training sample be $S = ((x_1, y_1), \ldots, (x_m, y_m))$. Suppose we wish to penalize differently errors made on x_i versus x_j. To do that, we associate some non-negative importance weight w_i to each point x_i and define the objective function $F(\boldsymbol{\alpha}) = \sum_{i=1}^{m} w_i e^{-y_i g(x_i)}$, where $g = \sum_{t=1}^{T} \alpha_t h_t$. Show that this function is convex and differentiable and use it to derive a boosting-type algorithm.

6.5 Define the unnormalized correlation of two vectors $\mathbf{x}$ and $\mathbf{x}'$ as the inner product between these vectors. Prove that the distribution vector $(D_{t+1}(1), \ldots, D_{t+1}(m))$ defined by AdaBoost and the vector of components $y_i h_t(x_i)$ are uncorrelated.

6.6 Fix $\epsilon \in (0, 1/2)$. Let the training sample be defined by m points in the plane with $\frac{m}{4}$ negative points all at coordinate $(1, 1)$, another $\frac{m}{4}$ negative points all at coordinate $(-1, -1)$, $\frac{m(1-\epsilon)}{4}$ positive points all at coordinate $(1, -1)$, and $\frac{m(1+\epsilon)}{4}$ positive points all at coordinate $(-1, +1)$. Describe the behavior of AdaBoost when

run on this sample using boosting stumps. What solution does the algorithm return after T rounds?

6.7 Noise-tolerant AdaBoost. AdaBoost may significantly overfitting in the presence of noise, in part due to the high penalization of misclassified examples. To reduce this effect, one could use instead the following objective function:

$$F = \sum_{i=1}^{m} G(-y_i g(x_i)), \tag{6.29}$$

where G is the function defined on $\mathbb{R}$ by

$$G(x) = \begin{cases} e^x & \text{if } x \leq 0 \\ x+1 & \text{otherwise.} \end{cases} \tag{6.30}$$

(a) Show that the function G is convex and differentiable.

(b) Use F and greedy coordinate descent to derive an algorithm similar to AdaBoost.

(c) Compare the reduction of the empirical error rate of this algorithm with that of AdaBoost.

6.8 Simplified AdaBoost. Suppose we simplify AdaBoost by setting the parameter α_t to a fixed value $\alpha_t = \alpha > 0$, independent of the boosting round t.

(a) Let γ be such that $(\frac{1}{2} - \epsilon_t) \geq \gamma > 0$. Find the best value of α as a function of γ by analyzing the empirical error.

(b) For this value of α, does the algorithm assign the same probability mass to correctly classified and misclassified examples at each round? If not, which set is assigned a higher probability mass?

(c) Using the previous value of α, give a bound on the empirical error of the algorithm that depends only on γ and the number of rounds of boosting T.

(d) Using the previous bound, show that for $T > \frac{\log m}{2\gamma^2}$, the resulting hypothesis is consistent with the sample of size m.

(e) Let s be the VC-dimension of the base learners used. Give a bound on the generalization error of the consistent hypothesis obtained after $T = \left\lfloor \frac{\log m}{2\gamma^2} \right\rfloor + 1$ rounds of boosting. (*Hint*: Use the fact that the VC-dimension of the family of functions $\{\text{sgn}(\sum_{t=1}^{T} \alpha_t h_t) : \alpha_t \in \mathbb{R}\}$ is bounded by $2(s+1)T\log_2(eT)$). Suppose now that γ varies with m. Based on the bound derived, what can be said if $\gamma(m) = O(\sqrt{\frac{\log m}{m}})$?)

MATRIX-BASED ADABOOST($\mathbf{M}, t_{\max}$)

1 $\lambda_{1,j} \leftarrow 0$ for $i = 1, \ldots, m$
2 **for** $t \leftarrow 1$ **to** $t_{\max}$ **do**
3 $\quad d_{t,i} \leftarrow \frac{\exp(-(\mathbf{M}\boldsymbol{\lambda}_t)_i)}{\sum_{k=1}^{m} \exp(-(\mathbf{M}\boldsymbol{\lambda}_t)_k)}$ for $i = 1, \ldots, m$
4 $\quad j_t \leftarrow \text{argmax}_j(\mathbf{d}_t^\top \mathbf{M})_j$
5 $\quad r_t \leftarrow (\mathbf{d}_t^\top \mathbf{M})_{j_t}$
6 $\quad \alpha_t \leftarrow \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
7 $\quad \boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t + \alpha_t \mathbf{e}_{j_t}$, where $\mathbf{e}_{j_t}$ is 1 in position j_t and 0 elsewhere.
8 **return** $\frac{\boldsymbol{\lambda}_{t_{\max}}}{\|\boldsymbol{\lambda}_{t_{\max}}\|_1}$

Figure 6.7 Matrix-based AdaBoost.

6.9 Matrix-based AdaBoost.

(a) Define an $m \times n$ matrix $\mathbf{M}$ where $\mathbf{M}_{ij} = y_i h_j(\mathbf{x}_i)$, i.e., $\mathbf{M}_{ij} = +1$ if training example i is classified correctly by weak classifier h_j, and -1 otherwise. Let $\mathbf{d}_t, \boldsymbol{\lambda}_t \in \mathbb{R}^n$, $\|\mathbf{d}_t\|_1 = 1$ and $d_{t,i}$ (respectively $\lambda_{t,i}$) equal the i^{th} component of $\mathbf{d}_t$ (respectively $\boldsymbol{\lambda}_t$). Now, consider the matrix-based form of AdaBoost described in figure 6.7 and define $\mathbf{M}$ as below with eight training points and eight weak classifiers.

$$\mathbf{M} = \begin{pmatrix} -1 & 1 & 1 & 1 & 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 & -1 & 1 & 1 \\ 1 & -1 & 1 & 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & 1 & -1 \end{pmatrix}$$

Assume that we start with the following initial distribution over the datapoints:

$$\mathbf{d}_1 = \left(\frac{3-\sqrt{5}}{8}, \frac{3-\sqrt{5}}{8}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{\sqrt{5}-1}{8}, \frac{\sqrt{5}-1}{8}, 0\right)^\top$$

Compute the first few steps of the matrix-based AdaBoost algorithm using $\mathbf{M}$, $\mathbf{d}_1$, and $t_{max} = 7$. What weak classifier is picked at each round of boosting?

Do you notice any pattern?

(b) What is the L_1 norm margin produced by AdaBoost for this example?

(c) Instead of using AdaBoost, imagine we combined our classifiers using the following coefficients: $[2, 3, 4, 1, 2, 2, 1, 1] \times \frac{1}{16}$. What is the margin in this case? Does AdaBoost maximize the margin?

7 On-Line Learning

This chapter presents an introduction to on-line learning, an important area with a rich literature and multiple connections with game theory and optimization that is increasingly influencing the theoretical and algorithmic advances in machine learning. In addition to the intriguing novel learning theory questions that they raise, on-line learning algorithms are particularly attractive in modern applications since they form an attractive solution for large-scale problems.

These algorithms process one sample at a time and can thus be significantly more efficient both in time and space and more practical than batch algorithms, when processing modern data sets of several million or billion points. They are also typically easy to implement. Moreover, on-line algorithms do not require any distributional assumption; their analysis assumes an adversarial scenario. This makes them applicable in a variety of scenarios where the sample points are not drawn i.i.d. or according to a fixed distribution.

We first introduce the general scenario of on-line learning, then present and analyze several key algorithms for on-line learning with expert advice, including the deterministic and randomized weighted majority algorithms for the zero-one loss and an extension of these algorithms for convex losses. We also describe and analyze two standard on-line algorithms for linear classifications, the Perceptron and Winnow algorithms, as well as some extensions. While on-line learning algorithms are designed for an adversarial scenario, they can be used, under some assumptions, to derive accurate predictors for a distributional scenario. We derive learning guarantees for this on-line to batch conversion. Finally, we briefly point out the connection of on-line learning with game theory by describing their use to derive a simple proof of von Neumann's minimax theorem.

7.1 Introduction

The learning framework for on-line algorithms is in stark contrast to the PAC learning or stochastic models discussed up to this point. First, instead of learning from a training set and then testing on a test set, the on-line learning scenario mixes

the training and test phases. Second, PAC learning follows the key assumption that the distribution over data points is fixed over time, both for training and test points, and that points are sampled in an i.i.d. fashion. Under this assumption, the natural goal is to learn a hypothesis with a small expected loss or generalization error. In contrast, with on-line learning, *no distributional assumption* is made, and thus there is no notion of generalization. Instead, the performance of on-line learning algorithms is measured using a *mistake model* and the notion of *regret*. To derive guarantees in this model, theoretical analyses are based on a worst-case or adversarial assumption.

The general on-line setting involves T rounds. At the tth round, the algorithm receives an instance $x_t \in \mathcal{X}$ and makes a prediction $\widehat{y}_t \in \mathcal{Y}$. It then receives the true label $y_t \in Y$ and incurs a loss $L(\widehat{y}_t, y_t)$, where $L\colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+$ is a loss function. More generally, the prediction domain for the algorithm may be $\mathcal{Y}' \neq \mathcal{Y}$ and the loss function defined over $\mathcal{Y}' \times \mathcal{Y}$. For classification problems, we often have $\mathcal{Y} = \{0, 1\}$ and $L(y, y') = |y' - y|$, while for regression $\mathcal{Y} \subseteq \mathbb{R}$ and typically $L(y, y') = (y' - y)^2$. The objective in the on-line setting is to minimize the cumulative loss: $\sum_{t=1}^{T} L(\widehat{y}_t, y_t)$ over T rounds.

7.2 Prediction with expert advice

We first discuss the setting of online learning with expert advice, and the associated notion of *regret*. In this setting, at the tth round, in addition to receiving $x_t \in \mathcal{X}$, the algorithm also receives *advice* $y_{t,i} \in Y$, $i \in [1, N]$, from N experts. Following the general framework of on-line algorithms, it then makes a prediction, receives the true label, and incurs a loss. After T rounds, the algorithm has incurred a cumulative loss. The objective in this setting is to minimize the *regret* R_T, also called *external regret*, which compares the cumulative loss of the algorithm to that of the best expert in hindsight after T rounds:

$$R_T = \sum_{t=1}^{T} L(\widehat{y}_t, y_t) - \min_{i=1}^{N} \sum_{t=1}^{T} L(\widehat{y}_{t,i}, y_t). \tag{7.1}$$

This problem arises in a variety of different domains and applications. Figure 7.1 illustrates the problem of predicting the weather using several forecasting sources as experts.

7.2.1 Mistake bounds and Halving algorithm

Here, we assume that the loss function is the standard zero-one loss used in classification. To analyze the expert advice setting, we first consider the realizable

Figure 7.1 Weather forecast: an example of a prediction problem based on expert advice.

case. As such, we discuss the *mistake bound model*, which asks the simple question "How many mistakes before we learn a particular concept?" Since we are in the realizable case, after some number of rounds T, we will learn the concept and no longer make errors in subsequent rounds. For any fixed concept c, we define the maximum number of mistakes a learning algorithm $\mathcal{A}$ makes as

$$M_{\mathcal{A}}(c) = \max_{x_1,\ldots,x_T} |\text{mistakes}(\mathcal{A}, c)|. \tag{7.2}$$

Further, for any concept in a concept class C, the maximum number of mistakes a learning algorithm makes is

$$M_{\mathcal{A}}(C) = \max_{c \in C} M_{\mathcal{A}}(c). \tag{7.3}$$

Our goal in this setting is to derive *mistake bounds*, that is, a bound M on $M_{\mathcal{A}}(C)$. We will first do this for the Halving algorithm, an elegant and simple algorithm for which we can generate surprisingly favorable mistake bounds. At each round, the Halving algorithm makes its prediction by taking the majority vote over all *active* experts. After any incorrect prediction, it deactivates all experts that gave faulty advice. Initially, all experts are active, and by the time the algorithm has converged to the correct concept, the active set contains only those experts that are consistent with the target concept. The pseudocode for this algorithm is shown in figure 7.2. We also present straightforward mistake bounds in theorems 7.1 and 7.2, where the former deals with finite hypothesis sets and the latter relates mistake bounds to VC-dimension. Note that the hypothesis complexity term in theorem 7.1 is identical to the corresponding complexity term in the PAC model bound of theorem 2.1.

Theorem 7.1
Let H be a finite hypothesis set. Then

$$M_{Halving}(H) \leq \log_2 |H|. \tag{7.4}$$

Proof Since the algorithm makes predictions using majority vote from the active set, at each mistake, the active set is reduced by at least half. Hence, after $\log_2 |H|$ mistakes, there can only remain one active hypothesis, and since we are in the

```
HALVING(H)
1  H_1 ← H
2  for t ← 1 to T do
3      RECEIVE(x_t)
4      ŷ_t ← MAJORITYVOTE(H_t, x_t)
5      RECEIVE(y_t)
6      if (ŷ_t ≠ y_t) then
7          H_{t+1} ← {c ∈ H_t : c(x_t) = y_t}
8  return H_{T+1}
```

Figure 7.2 Halving algorithm.

realizable case, this hypothesis must coincide with the target concept. ■

Theorem 7.2
Let opt(H) be the optimal mistake boundfor H. Then,

$$VCdim(H) \leq opt(H) \leq M_{Halving}(H) \leq \log_2 |H|. \tag{7.5}$$

Proof The second inequality is true by definition and the third inequality holds based on theorem 7.1. To prove the first inequality, we let $d = \text{VCdim}(H)$. Then there exists a shattered set of d points, for which we can form a complete binary tree of the mistakes with height d, and we can choose labels at each round of learning to ensure that d mistakes are made. Note that this adversarial argument is valid since the on-line setting makes no statistical assumptions about the data. ■

7.2.2 Weighted majority algorithm

In the previous section, we focused on the realizable setting in which the Halving algorithm simply discarded experts after a single mistake. We now move to the non-realizable setting and use a more general and less extreme algorithm, the Weighted Majority (WM) algorithm, that *weights* the importance of experts as a function of their mistake rate. The WM algorithm begins with uniform weights over all N experts. At each round, it generates predictions using a weighted majority vote. After receiving the true label, the algorithm then reduces the weight of each incorrect expert by a factor of $\beta \in [0, 1)$. Note that this algorithm reduces to the Halving algorithm when $\beta = 0$. The pseudocode for the WM algorithm is shown in

```
WEIGHTED-MAJORITY(N)
 1  for i ← 1 to N do
 2        w_{1,i} ← 1
 3  for t ← 1 to T do
 4        RECEIVE(x_t)
 5        if Σ_{i: y_{t,i}=1} w_{t,i} ≥ Σ_{i: y_{t,i}=0} w_{t,i} then
 6             ŷ_t ← 1
 7        else ŷ_t ← 0
 8        RECEIVE(y_t)
 9        if (ŷ_t ≠ y_t) then
10             for i ← 1 to N do
11                  if (y_{t,i} ≠ y_t) then
12                       w_{t+1,i} ← β w_{t,i}
13                  else w_{t+1,i} ← w_{t,i}
14  return w_{T+1}
```

Figure 7.3 Weighted majority algorithm, $y_t, y_{t,i} \in \{0, 1\}$.

figure 7.3.

Since we are not in the realizable setting, the mistake bounds of theorem 7.1 cannot apply. However, the following theorem presents a bound on the number of mistakes m_T made by the WM algorithm after $T \geq 1$ rounds of on-line learning as a function of the number of mistakes made by the best expert, that is the expert who achieves the smallest number of mistakes for the sequence $y_1, \ldots, y_T$. Let us emphasize that this is the best expert *in hindsight*.

Theorem 7.3

Fix $\beta \in (0, 1)$. Let m_T be the number of mistakes made by algorithm WM after $T \geq 1$ rounds, and m_T^* be the number of mistakes made by the best of the N experts. Then, the following inequality holds:

$$m_T \leq \frac{\log N + m_T^* \log \frac{1}{\beta}}{\log \frac{2}{1+\beta}}. \tag{7.6}$$

Proof To prove this theorem, we first introduce a potential function. We then derive upper and lower bounds for this function, and combine them to obtain our

result. This potential function method is a general proof technique that we will use throughout this chapter.

For any $t \geq 1$, we define our potential function as $W_t = \sum_{i=1}^{N} w_{t,i}$. Since predictions are generated using weighted majority vote, if the algorithm makes an error at round t, this implies that

$$W_{t+1} \leq \big[1/2 + (1/2)\beta\big] W_t = \left[\frac{1+\beta}{2}\right] W_t. \tag{7.7}$$

Since $W_1 = N$ and m_T mistakes are made after T rounds, we thus have the following upper bound:

$$W_T \leq \left[\frac{1+\beta}{2}\right]^{m_T} N. \tag{7.8}$$

Next, since the weights are all non-negative, it is clear that for any expert i, $W_T \geq w_{T,i} = \beta^{m_{T,i}}$, where $m_{T,i}$ is the number of mistakes made by the ith expert after T rounds. Applying this lower bound to the best expert and combining it with the upper bound in (7.8) gives us:

$$\begin{aligned} \beta^{m_T^*} \leq W_T &\leq \left[\frac{1+\beta}{2}\right]^{m_T} N \\ \Rightarrow m_T^* \log\beta &\leq \log N + m_T \log\left[\frac{1+\beta}{2}\right] \\ \Rightarrow m_T \log\left[\frac{2}{1+\beta}\right] &\leq \log N + m_T^* \log\frac{1}{\beta}, \end{aligned}$$

which concludes the proof. ■

Thus, the theorem guarantees a bound of the following form for algorithm WM:

$$m_T \leq O(\log N) + \text{constant} \times |\text{mistakes of best expert}|.$$

Since the first term varies only logarithmically as a function of N, the theorem guarantees that the number of mistakes is roughly a constant times that of the best expert in hindsight. This is a remarkable result, especially because it requires no assumption about the sequence of points and labels generated. In particular, the sequence could be chosen adversarially. In the realizable case where $m_T^* = 0$, the bound reduces to $m_T \leq O(\log N)$ as for the Halving algorithm.

7.2.3 Randomized weighted majority algorithm

In spite of the guarantees just discussed, the WM algorithm admits a drawback that affects all deterministic algorithms in the case of the zero-one loss: no deterministic algorithm can achieve a regret $R_T = o(T)$ over all sequences. Clearly, for any

RANDOMIZED-WEIGHTED-MAJORITY (N)

```
for i ← 1 to N do
    w_{1,i} ← 1
    p_{1,i} ← 1/N
for t ← 1 to T do
    for i ← 1 to N do
        if (l_{t,i} = 1) then
            w_{t+1,i} ← β w_{t,i}
        else w_{t+1,i} ← w_{t,i}
    W_{t+1} ← Σ_{i=1}^{N} w_{t+1,i}
    for i ← 1 to N do
        p_{t+1,i} ← w_{t+1,i} / W_{t+1}
return w_{T+1}
```

Figure 7.4 Randomized weighted majority algorithm.

deterministic algorithm $\mathcal{A}$ and any $t \in [1, T]$, we can adversarially select y_t to be 1 if the algorithm predicts 0, and choose it to be 0 otherwise. Thus, $\mathcal{A}$ errs at every point of such a sequence and its cumulative mistake is $m_T = T$. Assume for example that $N = 2$ and that one expert always predicts 0, the other one always 1. The error of the best expert over that sequence (and in fact any sequence of that length) is then at most $m_T^* \leq T/2$. Thus, for that sequence, we have

$$R_T = m_T - m_T^* \geq T/2,$$

which shows that $R_T = o(T)$ cannot be achieved in general. Note that this does not contradict the bound proven in the previous section, since for any $\beta \in (0, 1)$, $\frac{\log \frac{1}{\beta}}{\log \frac{2}{1+\beta}} \geq 2$. As we shall see in the next section, this negative result does not hold for any loss that is convex with respect to one of its arguments. But for the zero-one loss, this leads us to consider randomized algorithms instead.

In the randomized scenario of on-line learning, we assume that a set $A = \{1, \ldots, N\}$ of N actions is available. At each round $t \in [1, T]$, an on-line algorithm $\mathcal{A}$ selects a distribution $\mathbf{p}_t$ over the set of actions, receives a loss vector $\mathbf{l}_t$, whose ith component $l_{t,i} \in [0, 1]$ is the loss associated with action i, and incurs the expected loss $L_t = \sum_{i=1}^{N} p_{t,i}\, l_{t,i}$. The total loss incurred by the algorithm over T rounds is $\mathcal{L}_T = \sum_{t=1}^{T} L_t$. The total loss associated to action i is $\mathcal{L}_{T,i} = \sum_{t=1}^{T} l_{t,i}$. The

minimal loss of a single action is denoted by $\mathcal{L}_T^{\min} = \min_{i \in A} \mathcal{L}_{T,i}$. The regret R_T of the algorithm after T rounds is then typically defined by the difference of the loss of the algorithm and that of the best single action:[1]

$$R_T = \mathcal{L}_T - \mathcal{L}_T^{\min}.$$

Here, we consider specifically the case of zero-one losses and assume that $l_{t,i} \in \{0, 1\}$ for all $t \in [1, T]$ and $i \in A$.

The WM algorithm admits a straightforward randomized version, the randomized weighted majority (RWM) algorithm. The pseudocode of this algorithm is given in figure 7.4. The algorithm updates the weight $w_{t,i}$ of expert i as in the case of the WM algorithm by multiplying it by β. The following theorem gives a strong guarantee on the regret R_T of the RWM algorithm, showing that it is in $O(\sqrt{T \log N})$.

Theorem 7.4
Fix $\beta \in [1/2, 1)$. Then, for any $T \geq 1$, the loss of algorithm RWM on any sequence can be bounded as follows:

$$\mathcal{L}_T \leq \frac{\log N}{1 - \beta} + (2 - \beta)\mathcal{L}_T^{\min}. \tag{7.9}$$

In particular, for $\beta = \max\{1/2, 1 - \sqrt{(\log N)/T}\}$, the loss can be bounded as:

$$\mathcal{L}_T \leq \mathcal{L}_T^{\min} + 2\sqrt{T \log N}. \tag{7.10}$$

Proof As in the proof of theorem 7.3, we derive upper and lower bounds for the potential function $W_t = \sum_{i=1}^N w_{t,i}$, $t \in [1, T]$, and combine these bounds to obtain the result. By definition of the algorithm, for any $t \in [1, T]$, W_{t+1} can be expressed as follows in terms of W_t:

$$\begin{aligned} W_{t+1} = \sum_{i:\, l_{t,i}=0} w_{t,i} + \beta \sum_{i:\, l_{t.i}=1} w_{t,i} &= W_t + (\beta - 1) \sum_{i:\, l_{t,i}=1} w_{t,i} \\ &= W_t + (\beta - 1) W_t \sum_{i:\, l_{t,i}=1} p_{t,i} \\ &= W_t + (\beta - 1) W_t L_t \\ &= W_t (1 - (1 - \beta) L_t). \end{aligned}$$

Thus, since $W_1 = N$, it follows that $W_{T+1} = N \prod_{t=1}^T (1 - (1 - \beta) L_t)$. On the other hand, the following lower bound clearly holds: $W_{T+1} \geq \max_{i \in [1,N]} w_{T+1,i} = \beta^{\mathcal{L}_T^{\min}}$. This leads to the following inequality and series of derivations after taking the log

1. Alternative definitions of the regret with comparison classes different from the set of single actions can be considered.

and using the inequalities $\log(1-x) \leq -x$ valid for all $x < 1$, and $-\log(1-x) \leq x + x^2$ valid for all $x \in [0, 1/2]$:

$$\begin{aligned}
\beta^{\mathcal{L}_T^{\min}} \leq N \prod_{t=1}^{T} (1-(1-\beta)L_t) &\implies \mathcal{L}_T^{\min} \log\beta \leq \log N + \sum_{t=1}^{T} \log(1-(1-\beta)L_t) \\
&\implies \mathcal{L}_T^{\min} \log\beta \leq \log N - (1-\beta) \sum_{t=1}^{T} L_t \\
&\implies \mathcal{L}_T^{\min} \log\beta \leq \log N - (1-\beta)\mathcal{L}_T \\
&\implies \mathcal{L}_T \leq \frac{\log N}{1-\beta} - \frac{\log\beta}{1-\beta}\mathcal{L}_T^{\min} \\
&\implies \mathcal{L}_T \leq \frac{\log N}{1-\beta} - \frac{\log(1-(1-\beta))}{1-\beta}\mathcal{L}_T^{\min} \\
&\implies \mathcal{L}_T \leq \frac{\log N}{1-\beta} + (2-\beta)\mathcal{L}_T^{\min}.
\end{aligned}$$

This shows the first statement. Since $\mathcal{L}_T^{\min} \leq T$, this also implies

$$\mathcal{L}_T \leq \frac{\log N}{1-\beta} + (1-\beta)T + \mathcal{L}_T^{\min}. \tag{7.11}$$

Differentiating the upper bound with respect to β and setting it to zero gives $\frac{\log N}{(1-\beta)^2} - T = 0$, that is $\beta = \beta_0 = 1 - \sqrt{(\log N)/T}$, since $\beta < 1$. Thus, if $1 - \sqrt{(\log N)/T} \geq 1/2$, β_0 is the minimizing value of β, otherwise $1/2$ is the optimal value. The second statement follows by replacing β with β_0 in (7.11). ■

The bound (7.10) assumes that the algorithm additionally receives as a parameter the number of rounds T. As we shall see in the next section, however, there exists a general *doubling trick* that can be used to relax this requirement at the price of a small constant factor increase. Inequality 7.10 can be written directly in terms of the regret R_T of the RWM algorithm:

$$R_T \leq 2\sqrt{T \log N}. \tag{7.12}$$

Thus, for N constant, the regret verifies $R_T = O(\sqrt{T})$ and the *average regret* or *regret per round* R_T/T decreases as $O(1/\sqrt{T})$. These results are optimal, as shown by the following theorem.

Theorem 7.5
Let $N = 2$. There exists a stochastic sequence of losses for which the regret of any on-line learning algorithm verifies $\mathrm{E}[R_T] \geq \sqrt{T/8}$.

Proof For any $t \in [1, T]$, let the vector of losses $\mathbf{l}_t$ take the values $\mathbf{l}_{01} = (0,1)^\top$ and $\mathbf{l}_{10} = (1,0)^\top$ with equal probability. Then, the expected loss of any randomized

algorithm $\mathcal{A}$ is

$$\mathrm{E}[\mathcal{L}_T] = \mathrm{E}\Big[\sum_{t=1}^{T} \mathbf{p}_t \cdot \mathbf{l}_t\Big] = \sum_{t=1}^{T} \mathbf{p}_t \cdot \mathrm{E}[\mathbf{l}_t] = \sum_{t=1}^{T} \frac{1}{2} p_{t,1} + \frac{1}{2}(1 - p_{t,1}) = T/2,$$

where we denoted by $\mathbf{p}_t$ the distribution selected by $\mathcal{A}$ at round t. By definition, $\mathcal{L}_T^{\min}$ can be written as follows:

$$\mathcal{L}_T^{\min} = \min\{\mathcal{L}_{T,1}, \mathcal{L}_{T,2}\} = \frac{1}{2}(\mathcal{L}_{T,1} + \mathcal{L}_{T,2} - |\mathcal{L}_{T,1} - \mathcal{L}_{T,2}|) = T/2 - |\mathcal{L}_{T,1} - T/2|,$$

using the fact that $\mathcal{L}_{T,1} + \mathcal{L}_{T,2} = T$. Thus, the expected regret of $\mathcal{A}$ is

$$\mathrm{E}[R_T] = \mathrm{E}[\mathcal{L}_T] - \mathrm{E}[\mathcal{L}_T^{\min}] = \mathrm{E}[|\mathcal{L}_{T,1} - T/2|].$$

Let σ_t, $t \in [1, T]$, denote Rademacher variables taking values in $\{-1, +1\}$, then $\mathcal{L}_{T,1}$ can be rewritten as $\mathcal{L}_{T,1} = \sum_{t=1}^{T} \frac{1+\sigma_t}{2} = T/2 + \frac{1}{2}\sum_{t=1}^{T} \sigma_t$. Thus, introducing scalars $x_t = 1/2$, $t \in [1, T]$, by the Khintchine-Kahane inequality (D.22), we have:

$$\mathrm{E}[R_T] = \mathrm{E}\Big[|\sum_{t=1}^{T} \sigma_t x_t|\Big] \geq \sqrt{\frac{1}{2}\sum_{t=1}^{T} x_t^2} = \sqrt{T/8},$$

which concludes the proof. ■

More generally, for $T \geq N$, a lower bound of $R_T = \Omega(\sqrt{T \log N})$ can be proven for the regret of any algorithm.

7.2.4 Exponential weighted average algorithm

The WM algorithm can be extended to other loss functions L taking values in $[0, 1]$. The Exponential Weighted Average algorithm presented here can be viewed as that extension for the case where L is convex in its first argument. Note that this algorithm is deterministic and yet, as we shall see, admits a very favorable regret guarantee. Figure 7.5 gives its pseudocode. At round $t \in [1, T]$, the algorithm's prediction is

$$\widehat{y}_t = \frac{\sum_{i=1}^{N} w_{t,i} y_{t,i}}{\sum_{i=1}^{N} w_{t,i}}, \tag{7.13}$$

where $y_{t,i}$ is the prediction by expert i and $w_{t,i}$ the weight assigned by the algorithm to that expert. Initially, all weights are set to one. The algorithm then updates the weights at the end of round t according to the following rule:

$$w_{t+1,i} \leftarrow w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)} = e^{-\eta L_{t,i}}, \tag{7.14}$$

EXPONENTIAL-WEIGHTED-AVERAGE (N)

1 **for** $i \leftarrow 1$ **to** N **do**
2 $\quad w_{1,i} \leftarrow 1$
3 **for** $t \leftarrow 1$ **to** T **do**
4 $\quad$ RECEIVE(x_t)
5 $\quad \widehat{y}_t \leftarrow \frac{\sum_{i=1}^{N} w_{t,i} y_{t,i}}{\sum_{i=1}^{N} w_{t,i}}$
6 $\quad$ RECEIVE(y_t)
7 $\quad$ **for** $i \leftarrow 1$ **to** N **do**
8 $\quad\quad w_{t+1,i} \leftarrow w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)}$
9 **return** $\mathbf{w}_{T+1}$

Figure 7.5 Exponential weighted average, $L(\widehat{y}_{t,i}, y_t) \in [0,1]$.

where $L_{t,i}$ is the total loss incurred by expert i after t rounds. Note that this algorithm, as well as the others presented in this chapter, are simple, since they do not require keeping track of the losses incurred by each expert at all previous rounds but only of their cumulative performance. Furthermore, this property is also computationally advantageous. The following theorem presents a regret bound for this algorithm.

Theorem 7.6
Assume that the loss function L is convex in its first argument and takes values in $[0,1]$. Then, for any $\eta > 0$ and any sequence $y_1, \ldots, y_T \in Y$, the regret of the Exponential Weighted Average algorithm after T rounds satisfies

$$R_T \leq \frac{\log N}{\eta} + \frac{\eta T}{8}. \tag{7.15}$$

In particular, for $\eta = \sqrt{8 \log N / T}$, the regret is bounded as

$$R_T \leq \sqrt{(T/2) \log N}. \tag{7.16}$$

Proof We apply the same potential function analysis as in previous proofs but using as potential $\Phi_t = \log \sum_{i=1}^{N} w_{t,i}$, $t \in [1, T]$. Let $\mathbf{p}_t$ denote the distribution over $\{1, \ldots, N\}$ with $p_{t,i} = \frac{w_{t,i}}{\sum_{i=1}^{N} w_{t,i}}$. To derive an upper bound on Φ_t, we first examine

the difference of two consecutive potential values:

$$\Phi_{t+1} - \Phi_t = \log \frac{\sum_{i=1}^N w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)}}{\sum_{i=1}^N w_{t,i}} = \log \big(\mathop{\mathrm{E}}_{\mathbf{p}_t}[e^{\eta X}] \big),$$

with $X = -L(\widehat{y}_{t,i}, y_t) \in [-1, 0]$. To upper bound the expression appearing in the right-hand side, we apply Hoeffding's lemma (lemma D.1) to the centered random variable $X - \mathrm{E}_{\mathbf{p}_t}[X]$, then Jensen's inequality (theorem B.4) using the convexity of L with respect to its first argument:

$$\begin{aligned}
\Phi_{t+1} - \Phi_t &= \log \big(\mathop{\mathrm{E}}_{\mathbf{p}_t} \big[e^{\eta(X - \mathrm{E}[X]) + \eta\, \mathrm{E}[X]} \big] \big) \\
&\leq \frac{\eta^2}{8} + \eta \mathop{\mathrm{E}}_{\mathbf{p}_t}[X] = \frac{\eta^2}{8} - \eta \mathop{\mathrm{E}}_{\mathbf{p}_t}[L(\widehat{y}_{t,i}, y_t)] && \text{(Hoeffding's lemma)} \\
&\leq -\eta L\big(\mathop{\mathrm{E}}_{\mathbf{p}_t}[\widehat{y}_{t,i}], y_t \big) + \frac{\eta^2}{8} && \text{(convexity of first arg. of } L\text{)} \\
&= -\eta L(\widehat{y}_t, y_t) + \frac{\eta^2}{8}.
\end{aligned}$$

Summing up these inequalities yields the following upper bound:

$$\Phi_{T+1} - \Phi_1 \leq -\eta \sum_{t=1}^T L(\widehat{y}_t, y_t) + \frac{\eta^2 T}{8}. \tag{7.17}$$

We obtain a lower bound for the same quantity as follows:

$$\Phi_{T+1} - \Phi_1 = \log \sum_{i=1}^N e^{-\eta L_{T,i}} - \log N \geq \log \max_{i=1}^N e^{-\eta L_{T,i}} - \log N = -\eta \min_{i=1}^N L_{T,i} - \log N.$$

Combining the upper and lower bounds yields:

$$\begin{aligned}
&-\eta \min_{i=1}^N L_{T,i} - \log N \leq -\eta \sum_{t=1}^T L(\widehat{y}_t, y_t) + \frac{\eta^2 T}{8} \\
\implies &\sum_{t=1}^T L(\widehat{y}_t, y_t) - \min_{i=1}^N L_{T,i} \leq \frac{\log N}{\eta} + \frac{\eta T}{8},
\end{aligned}$$

and concludes the proof. ■

The optimal choice of η in theorem 7.6 requires knowledge of the horizon T, which is an apparent disadvantage of this analysis. However, we can use a standard *doubling trick* to eliminate this requirement, at the price of a small constant factor. This consists of dividing time into periods $[2^k, 2^{k+1} - 1]$ of length 2^k with $k = 0, \ldots, n$ and $T \geq 2^n - 1$, and then choose $\eta_k = \sqrt{\frac{8 \log N}{2^k}}$ in each period. The following theorem presents a regret bound when using the doubling trick to select η. A more general

method consists of interpreting η as a function of time, i.e., $\eta_t = \sqrt{(8 \log N)/t}$, which can lead to a further constant factor improvement over the regret bound of the following theorem.

Theorem 7.7
Assume that the loss function L is convex in its first argument and takes values in $[0,1]$. Then, for any $T \geq 1$ and any sequence $y_1, \ldots, y_T \in Y$, the regret of the Exponential Weighted Average algorithm after T rounds is bounded as follows:

$$R_T \leq \frac{\sqrt{2}}{\sqrt{2}-1}\sqrt{(T/2)\log N} + \sqrt{\log N/2}. \tag{7.18}$$

Proof Let $T \geq 1$ and let $I_k = [2^k, 2^{k+1}-1]$, for $k \in [0,n]$, with $n = \lfloor \log(T+1) \rfloor$. Let L_{I_k} denote the loss incurred in the interval I_k. By theorem 7.6 (7.16), for any $k \in [0,n]$, we have

$$L_{I_k} - \min_{i=1}^{N} L_{I_k,i} \leq \sqrt{2^k/2 \log N}. \tag{7.19}$$

Thus, we can bound the total loss incurred by the algorithm after T rounds as:

$$\begin{aligned} L_T = \sum_{k=0}^{n} L_{I_k} &\leq \sum_{k=0}^{n} \min_{i=1}^{N} L_{I_k,i} + \sum_{k=0}^{n} \sqrt{2^k (\log N)/2} \\ &\leq \min_{i=1}^{N} L_{T,i} + \sqrt{(\log N)/2} \cdot \sum_{k=0}^{n} 2^{\frac{k}{2}}, \end{aligned} \tag{7.20}$$

where the second inequality follows from the super-additivity of min, that is $\min_i X_i + \min_i Y_i \leq \min_i (X_i + Y_i)$ for any sequences $(X_i)_i$ and $(Y_i)_i$, which implies $\sum_{k=0}^{n} \min_{i=1}^{N} L_{I_k,i} \leq \min_{i=1}^{N} \sum_{k=0}^{n} L_{I_k,i}$. The geometric sum appearing in the right-hand side of (7.20) can be expressed as follows:

$$\sum_{k=0}^{n} 2^{\frac{k}{2}} = \frac{2^{(n+1)/2}-1}{\sqrt{2}-1} \leq \frac{\sqrt{2}\sqrt{T+1}-1}{\sqrt{2}-1} \leq \frac{\sqrt{2}(\sqrt{T}+1)-1}{\sqrt{2}-1} = \frac{\sqrt{2}\sqrt{T}}{\sqrt{2}-1} + 1.$$

Plugging back into (7.20) and rearranging terms yields (7.18). ■

The $O(\sqrt{T})$ dependency on T presented in this bound cannot be improved for general loss functions.

7.3 Linear classification

This section presents two well-known on-line learning algorithms for linear classification: the Perceptron and Winnow algorithms.

```
PERCEPTRON(w_0)
1  w_1 ← w_0        ▷ typically w_0 = 0
2  for t ← 1 to T do
3      RECEIVE(x_t)
4      ŷ_t ← sgn(w_t · x_t)
5      RECEIVE(y_t)
6      if (ŷ_t ≠ y_t) then
7          w_{t+1} ← w_t + y_t x_t    ▷ more generally η y_t x_t, η > 0.
8      else w_{t+1} ← w_t
9  return w_{T+1}
```

Figure 7.6 Perceptron algorithm.

7.3.1 Perceptron algorithm

The Perceptron algorithm is one of the earliest machine learning algorithms. It is an on-line linear classification algorithm. Thus, it learns a decision function based on a hyperplane by processing training points one at a time. Figure 7.6 gives its pseudocode.

The algorithm maintains a weight vector $\mathbf{w}_t \in \mathbb{R}^N$ defining the hyperplane learned, starting with an arbitrary vector $\mathbf{w}_0$. At each round $t \in [1, T]$, it predicts the label of the point $\mathbf{x}_t \in \mathbb{R}^N$ received, using the current vector $\mathbf{w}_t$ (line 4). When the prediction made does not match the correct label (lines 6-7), it updates $\mathbf{w}_t$ by adding $y_t\mathbf{x}_t$. More generally, when a learning rate $\eta > 0$ is used, the vector added is $\eta y_t \mathbf{x}_t$. This update can be partially motivated by examining the inner product of the current weight vector with $y_t\mathbf{x}_t$, whose sign determines the classification of $\mathbf{x}_t$. Just before an update, $\mathbf{x}_t$ is misclassified and thus $y_t\mathbf{w}_t \cdot \mathbf{x}_t$ is negative; afterward, $y_t\mathbf{w}_{t+1} \cdot \mathbf{x}_t = y_t\mathbf{w}_t \cdot y_t\mathbf{x}_t + \eta\|\mathbf{x}_t\|^2$, thus, the update corrects the weight vector in the direction of making the inner product positive by augmenting it with this quantity with $\eta\|\mathbf{x}_t\|^2 > 0$.

The Perceptron algorithm can be shown in fact to seek a weight vector $\mathbf{w}$ minimizing an objective function F precisely based on the quantities $(-y_t\mathbf{w} \cdot \mathbf{x}_t)$, $t \in [1, T]$. Since $(-y_t\mathbf{w} \cdot \mathbf{x}_t)$ is positive when $\mathbf{x}_t$ is misclassified by $\mathbf{w}$, F is defined

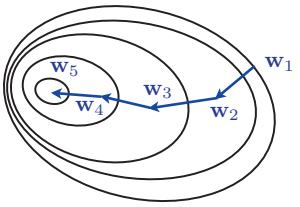


Figure 7.7 An example path followed by the iterative stochastic gradient descent technique. Each inner contour indicates a region of lower elevation.

for all $\mathbf{w} \in \mathbb{R}^N$ by

$$F(\mathbf{w}) = \frac{1}{T}\sum_{t=1}^{T} \max\Big(0, -y_t(\mathbf{w}\cdot\mathbf{x}_t)\Big) = \mathop{\mathrm{E}}_{\mathbf{x}\sim\widehat{D}}[\widetilde{F}(\mathbf{w},\mathbf{x})], \tag{7.21}$$

where $\widetilde{F}(\mathbf{w},\mathbf{x}) = \max\big(0, -f(\mathbf{x})(\mathbf{w}\cdot\mathbf{x})\big)$ with $f(\mathbf{x})$ denoting the label of $\mathbf{x}$, and $\widehat{D}$ is the empirical distribution associated with the sample $(\mathbf{x}_1, \ldots, \mathbf{x}_T)$. For any $t \in [1, T]$, $\mathbf{w} \mapsto -y_t(\mathbf{w}\cdot\mathbf{x}_t)$ is linear and thus convex. Since the max operator preserves convexity, this shows that F is convex. However, F is not differentiable. Nevertheless, the Perceptron algorithm coincides with the application of the *stochastic gradient descent* technique to F.

The stochastic (or on-line) gradient descent technique examines one point $\mathbf{x}_t$ at a time. For a function $\widetilde{F}$, a generalized version of this technique can be defined by the execution of the following update for each point $\mathbf{x}_t$:

$$\mathbf{w}_{t+1} \leftarrow \begin{cases} \mathbf{w}_t - \eta\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w}_t,\mathbf{x}_t) & \text{if } \mathbf{w}\mapsto\widetilde{F}(\mathbf{w},\mathbf{x}_t) \text{ differentiable at } \mathbf{w}_t, \\ \mathbf{w}_t & \text{otherwise,} \end{cases} \tag{7.22}$$

where $\eta > 0$ is a learning rate parameter. Figure 7.7 illustrates an example path the gradient descent follows. In the specific case we are considering, $\mathbf{w} \mapsto \widetilde{F}(\mathbf{w},\mathbf{x}_t)$ is differentiable at any $\mathbf{w}$ such that $y_t(\mathbf{w}\cdot\mathbf{x}_t) \neq 0$ with $\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w},\mathbf{x}_t) = -y\mathbf{x}_t$ if $y_t(\mathbf{w}\cdot\mathbf{x}_t) < 0$ and $\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w},\mathbf{x}_t) = 0$ if $y_t(\mathbf{w}\cdot\mathbf{x}_t) > 0$. Thus, the stochastic gradient descent update becomes

$$\mathbf{w}_{t+1} \leftarrow \begin{cases} \mathbf{w}_t + \eta y_t\mathbf{x}_t & \text{if } y_t(\mathbf{w}\cdot\mathbf{x}_t) < 0; \\ \mathbf{w}_t & \text{if } y_t(\mathbf{w}\cdot\mathbf{x}_t) > 0; \\ \mathbf{w}_t & \text{otherwise,} \end{cases} \tag{7.23}$$

which coincides exactly with the update of the Perceptron algorithm.

The following theorem gives a margin-based upper bound on the number of mistakes or updates made by the Perceptron algorithm when processing a sequence

of T points that can be linearly separated by a hyperplane with margin $\rho > 0$.

Theorem 7.8
Let $\mathbf{x}_1, \ldots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|\mathbf{x}_t\| \leq r$ for all $t \in [1,T]$, for some $r > 0$. Assume that there exist $\rho > 0$ and $\mathbf{v} \in \mathbb{R}^N$ such that for all $t \in [1,T]$, $\rho \leq \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\|\mathbf{v}\|}$. Then, the number of updates made by the Perceptron algorithm when processing $\mathbf{x}_1, \ldots, \mathbf{x}_T$ is bounded by r^2/ρ^2.

Proof Let I be the subset of the T rounds at which there is an update, and let M be the total number of updates, i.e., $|I| = M$. Summing up the assumption inequalities yields:

$$\begin{aligned}
M\rho \leq \frac{\mathbf{v} \cdot \sum_{t \in I} y_t \mathbf{x}_t}{\|\mathbf{v}\|} &\leq \Big\| \sum_{t \in I} y_t \mathbf{x}_t \Big\| && \text{(Cauchy-Schwarz inequality)} \\
&= \Big\| \sum_{t \in I} (\mathbf{w}_{t+1} - \mathbf{w}_t) \Big\| && \text{(definition of updates)} \\
&= \|\mathbf{w}_{T+1}\| && \text{(telescoping sum, } \mathbf{w}_0 = 0) \\
&= \sqrt{\sum_{t \in I} \|\mathbf{w}_{t+1}\|^2 - \|\mathbf{w}_t\|^2} && \text{(telescoping sum, } \mathbf{w}_0 = 0) \\
&= \sqrt{\sum_{t \in I} \|\mathbf{w}_t + y_t \mathbf{x}_t\|^2 - \|\mathbf{w}_t\|^2} && \text{(definition of updates)} \\
&= \sqrt{\sum_{t \in I} 2 \underbrace{y_t \mathbf{w}_t \cdot \mathbf{x}_t}_{\leq 0} + \|\mathbf{x}_t\|^2} \\
&\leq \sqrt{\sum_{t \in I} \|\mathbf{x}_t\|^2} \leq \sqrt{M r^2}.
\end{aligned}$$

Comparing the left- and right-hand sides gives $\sqrt{M} \leq r/\rho$, that is, $M \leq r^2/\rho^2$. ■

By definition of the algorithm, the weight vector $\mathbf{w}_T$ after processing T points is a linear combination of the vectors $\mathbf{x}_t$ at which an update was made: $\mathbf{w}_T = \sum_{t \in I} y_t \mathbf{x}_t$. Thus, as in the case of SVMs, these vectors can be referred to as *support vectors* for the Perceptron algorithm.

The bound of theorem 7.8 is remarkable, since it depends only on the normalized margin ρ/r and not on the dimension N of the space. This bound can be shown to be tight, that is the number of updates can be equal to r^2/ρ^2 in some instances (see exercise 7.3 to show the upper bound is tight).

The theorem required no assumption about the sequence of points $\mathbf{x}_1, \ldots, \mathbf{x}_T$. A standard setting for the application of the Perceptron algorithm is one where a finite sample S of size $m < T$ is available and where the algorithm makes multiple

passes over these m points. The result of the theorem implies that when S is linearly separable, the Perceptron algorithm converges after a finite number of updates and thus passes. For a small margin ρ, the convergence of the algorithm can be quite slow, however. In fact, for some samples, regardless of the order in which the points in S are processed, the number of updates made by the algorithm is in $\Omega(2^N)$ (see exercise 7.1). Of course, if S is not linearly separable, the Perceptron algorithm does not converge. In practice, it is stopped after some number of passes over S.

There are many variants of the standard Perceptron algorithm which are used in practice and have been theoretically analyzed. One notable example is the *voted Perceptron algorithm*, which predicts according to the rule $\operatorname{sgn}\left(\left(\sum_{t\in I} c_t \mathbf{w}_t\right) \cdot \mathbf{x}\right)$, where c_t is a weight proportional to the number of iterations that $\mathbf{w}_t$ survives, i.e., the number of iterations between $\mathbf{w}_t$ and $\mathbf{w}_{t+1}$.

For the following theorem, we consider the case where the Perceptron algorithm is trained via multiple passes till convergence over a finite sample that is linearly separable. In view of theorem 7.8, convergence occurs after a finite number of updates.

For a linearly separable sample S, we denote by r_S the radius of the smallest sphere containing all points in S and by ρ_S the largest margin of a separating hyperplane for S. We also denote by $M(S)$ the number of updates made by the algorithm after training over S.

Theorem 7.9

Assume that the data is linearly separable. Let h_S be the hypothesis returned by the Perceptron algorithm after training over a sample S of size m drawn according to some distribution D. Then, the expected error of h_S is bounded as follows:

$$\mathop{\mathrm{E}}_{S\sim D^m}[R(h_S)] \leq \mathop{\mathrm{E}}_{S\sim D^{m+1}}\left[\frac{\min\left(M(S), r_S^2/\rho_S^2\right)}{m+1}\right].$$

Proof Let S be a linearly separable sample of size $m+1$ drawn i.i.d. according to D and let $\mathbf{x}$ be a point in S. If $h_{S-\{\mathbf{x}\}}$ misclassifies $\mathbf{x}$, then $\mathbf{x}$ must be a support vector for h_S. Thus, the leave-one-out error of the Perceptron algorithm on sample S is at most $\frac{M(S)}{m+1}$. The result then follows lemma 4.1, which relates the expected leave-one-out error to the expected error, along with the upper bound on $M(S)$ given by theorem 7.8. ■

This result can be compared with a similar one given for the SVM algorithm (with no offset) in the following theorem, which is an extension of theorem 4.1. We denote by $N_{\mathrm{SV}}(S)$ the number of support vectors that define the hypothesis h_S returned by SVMs when trained on a sample S.

Theorem 7.10
Assume that the data is linearly separable. Let h_S be the hypothesis returned by SVMs used with no offset ($b = 0$) after training over a sample S of size m drawn according to some distribution D. Then, the expected error of h_S is bounded as follows:

$$\mathop{\mathrm{E}}_{S\sim D^m}[R(h_S)] \leq \mathop{\mathrm{E}}_{S\sim D^{m+1}}\left[\frac{\min\left(N_{SV}(S), r_S^2/\rho_S^2\right)}{m+1}\right].$$

Proof The fact that the expected error can be upper bounded by the average fraction of support vectors ($N_{\mathrm{SV}}(S)/(m+1)$) was already shown by theorem 4.1. Thus, it suffices to show that it is also upper bounded by the expected value of $(r_S^2/\rho_S^2)/(m+1)$. To do so, we will bound the leave-one-out error of the SVM algorithm for a sample S of size $m+1$ by $(r_S^2/\rho_S^2)/(m+1)$. The result will then follow by lemma 4.1, which relates the expected leave-one-out error to the expected error.

Let $S = (\mathbf{x}_1, \ldots, \mathbf{x}_{m+1})$ be a linearly separable sample drawn i.i.d. according to D and let $\mathbf{x}$ be a point in S that is misclassified by $h_{S-\{\mathbf{x}\}}$. We will analyze the case where $\mathbf{x} = \mathbf{x}_{m+1}$, the analysis of other cases is similar. We denote by S' the sample $(\mathbf{x}_1, \ldots, \mathbf{x}_m)$.

For any $q \in [1, m+1]$, let G_q denote the function defined over $\mathbb{R}^q$ by $G_q \colon \boldsymbol{\alpha} \mapsto \sum_{i=1}^q \alpha_i - \frac{1}{2}\sum_{i,j=1}^q \alpha_i\alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$. Then, G_{m+1} is the objective function of the dual optimization problem for SVMs associated to the sample S and G_m the one for the sample S'. Let $\boldsymbol{\alpha} \in \mathbb{R}^{m+1}$ denote a solution of the dual SVM problem $\max_{\boldsymbol{\alpha}\geq 0} G_{m+1}(\boldsymbol{\alpha})$ and $\boldsymbol{\alpha}' \in \mathbb{R}^{m+1}$ the vector such that $(\alpha'_1, \ldots, \alpha'_m)^\top \in \mathbb{R}^m$ is a solution of $\max_{\boldsymbol{\alpha}\geq 0} G_m(\boldsymbol{\alpha})$ and $\alpha'_{m+1} = 0$. Let $\mathbf{e}_{m+1}$ denote the $(m+1)$th unit vector in $\mathbb{R}^{m+1}$. By definition of $\boldsymbol{\alpha}$ and $\boldsymbol{\alpha}'$ as maximizers, $\max_{\beta\geq 0} G_{m+1}(\boldsymbol{\alpha}' + \beta\mathbf{e}_{m+1}) \leq G_{m+1}(\boldsymbol{\alpha})$ and $G_{m+1}(\boldsymbol{\alpha} - \alpha_{m+1}\mathbf{e}_{m+1}) \leq G_m(\boldsymbol{\alpha}')$. Thus, the quantity $A = G_{m+1}(\boldsymbol{\alpha}) - G_m(\boldsymbol{\alpha}')$ admits the following lower and upper bounds:

$$\max_{\beta\geq 0} G_{m+1}(\boldsymbol{\alpha}' + \beta\mathbf{e}_{m+1}) - G_m(\boldsymbol{\alpha}') \leq A \leq G_{m+1}(\boldsymbol{\alpha}) - G_{m+1}(\boldsymbol{\alpha} - \alpha_{m+1}\mathbf{e}_{m+1}).$$

Let $\mathbf{w} = \sum_{i=1}^{m+1} y_i\alpha_i\mathbf{x}_i$ denote the weight vector returned by SVMs for the sample S. Since $h_{S'}$ misclassifies $\mathbf{x}_{m+1}$, $\mathbf{x}_{m+1}$ must be a support vector for h_S, thus

$y_{m+1}\mathbf{w}\cdot\mathbf{x}_{m+1}=1$. In view of that, the upper bound can be rewritten as follows:

$$\begin{aligned}
&G_{m+1}(\boldsymbol{\alpha})-G_{m+1}(\boldsymbol{\alpha}-\alpha_{m+1}\mathbf{e}_{m+1})\\
&\qquad=\alpha_{m+1}-\sum_{i=1}^{m+1}(y_i\alpha_i\mathbf{x}_i)\cdot(y_{m+1}\alpha_{m+1}\mathbf{x}_{m+1})+\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\alpha_{m+1}(1-y_{m+1}\mathbf{w}\cdot\mathbf{x}_{m+1})+\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2.
\end{aligned}$$

Similarly, let $\mathbf{w}'=\sum_{i=1}^{m}y_i\alpha_i'\mathbf{x}_i$. Then, for any $\beta\geq 0$, the quantity maximized in the lower bound can be written as

$$\begin{aligned}
&G_{m+1}(\boldsymbol{\alpha}'+\beta\mathbf{e}_{m+1})-G_m(\boldsymbol{\alpha}')\\
&\qquad=\beta\big(1-y_{m+1}(\mathbf{w}'+\beta\mathbf{x}_{m+1})\cdot\mathbf{x}_{m+1}\big)+\frac{1}{2}\beta^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\beta(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})-\frac{1}{2}\beta^2\|\mathbf{x}_{m+1}\|^2.
\end{aligned}$$

The right-hand side is maximized for the following value of β: $\frac{1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1}}{\|\mathbf{x}_{m+1}\|^2}$. Plugging in this value in the right-hand side gives $\frac{1}{2}\frac{(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})}{\|\mathbf{x}_{m+1}\|^2}$. Thus,

$$A\geq\frac{1}{2}\frac{(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})}{\|\mathbf{x}_{m+1}\|^2}\geq\frac{1}{2\|\mathbf{x}_{m+1}\|^2},$$

using the fact that $y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1}<0$, since $\mathbf{x}_{m+1}$ is misclassified by $\mathbf{w}'$. Comparing this lower bound on A with the upper bound previously derived leads to $\frac{1}{2\|\mathbf{x}_{m+1}\|^2}\leq\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2$, that is

$$\alpha_{m+1}\geq\frac{1}{\|\mathbf{x}_{m+1}\|^2}\geq\frac{1}{r_S^2}.$$

The analysis carried out in the case $\mathbf{x}=\mathbf{x}_{m+1}$ holds similarly for any $\mathbf{x}_i$ in S that is misclassified by $h_{S-\{\mathbf{x}_i\}}$. Let I denote the set of such indices i. Then, we can write:

$$\sum_{i\in I}\alpha_i\geq\frac{|I|}{r_S^2}.$$

By (4.18), the following simple expression holds for the margin: $\sum_{i=1}^{m+1}\alpha_i=1/\rho_S^2$. Using this identity leads to

$$|I|\leq r_S^2\sum_{i\in I}\alpha_i\leq r_S^2\sum_{i=1}^{m+1}\alpha_i=\frac{r_S^2}{\rho_S^2}.$$

Since by definition $|I|$ is the total number of leave-one-out errors, this concludes the proof. ■

Thus, the guarantees given by theorem 7.9 and theorem 7.10 in the separable case have a similar form. These bounds do not seem sufficient to distinguish the effectiveness of the SVM and Perceptron algorithms. Note, however, that while the same margin quantity ρ_S appears in both bounds, the radius r_S can be replaced by a finer quantity that is different for the two algorithms: in both cases, instead of the radius of the sphere containing all sample points, r_S can be replaced by the radius of the sphere containing the support vectors, as can be seen straightforwardly from the proof of the theorems. Thus, the position of the support vectors in the case of SVMs can provide a more favorable guarantee than that of the support vectors (update vectors) for the Perceptron algorithm. Finally, the guarantees given by these theorems are somewhat weak. These are not high probability bounds, they hold only for the expected error of the hypotheses returned by the algorithms and in particular provide no information about the variance of their error.

The following theorem presents a bound on the number of updates or mistakes made by the Perceptron algorithm in the more general scenario of a non-linearly separable sample.

Theorem 7.11
Let $\mathbf{x}_1, \ldots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|\mathbf{x}_t\| \leq r$ for all $t \in [1, T]$, for some $r > 0$. Let $\mathbf{v} \in \mathbb{R}^N$ be any vector with $\|\mathbf{v}\| = 1$ and let $\rho > 0$. Define the deviation of $\mathbf{x}_t$ by $d_t = \max\{0, \rho - y_t(\mathbf{v} \cdot \mathbf{x}_t)\}$, and let $\delta = \sqrt{\sum_{t=1}^{T} d_t^2}$. Then, the number of updates made by the Perceptron algorithm when processing $\mathbf{x}_1, \ldots, \mathbf{x}_T$ is bounded by $(r + \delta)^2/\rho^2$.

Proof We first reduce the problem to the separable case by mapping each input vector $\mathbf{x}_t \in \mathbb{R}^N$ to a vector in $\mathbf{x}'_t \in \mathbb{R}^{N+T}$ as follows:

$$\mathbf{x}_t = \begin{bmatrix} x_{t,1} \\ \vdots \\ x_{t,N} \end{bmatrix} \mapsto \mathbf{x}'_t = \begin{bmatrix} x_{t,1} & \ldots & x_{t,N} & 0 & \ldots & 0 & \underbrace{\Delta}_{\substack{(N+t)\text{th} \\ \text{component}}} & 0 & \ldots & 0 \end{bmatrix}^\top,$$

where the first N components of $\mathbf{x}'_t$ are identical to those of $\mathbf{x}$ and the only other non-zero component is the $(N + t)$th component and is equal to Δ. The value of the parameter Δ will be set later. The vector $\mathbf{v}$ is replaced by the vector $\mathbf{v}'$ defined as follows:

$$\mathbf{v}' = \begin{bmatrix} v_1/Z & \ldots & v_N/Z & y_1 d_1/(\Delta Z) & \ldots & y_T d_T/(\Delta Z) \end{bmatrix}^\top.$$

The first N components of $\mathbf{v}'$ are equal to the components of $\mathbf{v}/Z$ and the remaining

DualPerceptron($\boldsymbol{\alpha}_0$)

$\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha}_0$ ▷ typically $\boldsymbol{\alpha}_0 = \mathbf{0}$
for $t \leftarrow 1$ **to** T **do**
  Receive($\mathbf{x}_t$)
  $\widehat{y}_t \leftarrow \text{sgn}(\sum_{s=1}^{T} \alpha_s y_s (\mathbf{x}_s \cdot \mathbf{x}_t))$
  Receive(y_t)
  if $(\widehat{y}_t \neq y_t)$ **then**
    $\alpha_{t+1} \leftarrow \alpha_t + 1$
  else $\alpha_{t+1} \leftarrow \alpha_t$
return $\boldsymbol{\alpha}$

Figure 7.8 Dual Perceptron algorithm.

T components are functions of the labels and deviations. Z is chosen to guarantee that $\|\mathbf{v}'\| = 1$: $Z = \sqrt{1 + \frac{\delta^2}{\Delta^2}}$. The predictions made by the Perceptron algorithm for $\mathbf{x}'_t$, $t \in [1, T]$ coincide with those made in the original space for $\mathbf{x}_t$, $t \in [1, T]$. Furthermore, by definition of $\mathbf{v}'$ and $\mathbf{x}'_t$, we can write for any $t \in [1, T]$:

$$\begin{aligned} y_t(\mathbf{v}' \cdot \mathbf{x}'_t) &= y_t\Big(\frac{\mathbf{v} \cdot \mathbf{x}_t}{Z} + \Delta \frac{y_t d_t}{Z\Delta}\Big) \\ &= \frac{y_t \mathbf{v} \cdot \mathbf{x}_t}{Z} + \frac{d_t}{Z} \\ &\geq \frac{y_t \mathbf{v} \cdot \mathbf{x}_t}{Z} + \frac{\rho - y_t(\mathbf{v} \cdot \mathbf{x}_t)}{Z} = \frac{\rho}{Z}, \end{aligned}$$

where the inequality results from the definition of the deviation d_t. This shows that the sample formed by $\mathbf{x}'_1, \ldots, \mathbf{x}'_T$ is linearly separable with margin ρ/Z. Thus, in view of theorem 7.8, since $||\mathbf{x}'_t||^2 \leq r^2 + \Delta^2$, the number of updates made by the Perceptron algorithm is bounded by $\frac{(r^2+\Delta^2)(1+\delta^2/\Delta^2)}{\rho^2}$. Choosing Δ^2 to minimize this bound leads to $\Delta^2 = r\delta$. Plugging in this value yields the statement of the theorem. ■

The main idea behind the proof of the theorem just presented is to map input points to a higher-dimensional space where linear separation is possible, which coincides with the idea of kernel methods. In fact, the particular kernel used in the proof is close to a straightforward one with a feature mapping that maps each data point to a distinct dimension.

The Perceptron algorithm can in fact be generalized, as in the case of SVMs,

KernelPerceptron($\boldsymbol{\alpha}_0$)

$\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha}_0$ ▷ typically $\boldsymbol{\alpha}_0 = \mathbf{0}$
for $t \leftarrow 1$ **to** T **do**
  Receive(x_t)
  $\widehat{y}_t \leftarrow \operatorname{sgn}(\sum_{s=1}^{T} \alpha_s y_s K(x_s, x_t))$
  Receive(y_t)
  if $(\widehat{y}_t \neq y_t)$ **then**
    $\alpha_{t+1} \leftarrow \alpha_t + 1$
  else $\alpha_{t+1} \leftarrow \alpha_t$
return $\boldsymbol{\alpha}$

Figure 7.9 Kernel Perceptron algorithm for PDS kernel K.

to define a linear separation in a high-dimensional space. It admits an equivalent dual form, the dual Perceptron algorithm, which is presented in figure 7.8. The dual Perceptron algorithm maintains a vector $\boldsymbol{\alpha} \in \mathbb{R}^T$ of coefficients assigned to each point $\mathbf{x}_t$, $t \in [1, T]$. The label of a point $\mathbf{x}_t$ is predicted according to the rule $\operatorname{sgn}(\mathbf{w} \cdot \mathbf{x}_t)$, where $\mathbf{w} = \sum_{s=1}^{T} \alpha_s y_s \mathbf{x}_s$. The coefficient α_t is incremented by one when this prediction does not match the correct label. Thus, an update for $\mathbf{x}_t$ is equivalent to augmenting the weight vector $\mathbf{w}$ with $y_t \mathbf{x}_t$, which shows that the dual algorithm matches exactly the standard Perceptron algorithm. The dual Perceptron algorithm can be written solely in terms of inner products between training instances. Thus, as in the case of SVMs, instead of the inner product between points in the input space, an arbitrary PDS kernel can be used, which leads to the kernel Perceptron algorithm detailed in figure 7.9. The kernel Perceptron algorithm and its average variant, i.e., voted Perceptron with uniform weights c_t, are commonly used algorithms in a variety of applications.

7.3.2 Winnow algorithm

This section presents an alternative on-line linear classification algorithm, the *Winnow algorithm*. Thus, it learns a weight vector defining a separating hyperplane by sequentially processing the training points. As suggested by the name, the algorithm is particularly well suited to cases where a relatively small number of dimensions or experts can be used to define an accurate weight vector. Many of the other dimensions may then be irrelevant.

WINNOW(η)

1 $w_1 \leftarrow \mathbf{1}/N$
2 **for** $t \leftarrow 1$ **to** T **do**
3 RECEIVE($\mathbf{x}_t$)
4 $\widehat{y}_t \leftarrow \text{sgn}(\mathbf{w}_t \cdot \mathbf{x}_t)$
5 RECEIVE(y_t)
6 **if** $(\widehat{y}_t \neq y_t)$ **then**
7 $Z_t \leftarrow \sum_{i=1}^{N} w_{t,i} \exp(\eta y_t x_{t,i})$
8 **for** $i \leftarrow 1$ **to** N **do**
9 $w_{t+1,i} \leftarrow \frac{w_{t,i} \exp(\eta y_t x_{t,i})}{Z_t}$
10 **else** $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t$
11 **return** $\mathbf{w}_{T+1}$

Figure 7.10 Winnow algorithm, with $y_t \in \{-1, +1\}$ for all $t \in [1, T]$.

The Winnow algorithm is similar to the Perceptron algorithm, but, instead of the additive update of the weight vector in the Perceptron case, Winnow's update is multiplicative. The pseudocode of the algorithm is given in figure 7.10. The algorithm takes as input a learning parameter $\eta > 0$. It maintains a non-negative weight vector $\mathbf{w}_t$ with components summing to one ($\|\mathbf{w}_t\|_1 = 1$) starting with the uniform weight vector (line 1). At each round $t \in [1, T]$, if the prediction does not match the correct label (line 6), each component $w_{t,i}$, $i \in [1, N]$, is updated by multiplying it by $\exp(\eta y_t x_{t,i})$ and dividing by the normalization factor Z_t to ensure that the weights sum to one (lines 7–9). Thus, if the label y_t and $x_{t,i}$ share the same sign, then $w_{t,i}$ is increased, while, in the opposite case, it is significantly decreased.

The Winnow algorithm is closely related to the WM algorithm: when $\mathbf{x}_{t,i} \in \{-1, +1\}$, $\text{sgn}(\mathbf{w}_t \cdot \mathbf{x}_t)$ coincides with the majority vote, since multiplying the weight of correct or incorrect experts by e^{η} or $e^{-\eta}$ is equivalent to multiplying the weight of incorrect ones by $\beta = e^{-2\eta}$. The multiplicative update rule of Winnow is of course also similar to that of AdaBoost.

The following theorem gives a mistake bound for the Winnow algorithm in the separable case, which is similar in form to the bound of theorem 7.8 for the Perceptron algorithm.

Theorem 7.12
Let $\mathbf{x}_1, \ldots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|x_t\|_\infty \leq r_\infty$ for all $t \in [1,T]$, for some $r_\infty > 0$. Assume that there exist $\mathbf{v} \in \mathbb{R}^N$, $\mathbf{v} \geq 0$, and $\rho_\infty > 0$ such that for all $t \in [1,T]$, $\rho_\infty \leq \frac{y_t(\mathbf{v}\cdot\mathbf{x}_t)}{\|\mathbf{v}\|_1}$. Then, for $\eta = \frac{\rho_\infty}{r_\infty^2}$, the number of updates made by the Winnow algorithm when processing $\mathbf{x}_1, \ldots, \mathbf{x}_T$ is upper bounded by $2\,(r_\infty^2/\rho_\infty^2)\log N$.

Proof Let $I \subseteq \{1, \ldots, T\}$ be the set of iterations at which there is an update, and let M be the total number of updates, i.e., $|I| = M$. The potential function Φ_t, $t \in [1,T]$, used for this proof is the relative entropy of the distribution defined by the normalized weights $v_i/\|\mathbf{v}\|_1 \geq 0$, $i \in [1,N]$, and the one defined by the components of the weight vector $w_{t,i}$, $i \in [1,N]$:

$$\Phi_t = \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i/\|\mathbf{v}\|_1}{w_{t,i}}\,.$$

To derive an upper bound on Φ_t, we analyze the difference of the potential functions at two consecutive rounds. For all $t \in I$, this difference can be expressed and bounded as follows:

$$\begin{aligned}
\Phi_{t+1} - \Phi_t &= \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{w_{t,i}}{w_{t+1,i}} \\
&= \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{Z_t}{\exp(\eta y_t x_{t,i})} \\
&= \log Z_t - \eta \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} y_t x_{t,i} \\
&\leq \log \Big[\sum_{i=1}^{N} w_{t,i} \exp(\eta y_t x_{t,i}) \Big] - \eta\rho_\infty \\
&= \log \mathop{\mathrm{E}}_{\mathbf{w}_t} \big[\exp(\eta y_t x_t) \big] - \eta\rho_\infty \\
&\leq \log \big[\exp(\eta^2 (2r_\infty)^2/8) \big] - \eta\rho_\infty \\
&= \eta^2 r_\infty^2/2 - \eta\rho_\infty.
\end{aligned}$$

The first inequality follows the definition ρ_∞. The subsequent equality rewrites the summation as an expectation over the distribution defined by $\mathbf{w}_t$. The next inequality uses Hoeffding's lemma (lemma D.1). Summing up these inequalities over all $t \in I$ yields:

$$\Phi_{T+1} - \Phi_1 \leq M(\eta^2 r_\infty^2/2 - \eta\rho_\infty).$$

Next, we derive a lower bound by noting that

$$\Phi_1 = \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i/\|\mathbf{v}\|_1}{1/N} = \log N + \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i}{\|\mathbf{v}\|_1} \leq \log N \,.$$

Additionally, since the relative entropy is always non-negative, we have $\Phi_{T+1} \geq 0$. This yields the following lower bound:

$$\Phi_{T+1} - \Phi_1 \geq 0 - \log N = -\log N \,.$$

Combining the upper and lower bounds we see that $-\log N \leq M(\eta^2 r_\infty^2/2 - \eta\rho_\infty)$. Setting $\eta = \frac{\rho_\infty}{r_\infty^2}$ yields the statement of the theorem. ■

The margin-based mistake bounds of theorem 7.8 and theorem 7.12 for the Perceptron and Winnow algorithms have a similar form, but they are based on different norms. For both algorithms, the norm $\|\cdot\|_p$ used for the input vectors $\mathbf{x}_t$, $t \in [1, T]$, is the dual of the norm $\|\cdot\|_q$ used for the margin vector $\mathbf{v}$, that is p and q are conjugate: $1/p + 1/q = 1$: in the case of the Perceptron algorithm $p = q = 2$, while for Winnow $p = \infty$ and $q = 1$.

These bounds imply different types of guarantees. The bound for Winnow is favorable when a sparse set of the experts $i \in [1, N]$ can predict well. For example, if $\mathbf{v} = \mathbf{e}_1$ where $\mathbf{e}_1$ is the unit vector along the first axis in $\mathbb{R}^N$ and if $\mathbf{x}_t \in \{-1, +1\}^N$ for all t, then the upper bound on the number of mistakes given for Winnow by theorem 7.12 is only $\log N$, while the upper bound of theorem 7.8 for the Perceptron algorithm is N. The guarantee for the Perceptron algorithm is more favorable in the opposite situation, where sparse solutions are not effective.

7.4 On-line to batch conversion

The previous sections presented several algorithms for the scenario of on-line learning, including the Perceptron and Winnow algorithms, and analyzed their behavior within the mistake model, where no assumption is made about the way the training sequence is generated. Can these algorithms be used to derive hypotheses with small generalization error in the standard stochastic setting? How can the intermediate hypotheses they generate be combined to form an accurate predictor? These are the questions addressed in this section.

Let H be a hypothesis of functions mapping $\mathcal{X}$ to $\mathcal{Y}'$, and let $L \colon \mathcal{Y}' \times \mathcal{Y} \to \mathbb{R}_+$ be a bounded loss function, that is $L \leq M$ for some $M \geq 0$. We assume a standard supervised learning setting where a labeled sample $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ is drawn i.i.d. according to some fixed but unknown distribution D. The sample is sequentially processed by an on-line learning algorithm $\mathcal{A}$. The algorithm

starts with an initial hypothesis $h_1 \in H$ and generates a new hypothesis $h_{i+1} \in H$, after processing pair (x_i, y_i), $i \in [1, m]$. The regret of the algorithm is defined as before by

$$R_T = \sum_{i=1}^{T} L(h_i(x_i), y_i) - \min_{h \in H} \sum_{i=1}^{T} L(h(x_i), y_i). \tag{7.24}$$

The generalization error of a hypothesis $h \in H$ is its expected loss $R(h) = \mathrm{E}_{(x,y)\sim D}[L(h(x), y)]$.

The following lemma gives a bound on the average of the generalization errors of the hypotheses generated by $\mathcal{A}$ in terms of its average loss $\frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i)$.

Lemma 7.1
Let $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ be a labeled sample drawn i.i.d. according to D, L a loss bounded by M and $h_1, \ldots, h_{T+1}$ the sequence of hypotheses generated by an on-line algorithm $\mathcal{A}$ sequentially processing S. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds:

$$\frac{1}{T}\sum_{i=1}^{T} R(h_i) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i) + M\sqrt{\frac{2\log\frac{1}{\delta}}{T}}. \tag{7.25}$$

Proof For any $i \in [1, T]$, let V_i be the random variable defined by $V_i = R(h_i) - L(h_i(x_i), y_i)$. Observe that for any $i \in [1, T]$,

$$\mathrm{E}[V_i | x_1, \ldots, x_{i-1}] = R(h_i) - \mathrm{E}[L(h_i(x_i), y_i) | h_i] = R(h_i) - R(h_i) = 0.$$

Since the loss is bounded by M, V_i takes values in the interval $[-M, +M]$ for all $i \in [1, T]$. Thus, by Azuma's inequality (theorem D.2), $\Pr[\frac{1}{T}\sum_{i=1}^{T} V_i \geq \epsilon] \leq \exp(-2T\epsilon^2/(2M)^2))$. Setting the right-hand side to be equal to $\delta > 0$ yields the statement of the lemma. ■

When the loss function is convex with respect to its first argument, the lemma can be used to derive a bound on the generalization error of the average of the hypotheses generated by $\mathcal{A}$, $\frac{1}{T}\sum_{t=1}^{T} h_i$, in terms of the average loss of $\mathcal{A}$ on S, or in terms of the regret R_T and the infimum error of hypotheses in H.

Theorem 7.13
Let $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ be a labeled sample drawn i.i.d. according to D, L a loss bounded by M and convex with respect to its first argument, and $h_1, \ldots, h_{T+1}$ the sequence of hypotheses generated by an on-line algorithm $\mathcal{A}$ sequentially processing S. Then, for any $\delta > 0$, with probability at least $1 - \delta$, each

of the following holds:

$$R\left(\frac{1}{T}\sum_{i=1}^{T} h_i\right) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i) + M\sqrt{\frac{2\log\frac{1}{\delta}}{T}} \tag{7.26}$$

$$R\left(\frac{1}{T}\sum_{i=1}^{T} h_i\right) \leq \inf_{h\in H} R(h) + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \tag{7.27}$$

Proof By the convexity of L with respect to its first argument, for any $(x, y) \in \mathcal{X} \times \mathcal{Y}$, we have $L(\frac{1}{T}\sum_{i=1}^{T} h_i(x), y) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x), y)$. Taking the expectation gives $R(\frac{1}{T}\sum_{i=1}^{T} h_i) \leq \frac{1}{T}\sum_{i=1}^{T} R(h_i)$. The first inequality then follows by lemma 7.1. Thus, by definition of the regret R_T, for any $\delta > 0$, the following holds with probability at least $1 - \delta/2$:

$$\begin{aligned} R\left(\frac{1}{T}\sum_{i=1}^{T} h_i\right) &\leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\leq \min_{h\in H} \frac{1}{T}\sum_{i=1}^{T} L(h(x_i), y_i) + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \end{aligned}$$

By definition of $\inf_{h\in H} R(h)$, for any $\epsilon > 0$, there exists $h^* \in H$ with $R(h^*) \leq \inf_{h\in H} R(h) + \epsilon$. By Hoeffding's inequality, for any $\delta > 0$, with probability at least $1 - \delta/2$, $\frac{1}{T}\sum_{i=1}^{T} L(h^*(x_i), y_i) \leq R(h^*) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}$. Thus, for any $\epsilon > 0$, by the union bound, the following holds with probability at least $1 - \delta$:

$$\begin{aligned} R\left(\frac{1}{T}\sum_{i=1}^{T} h_i\right) &\leq \frac{1}{T}\sum_{i=1}^{T} L(h^*(x_i), y_i) + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\leq R(h^*) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &= R(h^*) + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\leq \inf_{h\in H} R(h) + \epsilon + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \end{aligned}$$

Since this inequality holds for all $\epsilon > 0$, it implies the second statement of the theorem. ■

The theorem can be applied to a variety of on-line regret minimization algorithms, for example when $R_T/T = O(1/\sqrt{T})$. In particular, we can apply the theorem to the exponential weighted average algorithm. Assuming that the loss L is bounded

by $M = 1$ and that the number of rounds T is known to the algorithm, we can use the regret bound of theorem 7.6. The doubling trick (used in theorem 7.7) can be used to derive a similar bound if T is not known in advance. Thus, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for the generalization error of the average of the hypotheses generated by exponential weighted average:

$$R\left(\frac{1}{T}\sum_{i=1}^{T} h_i\right) \leq \inf_{h \in H} R(h) + \sqrt{\frac{\log N}{2T}} + 2\sqrt{\frac{2\log \frac{2}{\delta}}{T}},$$

where N is the number of experts, or the dimension of the weight vectors.

7.5 Game-theoretic connection

The existence of regret minimization algorithms can be used to give a simple proof of von Neumann's theorem. For any $m \geq 1$, we will denote by Δ_m the set of all distributions over $\{1, \ldots, m\}$, that is $\Delta_m = \{\mathbf{p} \in \mathbb{R}^m \colon \mathbf{p} \geq 0 \wedge \|\mathbf{p}\|_1 = 1\}$.

Theorem 7.14 **Von Neumann's minimax theorem**
Let $m, n \geq 1$. Then, for any two-person zero-sum game defined by matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$,

$$\min_{\mathbf{p} \in \Delta_m} \max_{\mathbf{q} \in \Delta_n} \mathbf{p}^\top \mathbf{M} \mathbf{q} = \max_{\mathbf{q} \in \Delta_n} \min_{\mathbf{p} \in \Delta_m} \mathbf{p}^\top \mathbf{M} \mathbf{q}. \tag{7.28}$$

Proof The inequality $\max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \min_{\mathbf{p}} \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M} \mathbf{q}$ is straightforward, since by definition of min, for all $\mathbf{p} \in \Delta_m, \mathbf{q} \in \Delta_n$, we have $\min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \mathbf{p}^\top \mathbf{M} \mathbf{q}$. Taking the maximum over $\mathbf{q}$ of both sides gives: $\max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M} \mathbf{q}$ for all $\mathbf{p}$, subsequently taking the minimum over $\mathbf{p}$ proves the inequality.[2]

To show the reverse inequality, consider an on-line learning setting where at each round $t \in [1, T]$, algorithm $\mathcal{A}$ returns $\mathbf{p}_t$ and incurs loss $\mathbf{M}\mathbf{q}_t$. We can assume that $\mathbf{q}_t$ is selected in the optimal adversarial way, that is $\mathbf{q}_t \in \operatorname{argmax}_{q \in \Delta_m} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}$, and that $\mathcal{A}$ is a regret minimization algorithm, that is $R_T / T \to 0$, where $R_T = \sum_{t=1}^{T} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}_t - \min_{\mathbf{p} \in \Delta_m} \sum_{t=1}^{T} \mathbf{p}^\top \mathbf{M} \mathbf{q}_t$. Then, the following holds:

$$\min_{\mathbf{p} \in \Delta_m} \max_{\mathbf{q} \in \Delta_n} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \max_{\mathbf{q}} \left(\frac{1}{T}\sum_{t=1}^{T} \mathbf{p}_t\right)^\top \mathbf{M} \mathbf{q} \leq \frac{1}{T}\sum_{t=1}^{T} \max_{\mathbf{q}} \mathbf{p}_t^\top \mathbf{M} \mathbf{q} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}_t.$$

2. More generally, the maxmin is always upper bounded by the minmax for any function or two arguments and any constraint sets, following the same proof.

By definition of regret, the right-hand side can be expressed and bounded as follows:

$$\frac{1}{T}\sum_{t=1}^{T}\mathbf{p}_t^\top\mathbf{M}\mathbf{q}_t = \min_{\mathbf{p}\in\Delta_m}\frac{1}{T}\sum_{t=1}^{T}\mathbf{p}^\top\mathbf{M}\mathbf{q}_t + \frac{R_T}{T} = \min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\Big(\frac{1}{T}\sum_{t=1}^{T}\mathbf{q}_t\Big) + \frac{R_T}{T}$$
$$\leq \max_{\mathbf{q}\in\Delta_n}\min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\mathbf{q} + \frac{R_T}{T}.$$

This implies that the following bound holds for the minmax for all $T \geq 1$:

$$\min_{\mathbf{p}\in\Delta_m}\max_{\mathbf{q}\in\Delta_n}\mathbf{p}^\top\mathbf{M}\mathbf{q} \leq \max_{\mathbf{q}\in\Delta_n}\min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\mathbf{q} + \frac{R_T}{T}$$

Since $\lim_{T\to+\infty}\frac{R_T}{T} = 0$, this shows that $\min_{\mathbf{p}}\max_{\mathbf{q}}\mathbf{p}^\top\mathbf{M}\mathbf{q} \leq \max_{\mathbf{q}}\min_{\mathbf{p}}\mathbf{p}^\top\mathbf{M}\mathbf{q}$. ■

7.6 Chapter notes

Algorithms for regret minimization were initiated with the pioneering work of Hannan [1957] who gave an algorithm whose regret decreases as $O(\sqrt{T})$ as a function of T but whose dependency on N is linear. The weighted majority algorithm and the randomized weighted majority algorithm, whose regret is only logarithmic in N, are due to Littlestone and Warmuth [1989]. The exponentiated average algorithm and its analysis, which can be viewed as an extension of the WM algorithm to convex non-zero-one losses is due to the same authors [Littlestone and Warmuth, 1989, 1994]. The analysis we presented follows Cesa-Bianchi [1999] and Cesa-Bianchi and Lugosi [2006]. The doubling trick technique appears in Vovk [1990] and Cesa-Bianchi et al. [1997]. The algorithm of exercise 7.7 and the analysis leading to a second-order bound on the regret are due to Cesa-Bianchi et al. [2005]. The lower bound presented in theorem 7.5 is from Blum and Mansour [2007].

While the regret bounds presented are logarithmic in the number of the experts N, when N is exponential in the size of the input problem, the computational complexity of an expert algorithm could be exponential. For example, in the online shortest paths problem, N is the number of paths between two vertices of a directed graph. However, several computationally efficient algorithms have been presented for broad classes of such problems by exploiting their structure [Takimoto and Warmuth, 2002, Kalai and Vempala, 2003, Zinkevich, 2003].

The notion of regret (or *external regret*) presented in this chapter can be generalized to that of *internal regret* or even *swap regret*, by comparing the loss of the algorithm not just to that of the best expert in retrospect, but to that of any modification of the actions taken by the algorithm by replacing each occurrence of some specific action with another one (internal regret), or even replacing actions via an ar-

bitrary mapping (swap regret) [Foster and Vohra, 1997, Hart and Mas-Colell, 2000, Lehrer, 2003]. Several algorithms for low internal regret have been given [Foster and Vohra, 1997, 1998, 1999, Hart and Mas-Colell, 2000, Cesa-Bianchi and Lugosi, 2001, Stoltz and Lugosi, 2003], including a conversion of low external regret to low swap regret by Blum and Mansour [2005].

The Perceptron algorithm was introduced by Rosenblatt [1958]. The algorithm raised a number of reactions, in particular by Minsky and Papert [1969], who objected that the algorithm could not be used to recognize the XOR function. Of course, the kernel Perceptron algorithm already given by Aizerman et al. [1964] could straightforwardly succeed to do so using second-degree polynomial kernels. The margin bound for the Perceptron algorithm was proven by Novikoff [1962] and is one of the first results in learning theory. The leave-one-out analysis for SVMs is described by Vapnik [1998]. The upper bound presented for the Perceptron algorithm in the non-separable case is by Freund and Schapire [1999a]. The Winnow algorithm was introduced by Littlestone [1987].

The analysis of the on-line to batch conversion and exercise 7.10 are from Cesa-Bianchi et al. [2001, 2004] (see also Littlestone [1989]). Von Neumann's minimax theorem admits a number of different generalizations. See Sion [1958] for a generalization to quasi-concave-convex functions semi-continuous in each argument and the references therein. The simple proof of von Neumann's theorem presented here is entirely based on learning-related techniques. A proof of a more general version using multiplicative updates was presented by Freund and Schapire [1999b].

On-line learning is a very broad and fast-growing research area in machine learning. The material presented in this chapter should be viewed only as an introduction to the topic, but the proofs and techniques presented should indicate the flavor of most results in this area. For a more comprehensive presentation of on-line learning and related game theory algorithms and techniques, the reader could consult the book of Cesa-Bianchi and Lugosi [2006].

7.7 Exercises

7.1 Perceptron lower bound. Let S be a labeled sample of m points in $\mathbb{R}^N$ with

$$x_i = (\underbrace{(-1)^i, \ldots, (-1)^i, (-1)^{i+1}}_{i \text{ first components}}, 0, \ldots, 0) \quad \text{and} \quad y_i = (-1)^{i+1}. \tag{7.29}$$

Show that the Perceptron algorithm makes $\Omega(2^N)$ updates before finding a separating hyperplane, regardless of the order in which it receives the points.

7.2 Generalized mistake bound. Theorem 7.8 presents a margin bound on the

```
ON-LINE-SVM(w_0)
1  w_1 ← w_0        ▷ typically w_0 = 0
2  for t ← 1 to T do
3       RECEIVE(x_t, y_t)
4       if y_t(w_t · x_t) < 1 then
5            w_{t+1} ← w_t − η(w_t − C y_t x_t)
6       elseif y_t(w_t · x_t) > 1 then
7            w_{t+1} ← w_t − η w_t
8       else w_{t+1} ← w_t
9  return w_{T+1}
```

Figure 7.11 On-line SVM algorithm.

maximum number of updates for the Perceptron algorithm for the special case $\eta = 1$. Consider now the general Perceptron update $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta y_t \mathbf{x}_t$, where $\eta > 0$. Prove a bound on the maximum number of mistakes. How does η affect the bound?

7.3 Sparse instances. Suppose each input vector $\mathbf{x}_t$, $t \in [1, T]$, coincides with the tth unit vector of $\mathbb{R}^T$. How many updates are required for the Perceptron algorithm to converge? Show that the number of updates matches the margin bound of theorem 7.8.

7.4 Tightness of lower bound. Is the lower bound of theorem 7.5 tight? Explain why or show a counter-example.

7.5 On-line SVM algorithm. Consider the algorithm described in figure 7.11. Show that this algorithm corresponds to the stochastic gradient descent technique applied to the SVM problem (4.23) with hinge loss and no offset (i.e., fix $p = 1$ and $b = 0$).

7.6 Margin Perceptron. Given a training sample S that is linearly separable with a maximum margin $\rho > 0$, theorem 7.8 states that the Perceptron algorithm run cyclically over S is guaranteed to converge after at most R^2/ρ^2 updates, where R is the radius of the sphere containing the sample points. However, this theorem does not guarantee that the hyperplane solution of the Perceptron algorithm achieves a margin close to ρ. Suppose we modify the Perceptron algorithm to ensure that

MarginPerceptron()

1 $\mathbf{w}_1 \leftarrow \mathbf{0}$
2 **for** $t \leftarrow 1$ **to** T **do**
3 Receive($\mathbf{x}_t$)
4 Receive(y_t)
5 **if** $\left((\mathbf{w}_t = 0)$ **or** $(\frac{y_t \mathbf{w}_t \cdot \mathbf{x}_t}{\|\mathbf{w}_t\|} < \frac{\rho}{2})\right)$ **then**
6 $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_t \mathbf{x}_t$
7 **else** $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t$
8 **return** $\mathbf{w}_{T+1}$

Figure 7.12 Margin Perceptron algorithm.

the margin of the hyperplane solution is at least $\rho/2$. In particular, consider the algorithm described in figure 7.12. In this problem we show that this algorithm converges after at most $16R^2/\rho^2$ updates. Let I denote the set of times $t \in [1, T]$ at which the algorithm makes an update and let $M = |I|$ be the total number of updates.

(a) Using an analysis similar to the one given for the Perceptron algorithm, show that $M\rho \leq \|\mathbf{w}_{T+1}\|$. Conclude that if $\|\mathbf{w}_{T+1}\| < \frac{4R^2}{\rho}$, then $M < 4R^2/\rho^2$. (For the remainder of this problem, we will assume that $\|\mathbf{w}_{T+1}\| \geq \frac{4R^2}{\rho}$.)

(b) Show that for any $t \in I$ (including $t = 0$), the following holds:

$$\|\mathbf{w}_{t+1}\|^2 \leq (\|\mathbf{w}_t\| + \rho/2)^2 + R^2.$$

(c) From (b), infer that for any $t \in I$ we have

$$\|\mathbf{w}_{t+1}\| \leq \|\mathbf{w}_t\| + \rho/2 + \frac{R^2}{\|\mathbf{w}_t\| + \|\mathbf{w}_{t+1}\| + \rho/2}.$$

(d) Using the inequality from (c), show that for any $t \in I$ such that either $\|\mathbf{w}_t\| \geq \frac{4R^2}{\rho}$ or $\|\mathbf{w}_{t+1}\| \geq \frac{4R^2}{\rho}$, we have

$$\|\mathbf{w}_{t+1}\| \leq \|\mathbf{w}_t\| + \frac{3}{4}\rho.$$

(e) Show that $\|\mathbf{w}_1\| \leq R \leq 4R^2/\rho$. Since by assumption we have $\|\mathbf{w}_{T+1}\| \geq \frac{4R^2}{\rho}$, conclude that there must exist a largest time $t_0 \in I$ such that $\|\mathbf{w}_{t_0}\| \leq \frac{4R^2}{\rho}$

and $\|\mathbf{w}_{t_0+1}\| \geq \frac{4R^2}{\rho}$.

(f) Show that $\|\mathbf{w}_{T+1}\| \leq \|\mathbf{w}_{t_0}\| + \frac{3}{4}M\rho$. Conclude that $M \leq 16R^2/\rho^2$.

7.7 Second-order regret bound. Consider the randomized algorithm that differs from the RWM algorithm only by the weight update, i.e., $w_{t+1,i} \leftarrow (1-(1-\beta)l_{t,i})w_{t,i}$, $t \in [1,T]$, which is applied to all $i \in [1,N]$ with $1/2 \leq \beta < 1$. This algorithm can be used in a more general setting than RWM since the losses $l_{t,i}$ are only assumed to be in $[0,1]$. The objective of this problem is to show that a similar upper bound can be shown for the regret.

(a) Use the same potential W_t as for the RWM algorithm and derive a simple upper bound for $\log W_{T+1}$:

$$\log W_{T+1} \leq \log N - (1-\beta)\mathcal{L}_T\,.$$

(*Hint*: Use the identity $\log(1-x) \leq -x$ for $x \in [0,1/2]$.)

(b) Prove the following lower bound for the potential for all $i \in [1,N]$:

$$\log W_{T+1} \geq -(1-\beta)\mathcal{L}_{T,i} - (1-\beta)^2 \sum_{t=1}^{T} l_{t,i}^2\,.$$

(*Hint*: Use the identity $\log(1-x) \geq -x-x^2$, which is valid for all $x \in [0,1/2]$.)

(c) Use upper and lower bounds to derive the following regret bound for the algorithm: $R_T \leq 2\sqrt{T\log N}$.

7.8 Polynomial weighted algorithm. The objective of this problem is to show how another regret minimization algorithm can be defined and studied. Let L be a loss function convex in its first argument and taking values in $[0,M]$.

We will assume $N > e^2$ and then for any expert $i \in [1,N]$, we denote by $r_{t,i}$ the instantaneous regret of that expert at time $t \in [1,T]$, $r_{t,i} = L(\widehat{y}_t, y_t) - L(y_{t,i}, y_t)$, and by $R_{t,i}$ his cumulative regret up to time t: $R_{t,i} = \sum_{s=1}^{t} r_{t,i}$. For convenience, we also define $R_{0,i} = 0$ for all $i \in [1,N]$. For any $x \in \mathbb{R}$, $(x)_+$ denotes $\max(x,0)$, that is the positive part of x, and for $\mathbf{x} = (x_1,\ldots,x_N)^\top \in \mathbb{R}^N$, $(\mathbf{x})_+ = ((x_1)_+,\ldots,(x_N)_+)^\top$.

Let $\alpha > 2$ and consider the algorithm that predicts at round $t \in [1,T]$ according to $\widehat{y}_t = \frac{\sum_{i=1}^{n} w_{t,i}y_{t,i}}{\sum_{i=1}^{n} w_{t,i}}$, with the weight $w_{t,i}$ defined based on the αth power of the regret up to time $(t-1)$: $w_{t,i} = (R_{t-1,i})_+^{\alpha-1}$. The potential function we use to analyze the algorithm is based on the function Φ defined over $\mathbb{R}^N$ by $\Phi\colon \mathbf{x} \mapsto \|(\mathbf{x})_+\|_\alpha^2 = \left[\sum_{i=1}^{N}(x_i)_+^\alpha\right]^{\frac{2}{\alpha}}$.

(a) Show that Φ is twice differentiable over $\mathbb{R}^N - B$, where B is defined as follows:

$$B = \{\mathbf{u} \in \mathbb{R}^N : (\mathbf{u})_+ = 0\}.$$

(b) For any $t \in [1, T]$, let $\mathbf{r}_t$ denote the vector of instantaneous regrets, $\mathbf{r}_t = (r_{t,1}, \ldots, r_{t,N})^\top$, and similarly $\mathbf{R}_t = (R_{t,1}, \ldots, R_{t,N})^\top$. We define the potential function as $\Phi(\mathbf{R}_t) = \|(\mathbf{R}_t)_+\|_\alpha^2$. Compute $\nabla\Phi(\mathbf{R}_{t-1})$ for $\mathbf{R}_{t-1} \notin B$ and show that $\nabla\Phi(\mathbf{R}_{t-1}) \cdot \mathbf{r}_t \leq 0$ (*Hint*: use the convexity of the loss with respect to the first argument).

(c) Prove the inequality $\mathbf{r}^\top[\nabla^2\Phi(\mathbf{u})]\mathbf{r} \leq 2(\alpha - 1)\|\mathbf{r}\|_\alpha^2$ valid for all $\mathbf{r} \in \mathbb{R}^N$ and $\mathbf{u} \in \mathbb{R}^N - B$ (*Hint*: write the Hessian $\nabla^2\Phi(\mathbf{u})$ as a sum of a diagonal matrix and a positive semi-definite matrix multiplied by $(2 - \alpha)$. Also, use Hölder's inequality generalizing Cauchy-Schwarz : for any $p > 1$ and $q > 1$ with $\frac{1}{p} + \frac{1}{q} = 1$ and $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$, $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|_p \|\mathbf{v}\|_q$).

(d) Using the answers to the two previous questions and Taylor's formula, show that for all $t \geq 1$, $\Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha - 1)\|\mathbf{r}_t\|_\alpha^2$, if $\gamma\mathbf{R}_{t-1} + (1 - \gamma)\mathbf{R}_t \notin B$ for all $\gamma \in [0, 1]$.

(e) Suppose there exists $\gamma \in [0, 1]$ such that $(1-\gamma)\mathbf{R}_{t-1} + \gamma\mathbf{R}_t \in B$. Show that $\Phi(\mathbf{R}_t) \leq (\alpha - 1)\|\mathbf{r}_t\|_\alpha^2$.

(f) Using the two previous questions, derive an upper bound on $\Phi(\mathbf{R}_T)$ expressed in terms of T, N, and M.

(g) Show that $\Phi(\mathbf{R}_T)$ admits as a lower bound the square of the regret R_T of the algorithm.

(h) Using the two previous questions give an upper bound on the regret R_T. For what value of α is the bound the most favorable? Give a simple expression of the upper bound on the regret for a suitable approximation of that optimal value.

7.9 General inequality. In this exercise we generalize the result of exercise 7.7 by using a more general inequality: $\log(1 - x) \geq -x - \frac{x^2}{\alpha}$ for some $0 < \alpha < 2$.

(a) First prove that the inequality is true for $x \in [0, 1 - \frac{\alpha}{2}]$. What does this imply about the valid range of β?

(b) Give a generalized version of the regret bound derived in exercise 7.7 in terms of α, which shows:

$$R_T \leq \frac{\log N}{1 - \beta} + \frac{1 - \beta}{\alpha} T\,.$$

What is the optimal choice of β and the resulting bound in this case?

(c) Explain how α may act as a regularization parameter. What is the optimal choice of α?

7.10 On-line to batch. Consider the margin loss (4.3), which is convex. Our goal is to apply theorem 7.13 to the kernel Perceptron algorithm using the margin loss.

(a) Show that the regret R_T can be bounded as $R_T \leq \sqrt{\mathrm{Tr}[\mathbf{K}]/\rho^2}$ where ρ is the margin and $\mathbf{K}$ is the kernel matrix associated to the sequence $x_1, \ldots, x_T$.

(b) Apply theorem 7.13. How does this result compare with the margin bounds for kernel-based hypotheses given by corollary 5.1?

7.11 On-line to batch — non-convex loss. The on-line to batch result of theorem 7.13 heavily relies on the fact that the loss in convex in order to provide a generalization guarantee for the uniformly averaged hypothesis $\frac{1}{T}\sum_{i=1}^{T} h_i$. For general losses, instead of using the averaged hypothesis we will use a different strategy and try to estimate the best single base hypothesis and show the expected loss of this hypothesis is bounded.

Let m_i denote the number of errors of hypothesis h_i makes on the points $(x_i, \ldots, x_T)$, i.e. the subset of points in the sequence that are not used to train h_i. Then we define the *penalized risk estimate* of hypothesis h_i as,

$$\frac{m_i}{T-i+1} + c_\delta(T-i+1) \quad \text{where} \quad c_\delta(x) = \sqrt{\frac{1}{2x}\log\frac{T(T+1)}{\delta}}\,.$$

The term c_δ penalizes the empirical error when the test sample is small. Define $\widehat{h} = h_{i*}$ where $i* = \mathrm{argmin}_i\, m_i/(T-i) + c_\delta(T-i+1)$. We will then show under the same conditions of theorem 7.13 (with $M = 1$ for simplicity), but without requiring the convexity of L, that the following holds with probability at least $1-\delta$:

$$R(\widehat{h}) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i) + 6\sqrt{\frac{1}{T}\log\frac{2(T+1)}{\delta}}\,. \tag{7.30}$$

(a) Prove the following inequality:

$$\min_{i\in[1,T]} \left(R(h_i) + 2c_\delta(T-i+1)\right) \leq \frac{1}{T}\sum_{i=1}^{T} R(h_i) + 4\sqrt{\frac{1}{T}\log\frac{T+1}{\delta}}\,.$$

(b) Use part (a) to show that with probability at least $1-\delta$,

$$\min_{i\in[1,T]}(R(h_i)+2c_\delta(T-i+1)) < \sum_{i=1}^{T} L(h_i(x_i),y_i)+\sqrt{\frac{2}{T}\log\frac{1}{\delta}}+4\sqrt{\frac{1}{T}\log\frac{T+1}{\delta}}\,.$$

(c) By design, the definition of c_δ ensures that with probability at least $1-\delta$

$$R(\widehat{h}) \le \min_{i\in[1,T]}(R(h_i)+2c_\delta(T-i+1))\,.$$

Use this property to complete the proof of (7.30).

8 Multi-Class Classification

The classification problems we examined in the previous chapters were all binary. However, in most real-world classification problems the number of classes is greater than two. The problem may consist of assigning a topic to a text document, a category to a speech utterance or a function to a biological sequence. In all of these tasks, the number of classes may be on the order of several hundred or more.

In this chapter, we analyze the problem of multi-class classification. We first introduce the multi-class classification learning problem and discuss its multiple settings, and then derive generalization bounds for it using the notion of Rademacher complexity. Next, we describe and analyze a series of algorithms for tackling the multi-class classification problem. We will distinguish between two broad classes of algorithms: *uncombined algorithms* that are specifically designed for the multi-class setting such as multi-class SVMs, decision trees, or multi-class boosting, and *aggregated algorithms* that are based on a reduction to binary classification and require training multiple binary classifiers. We will also briefly discuss the problem of structured prediction, which is a related problem arising in a variety of applications.

8.1 Multi-class classification problem

Let $\mathcal{X}$ denote the input space and $\mathcal{Y}$ denote the output space, and let D be an unknown distribution over $\mathcal{X}$ according to which input points are drawn. We will distinguish between two cases: the *mono-label case*, where $\mathcal{Y}$ is a finite set of classes that we mark with numbers for convenience, $\mathcal{Y} = \{1, \ldots, k\}$, and the *multi-label case* where $\mathcal{Y} = \{-1, +1\}^k$. In the mono-label case, each example is labeled with a single class, while in the multi-label case it can be labeled with several. The latter can be illustrated by the case of text documents, which can be labeled with several different relevant topics, e.g., *sports*, *business*, and *society*. The positive components of a vector in $\{-1, +1\}^k$ indicate the classes associated with an example.

In either case, the learner receives a labeled sample $S = \big((x_1, y_1), \ldots, (x_m, y_m)\big) \in (\mathcal{X} \times \mathcal{Y})^m$ with $x_1, \ldots, x_m$ drawn i.i.d. according to D, and $y_i = f(x_i)$ for all $i \in [1, m]$, where $f \colon \mathcal{X} \to \mathcal{Y}$ is the target labeling function. Thus, we consider a

deterministic scenario, which, as discussed in section 2.4.1, can be straightforwardly extended to a stochastic one where we have a distribution over $\mathcal{X} \times \mathcal{Y}$.

Given a hypothesis set H of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, the multi-class classification problem consists of using the labeled sample S to find a hypothesis $h \in H$ with small generalization error $R(h)$ with respect to the target f:

$$R(h) = \mathop{\mathrm{E}}_{x \sim D}[1_{h(x) \neq f(x)}] \qquad \text{mono-label case} \tag{8.1}$$

$$R(h) = \mathop{\mathrm{E}}_{x \sim D}\Big[\sum_{l=1}^{k} 1_{[h(x)]_l \neq [f(x)]_l}\Big] \qquad \text{multi-label case.} \tag{8.2}$$

The notion of *Hamming distance* d_H, that is, the number of corresponding components in two vectors that differ, can be used to give a common formulation for both errors:

$$R(h) = \mathop{\mathrm{E}}_{x \sim D}\Big[d_H(h(x), f(x))\Big]. \tag{8.3}$$

The empirical error of $h \in H$ is denoted by $\widehat{R}(h)$ and defined by

$$\widehat{R}(h) = \frac{1}{m} \sum_{i=1}^{m} d_H(h(x_i), y_i)\,. \tag{8.4}$$

Several issues, both computational and learning-related, often arise in the multi-class setting. Computationally, dealing with a large number of classes can be problematic. The number of classes k directly enters the time complexity of the algorithms we will present. Even for a relatively small number of classes such as $k = 100$ or $k = 1{,}000$, some techniques may become prohibitive to use in practice. This dependency is even more critical in the case where k is very large or even infinite as in the case of some structured prediction problems.

A learning-related issue that commonly appears in the multi-class setting is the existence of unbalanced classes. Some classes may be represented by less than 5 percent of the labeled sample, while others may dominate a very large fraction of the data. When separate binary classifiers are used to define the multi-class solution, we may need to train a classifier distinguishing between two classes with only a small representation in the training sample. This implies training on a small sample, with poor performance guarantees. Alternatively, when a large fraction of the training instances belong to one class, it may be tempting to propose a hypothesis always returning that class, since its generalization error as defined earlier is likely to be relatively low. However, this trivial solution is typically not the one intended. Instead, the loss function may need to be reformulated by assigning different misclassification weights to each pair of classes.

Another learning-related issue is the relationship between classes, which can

be hierarchical. For example, in the case of document classification, the error of misclassifying a document dealing with *world politics* as one dealing with *real estate* should naturally be penalized more than the error of labeling a document with *sports* instead of the more specific label *baseball*. Thus, a more complex and more useful multi-class classification formulation would take into consideration the hierarchical relationships between classes and define the loss function in accordance with this hierarchy. More generally, there may be a graph relationship between classes as in the case of the GO ontology in computational biology. The use of hierarchical relationships between classes leads to a richer and more complex multi-class classification problem.

8.2 Generalization bounds

In this section, we present margin-based generalization bounds for multi-class classification in the mono-label case. In the binary setting, classifiers are often defined based on the sign of a scoring function. In the multi-class setting, a hypothesis is defined based on a scoring function $h\colon \mathcal{X}\times\mathcal{Y} \to \mathbb{R}$. The label associated to point x is the one resulting in the largest score $h(x, y)$, which defines the following mapping from $\mathcal{X}$ to $\mathcal{Y}$:

$$x \mapsto \operatorname*{argmax}_{y\in\mathcal{Y}} h(x, y).$$

This naturally leads to the following definition of the *margin* $\rho_h(x, y)$ *of the function* h *at a labeled example* (x, y):

$$\rho_h(x, y) = h(x, y) - \max_{y'\neq y} h(x, y').$$

Thus, h misclassifies (x, y) iff $\rho_h(x, y) \leq 0$. For any $\rho > 0$, we can define the *empirical margin loss of a hypothesis* h for multi-class classification as

$$\widehat{R}_\rho(h) = \frac{1}{m}\sum_{i=1}^{m} \Phi_\rho(\rho_h(x_i, y_i)), \tag{8.5}$$

where Φ_ρ is the margin loss function (definition 4.3). Thus, the empirical margin loss for multi-class classification is upper bounded by the fraction of the training points misclassified by h or correctly classified but with confidence less than or equal to ρ:

$$\widehat{R}_\rho(h) \leq \frac{1}{m}\sum_{i=1}^{m} 1_{\rho_h(x_i,y_i)\leq\rho}. \tag{8.6}$$

The following lemma will be used in the proof of the main result of this section.

Lemma 8.1
Let $\mathcal{F}_1, \ldots, \mathcal{F}_l$ be l hypothesis sets in $\mathbb{R}^{\mathcal{X}}$, $l \geq 1$, and let $\mathcal{G} = \{\max\{h_1, \ldots, h_l\} \colon h_i \in \mathcal{F}_i, i \in [1, l]\}$. Then, for any sample S of size m, the empirical Rademacher complexity of $\mathcal{G}$ can be upper bounded as follows:

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \sum_{j=1}^{l} \widehat{\mathfrak{R}}_S(\mathcal{F}_j). \tag{8.7}$$

Proof Let $S = (x_1, \ldots, x_m)$ be a sample of size m. We first prove the result in the case $l = 2$. By definition of the max operator, for any $h_1 \in \mathcal{F}_1$ and $h_2 \in \mathcal{F}_2$,

$$\max\{h_1, h_2\} = \frac{1}{2}[h_1 + h_2 + |h_1 - h_2|].$$

Thus, we can write:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{G}) &= \frac{1}{m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{\substack{h_1 \in \mathcal{F}_1 \\ h_2 \in \mathcal{F}_2}} \sum_{i=1}^{m} \sigma_i \max\{h_1(x_i), h_2(x_i)\} \Big] \\
&= \frac{1}{2m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{\substack{h_1 \in \mathcal{F}_1 \\ h_2 \in \mathcal{F}_2}} \sum_{i=1}^{m} \sigma_i \big(h_1(x_i) + h_2(x_i) + |(h_1 - h_2)(x_i)|\big) \Big] \\
&\leq \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_2) + \frac{1}{2m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{\substack{h_1 \in \mathcal{F}_1 \\ h_2 \in \mathcal{F}_2}} \sum_{i=1}^{m} \sigma_i |(h_1 - h_2)(x_i)| \Big],
\end{aligned} \tag{8.8}$$

using the sub-additivity of sup. Since $x \mapsto |x|$ is 1-Lipschitz, by Talagrand's lemma (lemma 4.2), the last term can be bounded as follows

$$\begin{aligned}
\frac{1}{2m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{\substack{h_1 \in \mathcal{F}_1 \\ h_2 \in \mathcal{F}_2}} \sum_{i=1}^{m} \sigma_i |(h_1 - h_2)(x_i)| \Big] &\leq \frac{1}{2m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{\substack{h_1 \in \mathcal{F}_1 \\ h_2 \in \mathcal{F}_2}} \sum_{i=1}^{m} \sigma_i (h_1 - h_2)(x_i) \Big] \\
&\leq \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \frac{1}{2m} \mathop{\mathrm{E}}_{\boldsymbol{\sigma}} \Big[\sup_{h_2 \in \mathcal{F}_2} \sum_{i=1}^{m} -\sigma_i h_2(x_i) \Big] \\
&= \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_2),
\end{aligned} \tag{8.9}$$

where we again use the sub-additivity of sup for the second inequality and the fact that σ_i and $-\sigma_i$ have the same distribution for any $i \in [1, m]$ for the last equality. Combining (8.8) and (8.9) yields $\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \widehat{\mathfrak{R}}_S(\mathcal{F}_2)$. The general case can be derived from the case $l = 2$ using $\max\{h_1, \ldots, h_l\} = \max\{h_1, \max\{h_2, \ldots, h_l\}\}$ and an immediate recurrence. ■

For any family of hypotheses mapping $\mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$, we define $\Pi_1(H)$ by

$$\Pi_1(H) = \{x \mapsto h(x, y) \colon y \in \mathcal{Y}, h \in H\}.$$

The following theorem gives a general margin bound for multi-class classification.

Theorem 8.1 **Margin bound for multi-class classification**
Let $H \subseteq \mathbb{R}^{\mathcal{X} \times \mathcal{Y}}$ be a hypothesis set with $\mathcal{Y} = \{1, \ldots, k\}$. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following multi-class classification generalization bound holds for all $h \in H$:

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2k^2}{\rho} \mathfrak{R}_m(\Pi_1(H)) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{8.10}$$

Proof The first part of the proof is similar to that of theorem 4.4. Let $\widetilde{H}$ be the family of hypotheses mapping $\mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$ defined by $\widetilde{H} = \{z = (x, y) \mapsto \rho_h(x, y) \colon h \in H\}$. Consider the family of functions $\widetilde{\mathcal{H}} = \{\Phi_\rho \circ r \colon r \in \widetilde{H}\}$ derived from $\widetilde{H}$, which take values in $[0, 1]$. By theorem 3.1, with probability at least $1 - \delta$, for all $h \in H$,

$$\mathrm{E}\left[\Phi_\rho(\rho_h(x, y))\right] \leq \widehat{R}_\rho(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

Since $1_{u \leq 0} \leq \Phi_\rho(u)$ for all $u \in \mathbb{R}$, the generalization error $R(h)$ is a lower bound on the left-hand side, $R(h) = \mathrm{E}[1_{y[h(x') - h(x)] \leq 0}] \leq \mathrm{E}\left[\Phi_\rho(\rho_h(x, y))\right]$, and we can write:

$$R(h) \leq \widehat{R}_\rho(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

As in the proof of theorem 4.4, we can show that $\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{H}\big) \leq \frac{1}{\rho}\mathfrak{R}_m(\widetilde{H})$ using

the $(1/\rho)$-Lipschitzness of Φ_ρ. Here, $\mathfrak{R}_m(\widetilde{H})$ can be upper bounded as follows:

$$\begin{aligned}
\mathfrak{R}_m(\widetilde{H}) &= \frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \rho_h(x_i, y_i)\Big] \\
&= \frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sum_{y\in\mathcal{Y}} \sigma_i \rho_h(x_i, y) 1_{y=y_i}\Big] \\
&\le \frac{1}{m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \rho_h(x_i, y) 1_{y=y_i}\Big] && \text{(sub-additivity of sup)} \\
&= \frac{1}{m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \rho_h(x_i, y)\big(\tfrac{2(1_{y=y_i})-1}{2} + \tfrac{1}{2}\big)\Big] \\
&\le \frac{1}{2m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \epsilon_i \rho_h(x_i, y)\Big] + && \big(\epsilon_i = 2(1_{y=y_i}) - 1\big) \\
&\qquad \frac{1}{2m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \rho_h(x_i, y)\Big] && \text{(sub-additivity of sup)} \\
&= \frac{1}{m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \rho_h(x_i, y)\Big],
\end{aligned}$$

where by definition $\epsilon_i \in \{-1, +1\}$ and we use the fact that σ_i and $\sigma_i\epsilon_i$ have the same distribution.

Let $\Pi_1(H)^{(k-1)} = \{\max\{h_1, \ldots, h_l\} \colon h_i \in \Pi_1(H), i \in [1, k-1]\}$. Now, rewriting $\rho_h(x_i, y)$ explicitly, using again the sub-additivity of sup, observing that $-\sigma_i$ and σ_i are distributed in the same way, and using lemma 8.1 leads to

$$\begin{aligned}
\mathfrak{R}_m(\widetilde{H}) &\le \frac{1}{m} \sum_{y\in\mathcal{Y}} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \big(h(x_i, y) - \max_{y'\neq y} h(x_i, y')\big)\Big] \\
&\le \sum_{y\in\mathcal{Y}} \left[\frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i h(x_i, y)\Big] + \frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} -\sigma_i \max_{y'\neq y} h(x_i, y')\Big]\right] \\
&= \sum_{y\in\mathcal{Y}} \left[\frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i h(x_i, y)\Big] + \frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in H} \sum_{i=1}^{m} \sigma_i \max_{y'\neq y} h(x_i, y')\Big]\right] \\
&\le \sum_{y\in\mathcal{Y}} \left[\frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in \Pi_1(H)} \sum_{i=1}^{m} \sigma_i h(x_i)\Big] + \frac{1}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in \Pi_1(H)^{(k-1)}} \sum_{i=1}^{m} \sigma_i h(x_i)\Big]\right] \\
&\le k \left[\frac{k}{m} \mathop{\mathrm{E}}_{S,\sigma}\Big[\sup_{h\in \Pi_1(H)} \sum_{i=1}^{m} \sigma_i h(x_i)\Big]\right] = k^2 \mathfrak{R}_m(\Pi_1(H)).
\end{aligned}$$

This concludes the proof. ■

These bounds can be generalized to hold uniformly for all $\rho > 0$ at the cost of an additional term $\sqrt{(\log \log_2(2/\rho))/m}$, as in theorem 4.5 and exercise 4.2. As for other margin bounds presented in previous sections, they show the conflict between two terms: the larger the desired pairwise ranking margin ρ, the smaller the middle term, at the price of a larger empirical multi-class classification margin loss $\widehat{R}_\rho$. Note, however, that here there is additionally a quadratic dependency on the number of classes k. This suggests weaker guarantees when learning with a large number of classes or the need for even larger margins ρ for which the empirical margin loss would be small.

For some hypothesis sets, a simple upper bound can be derived for the Rademacher complexity of $\Pi_1(H)$, thereby making theorem 8.1 more explicit. We will show this for kernel-based hypotheses. Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and let $\mathbf{\Phi}\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K. In multi-class classification, a kernel-based hypothesis is based on k weight vectors $\mathbf{w}_1, \ldots, \mathbf{w}_k \in \mathbb{H}$. Each weight vector $\mathbf{w}_l$, $l \in [1, k]$, defines a scoring function $x \mapsto \mathbf{w}_l \cdot \mathbf{\Phi}(x)$ and the class associated to point $x \in \mathcal{X}$ is given by

$$\operatorname*{argmax}_{y \in \mathcal{Y}} \mathbf{w}_y \cdot \mathbf{\Phi}(x).$$

We denote by $\mathbf{W}$ the matrix formed by these weight vectors: $\mathbf{W} = (\mathbf{w}_1^\top, \ldots, \mathbf{w}_k^\top)^\top$ and for any $p \geq 1$ denote by $\|\mathbf{W}\|_{\mathbb{H},p}$ the $L_{\mathbb{H},p}$ group norm of $\mathbf{W}$ defined by

$$\|\mathbf{W}\|_{\mathbb{H},p} = \Big(\sum_{l=1}^{k} \|\mathbf{w}_l\|_{\mathbb{H}}^p\Big)^{1/p}.$$

For any $p \geq 1$, the family of kernel-based hypotheses we will consider is[1]

$$H_{K,p} = \{(x, y) \in \mathcal{X} \times \{1, \ldots, k\} \mapsto \mathbf{w}_y \cdot \mathbf{\Phi}(x) \colon \mathbf{W} = (\mathbf{w}_1, \ldots, \mathbf{w}_k)^\top, \|\mathbf{W}\|_{\mathbb{H},p} \leq \Lambda\}.$$

Proposition 8.1 **Rademacher complexity of multi-class kernel-based hypotheses**

Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and let $\mathbf{\Phi}\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K. Assume that there exists $r > 0$ such that $K(x, x) \leq r^2$ for all $x \in \mathcal{X}$. Then, for any $m \geq 1$, $\mathfrak{R}_m(\Pi_1(H_{K,p}))$ can be bounded as follows:

$$\mathfrak{R}_m(\Pi_1(H_{K,p})) \leq \sqrt{\frac{r^2\Lambda^2}{m}}.$$

Proof Let $S = (x_1, \ldots, x_m)$ denote a sample of size m. Observe that for all

1. The hypothesis set H can also be defined via $H = \{h \in \mathbb{R}^{\mathcal{X} \times \mathcal{Y}} \colon h(\cdot, y) \in \mathbb{H} \wedge \|h\|_{K,p} \leq \Lambda\}$, where $\|h\|_{K,p} = \big(\sum_{y=1}^{k} \|h(\cdot, y)\|_{\mathbb{H}}^p\big)^{1/p}$, without referring to a feature mapping for K.

$l \in [1,k]$, the inequality $\|\mathbf{w}_l\|_{\mathbb{H}} \leq \left(\sum_{l=1}^{k} \|\mathbf{w}_l\|_{\mathbb{H}}^p\right)^{1/p} = \|\mathbf{W}\|_{\mathbb{H},p}$ holds. Thus, the condition $\|\mathbf{W}\|_{\mathbb{H},p} \leq \Lambda$ implies that $\|\mathbf{w}_l\|_{\mathbb{H}} \leq \Lambda$ for all $l \in [1,k]$. In view of that, the Rademacher complexity of the hypothesis set $\Pi_1(H_{K,p})$ can be expressed and bounded as follows:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\Pi_1(H_{K,p})) &= \frac{1}{m} \mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\sup_{\substack{y\in\mathcal{Y}\\ \|\mathbf{W}\|\leq\Lambda}} \left\langle \mathbf{w}_y, \sum_{i=1}^{m} \sigma_i \boldsymbol{\Phi}(x_i)\right\rangle\right] \\
&\leq \frac{1}{m} \mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\sup_{\substack{y\in\mathcal{Y}\\ \|\mathbf{W}\|\leq\Lambda}} \|\mathbf{w}_y\|_{\mathbb{H}} \left\| \sum_{i=1}^{m} \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}\right] \quad \text{(Cauchy-Schwarz ineq.)} \\
&\leq \frac{\Lambda}{m} \mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\left\| \sum_{i=1}^{m} \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}\right] \\
&\leq \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\left\| \sum_{i=1}^{m} \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}^2\right]\right]^{1/2} \quad \text{(Jensen's inequality)} \\
&= \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\sum_{i=1}^{m} \|\Phi(x_i)\|_{\mathbb{H}}^2\right]\right]^{1/2} \quad (i \neq j \Rightarrow \mathop{\mathrm{E}}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j] = 0) \\
&= \frac{\Lambda}{m}\left[\mathop{\mathrm{E}}_{S,\boldsymbol{\sigma}}\left[\sum_{i=1}^{m} K(x_i,x_i)\right]\right]^{1/2} \\
&\leq \frac{\Lambda\sqrt{mr^2}}{m} = \sqrt{\frac{r^2\Lambda^2}{m}},
\end{aligned}$$

which concludes the proof. ■

Combining theorem 8.1 and proposition 8.1 yields directly the following result.

Corollary 8.1 **Margin bound for multi-class classification with kernel-based hypotheses**
Let $K\colon \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ be a PDS kernel and let $\boldsymbol{\Phi}\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K. Assume that there exists $r > 0$ such that $K(x,x) \leq r^2$ for all $x \in \mathcal{X}$. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1-\delta$, the following multi-class classification generalization bound holds for all $h \in H_{K,p}$:

$$R(h) \leq \widehat{R}_\rho(h) + 2k^2\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}. \tag{8.11}$$

In the next two sections, we describe multi-class classification algorithms that belong to two distinct families: *uncombined algorithms*, which are defined by a single optimization problem, and *aggregated algorithms*, which are obtained by training multiple binary classifications and by combining their outputs.

8.3 Uncombined multi-class algorithms

In this section, we describe three algorithms designed specifically for multi-class classification. We start with a multi-class version of SVMs, then describe a boosting-type multi-class algorithm, and conclude with *decision trees*, which are often used as base learners in boosting.

8.3.1 Multi-class SVMs

We describe an algorithm that can be derived directly from the theoretical guarantees presented in the previous section. Proceeding as in section 4.4 for classification, the guarantee of corollary 8.1 can be expressed as follows: for any $\delta > 0$, with probability at least $1 - \delta$, for all $h \in H_{K,2} = \{(x, y) \to \mathbf{w}_y \cdot \mathbf{\Phi}(x) \colon \mathbf{W} = (\mathbf{w}_1, \ldots, \mathbf{w}_k)^\top, \sum_{l=1}^k \|\mathbf{w}_l\|^2 \leq \Lambda^2\}$,

$$R(h) \leq \frac{1}{m} \sum_{i=1}^m \xi_i + 4k^2 \sqrt{\frac{r^2 \Lambda^2}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}, \tag{8.12}$$

where $\xi_i = \max\left(1 - [\mathbf{w}_{y_i} \cdot \mathbf{\Phi}(x_i) - \max_{y' \neq y_i} \mathbf{w}_{y'} \cdot \mathbf{\Phi}(x_i)], 0\right)$ for all $i \in [1, m]$.

An algorithm based on this theoretical guarantee consists of minimizing the right-hand side of (8.12), that is, minimizing an objective function with a term corresponding to the sum of the slack variables ξ_i, and another one minimizing $\|\mathbf{W}\|_{\mathbb{H},2}$ or equivalently $\sum_{l=1}^k \|\mathbf{w}_l\|^2$. This is precisely the optimization problem defining the *multi-class SVM* algorithm:

$$\begin{aligned} \min_{\mathbf{W}, \boldsymbol{\xi}} \ & \frac{1}{2} \sum_{l=1}^k \|\mathbf{w}_l\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to: } & \forall i \in [1, m], \forall l \in \mathcal{Y} - \{y_i\}, \\ & \mathbf{w}_{y_i} \cdot \mathbf{\Phi}(x_i) \geq \mathbf{w}_l \cdot \mathbf{\Phi}(x_i) + 1 - \xi_i. \end{aligned}$$

The decision function learned is of the form $x \mapsto \operatorname{argmax}_{l \in Y} \mathbf{w}_l \cdot \mathbf{\Phi}(x)$. As with the primal problem of SVMs, this is a convex optimization problem: the objective function is convex, since it is a sum of convex functions, and the constraints are affine and thus qualified. The objective and constraint functions are differentiable, and the KKT conditions hold at the optimum. Defining the Lagrangian and applying these conditions leads to the equivalent dual optimization problem, which can be

expressed in terms of the kernel function K alone:

$$\max_{\boldsymbol{\alpha}\in\mathbb{R}^{m\times k}} \sum_{i=1}^{m} \boldsymbol{\alpha}_i \cdot \mathbf{e}_{y_i} - \frac{1}{2}\sum_{i=1}^{m}(\boldsymbol{\alpha}_i \cdot \boldsymbol{\alpha}_j)K(x_i, x_j)$$
$$\text{subject to: } 0 \leq \boldsymbol{\alpha}_i \leq \mathbf{C} \wedge \boldsymbol{\alpha}_i \cdot \mathbf{1} = 0, \forall i \in [1, m].$$

Here, $\boldsymbol{\alpha} \in \mathbb{R}^{m\times k}$ is a matrix, $\boldsymbol{\alpha}_i$ denotes the ith row of $\boldsymbol{\alpha}$, and $\mathbf{e}_l$ the lth unit vector in $\mathbb{R}^k$, $l \in [1, k]$. Both the primal and dual problems are simple QPs generalizing those of the standard SVM algorithm. However, the size of the solution and the number of constraints for both problems is in $\Omega(mk)$, which, for a large number of classes k, can make it difficult to solve. However, there exist specific optimization solutions designed for this problem based on a decomposition of the problem into m disjoint sets of constraints.

8.3.2 Multi-class boosting algorithms

We describe a boosting algorithm for multi-class classification called *AdaBoost.MH*, which in fact coincides with a special instance of AdaBoost. An alternative multi-class classification algorithm based on similar boosting ideas, AdaBoost.MR, is described and analyzed in exercise 9.5. AdaBoost.MH applies to the multi-label setting where $Y = \{-1, +1\}^k$. As in the binary case, it returns a convex combination of base classifiers selected from a hypothesis set H. Let F be the following objective function defined for all samples $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n$, $n \geq 1$, by

$$F(\boldsymbol{\alpha}) = \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l]g_n(x_i,l)} = \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l]\sum_{t=1}^{n}\alpha_t h_t(x_i,l)}, \tag{8.13}$$

where $g_n = \sum_{t=1}^{n}\alpha_t h_t$ and where $y_i[l]$ denotes the lth coordinate of y_i for any $i \in [1, m]$ and $l \in [1, k]$. F is a convex and differentiable upper bound on the multi-class multi-label loss:

$$\sum_{i=1}^{m}\sum_{l=1}^{k} 1_{y_i[l]\neq g_n(x_i,l)} \leq \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l]g_n(x_i,l)}, \tag{8.14}$$

since for any $x \in \mathcal{X}$ with label $y = f(x)$ and any $l \in [1, k]$, the inequality $1_{y[l]\neq g_n(x,l)} \leq e^{-y[l]g_n(x,l)}$ holds. AdaBoost.MH coincides exactly with the application of coordinate descent to the objective function F. Figure 8.1 gives the pseudocode of the algorithm in the case where the base classifiers are functions mapping from $\mathcal{X} \times \mathcal{Y}$ to $\{-1, +1\}$. The algorithm takes as input a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and maintains a distribution D_t over $\{1, \ldots, m\} \times Y$. The remaining details of the algorithm are similar to AdaBoost. In

AdaBoost.MH($S = ((x_1, y_1), \ldots, (x_m, y_m))$)

for $i \leftarrow 1$ **to** m **do**
  for $l \leftarrow 1$ **to** k **do**
    $D_1(i,l) \leftarrow \frac{1}{mk}$
for $t \leftarrow 1$ **to** T **do**
  $h_t \leftarrow$ base classifier in H with small error $\epsilon_t = \Pr_{(i,l)\sim D_t}[h_t(x_i,l) \neq y_i[l]]$
  $\alpha_t \leftarrow \frac{1}{2}\log\frac{1-\epsilon_t}{\epsilon_t}$
  $Z_t \leftarrow 2[\epsilon_t(1-\epsilon_t)]^{\frac{1}{2}}$ ▷ normalization factor
  for $i \leftarrow 1$ **to** m **do**
    for $l \leftarrow 1$ **to** k **do**
      $D_{t+1}(i,l) \leftarrow \frac{D_t(i,l)\exp(-\alpha_t y_i[l] h_t(x_i,l))}{Z_t}$
$g \leftarrow \sum_{t=1}^{T} \alpha_t h_t$
return $h = \operatorname{sgn}(g)$

Figure 8.1 AdaBoost.MH algorithm, for $H \subseteq (\{-1,+1\}^k)^{\mathcal{X}\times\mathcal{Y}}$.

fact, AdaBoost.MH exactly coincides with AdaBoost applied to the training sample derived from S by splitting each labeled point (x_i, y_i) into k labeled examples $((x_i, l), y_i[l])$, with each example (x_i, l) in $\mathcal{X} \times \mathcal{Y}$ and its label in $\{-1, +1\}$:

$$(x_i, y_i) \rightarrow ((x_i, 1), y_i[1]), \ldots, ((x_i, k), y_i[k]), i \in [1, m].$$

Let S' denote the resulting sample, then $S' = ((x_1, 1), y_1[1]), \ldots, (x_m, k), y_m[k]))$. S' contains mk examples and the expression of the objective function F in (8.13) coincides exactly with that of the objective function of AdaBoost for the sample S'. In view of this connection, the theoretical analysis along with the other observations we presented for AdaBoost in chapter 6 also apply here. Hence, we will focus on aspects related to the computational efficiency and to the weak learning condition that are specific to the multi-class scenario.

The complexity of the algorithm is that of AdaBoost applied to a sample of size mk. For $\mathcal{X} \subseteq \mathbb{R}^N$, using boosting stumps as base classifiers, the complexity of the algorithm is therefore in $O((mk)\log(mk) + mkNT)$. Thus, for a large number of classes k, the algorithm may become impractical using a single processor. The weak learning condition for the application of AdaBoost in this scenario requires that at each round there exists a base classifier $h_t \colon \mathcal{X} \times \mathcal{Y} \rightarrow \{-1, +1\}$ such that $\Pr_{(i,l)\sim D_t}[h_t(x_i, l) \neq y_i[l]] < 1/2$. This may be hard to achieve if classes are close

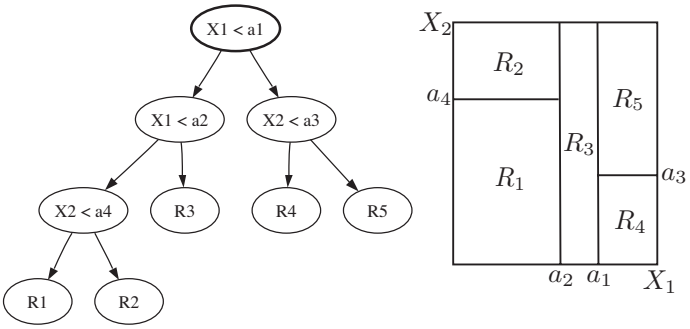


Figure 8.2 Left: example of a decision tree with numerical questions based on two variables X_1 and X_2. Here, each leaf is marked with the region it defines. The class labeling for a leaf is obtained via majority vote based on the training points falling in the region it defines. Right: Partition of the two-dimensional space induced by that decision tree.

and it is difficult to distinguish between them. It is also more difficult in this context to come up with "rules of thumb" h_t defined over $\mathcal{X} \times \mathcal{Y}$.

8.3.3 Decision trees

We present and discuss the general learning method of *decision trees* that can be used in multi-class classification, but also in other learning problems such as regression (chapter 10) and clustering. Although the empirical performance of decision trees often is not state-of-the-art, decision trees can be used as weak learners with boosting to define effective learning algorithms. Decision trees are also typically fast to train and evaluate and relatively easy to interpret.

Definition 8.1 **Binary decision tree**
A binary decision tree *is a tree representation of a partition of the feature space. Figure 8.2 shows a simple example in the case of a two-dimensional space based on two features X_1 and X_2, as well as the partition it represents. Each interior node of a decision tree corresponds to a question related to features. It can be a* numerical question *of the form $X_i \leq a$ for a feature variable X_i, $i \in [1, N]$, and some threshold $a \in \mathbb{R}$, as in the example of figure 8.2, or a* categorical question *such as $X_i \in \{blue, white, red\}$, when feature X_i takes a categorical value such as a color. Each leaf is labeled with a label $l \in \mathcal{Y}$.*

Decision trees can be defined using more complex node questions, resulting in partitions based on more complex decision surfaces. For example, *binary space*

GreedyDecisionTrees($S = ((x_1, y_1), \ldots, (x_m, y_m))$)

```
1  tree ← {n_0}  ▷ root node.
2  for t ← 1 to T do
3      (n_t, q_t) ← argmin_(n,q) F̃(n, q)
4      Split(tree, n_t, q_t)
5  return tree
```

Figure 8.3 Greedy algorithm for building a decision tree from a labeled sample S. The procedure Split($\mathsf{tree}, \mathsf{n}_t, \mathsf{q}_t$) splits node n_t by making it an internal node with question q_t and leaf children $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$ and $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$, each labeled with the dominating class of the region it defines, with ties broken arbitrarily.

partition (BSP) trees partition the space with convex polyhedral regions, based on questions of the form $\sum_{i=1}^{n} \alpha_i X_i \leq a$, and *sphere trees* partition with pieces of spheres based on questions of the form $\|X - a_0\| \leq a$, where X is a feature vector, a_0 a fixed vector, and a is a fixed positive real number. More complex tree questions lead to richer partitions and thus hypothesis sets, which can cause overfitting in the absence of a sufficiently large training sample. They also increase the computational complexity of prediction and training. Decision trees can also be generalized to branching factors greater than two, but binary trees are most commonly used due to computational considerations.

Prediction/partitioning: To predict the label of any point $x \in \mathcal{X}$ we start at the root node of the decision tree and go down the tree until a leaf is found, by moving to the right child of a node when the response to the node question is positive, and to the left child otherwise. When we reach a leaf, we associate x with the label of this leaf.

Thus, each leaf defines a *region* of $\mathcal{X}$ formed by the set of points corresponding exactly to the same node responses and thus the same traversal of the tree. By definition, no two regions intersect and all points belong to exactly one region. Thus, leaf regions define a partition of $\mathcal{X}$, as shown in the example of figure 8.2. In multi-class classification, the label of a leaf is determined using the training sample: the class with the majority representation among the training points falling in a leaf region defines the label of that leaf, with ties broken arbitrarily.

Learning: We will discuss two different methods for learning a decision tree using a labeled sample. The first method is a greedy technique. This is motivated by the fact that the general problem of finding a decision tree with the smallest error is NP-hard. The method consists of starting with a tree reduced to a single

(root) node, which is a leaf whose label is the class that has majority over the entire sample. Next, at each round, a node n_t is split based on some question q_t. The pair $(\mathsf{n}_t, \mathsf{q}_t)$ is chosen so that the *node impurity* is maximally decreased according to some measure of impurity F. We denote by $F(\mathsf{n})$ the impurity of n. The decrease in node impurity after a split of node n based on question q is defined as follows. Let $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$ denote the right child of n after the split, $\mathsf{n}_-(\mathsf{q}, \mathsf{n})$ the left child, and $\eta(\mathsf{n}, \mathsf{q})$ the fraction of the points in the region defined by n that are moved to $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$. The total impurity of the leaves $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$ and $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$ is therefore $\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))$. Thus, the decrease in impurity $\widetilde{F}(\mathsf{n}, \mathsf{q})$ by that split is given by

$$\widetilde{F}(\mathsf{n}, \mathsf{q}) = F(\mathsf{n}) - [\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))].$$

Figure 8.3 shows the pseudocode of this greedy construction based on $\widetilde{F}$. In practice, the algorithm is stopped once all nodes have reached a sufficient level of purity, when the number of points per leaf has become too small for further splitting or based on some other similar heuristic.

For any node n and class $l \in [1, k]$, let $p_l(\mathsf{n})$ denote the fraction of points at n that belong to class l. Then, the three most commonly used measures of node impurity F are defined as follows:

$$F(\mathsf{n}) = \begin{cases} 1 - \max_{l \in [1,k]} p_l(\mathsf{n}) & \textit{misclassification}; \\ -\sum_{l=1}^{k} p_l(\mathsf{n}) \log_2 p_l(\mathsf{n}) & \textit{entropy}; \\ \sum_{l=1}^{k} p_l(\mathsf{n})(1 - p_l(\mathsf{n})) & \textit{Gini index}. \end{cases}$$

Figure 8.4 illustrates these definitions in the special cases of two classes ($k = 2$). The entropy and Gini index impurity functions are upper bounds on the misclassification impurity function. All three functions are convex, which ensures that

$$F(\mathsf{n}) - [\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))] \geq 0.$$

However, the misclassification function is piecewise linear, so $\widetilde{F}(\mathsf{n}, \mathsf{q})$ is zero if the fraction of positive points remains less than (or more than) half after a split. In some cases, the impurity cannot be decreased by any split using that criterion. In contrast, the entropy and Gini functions are strictly convex, which guarantees a strict decrease in impurity. Furthermore, they are differentiable which is a useful feature for numerical optimization. Thus, the Gini index and the entropy criteria are typically preferred in practice.

The greedy method just described faces some issues. One issue relates to the greedy nature of the algorithm: a seemingly bad split may dominate subsequent useful splits, which could lead to trees with less impurity overall. This can be addressed to a certain extent by using a look-ahead of some depth d to determine

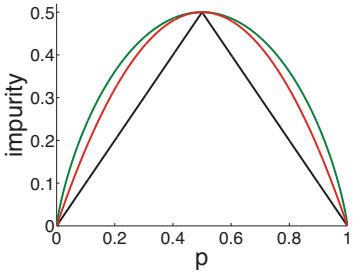


Figure 8.4 Node impurity plotted as a function of the fraction of positive examples in the binary case: misclassification (in black), entropy (in green, scaled by .5 to set the maximum to the same value for all three functions), and the Gini index (in red).

the splitting decisions, but such look-aheads can be computationally very costly. Another issue relates to the size of the resulting tree. To achieve some desired level of impurity, trees of relatively large sizes may be needed. But larger trees define overly complex hypotheses with high VC-dimensions (see exercise 9.6) and thus could overfit.

An alternative method for learning decision trees using a labeled training sample is based on the so-called *grow-then-prune strategy*. First a very large tree is grown until it fully fits the training sample or until no more than a very small number of points are left at each leaf. Then, the resulting tree, denoted as tree, is pruned back to minimize an objective function defined based on generalization bounds as the sum of an empirical error and a complexity term that can be expressed in terms of the size of $\widetilde{\mathsf{tree}}$, the set of leaves of tree:

$$G_\lambda(\mathsf{tree}) = \sum_{\mathsf{n} \in \widetilde{\mathsf{tree}}} |\mathsf{n}| F(\mathsf{n}) + \lambda |\widetilde{\mathsf{tree}}|. \tag{8.15}$$

$\lambda \geq 0$ is a regularization parameter determining the trade-off between misclassification, or more generally impurity, versus tree complexity. For any tree tree', we denote by $\widehat{R}(\mathsf{tree}')$ the total empirical error $\sum_{\mathsf{n} \in \widetilde{\mathsf{tree}}'} |\mathsf{n}| F(\mathsf{n})$. We seek a sub-tree tree_λ of tree that minimizes G_λ and that has the smallest size. tree_λ can be shown to be unique. To determine tree_λ, the following pruning method is used, which defines a finite sequence of nested sub-trees $\mathsf{tree}^{(0)}, \ldots, \mathsf{tree}^{(n)}$. We start with the full tree $\mathsf{tree}^{(0)} = \mathsf{tree}$ and for any $i \in [0, n-1]$, define $\mathsf{tree}^{(i+1)}$ from $\mathsf{tree}^{(i)}$ by collapsing an internal node n' of $\mathsf{tree}^{(i)}$, that is by replacing the sub-tree rooted at n' with a leaf, or equivalently by combining the regions of all the leaves dominated by n'. n' is chosen so that collapsing it causes the smallest per node increase in $\widehat{R}(\mathsf{tree}^{(i)})$,

that is the smallest $r(\mathsf{tree}^{(i)}, \mathsf{n}')$ defined by

$$r(\mathsf{tree}^{(i)}, \mathsf{n}') = \frac{|\mathsf{n}'|F(\mathsf{n}') - \widehat{R}(\mathsf{tree}')}{|\widetilde{\mathsf{tree}}'| - 1},$$

where n' is an internal node of $\mathsf{tree}^{(i)}$. If several nodes n' in $\mathsf{tree}^{(i)}$ cause the same smallest increase per node $r(\mathsf{tree}^{(i)}, \mathsf{n}')$, then all of them are pruned to define $\mathsf{tree}^{(i+1)}$ from $\mathsf{tree}^{(i)}$. This procedure continues until the tree $\mathsf{tree}^{(n)}$ obtained has a single node. The sub-tree tree_λ can be shown to be among the elements of the sequence $\mathsf{tree}^{(0)}, \ldots, \mathsf{tree}^{(n)}$. The parameter λ is determined via n-fold cross-validation.

Decision trees seem relatively easy to interpret, and this is often underlined as one of their most useful features. However, such interpretations should be carried out with care since decision trees are *unstable*: small changes in the training data may lead to very different splits and thus entirely different trees, as a result of their hierarchical nature. Decision trees can also be used in a natural manner to deal with the problem of *missing features* , which often appears in learning applications; in practice, some features values may be missing because the proper measurements were not taken or because of some noise source causing their systematic absence. In such cases, only those variables available at a node can be used in prediction. Finally, decision trees can be used and learned from data in a similar way in *regression* (see chapter 10).[2]

8.4 Aggregated multi-class algorithms

In this section, we discuss a different approach to multi-class classification that reduces the problem to that of multiple binary classification tasks. A binary classification algorithm is then trained for each of these tasks independently, and the multi-class predictor is defined as a combination of the hypotheses returned by each of these algorithms. We first discuss two standard techniques for the reduction of multi-class classification to binary classification, and then show that they are both special instances of a more general framework.

8.4.1 One-versus-all

Let $S = ((x_1, y_1), \ldots, x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ be a labeled training sample. A straightforward reduction of the multi-class classification to binary classification

2. The only changes to the description for classification are the following. For prediction, the label of a leaf is defined as the mean squared average of the labels of the points falling in that region. For learning, the impurity function is the mean squared error.

is based on the so-called *one-versus-all (OVA) or one-versus-the-rest technique*. This technique consists of learning k binary classifiers $h_l \colon \mathcal{X} \to \{-1, +1\}$, $l \in \mathcal{Y}$, each seeking to discriminate one class $l \in \mathcal{Y}$ from all the others. For any $l \in \mathcal{Y}$, h_l is obtained by training a binary classification algorithm on the full sample S after relabeling points in class l with 1 and all others with -1. For $l \in \mathcal{Y}$, assume that h_l is derived from the sign of a scoring function $f_l \colon \mathcal{X} \to \mathbb{R}$, that is $h_l = \operatorname{sgn}(f_l)$, as in the case of many of the binary classification algorithms discussed in the previous chapters. Then, the multi-class hypothesis $h \colon \mathcal{X} \to \mathcal{Y}$ defined by the OVA technique is given by:

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmax}_{l \in Y} f_l(x). \tag{8.16}$$

This formula may seem similar to those defining a multi-class classification hypothesis in the case of uncombined algorithms. Note, however, that for uncombined algorithms the functions f_l are learned together, while here they are learned independently. Formula (8.16) is well-founded when the scores given by functions f_l can be interpreted as confidence scores, that is when $f_l(x)$ is learned as an estimate of the probability of x conditioned on class l. However, in general, the scores given by functions f_l, $l \in \mathcal{Y}$, are not comparable and the OVA technique based on (8.16) admits *no principled justification*. This is sometimes referred to as a *calibration problem*. Clearly, this problem cannot be corrected by simply normalizing the scores of each function to make their magnitudes uniform, or by applying other similar heuristics. When it is justifiable, the OVA technique is simple and its computational cost is k times that of training a binary classification algorithm, which is similar to the computation costs for many uncombined algorithms.

8.4.2 One-versus-one

An alternative technique, known as the *one-versus-one (OVO) technique*, consists of using the training data to learn (independently), for each pair of distinct classes $(l, l') \in \mathcal{Y}^2$, $l \neq l'$, a binary classifier $h_{ll'} \colon \mathcal{X} \to \{-1, 1\}$ discriminating between classes l and l'. For any $(l, l') \in \mathcal{Y}^2$, $h_{ll'}$ is obtained by training a binary classification algorithm on the sub-sample containing exactly the points labeled with l or l', with the value $+1$ returned for class l' and -1 for class l. This requires training $\binom{k}{2} = k(k-1)/2$ classifiers, which are combined to define a multi-class classification hypothesis h via majority vote:

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmax}_{l' \in Y} \big|\{l \colon h_{ll'}(x) = 1\}\big|. \tag{8.17}$$

Thus, for a fixed point $x \in \mathcal{X}$, if we describe the prediction values $h_{ll'}(x)$ as the results of the matches in a tournament between two players l and l', with $h_{ll'}(x) = 1$

	Training	Testing
OVA	$O(km^\alpha)$	$O(kc_t)$
OVO	$O(k^{2-\alpha}m^\alpha)$	$O(k^2c_t)$

Table 8.1 Comparison of the time complexity the OVA and OVO techniques for both training and testing. The table assumes a full training sample of size m with each class represented by m/k points. The time for training a binary classification algorithm on a sample of size n is assumed to be in $O(n^\alpha)$. Thus, the training time for the OVO technique is in $O(k^2(m/k)^\alpha) = O(k^{2-\alpha}m^\alpha)$. c_t denotes the cost of testing a single classifier.

indicating l' winning over l, then the class predicted by h can be interpreted as the one with the largest number of wins in that tournament.

Let $x \in \mathcal{X}$ be a point belonging to class l'. By definition of the OVO technique, if $h_{ll'}(x) = 1$ for all $l \neq l'$, then the class associated to x by OVO is the correct class l' since $\big|\{l\colon h_{ll'}(x) = 1\}\big| = k-1$ and no other class can reach $(k-1)$ wins. By contraposition, if the OVO hypothesis misclassifies x, then at least one of the $(k-1)$ binary classifiers $h_{ll'}$, $l \neq l'$, incorrectly classifies x. Assume that the generalization error of all binary classifiers $h_{ll'}$ used by OVO is at most r, then, in view of this discussion, the generalization error of the hypothesis returned by OVO is at most $(k-1)r$.

The OVO technique is not subject to the calibration problem pointed out in the case of the OVA technique. However, when the size of the sub-sample containing members of the classes l and l' is relatively small, $h_{ll'}$ may be learned without sufficient data or with increased risk of overfitting. Another concern often raised for the use of this technique is the computational cost of training $k(k-1)/2$ binary classifiers versus that of the OVA technique.

Taking a closer look at the computational requirements of these two methods reveals, however, that the disparity may not be so great and that in fact under some assumptions the time complexity of training for OVO could be less than that of OVA. Table 8.1 compares the computational complexity of these methods both for training and testing assuming that the complexity of training a binary classifier on a sample of size m is in $O(m^\alpha)$ and that each class is equally represented in the training set, that is by m/k points. Under these assumptions, if $\alpha \in [2, 3)$ as in the case of some algorithms solving a QP problem, such as SVMs, then the time complexity of training for the OVO technique is in fact more favorable than that of OVA. For $\alpha = 1$, the two are comparable and it is only for sub-linear algorithms that the OVA technique would benefit from a better complexity. In all cases, at test time, OVO requires $k(k-1)/2$ classifier evaluations, which is $(k-1)$ times more than